DP-800-BATERIA-4

Number: 000-000
Passing Score: 700
Time Limit: 120 min
File Version: 1.0

Exam A

QUESTION 1
Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment -

Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.

Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:

AI workloads -

Vector search -

Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.

Requirements -

Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.

Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:

Vector similarity -

Keyword filtering or boosting -

Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.
Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to meet the development requirements for the FeedbackJson column.
How should you complete the Transact-SQL query? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
SELECT
  f.FeedbackId,
  f.VehicleId,
```

[▼] ,

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

```
  EDIT_DISTANCE_SIMILARITY(
    JSON_VALUE(f.FeedbackJson, '$.text'),
    @KnownIssueDescription
  ) AS SimilarityScore
FROM
  dbo.CustomerFeedback f
WHERE
```

[▼]

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

```
ORDER BY
```

[▼]

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

```
DESC;
```

Answer:

Answer Area

```
SELECT
  f.FeedbackId,
  f.VehicleId,
```

[▼] ,

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

```
  EDIT_DISTANCE_SIMILARITY(
    JSON_VALUE(f.FeedbackJson, '$.text'),
    @KnownIssueDescription
  ) AS SimilarityScore
FROM
  dbo.CustomerFeedback f
WHERE
```

[▼]

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

```
ORDER BY
```

[▼]

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

```
DESC;
```

Section: (none)

Explanation/Reference:
Q01 T01

Correct Answers:

Box 1 (SELECT clause): JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText

Box 2 (WHERE clause): CONTAINS(FeedbackJson, @Keyword)

Box 3 (ORDER BY clause): SimilarityScore

Explanation:

Box 1 — Extracting the Feedback Text:
El requisito es extraer el texto del feedback del cliente desde el documento JSON — un único valor de texto escalar. JSON_VALUE está diseñado exactamente para eso: extrae un valor escalar de una cadena JSON en una ruta determinada. JSON_QUERY, en cambio, extrae un objeto o un array (no un escalar), y solo acepta dos argumentos (expression, path) — la opción que ofrece un tercer argumento (@KnownIssueDescription) no es una sintaxis válida para JSON_QUERY en absoluto, lo cual la descarta tanto por motivos semánticos como sintácticos.

Box 2 — Filtering by Keyword:
El case study establece explícitamente que la columna FeedbackJson ya tiene un índice de texto completo (full-text index), y el requisito es filtrar las filas donde el texto JSON contiene una palabra clave — una búsqueda de contención literal, no una coincidencia aproximada/difusa. CONTAINS es el predicado de Full-Text Search creado específicamente para aprovechar un índice de texto completo en este tipo de búsqueda por palabra clave, y es mucho más eficiente que las opciones con EDIT_DISTANCE, las cuales miden la distancia de edición a nivel de carácter (una comparación de errores tipográficos difusa) en lugar de contención literal de una palabra clave — y una de esas dos opciones de EDIT_DISTANCE además apunta a la ruta JSON incorrecta ($.details.comment en vez de $.text), lo cual la hace doblemente incorrecta.

Box 3 — Ordering by Similarity Score:
El requisito es ordenar los resultados por puntuación de similitud, con la más alta primero. SimilarityScore es el alias ya definido en la lista del SELECT para EDIT_DISTANCE_SIMILARITY(...), y dado que esa función devuelve un valor de 0 a 100 donde 100 = mayor similitud, ordenar por SimilarityScore DESC (el DESC ya viene fijo en la plantilla) coloca correctamente la mayor similitud primero..

References:

Box 1 — JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText

https://learn.microsoft.com/en-us/sql/t-sql/functions/json-value-transact-sql

"Use the JSON_VALUE syntax to extract a scalar value from a JSON string... To extract an object or an array from a JSON string instead of a scalar value, see JSON_QUERY."

Box 2 — CONTAINS(FeedbackJson, @Keyword)

https://learn.microsoft.com/en-us/sql/t-sql/queries/contains-transact-sql AND https://learn.microsoft.com/en-us/sql/relational-databases/search/full-text-search

Box 3 — (ORDER BY clause) → SimilarityScore

https://learn.microsoft.com/en-us/sql/t-sql/functions/edit-distance-similarity-transact-sql — "Calculates a similarity value ranging from 0 (indicating no match) to 100 (indicating full match)."

QUESTION 2
You have an Azure SQL database that contains a table named dbo.Orders.

You have an application that calls a stored procedure named dbo.usp_CreateOrder to insert rows into dbo.Orders.

When an insert fails, the application receives inconsistent error details.

You need to implement error handling to ensure that any failures inside the procedure abort the transaction and return a consistent error to the caller.

How should you complete the stored procedure? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

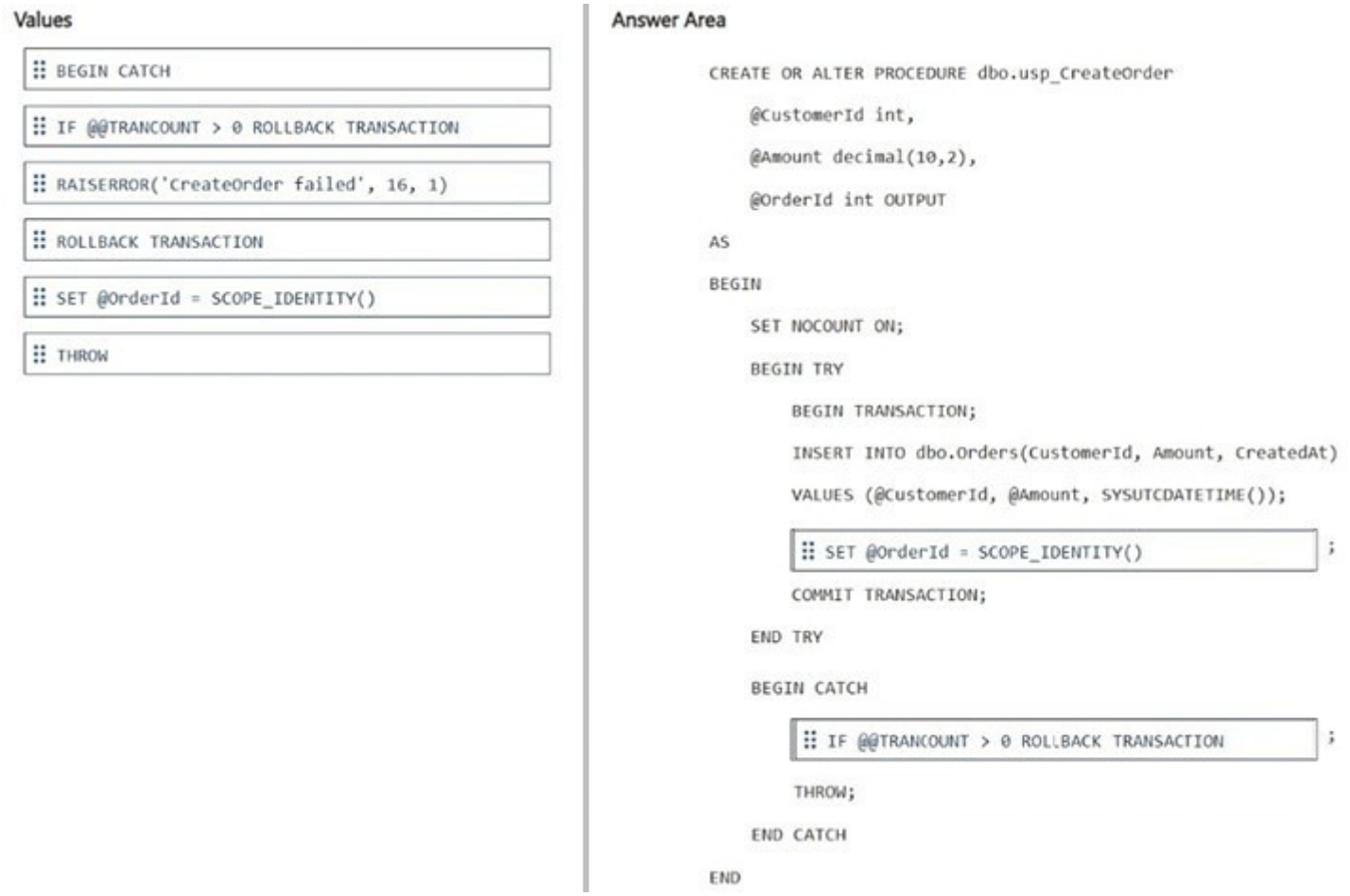
Values

- `BEGIN CATCH`
- `IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION`
- `RAISERROR('CreateOrder failed', 16, 1)`
- `ROLLBACK TRANSACTION`
- `SET @OrderId = SCOPE_IDENTITY()`
- `THROW`

Answer Area

```
CREATE OR ALTER PROCEDURE dbo.usp_CreateOrder
    @CustomerId int,
    @Amount decimal(10,2),
    @OrderId int OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        INSERT INTO dbo.Orders(CustomerId, Amount, CreatedAt)
        VALUES (@CustomerId, @Amount, SYSUTCDATETIME());
        [                                        ];
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        [                                        ];
        THROW;
    END CATCH
END
```

Answer:



Section: (none)

Explanation/Reference:
Q02 T01

Bloque TRY (antes de COMMIT TRANSACTION): SET @OrderId = SCOPE_IDENTITY()
Bloque CATCH (antes de THROW): IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION

Este orden garantiza que el OUTPUT se asigne dentro de la transacción exitosa, y que cualquier fallo revierta la transacción de forma segura antes de propagar el error original y consistente al llamador mediante THROW.

Justificación Técnica y Arquitectura

El patrón de manejo de errores transaccional recomendado por Microsoft para procedimientos almacenados en Azure SQL Database:

```
BEGIN TRY
   BEGIN TRANSACTION;
   INSERT INTO dbo.Orders(CustomerId, Amount, CreatedAt)
   VALUES (@CustomerId, @Amount, SYSUTCDATETIME());
   SET @OrderId = SCOPE_IDENTITY();
   COMMIT TRANSACTION;
END TRY
BEGIN CATCH
   IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
   THROW;
END CATCH
```

- SCOPE_IDENTITY() debe ejecutarse inmediatamente después del INSERT y ANTES del COMMIT, ya que captura el último valor de identidad generado dentro del scope actual del procedimiento (a diferencia de @@IDENTITY o IDENT_CURRENT).

- IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION en el bloque CATCH es el patrón oficial porque valida si la transacción sigue abierta antes de intentar el rollback, evitando el error 3903 en transacciones ya marcadas como "doomed" (XACT_STATE() = -1).

- THROW (sin parámetros) relanza la excepción original preservando número de error, severidad, estado y mensaje exacto, cumpliendo el requisito de "error consistente al llamador".

Desglose de Opciones Incorrectas

Opción BEGIN CATCH:
Ya es parte fija de la plantilla del área de respuesta (etiqueta estructural), no es un valor variable para los huecos.

Opción RAISERROR('CreateOrder failed', 16, 1):
Genera un mensaje genérico, perdiendo el número de error, la línea y el contexto original de la excepción. Contradice el requisito de error consistente.

Opción ROLLBACK TRANSACTION (sin el IF @@TRANCOUNT > 0):
Un ROLLBACK incondicional es inseguro: si la transacción ya fue revertida automáticamente (doomed transaction), lanza el error 3903 y enmascara el error real.

Opción THROW (usada en el hueco del bloque TRY):
No aplica dentro de TRY sin una excepción activa capturada. THROW sin parámetros solo es válido dentro de un bloque CATCH.

Opción SET @OrderId = SCOPE_IDENTITY() (usada en el hueco del CATCH):
Es funcionalmente inútil en el bloque CATCH, ya que en ese punto la inserción falló y no existe un valor de identidad válido que asignar.

Referencias Oficiales Microsoft Learn

- TRY...CATCH (Transact-SQL):
https://learn.microsoft.com/sql/t-sql/language-elements/try-catch-transact-sql?view=sql-server-ver17

- Uncommittable transactions y XACT_STATE:
https://learn.microsoft.com/sql/t-sql/language-elements/try-catch-transact-sql?view=sql-server-ver17#uncommittable-transactions-and-xact_state

- THROW (Transact-SQL):
https://learn.microsoft.com/sql/t-sql/language-elements/throw-transact-sql?view=sql-server-ver17

- Transaction locking and row versioning guide:
https://learn.microsoft.com/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?view=sql-server-ver17#transaction-basics

Advertencia sobre bancos de preguntas: algunas versiones de este reactivo en ExamTopics votan por ROLLBACK TRANSACTION sin validar @@TRANCOUNT. La documentación oficial confirma que ese patrón es incorrecto en transacciones "doomed", por lo que la respuesta correcta es la que incluye IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION.

QUESTION 3
Your team is developing an Azure SQL dataset solution from a locally cloned GitHub repository by using Microsoft Visual Studio Code and GitHub Copilot Chat.
You need to disable the GitHub Copilot repository-level instructions for yourself without affecting other

users.
What should you do?

A. From Visual Studio Code, modify your GitHub Copilot Chat user settings.
B. Add a -debug flag when you start the GitHub Copilot Chat extension.
C. Delete .github/copilot-instructions.md.

Answer: A
Section: (none)

Explanation/Reference:
Q03 T01

Correct Answer:
From Visual Studio Code, modify your GitHub Copilot Chat user settings.

Explanation:
SÍ es la opción A, porque en la documentación menciona específicamente que se pueden modificar desde el menú de opciones de Visual Studio Code.
NO es la opción B, porque la documentación no menciona ningún flag de arranque como mecanismo para controlar el uso de instrucciones personalizadas.
NO es la opción C, porque ".github/copilot-instructions.md" es un archivo compartido. Eliminarlo no solo afectaría al usuario actual.

References:
https://learn.microsoft.com/visualstudio/ide/copilot-chat-context?view=visualstudio (sección “Use a .github/copilot-instructions.md file”)

QUESTION 4
You have an Azure SQL database that contains the following SQL graph tables:
A NODE table named dbo.Person -
An EDGE table named dbo.Knows -
Each row in dbo.Person contains the following columns:
PersonID (int)
DisplayName (nvarchar(100))
You need to use a MATCH operator and exactly two directed Knows relationships to return the PersonID and DisplayName of people that are reachable from the person identified by an input parameter named @StartPersonId.
Which Transact-SQL query should you use?

A.
```
SELECT p2.PersonId, @StartPersonId
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.DisplayName = p2.DisplayName
   AND MATCH(p1-(k1)->p2-(k2)->p3);
```
B.
```
SELECT p3.PersonId, p3.DisplayName
FROM dbo.Person AS p1
JOIN dbo.Knows AS k1 ON 1 = 1
JOIN dbo.Person AS p2 ON 1 = 1
JOIN dbo.Knows AS k2 ON 1 = 1
JOIN dbo.Person AS p3 ON 1 = 1
WHERE p1.PersonId = @StartPersonId
   AND MATCH(p3<-(k2)-p2<-(k1)-p1);
```
C.
```
SELECT p3.PersonId, p3.DisplayName
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.PersonId = @StartPersonId
   AND MATCH(p1-(k1)->p2)
   AND MATCH(p2-(k2)->p3);
```

D. SELECT p3.PersonId, p3.DisplayName
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.PersonId = @StartPersonId
AND MATCH(p1-(k1)->p2-(k2)->p3);

Answer: D
Section: (none)

Explanation/Reference:
Q04 T01

Correct Answer:

```
SELECT p3.PersonId, p3.DisplayName
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2,
    dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.PersonId = @StartPersonId
   AND MATCH(p1-(k1)->p2-(k2)->p3);
```

Explanation:

La consulta correcta utiliza un único MATCH() para representar un recorrido de exactamente dos relaciones dirigidas Knows:

p1 --k1--> p2 --k2--> p3

p1 representa la persona inicial y se limita mediante:

p1.PersonId = @StartPersonId

La primera relación:

p1-(k1)->p2

indica que la persona inicial conoce a p2.

La segunda relación:

p2-(k2)->p3

indica que p2 conoce a p3.

Por tanto, p3 representa las personas situadas exactamente a dos saltos de la persona indicada mediante @StartPersonId. Finalmente:

SELECT p3.PersonId, p3.DisplayName

devuelve precisamente las dos columnas solicitadas.

Microsoft muestra prácticamente este mismo patrón para encontrar un "friend of a friend":

Person1 -(friend)-> Person2 -(friend2)-> Person3

es decir, dos relaciones dirigidas consecutivas dentro del mismo MATCH().

Por qué no las otras opciones:

```
SELECT p2.PersonId, @StartPersonId
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.DisplayName = p2.DisplayName
   AND MATCH(p1-(k1)->p2-(k2)->p3);
```

Aunque contiene el patrón de dos saltos, no utiliza @StartPersonId para localizar a p1. En su lugar compara p1.DisplayName = p2.DisplayName. Además, devuelve p2.PersonId y @StartPersonId, no p3. PersonId y p3.DisplayName. Por tanto, no satisface el resultado solicitado.

```
SELECT p3.PersonId, p3.DisplayName
FROM dbo.Person AS p1
JOIN dbo.Knows AS k1 ON 1 = 1
JOIN dbo.Person AS p2 ON 1 = 1
JOIN dbo.Knows AS k2 ON 1 = 1
JOIN dbo.Person AS p3 ON 1 = 1
WHERE p1.PersonId = @StartPersonId
   AND MATCH(p3<-(k2)-p2<-(k1)-p1);
```

El patrón gráfico escrito al revés, MATCH(p3<-(k2)-p2<-(k1)-p1) no es el problema; Microsoft indica que un patrón dirigido puede escribirse desde cualquiera de los extremos. Por ejemplo, Person1-(friend)->Person2 es equivalente a Person2<-(friend)-Person1.

El problema es que esta opción introduce las tablas mediante:

```
JOIN ... ON 1 = 1
```

y después usa esos aliases en MATCH. SQL Server documenta específicamente el error 13920: los identificadores usados en MATCH no pueden estar asociados a JOIN o APPLY.

```
SELECT p3.PersonId, p3.DisplayName
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.PersonId = @StartPersonId
   AND MATCH(p1-(k1)->p2)
   AND MATCH(p2-(k2)->p3);
```

Describe correctamente las dos relaciones, pero utiliza dos operadores MATCH() independientes:

```
MATCH(p1-(k1)->p2)
AND MATCH(p2-(k2)->p3)
```

El requisito pide utilizar un operador MATCH() con exactamente dos relaciones dirigidas. La sintaxis apropiada es expresar todo el camino de dos saltos dentro de un único patrón:

```
MATCH(p1-(k1)->p2-(k2)->p3)
```

Microsoft utiliza exactamente esta construcción para los recorridos multi-hop.

Por ello, la consulta correcta es la que realiza:

```
@StartPersonId → p1 → k1 → p2 → k2 → p3
                          ↓
                  PersonId + DisplayName
```

References:

https://learn.microsoft.com/es-es/sql/t-sql/queries/match-sql-graph?view=sql-server-ver17
Documenta la sintaxis de MATCH, la dirección de las aristas y el patrón para recorrer varios saltos. Se aplica a SQL Server, Azure SQL Database, Azure SQL Managed Instance y SQL Database in Microsoft Fabric.

https://learn.microsoft.com/en-us/sql/relational-databases/graphs/sql-graph-overview?view=sql-server-ver17
Explica las tablas NODE, EDGE y cómo MATCH permite realizar navegación multi-hop mediante SQL Graph. También documenta el soporte de SQL Graph en SQL Database de Microsoft Fabric.

https://learn.microsoft.com/en-us/sql/relational-databases/errors-events/database-engine-events-and-errors-13000-to-13999?view=sql-server-ver17

El error 13920 confirma que JOIN y APPLY no se admiten para identificadores utilizados dentro de una cláusula MATCH.

QUESTION 5
You have a SQL database in Microsoft Fabric that contains a column named Payload. Payload stores customer data in JSON documents that have the following format.
text
```
{
  "date": "2020-01-25",
  "customer_email": "user@contoso.com",
  ...
}
```
Data analysis shows that some customers have subaddressing in their email address, for example, user1 +promo@contoso.com.
You need to return a normalized email value that removes the subaddressing, for example, user1 +promo@contoso.com must be normalized to user1@contoso.com.
Which Transact-SQL expression should you use?

A. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*$', '')
B. REGEXP_SUBSTR(JSON_VALUE(Payload, '$.customer_email'), '^[^+]+@.*$=')
C. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*@', '@')
D. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*', '')

Answer: C
Section: (none)

Explanation/Reference:
Q05 T01

Correct Answer : REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '+.*@', '@')

Explanation:La opción correcta es REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '+.*@', '@') porque busca desde el signo + hasta el @ (incluido) y lo sustituye por un único @. De esta forma se elimina solo el subaddressing y se mantiene intacta la parte local del email junto con el dominio, obteniendo exactamente user1@contoso.com.

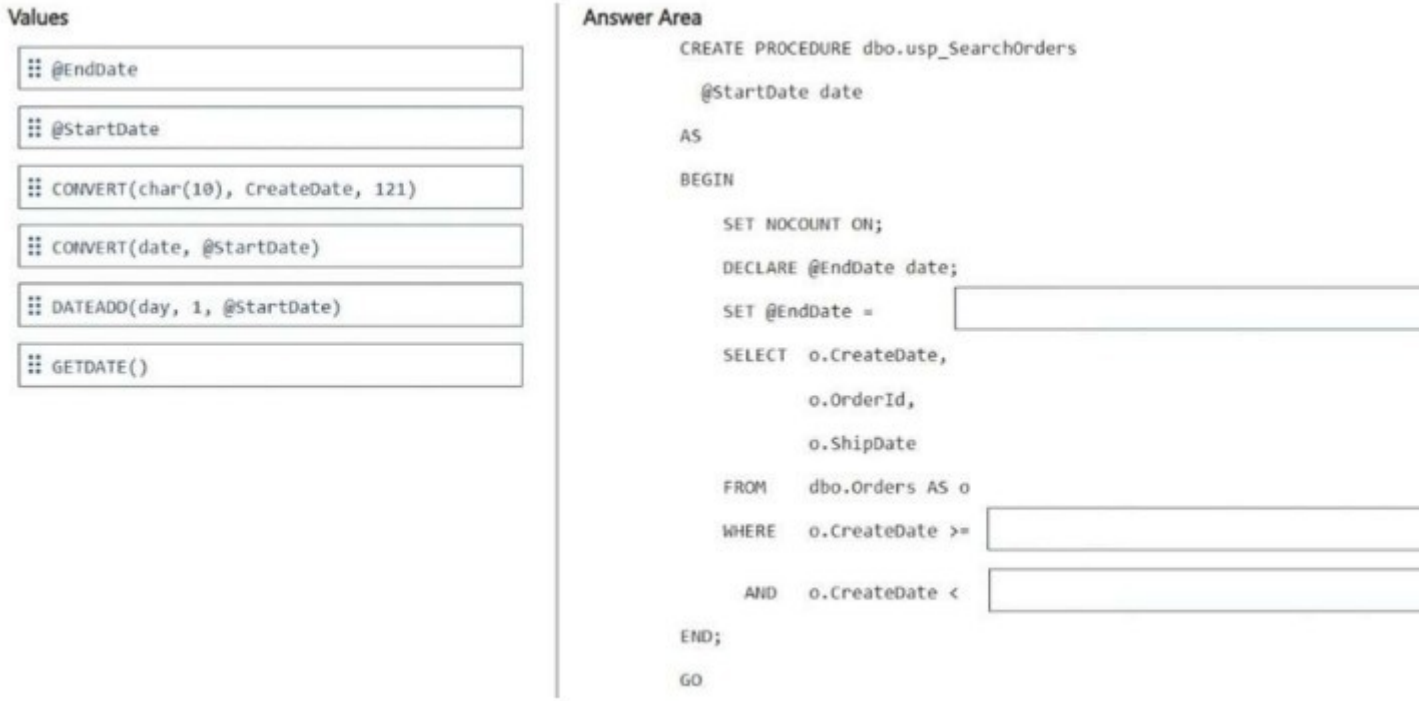
REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*$', '') es incorrecta: el patrón '\+.*$' elimina desde el + hasta el final de la cadena, por lo que se pierde el dominio y queda únicamente user1.
REGEXP_SUBSTR(JSON_VALUE(Payload, '.customer_email'), '^[^+]+@.*') es incorrecta: el patrón '^[^+]+@.*$' exige que no exista ningún + antes del @. Cuando hay subaddressing el patrón no hace match y la función devuelve NULL.
REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '+.*', '') es incorrecta: el patrón '\+.*' elimina desde el + en adelante (incluyendo el dominio), dejando solo user1.

References:

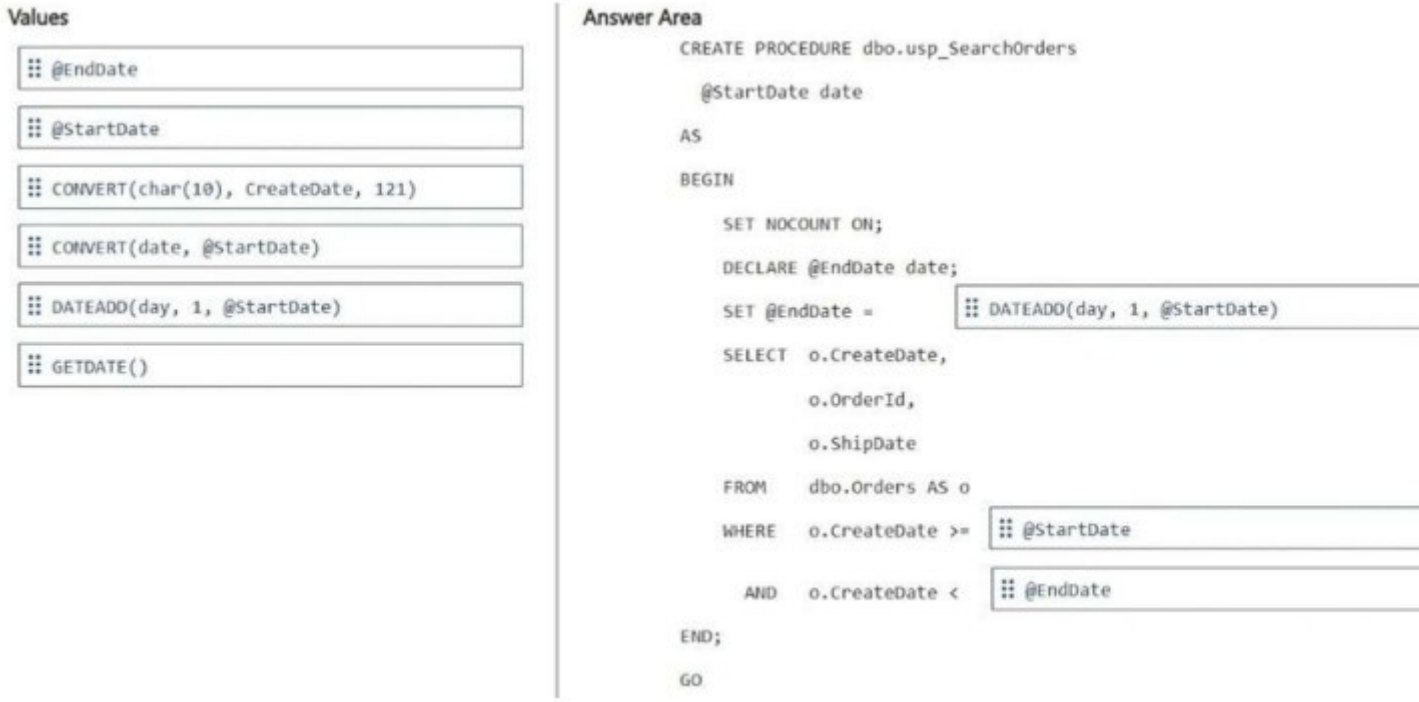
REGEXP_REPLACE (Transact-SQL) – Microsoft Learn (SQL database in Microsoft Fabric): https://learn.microsoft.com/en-us/sql/t-sql/functions/regexp-replace-transact-sql
REGEXP_SUBSTR (Transact-SQL) – Microsoft Learn: https://learn.microsoft.com/en-us/sql/t-sql/functions/regexp-substr-transact-sql
JSON_VALUE (Transact-SQL) – Microsoft Learn: https://learn.microsoft.com/en-us/sql/t-sql/functions/json-value-transact-sql
Regular Expressions Functions (Transact-SQL) – Microsoft Learn: https://learn.microsoft.com/en-us/sql/t-sql/functions/regular-expressions-functions-transact-sql

QUESTION 6
You have an Azure SQL database that containS a table named dbo_Orders. db0_Orders containS a col umn named CreateDate that Stores Order

creation dates.
You need to create a procedure that filters Orders by CreateDate for a single calendar day. The solution must be SARGable.
HOW Should complete the Transact-SQL cade? TO answer, drag the appropriate values to the correct targets. Each value may be used once,
more than once, or not at all _ You may need to drag the split bar between panes or scroll to View content.
NOTE: Each correct se' ection is worlh one point.



Answer:



Section: (none)

Explanation/Reference:
Q06 T01

1. Analisis del Escenario

Se necesita un procedimiento almacenado que filtre la tabla dbo.Orders por la columna CreateDate para un unico dia calendario.
La solucion debe ser SARGable (Search ARGument Able), es decir, permitir el uso eficiente de indices sin aplicar funciones sobre la columna.
Dominio evaluado: Diseno y desarrollo de soluciones de base de datos (T-SQL avanzado y optimizacion de consultas).

2. Solucion Correcta
Parametro de entrada: @StartDate date

Calculo del limite superior: SET @EndDate = DATEADD(day, 1, @StartDate)

Filtro SARGable:
WHERE o.CreateDate >= @StartDate

AND o.CreateDate < @EndDate
Codigo completo correcto:

```
CREATE PROCEDURE dbo.usp_SearchOrders
@StartDate date
AS
BEGIN
SET NOCOUNT ON;
DECLARE @EndDate date;
SET @EndDate = DATEADD(day, 1, @StartDate);
SELECT o.CreateDate,
o.OrderId,
o.ShipDate
FROM dbo.Orders AS o
WHERE o.CreateDate >= @StartDate
AND o.CreateDate < @EndDate;
END;
```

3. Justificacion Tecnica
Para que una prediccion sea SARGable, nunca se debe aplicar una funcion sobre la columna (CreateDate).

Al declarar el parametro como date y calcular el dia siguiente con DATEADD(day, 1, @StartDate), se obtiene un rango semi-abierto:

= @StartDate → incluye desde las 00:00:00 del dia solicitado
< @EndDate → excluye el dia siguiente

Este patron permite al optimizador de Azure SQL realizar un Index Seek (si existe un indice en CreateDate) en lugar de un Index Scan, garantizando rendimiento predecible incluso con grandes volumenes de datos.

4. Descarte de Alternativas

CONVERT(char(10), CreateDate, 121) o CONVERT(date, CreateDate) → Aplican una funcion sobre la columna → el predicado deja de ser SARGable y fuerza un scan.
GETDATE() → No tiene relacion con el filtrado de un dia concreto proporcionado por el usuario.
Comparaciones con igualdad (=) o rangos cerrados incorrectos → No cubren correctamente todo el dia o generan inclusiones/exclusiones erroneas de horas.
Cualquier expresion que envuelva CreateDate en funciones (CAST, CONVERT, DATEADD sobre la columna, etc.) invalida el uso del indice.

5. Referencias Oficiales

https://learn.microsoft.com/en-us/sql/t-sql/queries/select-transact-sql – Explica por que las funciones sobre

columnas impiden el uso de indices.
https://learn.microsoft.com/en-us/sql/t-sql/functions/dateadd-transact-sql – Documentacion oficial de la funcion utilizada para calcular el dia siguiente de forma segura.
https://learn.microsoft.com/en-us/azure/azure-sql/database/performance-guidance – Mejores practicas de Microsoft para mantener predicados SARGable en Azure SQL.

QUESTION 7
You have an Azure SQL database.
You need to create a scalar user-defined function (UDF) that returns the number of whole years between an input parameter named @OrderDate and the current date/time as a single positive integer. The function must be created in Azure SQL Database.
You write the following code.

```
01 CREATE FUNCTION dbo.ufnYearsSinceOrder (@OrderDate datetime2)
02 RETURNS int
03 AS
04 BEGIN
05
06 END
```

What should you insert at line 05?

A. RETURN DATEDIFF(year, GETDATE(), @OrderDate);
B. DATEDIFF(month, @orderdate, GETDATE()) / 12
C. DATEPART(year, GETDATE()) - DATEPART(year, @orderdate)
D. RETURN DATEDIFF(year, @OrderDate, GETDATE());

Answer: D
Section: (none)

Explanation/Reference:
Q07 T01

Correct Answer:

RETURN DATEDIFF(year, @OrderDate, GETDATE());

Explanation:

La opción correcta es **RETURN DATEDIFF(year, @OrderDate, GETDATE());**, colocando @OrderDate como startdate y GETDATE() como enddate, precedida de la sentencia RETURN. Según la documentación oficial de Transact-SQL, DATEDIFF devuelve el número de límites de la unidad de fecha (en este caso, años) cruzados entre startdate y enddate como un valor entero con signo. Al poner la fecha de pedido primero y la fecha/hora actual después, se obtiene un entero positivo que representa el número de años completos transcurridos. Además, la sintaxis de una función escalar definida por el usuario (scalar UDF) en Azure SQL Database exige, dentro del bloque BEGIN…END, una sentencia RETURN que devuelva la expresión escalar del tipo declarado (int).

La opción **RETURN DATEDIFF(year, GETDATE(), @OrderDate);** invierte el orden de los argumentos de DATEDIFF (GETDATE() como startdate y @OrderDate como enddate). Esto produce un valor negativo, lo cual no cumple el requisito de devolver un único entero positivo.

La opción **DATEDIFF(month, @orderdate, GETDATE()) / 12** carece de la sentencia RETURN obligatoria en el cuerpo de una función escalar multi-sentencia. Además, dividir la diferencia en meses entre 12 no garantiza el cálculo exacto de años completos (no tiene en cuenta el día del mes ni los años bisiestos de forma precisa).

La opción **DATEPART(year, GETDATE()) - DATEPART(year, @orderdate)** también omite la sentencia RETURN requerida. Además, restar únicamente las partes de año (DATEPART) no calcula los años completos: no verifica si la fecha de aniversario ya ha pasado en el año actual, por lo que puede devolver un valor incorrecto.

References:

https://learn.microsoft.com/sql/t-sql/functions/datediff-transact-sql?view=sql-server-ver17
https://learn.microsoft.com/sql/t-sql/statements/create-function-transact-sql?view=sql-server-ver17

QUESTION 8
You have an Azure SQL database.
You deploy Data API builder (DAB) to Azure Container Apps by using the mcr.microsoft.com/azure-databases/data-api-builder:latest image.
You have the following Container Apps secrets:
MSSQL_CONNECTION_STRING that maps to the SQL connection string
DAB_CONFIG_BASE64 that maps to the DAB configuration
You need to initialize the DAB configuration to read the SQL connection string.
Which command should you run?

A. dab init --database-type mssql --connection-string “secretref:DAB_CONFIG_BASE64” --host-mode Production --config dab-config.json
B. dab init --database-type mssql --connection-string “@env(‘MSSQL_CONNECTION_STRING’)” --host-mode Production --config dab-config.json
C. dab init --database-type mssql --connection-string “secretref:mssql-connection-string” --host-mode Production --config dab-config.json
D. dab init --database-type mssql --connection-string “@env(‘DAB_CONFIG_BASE64’)” --host-mode Production --config dab-config.json

Answer: B
Section: (none)

Explanation/Reference:
Q08 T01

Correct Answer:
dab init --database-type mssql --connection-string “@env(‘MSSQL_CONNECTION_STRING’)” --host-mode Production --config dab-config.json

Explanation:
Por qué es correcta

DAB usa la sintaxis @env('VARIABLE_NAME') para resolver, en tiempo de ejecución, el valor de una variable de entorno del proceso host. Cuando despliegas DAB en Azure Container Apps, los "secrets" que defines en Container Apps se exponen al contenedor como variables de entorno (normalmente mapeándolos explícitamente en la definición del container app). Por eso, para que DAB lea la cadena de conexión desde el secret MSSQL_CONNECTION_STRING, hay que referenciarlo con @env ('MSSQL_CONNECTION_STRING') en el parámetro --connection-string. Esto es exactamente lo que muestra la documentación oficial de DAB, incluida la página del comando init: "Database connection string. Supports @env()." con el ejemplo dab init --database-type mssql --connection-string "@env ('MSSQL_CONNECTION_STRING')", generando un dab-config.json con ese mismo valor literal.

También en la guía de despliegue a Azure App Service se sigue el mismo patrón: se recomienda usar la función @env() para referenciar la variable de entorno de la cadena de conexión, de modo que las credenciales no queden almacenadas en el archivo de configuración.

Por qué las otras opciones son incorrectas
A. --connection-string "secretref:DAB_CONFIG_BASE64" — Incorrecta por dos motivos: secretref: no es una sintaxis reconocida por DAB (es sintaxis de Container Apps YAML/ARM para mapear un secret a una variable de entorno o volumen, no algo que DAB interprete dentro de dab-config.json), y además apunta al secret equivocado (DAB_CONFIG_BASE64, que es la configuración completa en base64, no la

cadena de conexión).
C. --connection-string "secretref:mssql-connection-string" — Mismo problema de sintaxis que A: secretref: no es una función soportada por DAB para resolver valores dentro del JSON de configuración.
D. --connection-string "@env('DAB_CONFIG_BASE64')" — Usa correctamente la sintaxis @env(), pero referencia la variable equivocada. DAB_CONFIG_BASE64 contiene la configuración completa de DAB codificada en base64, no la cadena de conexión SQL, por lo que el resultado sería una connection string inválida.

References:
https://learn.microsoft.com/azure/data-api-builder/command-line/dab-init#%60--connection-string%60

https://learn.microsoft.com/azure/data-api-builder/concept/config/env-function

https://learn.microsoft.com/azure/data-api-builder/deployment/azure-app-service#build-the-configuration-file

https://learn.microsoft.com/azure/data-api-builder/concept/config/akv-function

QUESTION 9
You have a SQL database in Microsoft Fabric that contains a nvarchar (max) column named MessageText. An ID is always contained within the first paragraph of MessageText.
You need to write a Transact-SQL query that uses REGEXP_SUBSTR to extract the ID from MessageText.
What should you include in the query?

A. Apply STRING_ESCAPE(MessageText, 'json') before calling REGEXP_SUBSTR.
B. Cast MessageText to nvarchar (4000) before calling REGEXP_SUBSTR.
C. Add a COLLATE Latin1_General_CS_AS clause to MessageText before calling REGEXP_SUBSTR.
D. Run TRY_CONVERT(varchar(max), MessageText) before calling REGEXP_SUBSTR.

Answer: B
Section: (none)

Explanation/Reference:
Q09 T01

Respuesta Correcta:

Cast MessageText to nvarchar (4000) before calling REGEXP_SUBSTR.

Explicacion:

La documentación oficial de Microsoft Learn especifica que, dentro de la familia de funciones REGEXP_* de Transact-SQL, el soporte para tipos LOB (varchar(max) y nvarchar(max), hasta 2 MB) está limitado únicamente a REGEXP_LIKE, REGEXP_COUNT y REGEXP_INSTR. La función REGEXP_SUBSTR no admite tipos LOB como parámetro string_expression.
Como MessageText está definida como nvarchar(max), pasarla directamente a REGEXP_SUBSTR provocaría un error de tipo de dato no admitido. La solución consiste en convertir la columna a un tipo no-LOB compatible, como nvarchar(4000), antes de invocar la función. Dado que el enunciado indica que el ID siempre se encuentra en el primer párrafo del texto, truncar a 4000 caracteres no supone pérdida de la información necesaria.

Por que el resto son incorrectas?:

A. STRING_ESCAPE(MessageText, 'json') sirve para escapar caracteres especiales en cadenas JSON, no para resolver un problema de tipo de dato con funciones de expresión regular; no tiene relación con la restricción de REGEXP_SUBSTR sobre LOB.
C. Añadir una cláusula COLLATE Latin1_General_CS_AS solo modifica las reglas de ordenación/ comparación (sensibilidad a mayúsculas/acentos) de la cadena, pero no cambia su tipo de dato subyacente ni elimina la restricción de tamaño LOB.
D. TRY_CONVERT(varchar(max), MessageText) sigue produciendo un tipo (max), es decir, sigue siendo un tipo LOB, por lo que el problema de compatibilidad con REGEXP_SUBSTR persistiría; además,

convertir de nvarchar a varchar puede provocar pérdida de caracteres Unicode.

Referencias:

Microsoft Learn – REGEXP_SUBSTR (Transact-SQL): "The REGEXP_LIKE, REGEXP_COUNT, and REGEXP_INSTR functions support LOB types (varchar(max) and nvarchar(max)) up to 2 MB for the string_expression parameter" (nota que excluye explícitamente a REGEXP_SUBSTR de esta lista de compatibilidad LOB).
https://learn.microsoft.com/en-us/sql/t-sql/functions/regexp-substr-transact-sql?view=sql-server-ver17

QUESTION 10
You have an Azure SQL database that contains database-level Data Definition Language (DDL) triggers, including a trigger named ddl_Audit.
You need to prevent ddl_Audit from firing during the next deployment. The trigger object must remain in place.
Which Transact-SQL statement should you use?

A. ALTER TRIGGER
B. ALTER DATABASE
C. ALTER SERVER AUDIT SPECIFICATION
D. DISABLE TRIGGER
E. ALTER DATABASE AUDIT SPECIFICATION

Answer: D
Section: (none)

Explanation/Reference:
Q10 T01

Correcta Answer:
DISABLE TRIGGER

Explanation:
La pregunta requiere impedir que el trigger DDL de nivel de base de datos ddl_Audit se dispare durante el próximo despliegue, sin eliminar el objeto.
Según la documentación oficial de Microsoft Learn:

DISABLE TRIGGER deshabilita un trigger (incluyendo DDL de alcance DATABASE). El trigger sigue existiendo como objeto en la base de datos, pero no se ejecuta cuando ocurren las instrucciones Transact-SQL para las que fue programado. Se puede volver a habilitar más tarde con ENABLE TRIGGER.
Ejemplo de sintaxis aplicable: DISABLE TRIGGER ddl_Audit ON DATABASE;

Por qué las demas son incorrectas?:

A. ALTER TRIGGER: modifica la definición del trigger y, según la documentación, habilita el trigger.
B. ALTER DATABASE: se usa para cambiar opciones de base de datos, no para deshabilitar un trigger concreto.
C. ALTER SERVER AUDIT SPECIFICATION y E. ALTER DATABASE AUDIT SPECIFICATION: pertenecen al sistema de SQL Server Audit, no a la gestión de triggers DDL.

References:
https://learn.microsoft.com/es-es/sql/t-sql/statements/disable-trigger-transact-sql?view=sql-server-ver17

https://learn.microsoft.com/es-es/sql/relational-databases/triggers/implement-ddl-triggers?view=sql-server-ver17#disabling-and-dropping-ddl-triggers

QUESTION 11
Your development team uses GitHub Copilot Chat in Microsoft SQL Server Management Studio (SSMS) to generate and run Transact-SQL queries against an Azure SQL database named DB1. DB1 contains tables

that store sensitive customer data.
You need to ensure that Transact-SQL queries that run from GitHub Copilot Chat in SSMS are restricted by the same permissions as the developer's login.
What prevents GitHub Copilot Chat from running queries accessing data beyond the developer's access?

A. GitHub Copilot Chat runs queries in a read-only sandbox that is isolated from production database permissions.
B. GitHub Copilot Chat runs queries by using the developer’s database identity and permissions.
C. GitHub Copilot Chat filters query results on the client side to remove rows the developer is unauthorized to see.
D. GitHub Copilot Chat uses different row-level security (RLS) policies than the developer.

Answer: B
Section: (none)

Explanation/Reference:
Q11-T01

The correct answer is: B

GitHub Copilot Chat runs queries by using the developer's database identity and permissions.

Explanation

Execution Context

Queries generated by GitHub Copilot in SSMS are executed under the context of the user's login and permissions, unless a custom database user or SQL login is specified in the front matter of the database's CONSTITUTION.md.

Layered Security

- Copilot Application Layer: Although this system blocks write operations, it is not a security boundary. Access control must be enforced at the database layer using least-privilege principles.

- Real Security Layer: The true security boundary is SQL Server. The database engine validates each permission according to configured authorization rules.
Alternative Configuration with CONSTITUTION.md

For greater control, administrators can configure the execution context to ensure that queries generated by Copilot run under a dedicated account with lower privileges instead of the user's own permissions.

This is specified in CONSTITUTION.md using agentExecuteAsUser, but the default behavior is to use the connected developer's permissions.

Why This Distinction Is Important

Answer B is correct because it identifies the fundamental mechanism: GitHub Copilot inherits the user's permissions. There is no magical filtering or sandbox isolation. It all comes down to SQL Server enforcing its security rules consistently, regardless of whether the query comes from a user typing directly or from Copilot.

Why the other options are incorrect:

- A (Isolated sandbox): Incorrect. GitHub Copilot does not run in an isolated sandbox. Queries are executed against the real database using the developer's context.

- C (Client-side filtering): Incorrect. Security is not implemented on the client side. The security boundary is the enforcement of SQL Server permissions, not Copilot's classification.

- D (Different RLS): Incorrect. GitHub Copilot uses the same RLS policies as the user. It does not apply different policies.

References:

- Microsoft Learn - Execution context for GitHub Copilot in SQL Server Management Studio
  URL: https://learn.microsoft.com/ssms/github-copilot/execution-context
  Topic: Default execution model and context security

- Microsoft Learn - Use the GitHub Copilot Chat experience in SQL Server Management Studio
  URL: https://learn.microsoft.com/ssms/github-copilot/chat
  Topic: How Copilot executes queries under user context

- Microsoft Learn - Best practices for GitHub Copilot in SQL Server Management Studio
  URL: https://learn.microsoft.com/ssms/github-copilot/best-practices
  Topic: Least privilege principle and permission security

- DP-800 Study Guide - Design and implement SQL solutions by using AI-assisted tools
  Topic: Interpretation of security impact when using AI-assisted tools

QUESTION 12
You have an Azure SQL database named AdventureWorksDB that contains a table named dbo.Employee.
You have a C# Azure Functions app that uses an HTTP-triggered function with an Azure SQL input binding to query dbo.Employee.
You are adding a second function that will react to row changes in dbo.Employee and write structured logs.
You need to configure AdventureWorksDB and the app to meet the following requirements:
Changes to dbo.Employee must trigger the new function within five seconds.
Each invocation must process no more than 100 changes.
Which two database configurations should you perform? Each correct answer presents part of the solution.

A. Create an AFTER trigger on dbo.Employee for Data Manipulation Language (DML).
B. Set Sql_Trigger_MaxBatchSize to 100.
C. Enable change tracking on the dbo.Employee table.
D. Enable change tracking at the database level.
E. Set Sql_Trigger_PollingIntervalMs to 5000.
F. Enable change data capture (CDC) for dbo.Employee table changes.

Answer: CD
Section: (none)

Explanation/Reference:
Q12-T01

---ES---

Respuesta

C. Enable change tracking on the dbo.Employee table.
D. Enable change tracking at the database level.

Explicación

El Azure SQL trigger de Azure Functions requiere **Change Tracking**.
Las dos configuraciones de base de datos necesarias son:

- Habilitar Change Tracking a nivel de base de datos.
- Habilitar Change Tracking en la tabla dbo.Employee.

Son las únicas opciones de la lista que son configuraciones de base de datos.

Por qué las demás no son correctas

- A: Un trigger AFTER no se usa con el Azure SQL trigger binding.
- B y E: Son ajustes de la aplicación (`Sql_Trigger_MaxBatchSize` y `Sql_Trigger_PollingIntervalMs`), no de la base de datos.
- F: CDC no es el mecanismo que utiliza el Azure SQL trigger.

Referencias

Microsoft Learn: Azure SQL trigger for Functions – Set up change tracking (required)
https://learn.microsoft.com/azure/azure-functions/functions-bindings-azure-sql-trigger#set-up-change-tracking-required

SQL Server: About Change Tracking
https://learn.microsoft.com/sql/relational-databases/track-changes/about-change-tracking-sql-server

---EN---

Answer

C. Enable change tracking on the dbo.Employee table.
D. Enable change tracking at the database level.

Explanation

The Azure SQL trigger for Azure Functions requires **Change Tracking**.
The two required database configurations are:

- Enable Change Tracking at the database level.
- Enable Change Tracking on the dbo.Employee table.

These are the only database-level options in the list.

Why the others are incorrect

- A: An AFTER trigger is not used by the Azure SQL trigger binding.
- B and E: These are application settings (`Sql_Trigger_MaxBatchSize` and `Sql_Trigger_PollingIntervalMs`), not database configurations.
- F: CDC is not the mechanism used by the Azure SQL trigger.

References

Microsoft Learn: Azure SQL trigger for Functions – Set up change tracking (required)
https://learn.microsoft.com/azure/azure-functions/functions-bindings-azure-sql-trigger#set-up-change-tracking-required

SQL Server: About Change Tracking
https://learn.microsoft.com/sql/relational-databases/track-changes/about-change-tracking-sql-server

QUESTION 13
You have a Microsoft SQL Server 2025 database that contains a table named dbo.CustomerMessages. dbo.CustomerMessages contains two columns named MessageID (int) and MessageRaw (nvarchar(max)). MessageRaw can contain a phone number in multiple formats, and some rows do NOT contain a phone number.

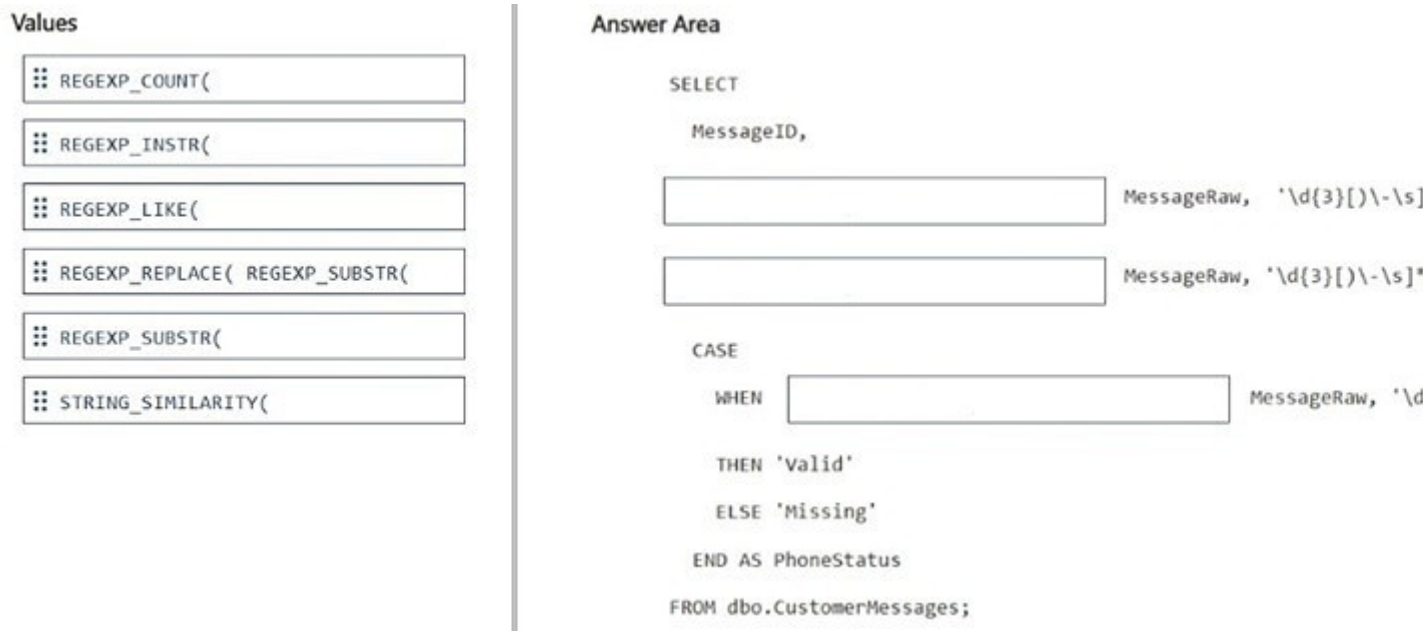
You need to write a single SELECT query that meets the following requirements:
The query must return MessageID, RawNumber, DigitsOnly, and PhoneStatus.
RawNumber must contain the first substring that matches a phone-number pattern, or NULL if no match

exists.

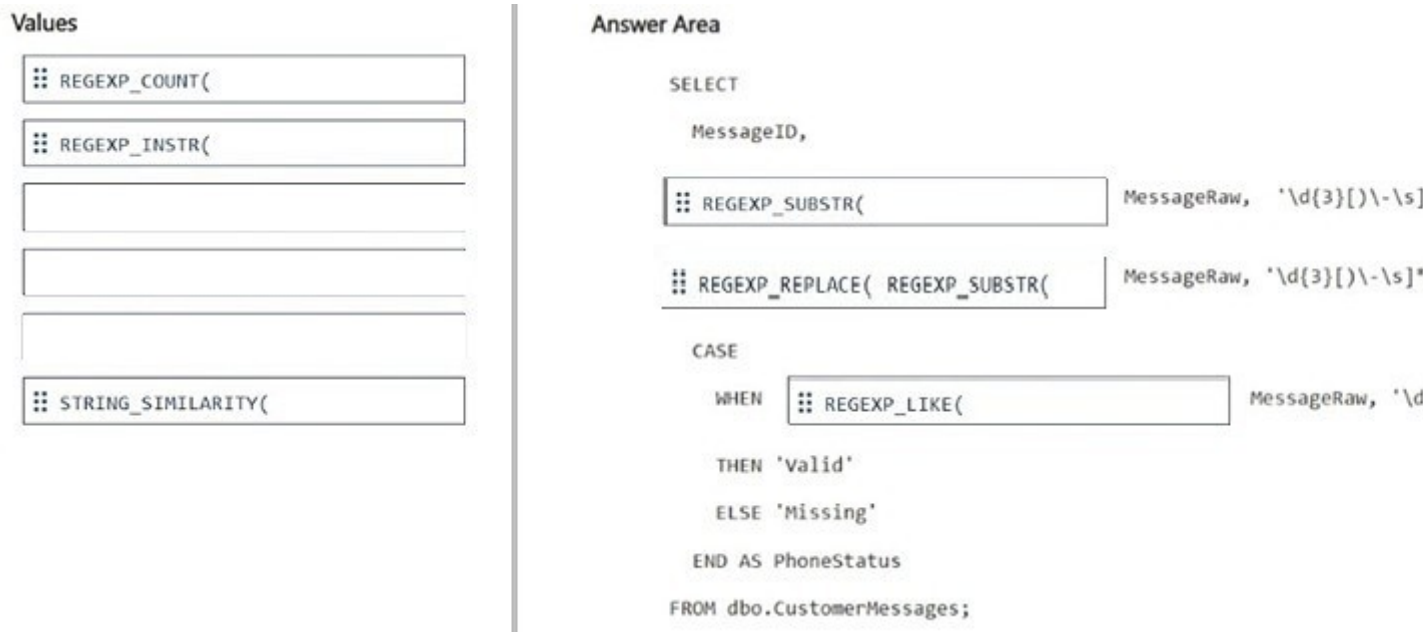
DigitsOnly must remove all non-digit characters from RawNumber, or return NULL.
PhoneStatus must return valid when a phone number exists in MessageRaw, otherwise return Missing.

How should you complete the Transact-SQL query? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q13 T01

Correct Answer:
- Primer espacio (RawNumber): **REGEXP_SUBSTR(**

- Segundo espacio (DigitsOnly): **REGEXP_REPLACE(REGEXP_SUBSTR(**
- Tercer espacio (CASE / PhoneStatus): **REGEXP_LIKE(**

Explanation:
REGEXP_SUBSTR extrae la primera subcadena que coincide con el patrón de número de teléfono de MessageRaw (devuelve NULL si no hay coincidencia).
Para obtener DigitsOnly se anida REGEXP_REPLACE sobre el resultado de REGEXP_SUBSTR y se eliminan todos los caracteres que no sean dígitos.
REGEXP_LIKE comprueba si el patrón existe en MessageRaw. Como en T-SQL el valor booleano true se evalúa como 1, la comparación = 1 funciona correctamente y permite devolver 'Valid' o 'Missing'.
Esta combinación cumple los requisitos del enunciado en una sola consulta SELECT.

English translation of the Explanation:
REGEXP_SUBSTR extracts the first matching phone-number substring from MessageRaw (returns NULL if none exists).
To obtain DigitsOnly, REGEXP_REPLACE is nested on the result of REGEXP_SUBSTR and all non-digit characters are removed.
REGEXP_LIKE checks whether the pattern exists in MessageRaw. Because in T-SQL the boolean value true evaluates to 1, the comparison = 1 works correctly and allows returning 'Valid' or 'Missing'.
This combination meets the requirements of the question in a single SELECT statement.

Reference:
- REGEXP_SUBSTR (Transact-SQL): https://learn.microsoft.com/es-es/sql/t-sql/functions/regexp-substr-transact-sql?view=sql-server-ver17
- REGEXP_REPLACE (Transact-SQL): https://learn.microsoft.com/es-es/sql/t-sql/functions/regexp-replace-transact-sql?view=sql-server-ver17
- REGEXP_LIKE (Transact-SQL): https://learn.microsoft.com/es-es/sql/t-sql/functions/regexp-like-transact-sql?view=sql-server-ver17
- Regular expressions functions (Transact-SQL): https://learn.microsoft.com/es-es/sql/t-sql/functions/regular-expressions-functions-transact-sql?view=sql-server-ver17

QUESTION 14
You have an Azure SQL database that contains a table named Rooms. Rooms was created by using the following Transact-SQL statement.

```
CREATE TABLE Rooms
(
    RoomID int PRIMARY KEY,
    Owner nvarchar(100),
    Capacity int
);
```

You discover that some records in the Rooms table contain NULL values for the Owner field.

You need to ensure that all future records have a value for the Owner field.
What should you add?

A. a foreign key
B. a check constraint
C. a nonclustered index
D. a unique constraint

Answer: B
Section: (none)

Explanation/Reference:
Q14 T01

Correct Answer:

B. a check constraint

Explanation:

Una restricción CHECK permite limitar los valores admitidos en una columna mediante una condición lógica. En este caso, la condición Owner IS NOT NULL impide que los nuevos registros de la tabla Rooms tengan un valor NULL en Owner.

Como ya existen registros con valores NULL, la restricción puede añadirse utilizando WITH NOCHECK para omitir la comprobación de los datos existentes, manteniendo la validación para las nuevas inserciones.

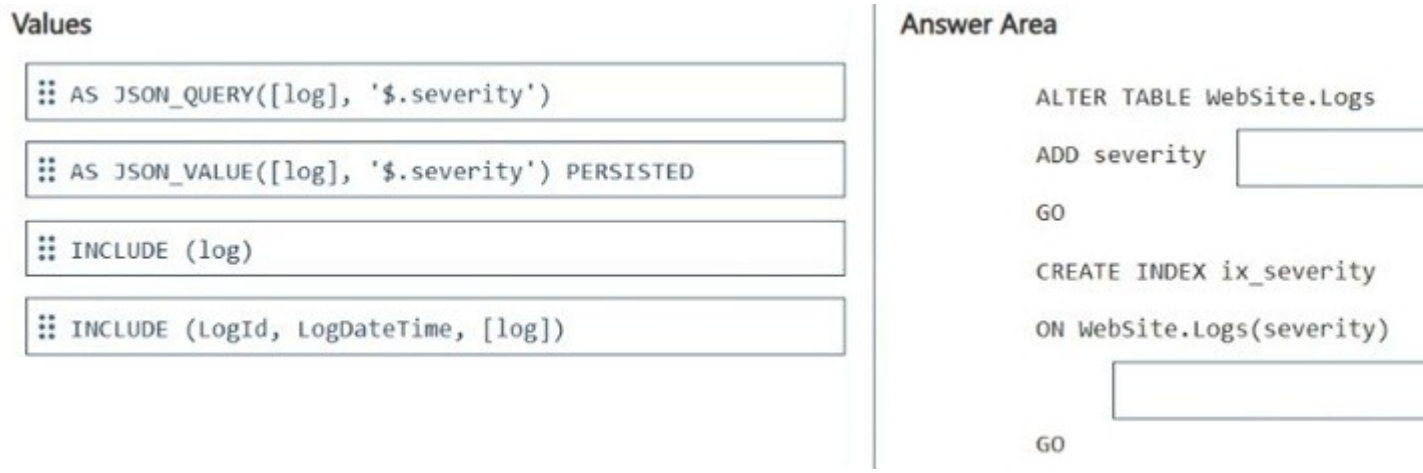
Reference:

https://learn.microsoft.com/en-us/sql/t-sql/statements/alter-table-table-constraint-transact-sql?view=sql-server-ver17#remarks

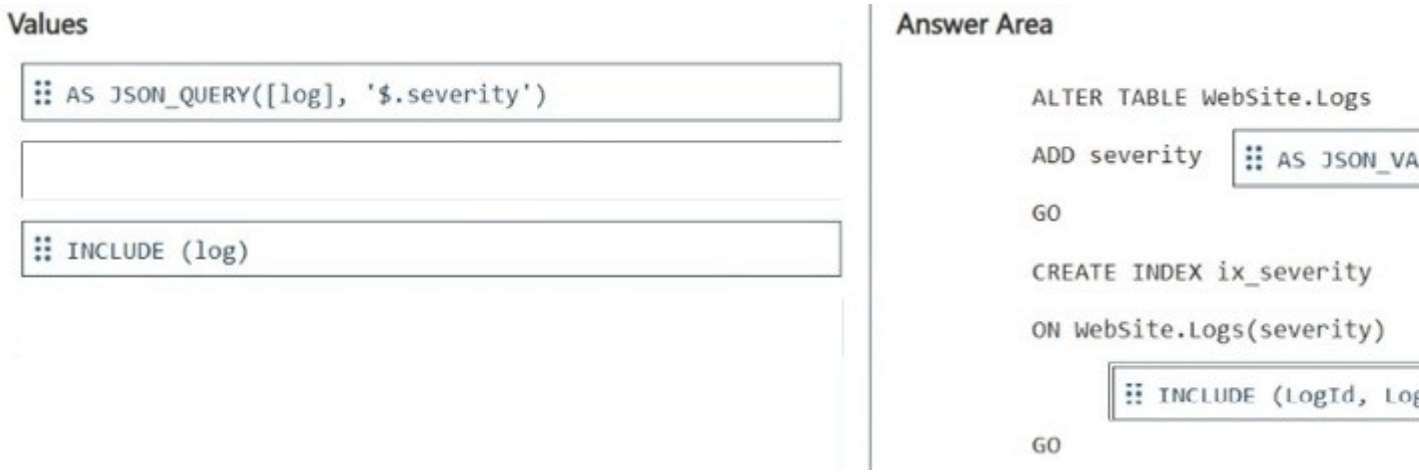
QUESTION 15
You have a SQL database in Microsoft Fabric that contains a table named WebSite.Logs. WebSite.Logs stores application telemetry data. WebSite.Logs contains a nvarchar (max) column named log that stores JSON documents.

You have a daily report that filters by the $.severity JSON property and returns LogId, LogDateTime, and log. The report frequently causes full table scans.
You need to modify WebSite.Logs to support efficient filtering by $.severity and avoid key lookups for the columns returned by the report.

How should you complete the Transact-SQL code to avoid full table scans? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.



Answer:



Section: (none)

Explanation/Reference:
Q15 T01

CORRECT ANSWER:

ALTER TABLE WebSite.Logs

ADD severity AS JSON_VALUE([log], '$.severity') PERSISTED;

GO

CREATE INDEX ix_severity

ON WebSite.Logs(severity)

INCLUDE (LogId, LogDateTime, [log]);

GO

EXPLANATION:

Para evitar full table scans al filtrar por la propiedad JSON $.severity y eliminar key lookups de las columnas del informe (LogId, LogDateTime y log), se añade una columna calculada persistida con JSON_VALUE que expone el valor escalar. A continuación se crea un índice no clusterizado sobre esa columna e se incluyen las columnas necesarias con INCLUDE. De esta forma el motor utiliza un index seek y obtiene todos los datos directamente del índice de cobertura, sin necesidad de acceder a la tabla base. JSON_VALUE se utiliza para propiedades escalares; JSON_QUERY solo para objetos o arrays.

REFERENCES:

https://learn.microsoft.com/sql/relational-databases/json/index-json-data?view=sql-server-ver17#index-json-properties-by-using-computed-columns
https://learn.microsoft.com/sql/relational-databases/json/store-json-documents-in-sql-tables?view=sql-server-ver17#indexes

QUESTION 16
HOTSPOT

Case Study

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the

time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment
Azure Environment

Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:

Patients
Employees
Procedures
Transactions
UsefulPrompts
ProcedureDocuments

You store a column master key as a secret in Azure Key Vault.

You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.

Problem Statements

Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes incosistent
Requirements
Planned Changes
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.
Security Requirements
Fabrikam identifies the following security requirements:

• The TransactionProcessing application must use a passwordless connection to DB1.

• The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.

• Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
AI Search, Embeddings, and Vector Indexing
Fabrikam identifies the following AI-related requirements:

• Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).

• Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.

• The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.

Development Requirements

Fabrikam identifies the following development requirements:

• Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.

• Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:

o Read data from the procedures table without authentication.

o Read and insert data into the Transactions table once authenticated.

o Execute the sp_UpdateProcedurePatient stored procedure.

• Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.

• Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

-

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You need to create a solution that meets the development requirements for retrieving the patient lists.

How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.

Answer Area

```
CREATE PROCEDURE dbo.GetActivePatients

AS

BEGIN

  SET NOCOUNT ON;

 SELECT p.Name

  FROM dbo.Patients AS p
```

[▼] (

- WHERE EXISTS
- WHERE NOT EXISTS
- WHERE p.PatienId IN
- WHERE p.PatienId NOT IN

```
    SELECT PatientId

    FROM dbo.Procedures AS pr
```

[▼]

- HAVING p.PatientId = pr.PatientId
- HAVING p.TransactionId = pr.TransactionId
- WHERE p.PatientId = pr.PatientId
- WHERE p.TransactionId = pr.TransactionId

```
      AND pr.TransactionDate >= DATEADD(DAY, -30, SYSUTCDATETIME())

 );

END;
```

Answer:

Answer Area

```
CREATE PROCEDURE dbo.GetActivePatients

AS

BEGIN

  SET NOCOUNT ON;

 SELECT p.Name

  FROM dbo.Patients AS p
```

[WHERE EXISTS | WHERE NOT EXISTS | WHERE p.PatientId IN | WHERE p.PatientId NOT IN] (

```
    SELECT PatientId

    FROM dbo.Procedures AS pr
```

[HAVING p.PatientId = pr.PatientId | HAVING p.TransactionId = pr.TransactionId | WHERE p.PatientId = pr.PatientId | WHERE p.TransactionId = pr.TransactionId]

```
      AND pr.TransactionDate >= DATEADD(DAY, -30, SYSUTCDATETIME())

 );

END;
```

Section: (none)

Explanation/Reference:
Q16 T01
1. ANÁLISIS

Concepto principal: subconsulta correlacionada en Transact-SQL para filtrar filas de Patients que tienen coincidencias en Procedures dentro de los últimos 30 días.

Servicio: Azure SQL Database / SQL Server.

Escenario: cumplir el requisito de desarrollo “retrieve a list of the names of patients who underwent medical procedures during the last 30 days”.
2. VERIFICACIÓN (MCP)

Consultada la documentación oficial de Microsoft Learn sobre subconsultas y el operador EXISTS.
3. RESOLUCIÓN

Primera lista desplegable: WHERE EXISTS

Segunda lista desplegable: WHERE p.PatientId = pr.PatientId
4. JUSTIFICACIÓN

Correcta – WHERE EXISTS + WHERE p.PatientId = pr.PatientId
Según Microsoft Learn, WHERE EXISTS (subquery) realiza una prueba de existencia. La subconsulta debe

ser correlacionada (p.PatientId = pr.PatientId) para que, por cada fila de Patients, se verifique si existe al menos una fila en Procedures que cumpla la condición de fecha (pr.TransactionDate >= DATEADD(DAY, -30, SYSUTCDATETIME())). Esto devuelve exactamente los nombres de pacientes que tuvieron procedimientos en los últimos 30 días.
Incorrectas
WHERE NOT EXISTS → devolvería los pacientes sin procedimientos.
WHERE p.PatientId IN / NOT IN → sintácticamente posibles, pero la estructura del código y las opciones del segundo desplegable están diseñadas para el patrón de existencia correlacionada documentado.
Cualquier opción con HAVING → inválida: HAVING solo se usa después de GROUP BY y las reglas de subconsultas de Microsoft Learn prohíben su uso en este contexto.

5. REFERENCIA

https://learn.microsoft.com/sql/relational-databases/performance/subqueries

https://learn.microsoft.com/sql/t-sql/language-elements/exists-transact-sql

QUESTION 17
You have an Azure SQL database named ToDo that contains a table named dbo.ToDo.

Your company plans to develop an Azure Functions app to run whenever the rows in dbo.ToDo change. The app will process INSERT, UPDATE, and DELETE events by using the Azure SQL trigger binding.

You need to configure ToDo to support the planned app.

What should you do?

A. Enable change tracking on ToDo and dbo.ToDo.
B. On dbo.ToDo, create a Data Manipulation Language (DML) trigger that calls the Azure Functions HTTP endpoint.
C. Enable change data capture (CDC) on ToDo and dbo.ToDo.
D. On dbo.ToDo, create a Data Definition Language (DDL) trigger that calls the Azure Functions HTTP endpoint

Answer: A
Section: (none)

Explanation/Reference:
Q17 T01

Correct Answer: Enable change tracking on ToDo and dbo.ToDo.

Explanation:

Por qué la opción A es correcta.

El Azure SQL trigger binding de Azure Functions se basa exclusivamente en la funcionalidad de Change Tracking de SQL Server / Azure SQL.

Según la documentación oficial, es obligatorio realizar dos pasos:

Habilitar Change Tracking a nivel de base de datos:SQLALTER DATABASE [ToDo]
SET CHANGE_TRACKING = ON
(CHANGE_RETENTION = 2 DAYS, AUTO_CLEANUP = ON);
Habilitar Change Tracking en la tabla:SQLALTER TABLE [dbo].[ToDo]
ENABLE CHANGE_TRACKING;

Una vez habilitado, el runtime del trigger realiza un polling periódico sobre las tablas de sistema de Change Tracking (CHANGETABLE) y entrega los cambios (Insert / Update / Delete) a la función de Azure Functions. Sin este requisito el binding no funciona.

B: Crear un trigger DML que invoque un endpoint HTTP de Azure Functions es un patrón completamente manual (webhook personalizado). No es el mecanismo que utiliza el binding nativo SqlTrigger. El binding no requiere ni utiliza triggers DML definidos por el usuario.

C: Change Data Capture (CDC) es una tecnología distinta (basada en el log de transacciones y jobs de captura). El Azure SQL trigger binding no utiliza CDC. La documentación oficial indica de forma explícita que se usa únicamente Change Tracking.

D: Un trigger DDL responde a cambios de esquema (CREATE, ALTER, DROP de objetos), no a modificaciones de filas de datos (DML). Es irrelevante para detectar INSERT/UPDATE/DELETE en la tabla dbo.ToDo

References:

Fuente oficial exacta utilizada (Microsoft Learn – documentación actualizada):

Título: Azure SQL trigger for Functions
Sección específica: Set up change tracking (required)
URL directa:
https://learn.microsoft.com/azure/azure-functions/functions-bindings-azure-sql-trigger#set-up-change-tracking-required

QUESTION 18
Question #18 – DRAG DROP
You have an Azure SQL database that contains a table named dbo.Customers.

dbo.Customers contains the following columns:

CustomerId (int) (primary key)
ProfileJson (nvarchar(max))

You have an application that returns phone numbers in a format of +000 000-000-0000.

The phone numbers are stored in ProfileJson.
You need to write a query that returns:

One row per customer
A PhoneNUmerals column that contains only the digits

How should you complete the Transact-SQL query?

To answer, drag the appropriate values to the correct targets.

Each value may be used once, more than once, or not at all.

You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

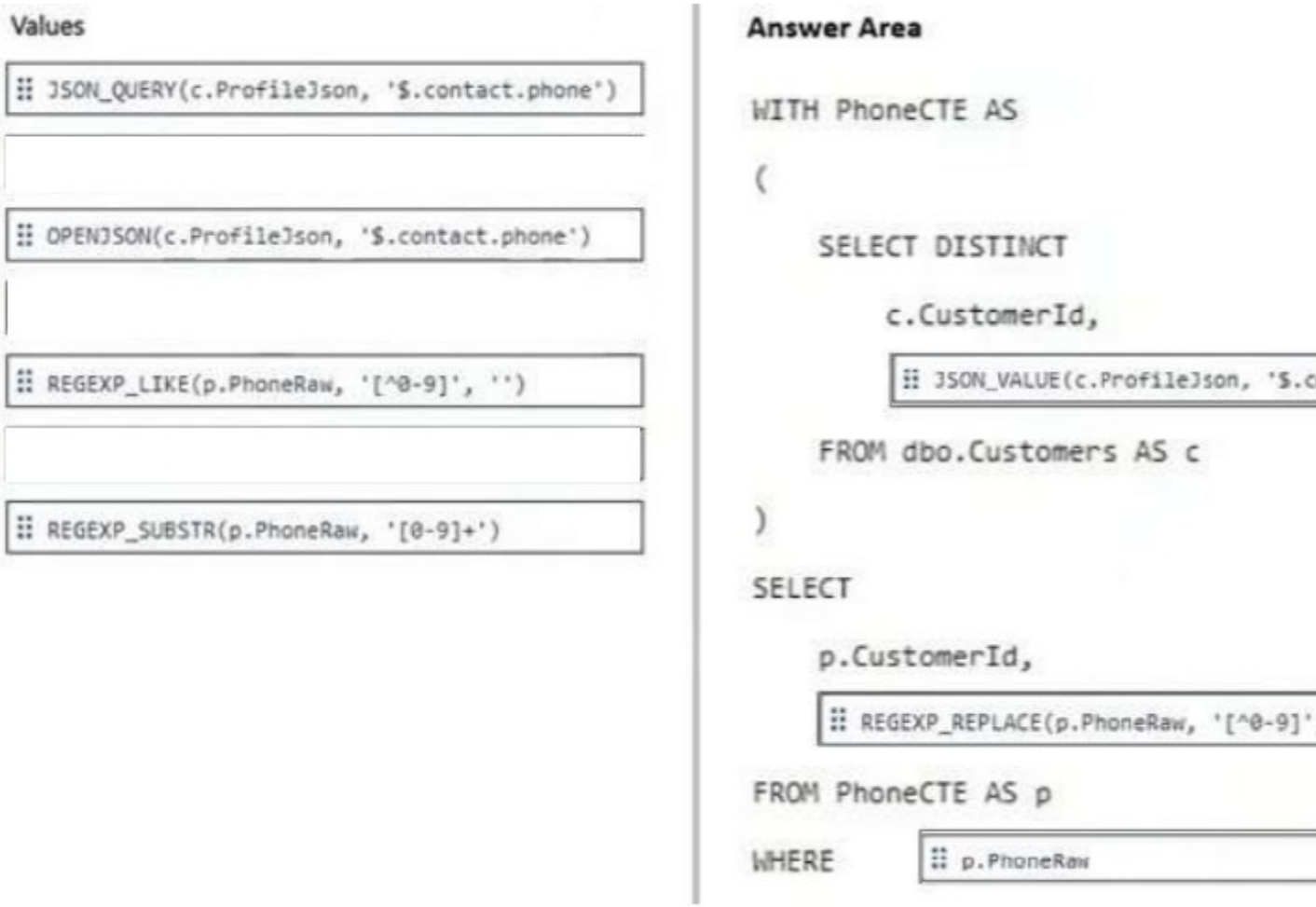
Values

- `JSON_QUERY(c.ProfileJson, '$.contact.phone')`
- `JSON_VALUE(c.ProfileJson, '$.contact.phone')`
- `OPENJSON(c.ProfileJson, '$.contact.phone')`
- `p.PhoneRaw`
- `REGEXP_LIKE(p.PhoneRaw, '[^0-9]', '')`
- `REGEXP_REPLACE(p.PhoneRaw, '[^0-9]', '')`
- `REGEXP_SUBSTR(p.PhoneRaw, '[0-9]+')`

Answer Area

```
WITH PhoneCTE AS
(
    SELECT DISTINCT
        c.CustomerId,
        [          ]
    FROM dbo.Customers AS c
)
SELECT
    p.CustomerId,
    [          ]
FROM PhoneCTE AS p
WHERE [          ]
```

Answer:



Section: (none)

Explanation/Reference:
Q18 T01

Correct Answer:

WITH PhoneCTE AS
(
SELECT DISTINCT
c.CustomerId,
JSON_VALUE(c.ProfileJson, '$.contact.phone') AS PhoneRaw
FROM dbo.Customers AS c
)
SELECT
p.CustomerId,
REGEXP_REPLACE(p.PhoneRaw, '[^0-9]', '') AS PhoneNUmerals
FROM PhoneCTE AS p
WHERE p.PhoneRaw IS NOT NULL;

Explanation:
JSON_VALUE extracts the scalar phone number value from the JSON path $.contact.phone.
REGEXP_REPLACE(p.PhoneRaw, '[^0-9]', '') removes every non-digit character, leaving only the numbers.
The CTE + DISTINCT ensures one row per customer.
The WHERE clause filters out rows without a phone number.

References:
https://learn.microsoft.com/en-us/sql/t-sql/functions/json-value-transact-sql

https://learn.microsoft.com/en-us/sql/t-sql/functions/regexp-replace-transact-sql
https://learn.microsoft.com/en-us/sql/relational-databases/json/json-data-sql-server

QUESTION 19
DRAG DROP
You have an Azure SQL database named SalesDB that supports an ecommerce application. SalesDB contains a table named dbo.Orders that has a clustered index on a column named OrderId.

dbo.Orders receives continuous OLTP inserts and updates during business hours.

Your analytics team runs hourly aggregate queries that scan dbo.Orders to calculate revenue trends for recent dates.

You need to improve the performance of the hourly analytics queries without significantly affecting OLTP throughput. The solution must meet the following requirements:

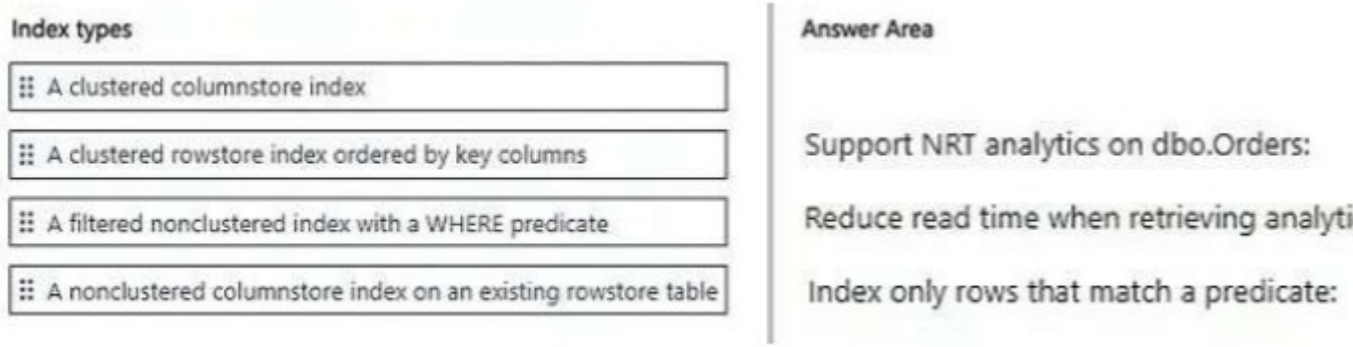
Support near-real-time (NRT) analytics on the dbo.Orders table.

Reduce read time when retrieving analytical data from dbo.Orders.

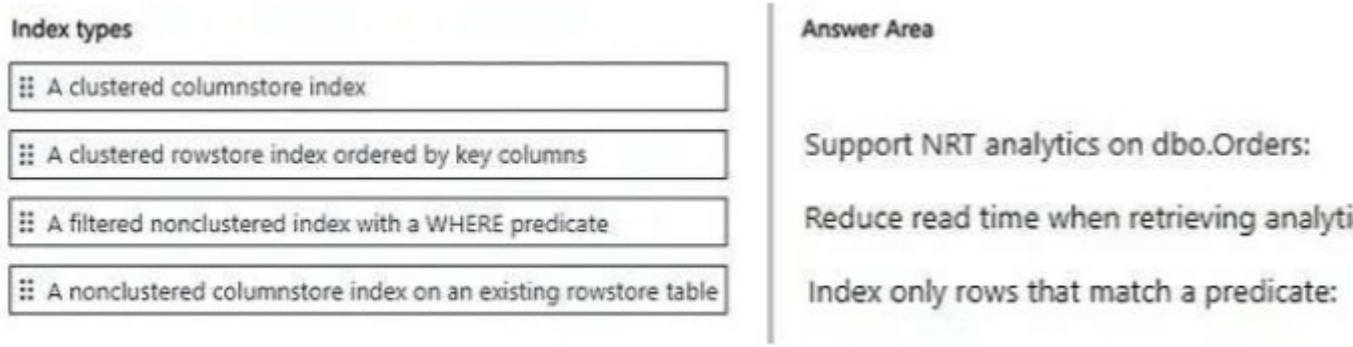
Support indexing only rows that match a predicate, such as Active = 1.

Which type of index should you use for each requirement? To answer, drag the appropriate index types to the correct requirements. Each index type may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q19-T01

Correct Answer:

Support NRT analytics on dbo.Orders → D. Nonclustered columnstore index on an existing rowstore table
Reduce read time when retrieving analytical data → D. Nonclustered columnstore index on an existing rowstore table
Index only rows that match a predicate → C. Filtered nonclustered index with a WHERE predicate

(D puede usarse más de una vez.)
Technical Justification:

dbo.Orders es una tabla rowstore con índice clustered existente y workload OLTP continuo.

Para soportar analytics near-real-time (NRT) y reducir el tiempo de las consultas analíticas (agregaciones) sin afectar significativamente el throughput OLTP, se crea un nonclustered columnstore index (NCCI) updateable. El optimizador dirige automáticamente las consultas analíticas al columnstore (batch mode + alta compresión) mientras el OLTP sigue usando los índices B-tree. Los cambios se reflejan en tiempo real. Para indexar solo las filas que cumplen un predicado (ej. Active = 1) se usa un filtered nonclustered index.

Why the other options are incorrect:
A (Clustered columnstore index): Convierte toda la tabla a columnstore, elimina el clustered index existente y tiene mayor impacto en inserts/updates OLTP.
B: No aporta rendimiento de columnstore ni filtrado.
C para NRT/analytics: Un filtered B-tree no ofrece el rendimiento de scans/agregaciones de un columnstore.

Official Microsoft references:

https://learn.microsoft.com/en-us/sql/relational-databases/indexes/get-started-with-columnstore-for-real-time-operational-analytics
https://learn.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-overview
https://learn.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-design-guidance

QUESTION 20
You have an Azure SQL database that stores sales data and contains tables named Sales and Products. Sales contains three columns named SalesDate, ProductKey, and TotalSale,

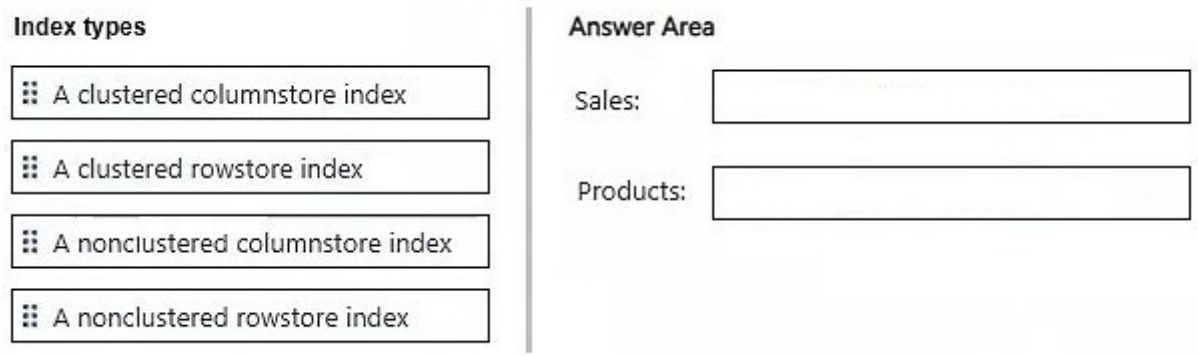
Sales is 10 TB and is loaded nightly by using a batch process. Most reporting queries scan large portions of Sales, filter on SalesDate or ProductKey, and use SUM() to aggregate TotalSale.

Products is relatively small and is used primarily for point lookups and joins to Sales.

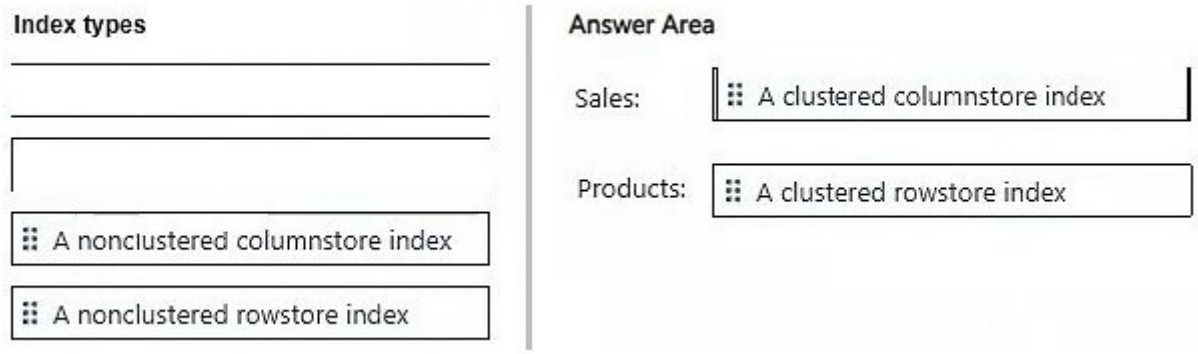
You need to recommend which indexes to create to optimize the reporting queries. The solution must minimize storage requirements.

Which type of index should you recommend for each table? To answer, drag the appropriate index types to the correct tables. Each index type may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q20 T01

Correct Answer:
Sales: A clustered columnstore index
Products: A clustered rowstore index

Why not the others:

Sales: nonclustered columnstore duplicaría almacenamiento; rowstore no da la compresión ni el rendimiento necesario para escaneos y agregaciones masivas.

Products: columnstore no aporta beneficio en tablas pequeñas con seeks puntuales; nonclustered rowstore añade almacenamiento extra innecesario.

Explanation:
Sales es grande y sus consultas escanean rangos y agregan con SUM, por lo que columnstore reduce el almacenamiento gracias a su compresión de aproximadamente 10x. Products es pequeña y se usa para lookups y joins puntuales, por lo que rowstore es más eficiente para ese tipo de búsquedas.

References:
https://learn.microsoft.com/sql/relational-databases/indexes/columnstore-indexes-design-guidance?view=sql-server-ver17

QUESTION 21
You have an Azure SQL database that contains a table named dbo.SupportTickets. dbo.SupportTickets contains a JSON column named Payload and a datetime column CreatedAt.

You need to generate a report for the last seven days that meets the following requirements:

• Returns exactly one row per customer per day
• For each customer and day, returns the earliest ticket
• Includes the customer ID stored in Payload

How should you complete the Transact-SQL query? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
WITH TicketRanks AS
(
    SELECT
        t.TicketId,
        CAST(t.CreatedAt AS date) AS TicketDate,
            [ ▼ ]  (t.Payload, '$.customer.id') AS CustomerId,
        t.CreatedAt,
            [ ▼ ]  () OVER
        (
            PARTITION BY
                CAST(t.CreatedAt AS date),
                JSON_VALUE(t.Payload, '$.customer.id')
            ORDER BY t.CreatedAt ASC
        ) AS rn
    FROM dbo.SupportTickets AS t
    WHERE t.CreatedAt >= DATEADD(day, -7, SYSUTCDATETIME())
)
SELECT
    TicketDate,
    CustomerId,
    TicketId,
    CreatedAt
```

DENSE_RANK
JSON_VALUE
OPENJSON
ROW_NUMBER

DENSE_RANK
JSON_VALUE
OPENJSON
ROW_NUMBER

Answer:

Answer Area

```
WITH TicketRanks AS
(
    SELECT
        t.TicketId,
        CAST(t.CreatedAt AS date) AS TicketDate,
        [ JSON_VALUE ▼ ] (t.Payload, '$.customer.id') AS CustomerId,
        t.CreatedAt,
        [ ROW_NUMBER ▼ ] () OVER
        (
            PARTITION BY
                CAST(t.CreatedAt AS date),
                JSON_VALUE(t.Payload, '$.customer.id')
            ORDER BY t.CreatedAt ASC
        ) AS rn
    FROM dbo.SupportTickets AS t
    WHERE t.CreatedAt >= DATEADD(day, -7, SYSUTCDATETIME())
)
SELECT
    TicketDate,
    CustomerId,
    TicketId,
    CreatedAt
```

DENSE_RANK
JSON_VALUE
OPENJSON
ROW_NUMBER

Section: (none)

Explanation/Reference:
Q21 T01

Explanation

Primer desplegable:

RESPUESTA CORRECTA:

- JSON_VALUE, ya que extrae un valor de un documento JSON usando un JSON path,el uso sería el siguiente: JSON_VALUE(t.Payload, '$.customer.id') ya que lo que denota esta expresion seria que de la columna **Payload** extrae el **customer.id.**

RESPUESTAS INCORRECTAS:

- DENSE_RANK no es correcta ya que es una funcion de ranking, y estas no extraen valores de un JSON.
- ROW_NUMBER no es correcta ya que es una funcion de ventana y no sirven para extraer valores de JSON.
- OPENJSON sirve para transformar un JSON en filas y columnas. No devuelve un valor escalar para usarlo directamente como columna en el SELECT.

Segundo desplegable:

RESPUESTA CORRECTA:

- ROW_NUMBER() asigna un número secuencial único empezando en 1 dentro de cada partición (fecha + CustomerId), ordenado por CreatedAt ASC. Así se identifica de forma fiable el primer ticket de cada cliente en cada día.

RESPUESTAS INCORRECTAS:

- JSON_VALUE y OPENJSON: no son funciones de ventana; no se pueden usar con la cláusula OVER.
- DENSE_RANK asigna el mismo rango a filas empatadas. Si dos tickets tienen exactamente el mismo CreatedAt, ambos recibirían rn = 1 y se devolverían ambos, lo que no es el comportamiento deseado cuando se quiere “el primero” de forma única.

Tercer desplegable:

RESPUESTA CORRECTA:

- 1 es la opccion correcta, ya que como la consulta ordena los tickets por fecha de creación ascendente, el ticket más antiguo de cada cliente en un día dado recibe siempre el valor rn = 1. Al filtrar en la consulta principal con WHERE rn = 1, se elimina el resto de tickets secundarios del día.

RESPUESTA INCORRECTA:

- el numero 7 no tiene sentido ya que **rn** es un número de fila generado por la función de ventana, no un valor de fecha.
- DATEDIFF(day, -7, SYSUTCDATETIME()): es una expresión de fecha/hora y no tiene ninguna relacion con el valor que representa **rn**.

Reference

Documentación oficial de Microsoft Learn – JSON_VALUE (Transact-SQL):
- https://learn.microsoft.com/en-us/sql/t-sql/functions/json-value-transact-sql

Documentación oficial de Microsoft Learn – OPENJSON (Transact-SQL) y comparación con JSON_VALUE:
- https://learn.microsoft.com/en-us/sql/relational-databases/json/convert-json-data-to-rows-and-columns-with-openjson-sql-server
- https://learn.microsoft.com/en-us/sql/relational-databases/json/validate-query-and-change-json-data-with-built-in-functions-sql-server

Documentación oficial de Microsoft Learn – ROW_NUMBER (Transact-SQL):
- https://learn.microsoft.com/en-us/sql/t-sql/functions/row-number-transact-sql

Documentación oficial de Microsoft Learn – OVER Clause y funciones de ranking:
- https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql

QUESTION 22
You are developing an Azure SQL database solution from a locally cloned GitHub repository by using Microsoft Visual Studio Code and GitHub Copilot Chat.

You need to ensure that GitHub Copilot Chat can call the hosted GitHub MCP Server tools by using OAuth. The MCP server configuration must be scoped to the repository.

What should you do in Visual Studio Code?

A. Create a personal access token (PAT) that has the repo scope and store the PAT in the .vscode\mcp. json file as the Authorization header.
B. From the Command Palette, enter MCP: add server, select HTTP (HTTP or Server-Sent Events), enter https://api.githubcopilot.com/mcp/, and then save the configuration to the workspace settings.
C. From the Command Palette, enter MCP: add server, select HTTP (HTTP or Server-Sent Events), enter https://api.githubcopilot.com/mcp/, and then save the configuration to the user settings.
D. Create a personal access token (PAT) that has the repo scope and store the PAT in the .github\mcp. json file as the Authorization header.

Answer: B
Section: (none)

Explanation/Reference:
Q22 T01

Correct Answer:
From the Command Palette, enter MCP: add server, select HTTP (HTTP or Server-Sent Events), enter https://api.githubcopilot.com/mcp/, and then save the configuration to the workspace settings.

Explanation:
The GitHub MCP Server hosted at https://api.githubcopilot.com/mcp/ supports remote HTTP/SSE transport and authenticates automatically via OAuth when added through VS Code's MCP: Add Server command — you are prompted to sign in and consent, rather than needing to supply any token yourself. Saving the configuration to workspace settings writes it to .vscode\mcp.json in the repository, which satisfies the requirement that the configuration be scoped only to that repository (rather than to all workspaces globally).

The two personal access token (PAT) options are incorrect because they rely on a manually created and stored secret (an Authorization header with a PAT) instead of OAuth — this contradicts the requirement to use OAuth, and hardcoding a PAT in a checked-in config file is also a security anti-pattern. The .github\mcp. json location is not a recognized VS Code MCP configuration path at all — VS Code looks for .vscode\mcp. json for workspace-scoped servers, not a .github folder.

Saving to user settings instead of workspace settings is incorrect because user/global settings apply the MCP server configuration across all workspaces in VS Code, not just the current repository — this violates the requirement that the configuration be scoped specifically to the repository.

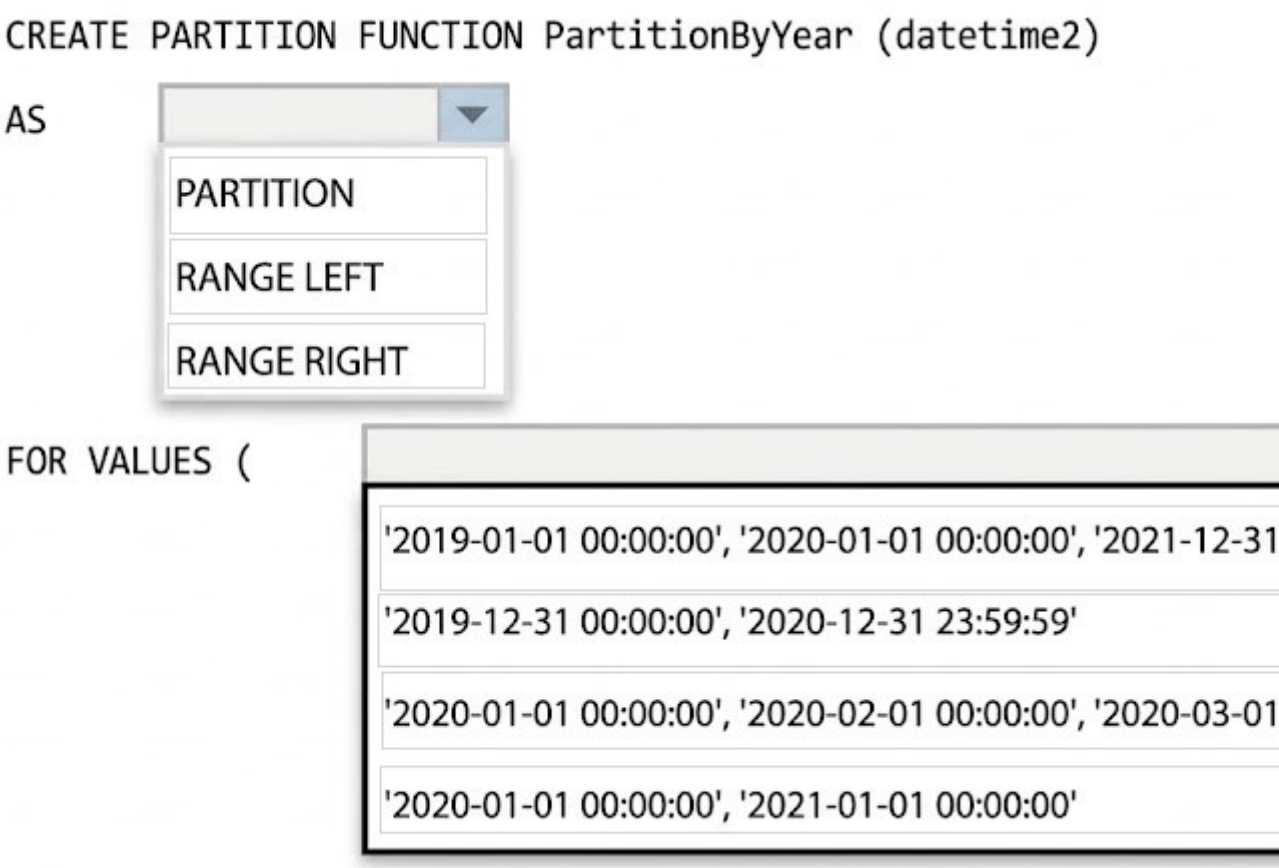
References:
https://code.visualstudio.com/docs/copilot/chat/mcp-servers
https://learn.microsoft.com/azure/api-management/expose-existing-mcp-server#validate-and-use-the-mcp-server
https://docs.github.com/copilot/customizing-copilot/using-model-context-protocol/using-the-github-mcp-server
https://learn.microsoft.com/en-us/training/modules/mcp-server/3-configure-connect

QUESTION 23
You have an Azure SQL database that contains a table named Table1. Table1 contains 25,000,000 rows of data and a datetime2 column named DateKey. The data in Table1 spans the years 2020 through 2021.

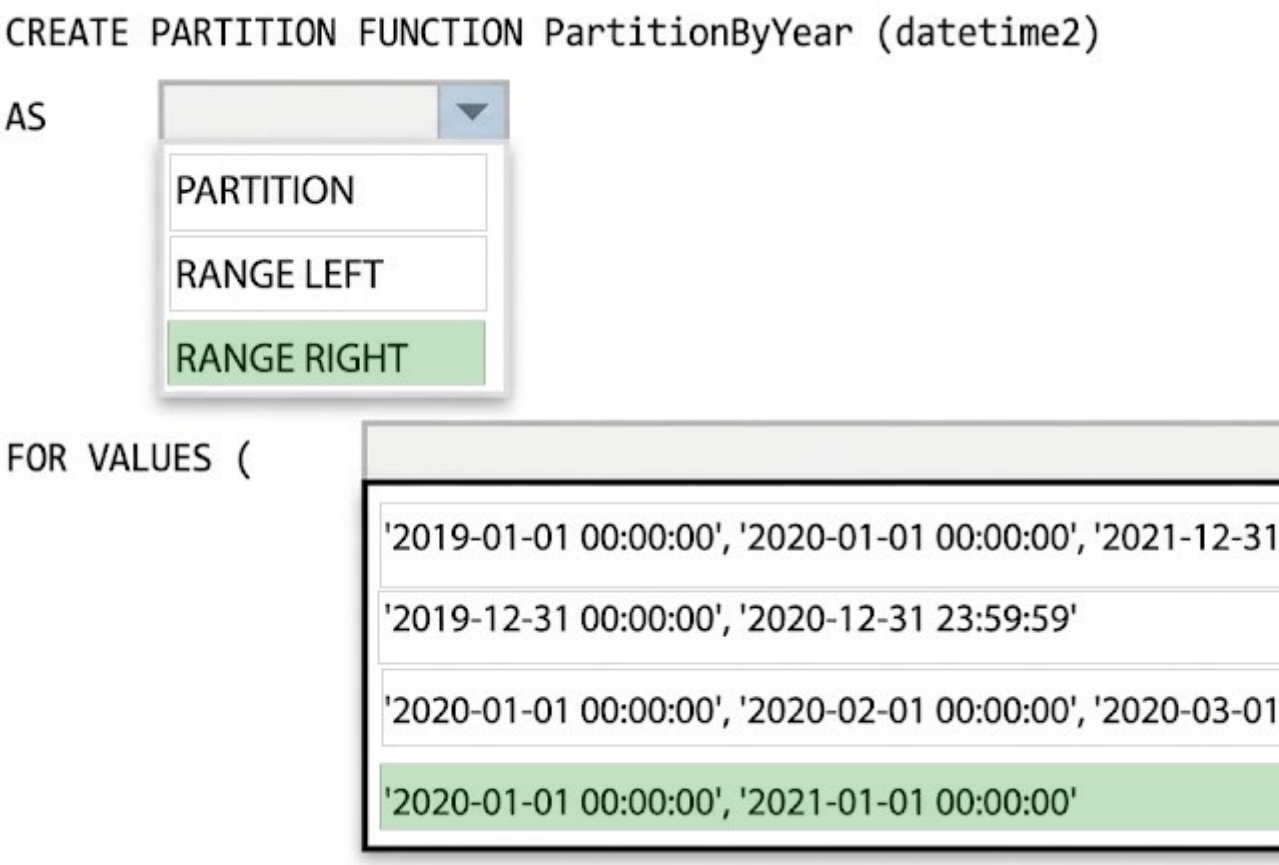
You need to partition the data in Table1 by year. The solution must minimize how long it takes to rebuild or reindex the table.

How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q23 T01

Correct answer:

RANGE RIGHT

'2020-01-01 00:00:00', '2021-01-01 00:00:00'
Explanation:

La sintaxis correcta de CREATE PARTITION FUNCTION es AS RANGE [LEFT | RIGHT] FOR VALUES (...). La opción “PARTITION” sola no es válida.
Se usa RANGE RIGHT porque es la práctica recomendada de Microsoft para columnas de tipo fecha/datetime2: el valor límite actúa como límite inferior inclusivo del rango derecho. Con los valores '2020-01-01 00:00:00' y '2021-01-01 00:00:00' se crean tres particiones:

datos < 2020-01-01
datos ≥ 2020-01-01 y < 2021-01-01 (todo el año 2020)
datos ≥ 2021-01-01 (año 2021 y posteriores)

Esto particiona exactamente por año los datos que abarcan 2020-2021.
Las otras opciones de valores no sirven:

'2019-01-01 00:00:00', '2020-01-01 00:00:00', '2021-12-31 23:59:59' mezcla años y usa un límite final de día que no es necesario ni limpio.
'2019-12-31 00:00:00', '2020-12-31 23:59:59' serían más propios de RANGE LEFT y exigen manejar el final exacto del día (problemas de precisión con datetime2).
'2020-01-01 00:00:00', '2020-02-01 00:00:00', '2020-03-01 00:00:00' crea particiones mensuales solo de

2020, no anuales.

RANGE LEFT con los valores de inicio de año colocaría el 1 de enero en la partición anterior, lo que no separa correctamente los años.
El particionamiento por año permite operaciones de mantenimiento (REBUILD / REORGANIZE) a nivel de partición en lugar de toda la tabla de 25 millones de filas, minimizando el tiempo requerido.

Reference:

https://learn.microsoft.com/en-us/sql/t-sql/statements/create-partition-function-transact-sql

https://learn.microsoft.com/en-us/sql/relational-databases/partitions/create-partitioned-tables-and-indexes

https://learn.microsoft.com/en-us/sql/relational-databases/partitions/partitioned-tables-and-indexes

QUESTION 24
You are developing an Azure SQL solution by using Microsoft Visual Studio 2026. The solution uses a GitHub repository.

You plan to use GitHub Copilot Chat to access the GitHub repository tools by connecting to the GitHub MCP Server.

You need to configure Visual Studio to support the planned configuration. The solution must rely on OAuth to access the MCP server.

What should you create?

A. the \.vs\settings.json file that contains the personal access token (PAT) that has the repo scope
B. the \.mcp.json file that contains a URL of https://api.githubcopilot.com/mcp/
C. the \.vscode\settings.json file that contains a URL of https://api.githubcopilot.com/mcp/
D. the \.github\mcp.json file that contains the personal access token (PAT) that has the repo scope

Answer: B
Section: (none)

Explanation/Reference:
Q24 / T01

1. Respuesta correcta.

B. the .mcp.json file that contains a URL of https://api.githubcopilot.com/mcp/

2. Explicación.
La documentación oficial de Microsoft Learn para Visual Studio indica que, para conectar GitHub Copilot Chat al servidor MCP remoto de GitHub y usar autenticación OAuth (sin necesidad de Personal Access Token), se debe crear un archivo *.mcp.json* (en la raíz de la solución *<SOLUTIONDIR>\.mcp.json* o en el perfil de usuario *%USERPROFILE%\.mcp.json*) con el siguiente contenido:

```
{
  "servers": {
    "github": {
      "url": "https://api.githubcopilot.com/mcp/"
    }
  }
}
```

Después de guardar el archivo, Visual Studio muestra un CodeLens que permite autenticarse mediante OAuth con una cuenta de GitHub. Esto cumple exactamente el requisito del enunciado: “The solution must rely on OAuth to access the MCP server”.

A es incorrecta: El archivo .vs\settings.json no es el mecanismo de configuración de servidores MCP en Visual Studio, y además exige un PAT (Personal Access Token) con scope repo. El enunciado exige OAuth, no PAT.
C es incorrecta: El archivo .vscode\settings.json (o .vscode\mcp.json) es el formato típico de Visual Studio Code, no de Visual Studio. Visual Studio usa específicamente .mcp.json.
D es incorrecta: No existe un archivo estándar .github\mcp.json para esta configuración en Visual Studio, y además contiene un PAT, lo que viola el requisito de usar OAuth.

3. Referencias.
https://learn.microsoft.com/en-us/visualstudio/ide/mcp-servers
https://docs.github.com/en/copilot/how-tos/provide-context/use-mcp-in-your-ide/set-up-the-github-mcp-server

QUESTION 25
You have an Azure SQL database that contains a table named Sales.Orders. Sales.Orders contains the following columns.

Column	Data type
OrderId	int
CustomerId	int
OrderDate	datetime2
TotalAmount	decimal(18,2)

Reporting queries frequently repeat logic to calculate the number of days since an order was placed.

You need to create a scalar user-defined function (UDF) that returns the number of days between an input value of @OrderDate and the current date and time.

How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

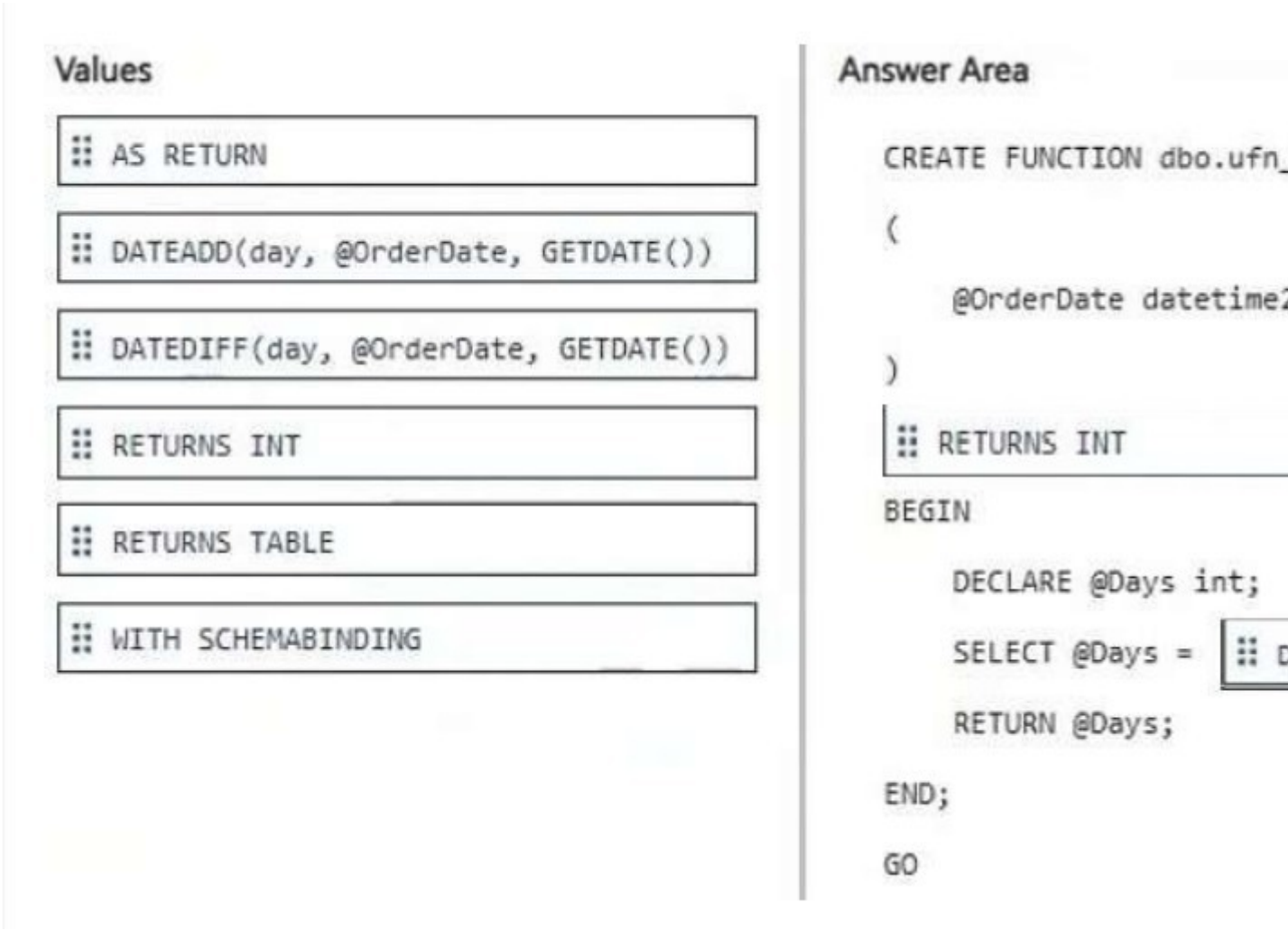
Values

- AS RETURN
- DATEADD(day, @OrderDate, GETDATE())
- DATEDIFF(day, @OrderDate, GETDATE())
- RETURNS INT
- RETURNS TABLE
- WITH SCHEMABINDING

Answer Area

```
CREATE FUNCTION dbo.ufn_
(
    @OrderDate datetime2
)
[                    ]
BEGIN
    DECLARE @Days int;
    SELECT @Days = [    ]
    RETURN @Days;
END;
GO
```

Answer:



Section: (none)

Explanation/Reference:
Q25 T01

Respuesta correcta
En el Answer Area hay que colocar:

Después de los parámetros → **RETURNS INT**
Dentro del SELECT @Days = → **DATEDIFF(day, @OrderDate, GETDATE())**

Por qué estas opciones
1. RETURNS INT

Es una función escalar: solo devuelve un valor único (el número de días).
Por eso se usa RETURNS INT.
RETURNS TABLE y AS RETURN son solo para funciones que devuelven tablas (Table-Valued Functions).
WITH SCHEMABINDING es opcional y el enunciado no lo pide.

2. DATEDIFF(day, @OrderDate, GETDATE())

DATEDIFF calcula la diferencia entre dos fechas.
El primer parámetro (day) indica que queremos el resultado en días.
@OrderDate es la fecha de inicio.
GETDATE() es la fecha y hora actual del servidor.
DATEADD no sirve aquí porque añade tiempo a una fecha, no calcula diferencias.

Referencias oficiales de Microsoft Learn

Sintaxis de funciones escalares
https://learn.microsoft.com/sql/t-sql/statements/create-function-transact-sql?view=sql-server-ver17#syntax
Función DATEDIFF
https://learn.microsoft.com/sql/t-sql/functions/datediff-transact-sql?view=sql-server-ver17
Funciones de fecha y hora
https://learn.microsoft.com/sql/t-sql/functions/date-and-time-data-types-and-functions-transact-sql?view=sql-server-ver17

QUESTION 26
You plan to implement changes to sp_UpdateProcedureForPatient to meet the performance requirements.
You change the stored procedure to run all its code within a transaction.
Which transaction level should you use?

Case Study Title (Case Study):
Q26 T01

Case Study -

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study -

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment -

Azure Environment -

Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:

Patients

Employees

Procedures

Transactions

UsefulPrompts

ProcedureDocuments

You store a column master key as a secret in Azure Key Vault.

You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.

Problem Statements -

Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.

Requirements -

Planned Changes -

Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.

Security Requirements -

Fabrikam identifies the following security requirements:

The TransactionProcessing application must use a passwordless connection to DB1.

The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.

Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.

Database Performance Requirements

Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.

AI Search, Embeddings, and Vector Indexing

Fabrikam identifies the following AI-related requirements:

Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).

Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.

Development Requirements -

Fabrikam identifies the following development requirements:

Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.

Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API: o Read data from the procedures table without authentication. o Read and insert data into the Transactions table once authenticated. o Execute the sp_UpdateProcedurePatient

stored procedure.

Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.

Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB -
You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

A. REPEATABLE READ

B. SERIALIZABLE
C. SNAPSHOT
D. READ COMMITTED SNAPSHOT

Answer: A
Section: (none)

Explanation/Reference:
T01 Q26

Explanation:
El requisito del case study dice: "Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs." Es decir, los registros que el procedimiento lee no pueden ser modificados por otras transacciones mientras el procedimiento se ejecuta.

REPEATABLE READ cumple exactamente eso:

Mantiene los shared locks sobre las filas leídas hasta que termina la transacción, no solo hasta el final de cada sentencia.

Ninguna otra transacción puede modificar ni borrar los datos que ya leíste hasta que hagas COMMIT.

Esto evita las lecturas no repetibles y las actualizaciones perdidas (lost updates). Eso encaja con el problema descrito: tras un procedimiento largo, los datos quedaban inconsistentes.

La documentación indica que hace falta al menos REPEATABLE READ para evitar lost updates.

Por qué las otras opciones son incorrectas

B. SERIALIZABLE: también cumpliría el requisito, pero es el nivel más restrictivo. Bloquea rangos completos de claves e impide insertar filas nuevas en esos rangos (evita phantoms). El requisito solo pide proteger los registros accedidos, no impedir inserciones nuevas. Usarlo reduce la concurrencia más de lo necesario. En un examen se elige el nivel mínimo que cumple el requisito.
C. SNAPSHOT: usa versionado de filas y no adquiere locks sobre los datos, así que otras transacciones sí pueden modificar las filas mientras corre tu procedimiento. Si tu transacción intenta confirmar cambios sobre datos que ya cambiaron, se hace rollback con error. No garantiza que nadie las cambie. Además, requiere activar ALLOW_SNAPSHOT_ISOLATION en la base de datos.
D. READ COMMITTED SNAPSHOT: es una opción de la base de datos (READ_COMMITTED_SNAPSHOT ON) que cambia el comportamiento de READ COMMITTED, no un nivel de aislamiento que se pueda fijar en la transacción con SET TRANSACTION ISOLATION LEVEL. Tampoco retiene bloqueos sobre lo leído.

References

SET TRANSACTION ISOLATION LEVEL (Transact-SQL): https://learn.microsoft.com/sql/t-sql/statements/set-transaction-isolation-level-transact-sql?view=sql-server-ver17

Snapshot isolation in SQL Server (describes each level, including REPEATABLE READ and SERIALIZABLE): https://learn.microsoft.com/sql/connect/ado-net/sql/snapshot-isolation-sql-server?view=sql-server-ver17

Understanding isolation levels (table of side effects and the note about lost updates): https://learn.microsoft.com/sql/connect/jdbc/understanding-isolation-levels?view=sql-server-ver17

QUESTION 27
HOTSPOT -

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the

time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment -

Azure Environment -

Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.

Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:

AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines

Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.

The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.

Requirements -

Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.

Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.

Database Performance and Requirements -
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. Latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.

Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.

AI Search, Embeddings, and Vector Indexing -
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -

Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.
Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.

Order the results by similarity score, with the highest score first.
You are creating a table that will store customer profiles.
You have the following Transact-SQL code.

```
CREATE TABLE dbo.VehicleTelemetry
(
  TelemetryId BIGINT IDENTITY(1,1) NOT NULL,
  VehicleId NVARCHAR(50) NOT NULL,
  TelemetryTimeUtc DATETIME2(3) NOT NULL,
  BatteryPercent TINYINT NULL,
  SpeedKmh SMALLINT NULL,
  LocationJson JSON NULL,
  ErrorCodesJson JSON NULL,
  RawPayload NVARCHAR(MAX) NULL,
  SysStartTime DATETIME2(7) GENERATED ALWAYS AS ROW START NOT NULL,
  SysEndTime DATETIME2(7) GENERATED ALWAYS AS ROW END NOT NULL,
  PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
  CONSTRAINT PK_VehicleTelemetry PRIMARY KEY CLUSTERED (TelemetryId)
)
WITH
(
  SYSTEM_VERSIONING = ON
  (
    HISTORY_TABLE = dbo.VehicleTelemetryHistory
  )
);
GO
CREATE INDEX IX_VehicleTelemetry_Time ON dbo.VehicleTelemetry (TelemetryTimeUtc);
CREATE JSON INDEX JI_VehicleTelemetry_Location ON dbo.VehicleTelemetry (LocationJson) FO
(
  '$.location. latitude', '$.location. longitude',
  '$.location. accuracy'
);
```

For each of the following statements, select Yes if the statement is true. Otherwise, select No.

NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes
The code meets the database performance requirements for partitioning.	○
The code meets the database performance requirements for JSON property querying.	○
Queries that filter on `$.location.heading` will use the `JI_VehicleTelemetry_Location` index.	○

Answer:

Answer Area

Statements	Yes
The code meets the database performance requirements for partitioning.	○
The code meets the database performance requirements for JSON property querying.	○
Queries that filter on `$.location.heading` will use the `JI_VehicleTelemetry_Location` index.	○

Section: (none)

Explanation/Reference:
Q27 T02

RESPUESTAS CORRECTAS:
Afirmación 1 - "El código cumple los requisitos de rendimiento de la base de datos para el particionamiento." = **NO**
Afirmación 2 - "El código cumple los requisitos de rendimiento de la base de datos para la consulta de propiedades JSON." = **SÍ**
Afirmación 3 - "Las consultas que filtran por $.location.heading usarán el índice JI_VehicleTelemetry_Location." = **NO**

EXPLICACIÓN:

Afirmación 1 (NO):
La sentencia CREATE TABLE de dbo.VehicleTelemetry **no tiene función de partición, ni esquema de partición, ni la cláusula "ON <partition_scheme>(<columna>)"** en la tabla ni en la clave primaria agrupada PK_VehicleTelemetry. SYSTEM_VERSIONING (tabla temporal) **no tiene relación con el particionamiento** — una tabla puede ser temporal y estar particionada al mismo tiempo, pero aquí simplemente **falta toda la infraestructura de partición**. El requisito del caso de estudio "Telemetry data must be stored in a partitioned table" **no se cumple**. Una implementación conforme necesitaría algo como:
CREATE PARTITION FUNCTION PF_TelemetryTime (DATETIME2(3)) AS RANGE RIGHT FOR VALUES (...);
CREATE PARTITION SCHEME PS_TelemetryTime AS PARTITION PF_TelemetryTime ALL TO ([PRIMARY]);
CREATE TABLE dbo.VehicleTelemetry (...) ON PS_TelemetryTime (TelemetryTimeUtc);
Nada de esto está presente, por lo que el requisito falla.

Afirmación 2 (SÍ):
CREATE JSON INDEX JI_VehicleTelemetry_Location ON dbo.VehicleTelemetry (LocationJson) FOR ('$.location.latitude', '$.location.longitude', '$.location.accuracy');
Esta es la sintaxis **nativa de SQL Server 2025** para CREATE JSON INDEX: CREATE JSON INDEX name

ON table_name (json_column_name) [FOR (sql_json_path [, ...n])]. Requiere una **clave agrupada** en la tabla, que existe (PK_VehicleTelemetry PRIMARY KEY CLUSTERED). Como el índice cubre explícitamente latitude, longitude y accuracy — exactamente las tres propiedades que exige el requisito — las consultas que usan JSON_VALUE(LocationJson,'$.location.latitude' RETURNING FLOAT) = @lat (y de forma equivalente para longitude/accuracy) pueden resolverse mediante **index seek** en lugar de escanear/parsear el documento JSON completo. Esto satisface el requisito "Latitude, longitude, and accuracy JSON properties must be filtered by using an index seek."

Afirmación 3 (NO):
Según la documentación oficial de CREATE JSON INDEX, cada sql_json_path de la cláusula FOR indexa **recursivamente solo a partir de esa ruta en adelante**, y las rutas **no pueden solaparse.** El índice se creó únicamente FOR ('$.location.latitude', '$.location.longitude', '$.location.accuracy') — **'$.location.heading' no forma parte de la definición del índice**. Por lo tanto, una consulta que filtre por JSON_VALUE (LocationJson,'$.location.heading') **no puede usar** JI_VehicleTelemetry_Location para un seek; recurriría a un escaneo/parseo del JSON. Para cubrir heading, el índice tendría que **recrearse** (según la documentación, "Modification of paths requires recreating the JSON index") incluyendo esa ruta, o crearse sin cláusula FOR (indexando todo el documento desde la raíz $).

REFERENCIAS:
- CREATE JSON INDEX (Transact-SQL): https://learn.microsoft.com/sql/t-sql/statements/create-json-index-transact-sql?view=sql-server-ver17
- JSON data in SQL Server - SQL Server 2025 changes: https://learn.microsoft.com/sql/relational-databases/json/json-data-sql-server?view=sql-server-ver17#sql-server-2025-changes
- Index JSON data: https://learn.microsoft.com/sql/relational-databases/json/index-json-data?view=sql-server-ver17
- sys.json_index_paths (Transact-SQL): https://learn.microsoft.com/sql/relational-databases/system-catalog-views/sys-json-index-paths-transact-sql?view=sql-server-ver17
- Partitioned tables and indexes: https://learn.microsoft.com/sql/relational-databases/partitions/partitioned-tables-and-indexes

QUESTION 28
Case Study

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer your questions. Clicking these buttons displays information about this case study, including environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment
Azure Environment

Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar(max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUtc (datetime2) • Description (nvarchar(256))
Supporttickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN (nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format:

```
{
  "text": "The battery drains too fast when driving uphill",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
```

}

The support staff at Contoso never has the UNMASK permission.

Problem Statements

Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:

-AI workloads

-Vector search

-Modernized API access

-Retrieval Augmented Generation (RAG) pipelines

Sometimes the ingestion pipeline fails to malformed JSON and duplicate payloads.

The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdateUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdateUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.

VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.

Requirements
Planned Changes

Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.

Security Requirements

Contoso identifies the following security requirements:

Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.

Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.

Database Performance and Requirements

Contoso identifies the following telemetry requirements:

Telemetry data must be stored in a partitioned table.

Telemetry data must provide predictable performance for ingestion and retention operations.

Latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.

Contoso identifies the following maintenance data requirements:

Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.

Avoid recursive updates.

AI Search, Embeddings, and Vector Indexing

Contoso plans to implement semantic search over incident data to meet the following requirements:

Embeddings must be stored in dedicated Azure SQL Database tables.

Embeddings must be generated from rich natural language fields.

Chunking must preserve semantic coherence.

Hybrid search must combine the following:

Vector similarity

Keyword filtering or boosting

Development Requirements

The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:

Extract the customer feedback text from the JSON document.

Filter rows where the JSON text contains a keyword.

Calculate a fuzzy similarity score between the feedback text and a known issue description.

Order the results by similarity score, with the highest score first.

You are creating a table that will store customer profiles.

You have the following Transact-SQL code.

```
CREATE TABLE dbo.CustomerProfiles
(
    CustomerId      BIGINT IDENTITY(1,1) PRIMARY KEY,
    FullName        NVARCHAR(200) MASKED WITH (FUNCTION = 'partial(1,"xxxx",1)'),
    EmailAddress    NVARCHAR(200) MASKED WITH (FUNCTION = 'email()'),
    PhoneNumber     NVARCHAR(50)  MASKED WITH (FUNCTION = 'default()'),
    RegionCode      NVARCHAR(10) NOT NULL
);
GO
CREATE FUNCTION dbo.fn_FilterByRegion(@RegionCode NVARCHAR(10))
RETURNS TABLE
AS
RETURN
(
    SELECT 1
    FROM dbo.UserRegionAccess ura
    WHERE ura.UserPrincipalName = SUSER_SNAME()
      AND ura.RegionCode = @RegionCode
);
GO
CREATE SECURITY POLICY CustomerRegionPolicy
ADD FILTER PREDICATE dbo.fn_FilterByRegion(RegionCode)
ON dbo.CustomerProfiles
WITH (STATE = ON);
GO
```

For each of the following statements, select Yes if the statement is true. Otherwise, select No.

NOTE: Each correct selection is worth one point.

Statements	Yes	No
The schema meets the security requirements for PII data.	○	○
Administrators of the Azure SQL server can see all the rows in `dbo.CustomerProfiles` when they use an application.	○	○
The masking rules will apply even when row-level security (RLS) filters out rows.	○	○

Answer:

Statements	Yes	No
The schema meets the security requirements for PII data.	●	○
Administrators of the Azure SQL server can see all the rows in `dbo.CustomerProfiles` when they use an application.	○	●
The masking rules will apply even when row-level security (RLS) filters out rows.	○	●

Section: (none)

Explanation/Reference:
Q28 T02

Explanation:
1 - The schema meets the security requirements for PII data → Sí
El requisito exige restringir al support staff (sin permiso UNMASK) de ver correos y teléfonos completos. EmailAddress usa email() (expone solo la primera letra y ".com") y PhoneNumber usa default() (enmascarado total). Ambas cumplen la definición de "PII" del caso.

2 - Administrators of the Azure SQL server can see all the rows in dbo.CustomerProfiles when they use an application → No
RLS se aplica a todos los usuarios, incluidos dbo/db_owner. Solo verían todas las filas si el predicado lo permitiera explícitamente; fn_FilterByRegion no tiene esa excepción, así que también se les filtra.

3 - The masking rules will apply even when row-level security (RLS) filters out rows → No
RLS filtra las filas antes de generar el resultado; DDM solo actúa sobre el conjunto de resultados ya devuelto. Una fila eliminada por RLS nunca llega a ese conjunto, por lo que no hay nada que enmascarar.

References:
https://learn.microsoft.com/sql/relational-databases/security/dynamic-data-masking?view=sql-server-ver17#define-a-dynamic-data-mask (Dynamic data masking – Define a mask)
https://learn.microsoft.com/sql/relational-databases/security/row-level-security?view=sql-server-ver17#permissions (Row-level security – Permissions)
https://learn.microsoft.com/fabric/data-warehouse/row-level-security#predicate-based-row-level-security (Row-level security in Fabric – Predicate-based RLS)

QUESTION 29
Case Study -

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study -

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment -

Azure Environment -

Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUtc (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (varchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • FleetId (int) • VIN (nvarchar(50)) • VehicleDescription (nvarchar(255))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector(1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.

Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:

AI workloads -

Vector search -

Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.

The engineers at Contoso report that the following dashboard query runs slowly.

SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth

```
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.

VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.

Requirements -

Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.

Security Requirements -
Contoso identifies the following security requirements:

Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.

Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.

Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.

Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.

Avoid recursive updates.

AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:

Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:

Vector similarity -

Keyword filtering or boosting -

Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:

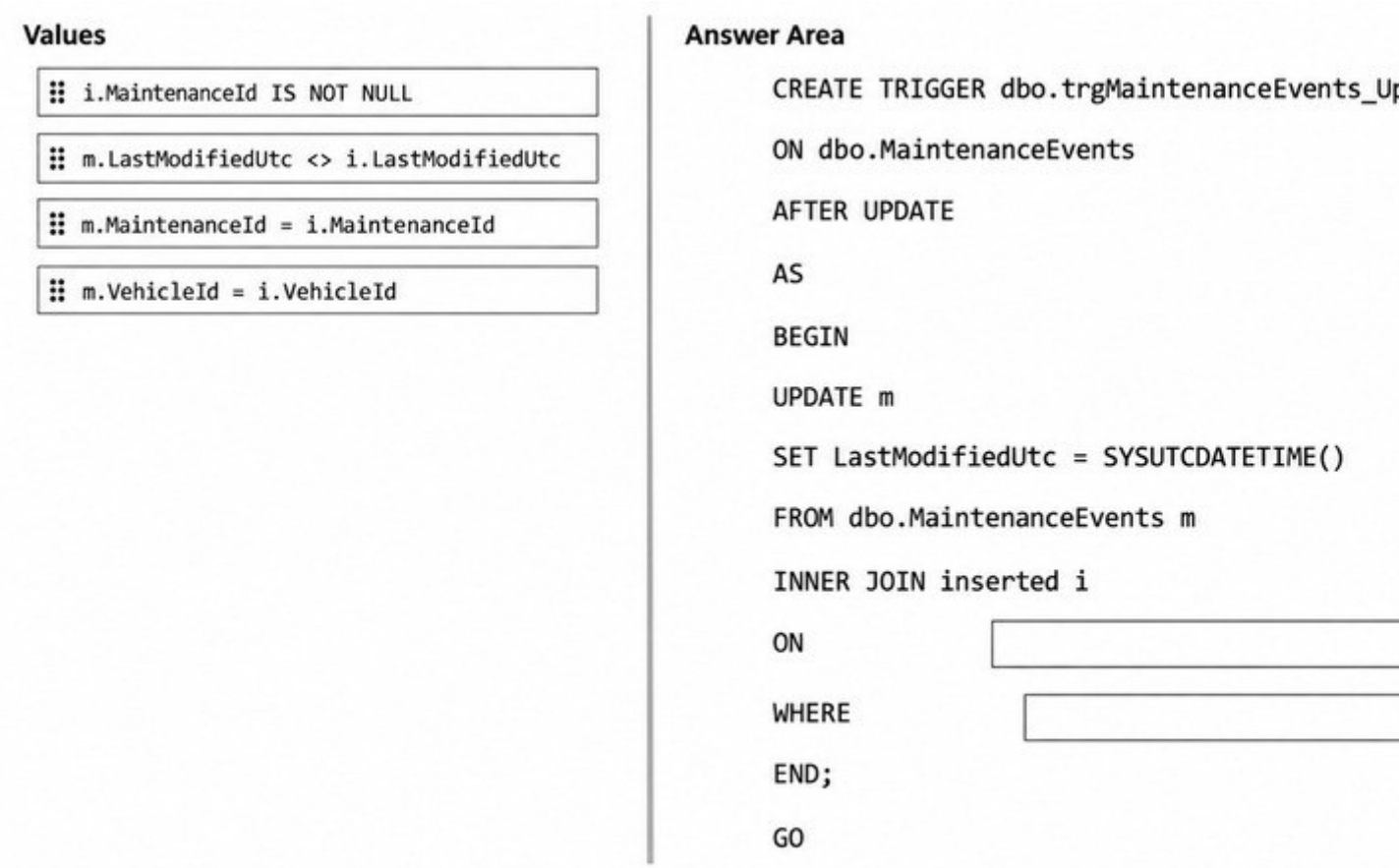
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.

Order the results by similarity score, with the highest score first.
You need to meet the database performance requirements for maintenance data.

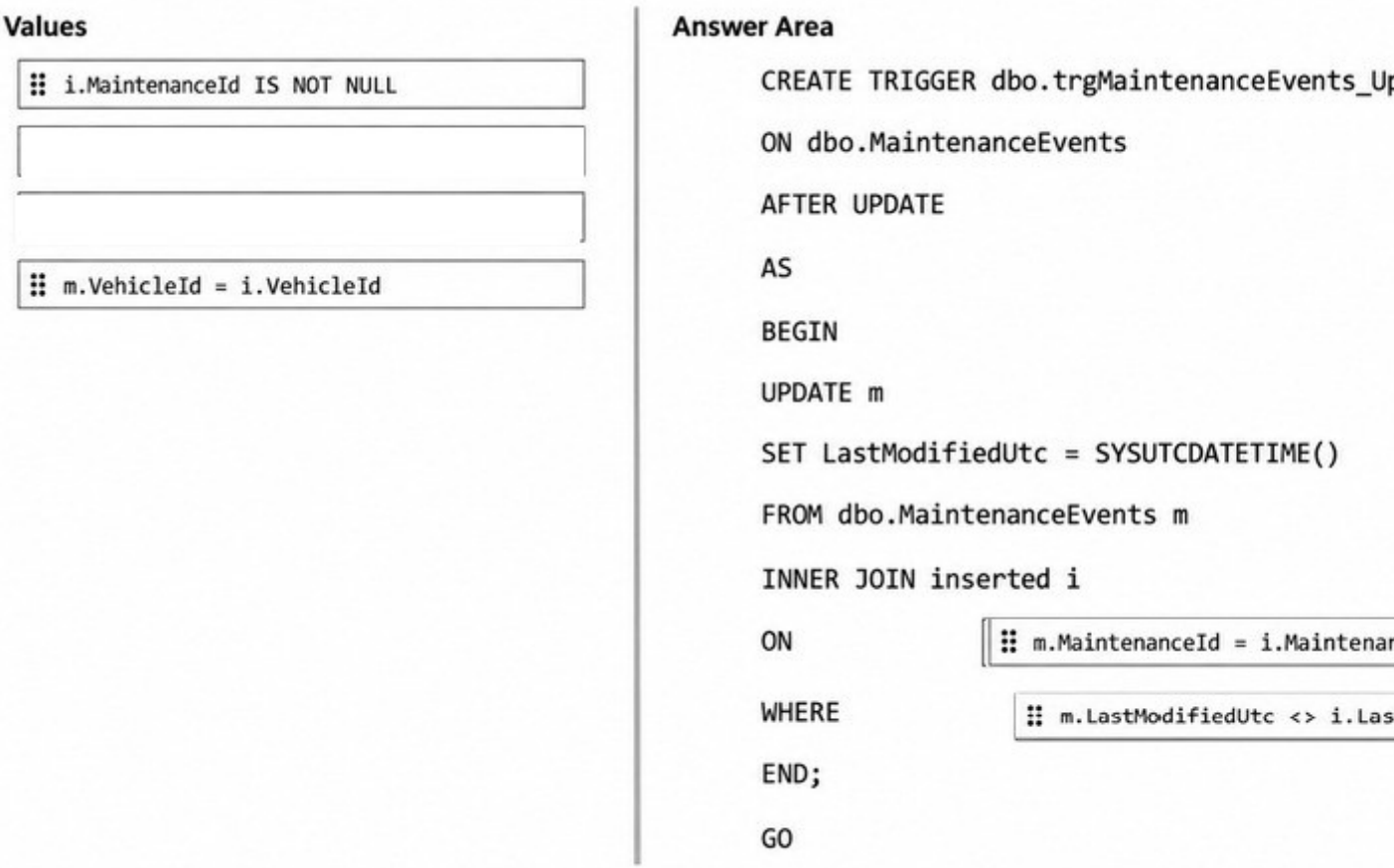
How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct

targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q29 T02

Correct Answer:

- ON (m.MaintenanceId = i.MaintenanceId)
- WHERE (m.LastModifiedUtc <> i.LastModifiedUtc)

Explanation:

El ejercicio establece dos requisitos para MaintenanceEvents:

Cuando se modifique una fila, actualizar LastModifiedUtc.
Evitar actualizaciones recursivas.

La primera selección importante es:

ON m.MaintenanceId = i.MaintenanceId

MaintenanceId es la clave primaria de MaintenanceEvents. La tabla especial inserted de un trigger contiene las filas afectadas por la operación UPDATE. Por tanto, relacionar ambas tablas mediante la clave primaria permite identificar exactamente qué fila de MaintenanceEvents corresponde a cada fila de inserted.
Microsoft recomienda implementar los triggers mediante lógica basada en conjuntos porque una sola sentencia UPDATE puede modificar varias filas.

Es decir:

MaintenanceEvents inserted
---------- --------

MaintenanceId = 15 <----------> MaintenanceId = 15
MaintenanceId = 27 <----------> MaintenanceId = 27

Por eso no se utiliza:

m.VehicleId = i.VehicleId

VehicleId no es la clave primaria. Un mismo vehículo puede disponer de varios eventos de mantenimiento, por lo que relacionar por VehicleId podría afectar a más de una fila cuando solo se pretende actualizar el evento concreto que fue modificado.

La segunda selección esperada por el ejercicio es:

WHERE m.LastModifiedUtc <> i.LastModifiedUtc

La intención de esta condición en la respuesta del ejercicio es evitar actualizaciones repetidas del timestamp. El valor que se asignará es:

SYSUTCDATETIME()

y Microsoft documenta que SYSUTCDATETIME() devuelve un datetime2 con la fecha y hora actuales en UTC, por lo que es apropiado para una columna denominada LastModifiedUtc.

No se selecciona:

i.MaintenanceId IS NOT NULL

porque MaintenanceId es la PRIMARY KEY. Esta condición no identifica cambios en LastModifiedUtc ni aporta un mecanismo para controlar actualizaciones repetidas.

Importante: matiz técnico de este ejercicio

Aquí hay un detalle relevante de cara a estudiar correctamente SQL Server: la combinación mostrada arriba es la respuesta esperada por este ejercicio, pero la justificación del WHERE es discutible técnicamente.

Microsoft documenta que, en un UPDATE, primero se actualiza la tabla base y después se copia la nueva versión de la fila a inserted. Por tanto, en un trigger AFTER UPDATE, inserted representa el nuevo estado de la fila.

La secuencia real es:

1. Estado antiguo → deleted
2. Se modifica la tabla
3. Estado nuevo → inserted
4. Se ejecuta el AFTER trigger

Esto significa que, en una implementación real, usar únicamente:

m.LastModifiedUtc <> i.LastModifiedUtc

no es un mecanismo sólido para controlar la recursividad, porque m e inserted normalmente representan ya el mismo estado después del UPDATE.

Microsoft documenta que la recursividad directa de un trigger AFTER depende de la opción de base de datos RECURSIVE_TRIGGERS; con ella habilitada, un trigger que actualice de nuevo su propia tabla puede volver a dispararse.

En código de producción, un mecanismo explícito como comprobar si el propio campo LastModifiedUtc fue actualizado sería más claro, por ejemplo mediante UPDATE(LastModifiedUtc), función que Microsoft proporciona específicamente para comprobar si una columna fue objeto de un UPDATE dentro de un trigger.

References:

Microsoft Learn — Use the inserted and deleted tables
Es la referencia principal para entender por qué se utiliza inserted, qué contiene después de un UPDATE y cómo se relaciona con la tabla original. Se aplica a SQL Server, Azure SQL Database y SQL Database en Microsoft Fabric.

Microsoft Learn — Create DML Triggers to Handle Multiple Rows of Data
Sirve para justificar que un trigger debe trabajar mediante lógica basada en conjuntos y considerar que un UPDATE puede afectar a múltiples filas, razón por la que es fundamental relacionar correctamente MaintenanceEvents con inserted.

Microsoft Learn — SYSUTCDATETIME (Transact-SQL)
Justifica específicamente por qué SYSUTCDATETIME() proporciona el valor adecuado para LastModifiedUtc: devuelve fecha y hora en UTC con tipo datetime2.

Microsoft Learn — Create Nested Triggers / Recursive Triggers
Esta referencia sirve exclusivamente para el requisito “Avoid recursive updates”. Explica cuándo un trigger AFTER puede volver a invocarse al actualizar su propia tabla.

QUESTION 30
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
PNG

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
PNG

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
PNG

To start the case study -

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
PNG

Existing Environment -
Azure Environment -

Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUtc (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN (nvarchar(50)) • VehicleDescrip(50) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format:
JSON{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
The support staff at Contoso never has the UNMASK permission. Problem Statements -Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following: AI workloads Modernized API access Retrieval Augmented Generation (RAG) pipelines Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads. The engineers at Contoso report that the following dashboard query runs slowly: SQLSELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
  You review the execution plan and discover that the plan shows a clustered index scan. VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details. Requirements -Planned Changes -Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports. Security Requirements -Contoso identifies the following security requirements: Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers. Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets. Database Performance and Requirements -Contoso identifies the following telemetry requirements: Telemetry data must be stored in a partitioned table. Telemetry data must provide predictable performance for ingestion and retention operations. Latitude, longitude, and accuracy JSON properties must be filtered by using an index seek. Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change. Avoid recursive updates. AI Search, Embeddings, and Vector Indexing -Contoso plans to implement semantic search over incident data to meet the following requirements: Embeddings must be stored in dedicated Azure SQL Database tables. Embeddings must be generated from rich natural language fields. Chunking must preserve semantic coherence. Hybrid

search must combine the following: Vector similarity Keyword filtering or boosting Development Requirements -The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases. Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table: Extract the customer feedback text from the JSON document. Filter rows where the JSON text contains a keyword. Calculate a fuzzy similarity score between the feedback text and a known issue description. Order the results by similarity score, with the highest score first. QuestionYou need to recommend a solution to resolve the slow dashboard query issue.What should you recommend? A. Create a clustered index on LastUpdatedUtc. B. On FleetId, create a nonclustered index that includes LastUpdatedUtc, EngineStatus, and BatteryHealth. C. On LastUpdatedUtc, create a nonclustered index that includes FleetId. D. On FleetId, create a filtered index where LastUpdatedUtc > DATEADD(DAY, -7, SYSDATETIME()).

A. Create a clustered index on LastUpdatedUtc.
B. On FleetId, create a nonclustered index that includes LastUpdatedUtc, EngineStatus, and BatteryHealth.
C. On LastUpdatedUtc, create a nonclustered index that includes FleetId.
D. On FleetId, create a filtered index where LastUpdatedUtc > DATEADD(DAY, -7, SYSUTCDATETIME()).

Answer: B
Section: (none)

Explanation/Reference:
Q30 T02
Correct Answer: On FleetId, create a nonclustered index that includes LastUpdatedUtc, EngineStatus, and BatteryHealth.
Explanation:
La consulta es:

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

El plan de ejecución muestra un clustered index scan, lo que significa que no existe un índice adecuado para filtrar de forma eficiente por FleetId ni para cubrir las columnas del SELECT y el ORDER BY.
La respuesta correcta crea un índice no agrupado (nonclustered) sobre la columna de filtrado (FleetId) e incluye las columnas LastUpdatedUtc, EngineStatus y BatteryHealth. De esta forma el índice se convierte en un covering index: el motor puede hacer un Index Seek sobre FleetId y obtener todas las columnas necesarias directamente del índice, sin necesidad de Key Lookups ni de acceder a la tabla base. Esto mejora notablemente el rendimiento. VehicleId se incluye automáticamente porque normalmente forma parte de la clave del índice clustered.

Create a clustered index on LastUpdatedUtc: Solo puede existir un índice clustered por tabla. Cambiarlo afectaría a todas las consultas y no optimiza el filtro por FleetId.
On LastUpdatedUtc, create a nonclustered index that includes FleetId: El filtro principal de la consulta es por FleetId (=). Un índice cuya clave principal es LastUpdatedUtc no permite un seek eficiente sobre FleetId.
On FleetId, create a filtered index where LastUpdatedUtc > DATEADD(DAY, -7, SYSUTCDATETIME()): El filtro de los últimos 7 días no aparece en la consulta original y limitaría el índice de forma innecesaria.

References:

Microsoft Learn: Index Architecture and Design Guide – Covering indexes y uso de INCLUDE
Microsoft Learn: Create Indexes with Included Columns
Microsoft Learn: CREATE INDEX (Transact-SQL) – cláusula INCLUDE
Documentación de Azure SQL Database / SQL Server sobre optimización de índices para consultas con WHERE + SELECT + ORDER BY

QUESTION 31
Order the results by similarity score, with the highest score first.
You need to recommend a solution that will resolve the ingestion pipeline failure issues.

Which two actions should you recommend? Each correct answer presents part of the solution.

Case Study Title (Case Study):
Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUtc (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN (nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
   "text": "The battery drains too fast when driving uphill.",
   "category": "Battery",
   "metadata": {
     "appVersion": "5.2.1",
     "device": "Android",
     "language": "en-GB"
   }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:

AI workloads -

Vector search -

Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.

Requirements -

Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.

Security Requirements -
Contoso identifies the following security requirements:

Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.

Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.

Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.

Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.

AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.

Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:

Vector similarity -

Keyword filtering or boosting -

Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.
Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.

A. Enable snapshot isolation on the database.
B. Use a trigger to automatically rewrite malformed JSON.
C. Add foreign key constraints on the table.
D. Create a unique index on a hash of the payload.
E. Add a check constraint that validates the JSON structure.

Answer: DE
Section: (none)

Explanation/Reference:
Q31 T02

Analisis del Escenario
El caso de estudio indica que el pipeline de ingesta falla ocasionalmente por dos causas concretas:

JSON mal formado (payloads que no son JSON válido).
Payloads duplicados.

Se necesita una solución declarativa a nivel de base de datos que rechace automáticamente ambos tipos de datos incorrectos en el momento de la inserción, sin modificar el código de la aplicación ni introducir lógica procedural frágil.

Dominio evaluado: Diseño y optimización de soluciones de base de datos (integridad de datos, restricciones CHECK e índices únicos en Azure SQL).

Solucion Correcta

Create a unique index on a hash of the payload..
Add a check constraint that validates the JSON structure.

Ejemplo de implementación típica:
SQL-- Columna calculada para el hash (persistida)
ALTER TABLE dbo.TelemetryData
ADD PayloadHash AS HASHBYTES('SHA2_256', RawPayload) PERSISTED;

-- Índice único sobre el hash (impide duplicados)
CREATE UNIQUE INDEX IX_TelemetryData_PayloadHash
ON dbo.TelemetryData (PayloadHash);

-- Restricción CHECK para validar JSON
ALTER TABLE dbo.TelemetryData
ADD CONSTRAINT CK_TelemetryData_RawPayload_IsJSON
CHECK (ISJSON(RawPayload) = 1);

Justificacion Tecnica
La restricción CHECK con ISJSON() se evalúa en cada INSERT/UPDATE. Si el texto no es JSON válido, la operación se rechaza de inmediato, eliminando los fallos por JSON mal formado.

El índice único sobre el hash del payload garantiza que no puedan existir dos filas con el mismo contenido. Como el hash es determinista y de longitud fija, el índice es eficiente y evita la comparación directa de columnas nvarchar(max).
Ambas técnicas son declarativas, se ejecutan en el motor de base de datos y no requieren triggers ni código de aplicación, lo que cumple el requisito de estabilidad del pipeline de ingesta.

Descarte de Alternativas

Habilitar aislamiento de instantáneas (snapshot isolation): solo afecta a la concurrencia y el bloqueo; no valida la estructura del JSON ni impide duplicados.
Usar un trigger para reescribir JSON mal formado: introduce lógica procedural, riesgo de actualizaciones recursivas y no es el patrón recomendado por Microsoft para validación de esquema.
Agregar restricciones de clave foránea: solo aplican integridad referencial entre tablas; no tienen efecto sobre la validez del contenido JSON ni sobre la unicidad de los payloads.

Referencias Oficiales
https://learn.microsoft.com/sql/t-sql/functions/isjson-transact-sql
https://learn.microsoft.com/azure/azure-sql/database/json-features
https://learn.microsoft.com/sql/relational-databases/json/store-json-documents-in-sql-tables
https://learn.microsoft.com/sql/relational-databases/indexes/create-unique-indexes
https://learn.microsoft.com/sql/relational-databases/tables/unique-constraints-and-check-constraints
https://learn.microsoft.com/sql/t-sql/functions/hashbytes-transact-sql

QUESTION 32
You have a Microsoft SQL Server 2025 instance that has a managed identity enabled.
You have a database that contains a table named dbo.ManualChunks. dbo.ManualChunks contains product manuals.

A retrieval query already returns the top five matching chunks as nvarchar(max) text.
You need to call an Azure OpenAI REST endpoint for chat completions. The solution must provide the highest level of security.

You write the following Transact-SQL code.

```
01  CREATE DATABASE SCOPED CREDENTIAL AzureOpenAIHeaders
02
03  GO
04  CREATE OR ALTER PROCEDURE dbo.AskManuals
05    ...
06    SELECT @chunks =
07    ...
08    SET @payload =
09    ...
10    EXEC @retval = sys.sp_invoke_external_rest_endpoint
11      @@url = N'https://<your-resource>.openai.azure.com/deployments/<your-deployment>/chat/co
12      @method = N'POST',
13      @credential = N'AzureOpenAIHeaders',
14      @payload = @payload,
15      @response = @response OUTPUT;
16  END;
17  GO
```

A. WITH IDENTITY = 'HTTPEndpointQuerystring',
SECRET = N'{"api-key":"<value>"}';
B. WITH IDENTITY = 'Managed Identity',
SECRET ='{"resourceid":"<value>"}';
C. WITH IDENTITY = 'Managed Identity',
SECRET = N'{"api-key":"<value>"}';
D. WITH IDENTITY = 'Managed Identity',
SECRET = N'{"api-key":"<value>"}';
E. WITH IDENTITY = 'AzureOpenAI',
SECRET = N'{"resourceid":"<value>"}';

Answer: B
Section: (none)

Explanation/Reference:
Q032 T02

Correct Answer:

WITH IDENTITY = 'Managed Identity',
SECRET ='{"resourceid":"<value>"}';

Explanation:
La solución requiere el nivel más alto de seguridad al llamar a un endpoint REST de Azure OpenAI desde Microsoft SQL Server 2025 usando sys.sp_invoke_external_rest_endpoint. Managed Identity es el método de autenticación recomendado porque evita almacenar o transmitir secretos como claves de API.

La sintaxis correcta para una credencial de ámbito de base de datos que usa Managed Identity es:

WITH IDENTITY = 'Managed Identity', SECRET = N'{"resourceid":"<value>"}';
Esto coincide exactamente con la opción indicada. El valor de IDENTITY 'Managed Identity' indica a SQL Server que obtenga un token de acceso de la identidad administrada asignada por el sistema (o la identidad primaria asignada por el usuario) de la instancia y lo pase en los encabezados de la solicitud. El objeto JSON de SECRET debe contener la clave resourceid cuyo valor es el identificador de audiencia/recurso esperado por el servicio de destino (comúnmente https://cognitiveservices.azure.com para Azure OpenAI).

WITH IDENTITY = 'HTTPEndpointQueryString', SECRET = N'{"api-key":""}';

es incorrecta porque HTTPEndpointQueryString coloca el secreto en la cadena de consulta y espera un valor de api-key. Esto es menos seguro que Managed Identity y no utiliza la identidad administrada de la instancia de SQL Server.

WITH IDENTITY = 'Managed Identity', SECRET = N'{"api-key":""}';
es incorrecta porque, aunque usa la identidad 'Managed Identity', el SECRET aún suministra una api-key. La autenticación con Managed Identity no utiliza claves de API; el SECRET debe contener un resourceid.

WITH IDENTITY = 'HTTPEndpointHeaders', SECRET = N'{"api-key":""}';
es incorrecta porque HTTPEndpointHeaders con una api-key almacena y transmite un secreto estático en los encabezados de la solicitud. Esto representa un nivel de seguridad inferior al de Managed Identity.

WITH IDENTITY = 'AzureOpenAI', SECRET = N'{"resourceid":""}';
es incorrecta porque el valor de IDENTITY 'AzureOpenAI' no es un tipo de identidad admitido para las credenciales de ámbito de base de datos usadas con sp_invoke_external_rest_endpoint. Los únicos valores de identidad admitidos son HTTPEndpointHeaders, HTTPEndpointQueryString, Managed Identity y Shared Access Signature.

References:
https://learn.microsoft.com/sql/relational-databases/system-stored-procedures/sp-invoke-external-rest-endpoint-transact-sql?view=sql-server-ver17#credentials
https://learn.microsoft.com/sql/t-sql/statements/create-external-model-transact-sql?view=sql-server-ver17#catalog-view

QUESTION 33
You have an Azure SQL database named SalesDB that contains a table named dbo.Articles. dbo.Articles contains two million articles with embeddings. The articles are updated frequently throughout the day.You query the embeddings by using VECTOR_SEARCH.Users report that semantic search results do NOT reflect the updates until the following day.You need to ensure that the embeddings are updated whenever the articles change. The solution must minimize CPU usage on SalesDB.Which embedding maintenance method should you implement?

A. Modify the query to use VECTOR_DISTANCE instead of VECTOR_SEARCH.
B. Enable change data capture (CDC) on dbo.Articles and use an Azure Functions app to process CDC changes.
C. Run an hourly Transact-SQL job that regenerates embeddings for all the rows in dbo.Articles.
D. On dbo.Articles, create a trigger that calls AI_GENERATE_EMBEDDINGS for each inserted or updated row.

Answer: B
Section: (none)

Explanation/Reference:
Q33 T02

Correct Answer:
Enable change data capture (CDC) on dbo.Articles and use an Azure Functions app to process CDC changes.

Explanation:

Por qué es correcta

CDC permite capturar de forma asíncrona y eficiente únicamente las filas que han cambiado (inserciones/actualizaciones) en dbo.Articles, sin necesidad de escanear ni recalcular las dos millones de filas completas. Una Azure Function puede leer periódicamente (o casi en tiempo real) los cambios registrados en las tablas de captura de CDC y regenerar embeddings solo para esas filas modificadas.

Esto cumple los dos requisitos del enunciado:

Actualidad: los embeddings se actualizan poco después de que el artículo cambia, en lugar de esperar al día siguiente.
Mínimo uso de CPU en SalesDB: CDC tiene un overhead muy bajo sobre la base de datos (se basa en leer el log de transacciones), y el procesamiento pesado (llamar al modelo de embeddings) se delega fuera de SQL, en la Azure Function, en lugar de ejecutarse dentro del motor de base de datos.

Por qué las otras opciones son incorrectas:

A. Usar VECTOR_DISTANCE en lugar de VECTOR_SEARCH — Esto solo cambia la forma de calcular similitud en la consulta, pero no soluciona el problema real: los embeddings almacenados siguen sin actualizarse cuando cambian los artículos. No resuelve la causa del retraso.
C. Job horario en T-SQL que regenera embeddings para todas las filas — Aunque mejoraría la frecuencia de actualización, regenerar los embeddings de las dos millones de filas cada hora (en vez de solo las filas modificadas) consume una cantidad de CPU muy alta e innecesaria en SalesDB, incumpliendo el requisito de minimizar el uso de CPU.
D. Trigger que llama a AI_GENERATE_EMBEDDINGS en cada inserción/actualización — Un trigger síncrono ejecuta el cálculo del embedding dentro de la misma transacción, en el propio motor de SQL, cada vez que se modifica una fila. Con actualizaciones frecuentes a lo largo del día sobre una tabla de dos millones de artículos, esto genera una carga de CPU alta y constante sobre SalesDB, además de aumentar la latencia de cada transacción.

References:
https://learn.microsoft.com/azure/azure-sql/database/change-data-capture-overview

https://learn.microsoft.com/azure/azure-functions/functions-bindings-azure-sql

https://learn.microsoft.com/azure/azure-sql/database/vector-search-overview

QUESTION 34
You have a GitHub Codespaces enviroment tha has GitHub Copilot Chat installed and is connected to a SQL database in Microsoft Fabric named DB1. DB1 contains tables named Sales.Orders and Sales. Customers
You use GitHub Copilot in the context of DB1.
A company policy prohibits sharing customer Personally Identifiable Information (PII), secrets, and query result sets with any AI service.
You are asked to use GitHub Copilot Chat to write and review Transact-SQL code for a new stored procedure that will join Sales.Customers and return customer names and email addresses. The solution must NOT share the actual data in the tables with GitHub Copilot Chat.
What should you do?

A. From Sales.Customers, paste several rows that include email addresses into a chat, so that GitHub Copilot Chat can infer edge cases.
B. Run a SELECT statement that returns customer names and email addresses and provide the result set to GitHub Copilot Chat so that GitHub Copilot Chat can generate the stored procedure.
C. Provide the database connection string to GitHub Copilot Chat so that GitHub Copilot Chat can validate the stored procedure.
D. Ask GitHub Copilot Chat to generate the stored procedure by using schema details only.

Answer: D
Section: (none)

Explanation/Reference:
Q34 T02
Respuesta Correcta:

Ask GitHub Copilot Chat to generate the stored procedure by using schema details only.

Explicación:

La opción Ask GitHub Copilot Chat to generate the stored procedure by using schema details only es correcta porque Copilot puede utilizar los detalles del esquema —tablas, columnas y relaciones— para

generar el código T-SQL sin necesidad de acceder a los datos reales.

Esto permite crear el procedimiento almacenado sin compartir PII, secretos ni resultados de consultas, cumpliendo la política de la compañía.

¿Por qué el resto son incorrectas?

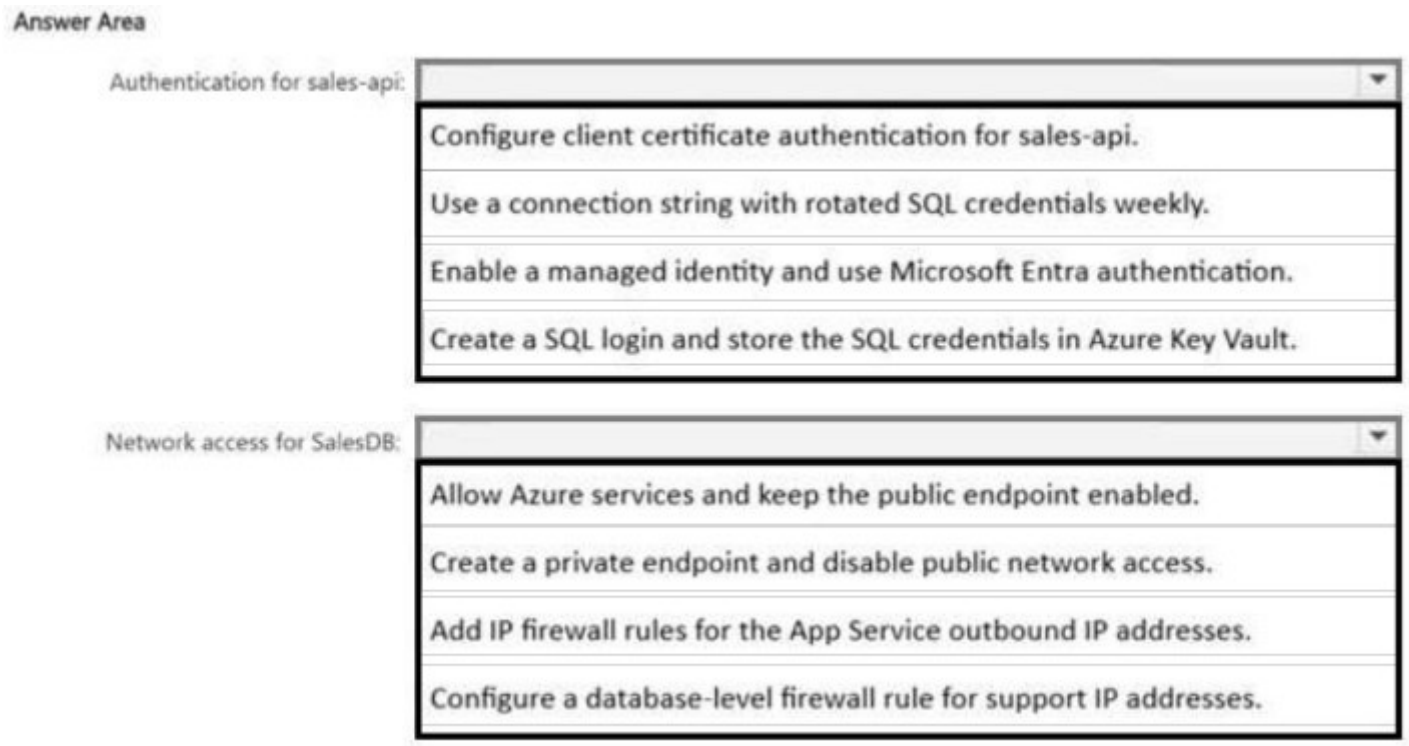
From Sales.Customers, paste several rows that include email addresses into a chat, so that GitHub Copilot Chat can infer edge cases. Comparte filas reales de Sales.Customers, incluyendo direcciones de correo electrónico, que constituyen PII.

Run a SELECT statement that returns customer names and email addresses and provide the result set to GitHub Copilot Chat so that GitHub Copilot Chat can generate the stored procedure. Comparte con Copilot el resultado de una consulta que contiene datos reales de clientes, algo expresamente prohibido por la política.

Provide the database connection string to GitHub Copilot Chat so that GitHub Copilot Chat can validate the stored procedure. Una cadena de conexión puede contener información sensible o credenciales, por lo que no debe compartirse con un servicio de IA.

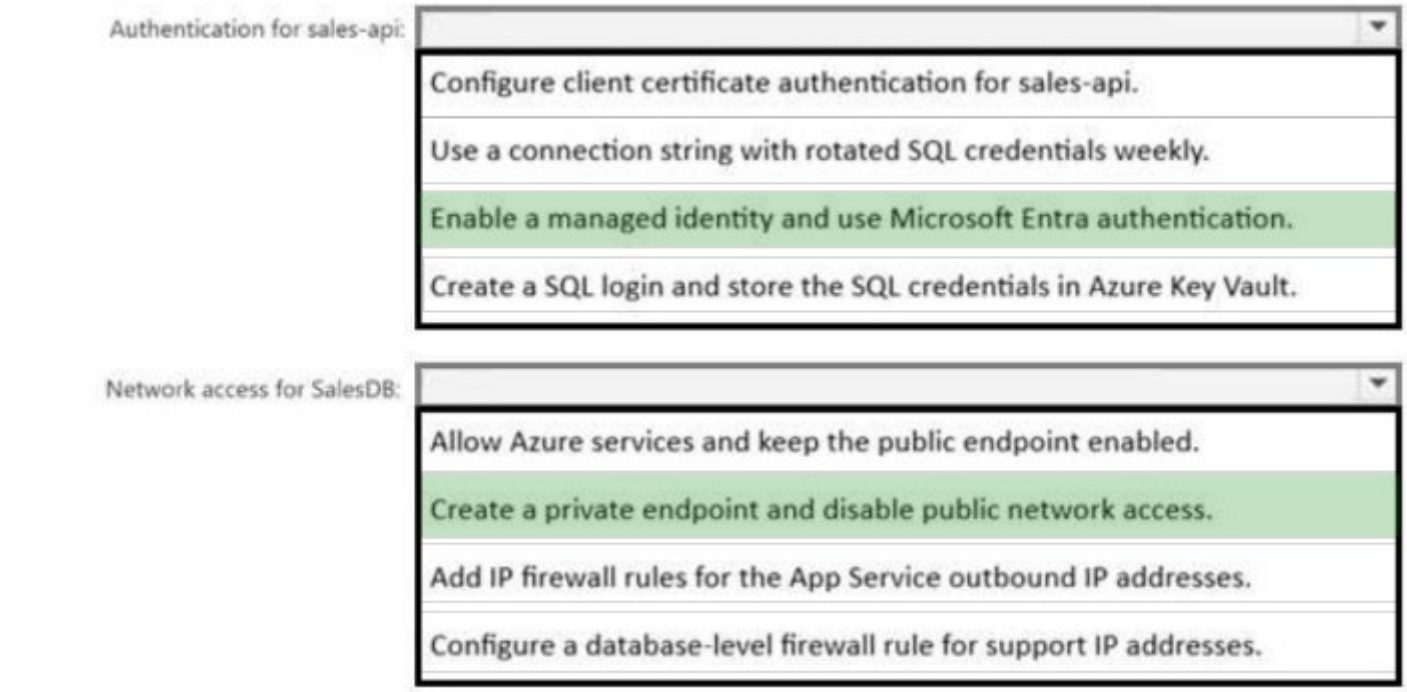
Referencia:
Microsoft Learn – GitHub Copilot con la extensión MSSQL: Microsoft Learn – https://learn.microsoft.com/en-us/sql/tools/visual-studio-code-extensions/github-copilot/overview?view=sql-server-ver17

QUESTION 35
HOTSPOT-
You have an Azure subscription. The subscription contains an Azure SQL database named SalesDB and an Azure App Service app named sales-api. sales-api uses virtual network integration to a subnet named vnet-prod/subnet-app and reads from SalesDB.
You need to configure authentication and network access to meet the following requirements:
Ensure that sales-api connects to SalesDB by using passwordless authentication.
Ensure that all the database traffic remains within the subscription.
The solution must minimize administrative effort.
What should you configure? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.



Answer:

Answer Area



Section: (none)

Explanation/Reference:
Q35 T02

Correct answer:

1.Enable a managed identity and use microsoft entra authentication

2.Create a private endpoint and disable public network access.

Explanation:

Authentication for sales-api

Correcta: "Enable a managed identity and use Microsoft Entra authentication." Una managed identity de App Service permite que sales-api se autentique en Azure SQL Database con un token de Microsoft Entra, sin contraseñas ni secretos en la connection string, que es el requisito de passwordless. Además Azure gestiona el ciclo de vida de la identidad, por lo que no hay que rotar ni custodiar credenciales y el esfuerzo administrativo es mínimo.

Incorrectas:
- "Configure client certificate authentication for sales-api." La autenticación por certificado de cliente en App Service sirve para autenticar a los clientes que llaman a la app, no para que la app se autentique contra Azure SQL Database. Además implicaría gestionar certificados.
- "Use a connection string with rotated SQL credentials weekly." Sigue usando usuario y contraseña, así que no es passwordless, y rotar credenciales cada semana aumenta el esfuerzo administrativo.
- "Create a SQL login and store the SQL credentials in Azure Key Vault." Key Vault protege el secreto, pero la autenticación sigue basada en contraseña, por lo que no cumple passwordless, y añade más componentes que administrar.

Network access for SalesDB

Correcta: "Create a private endpoint and disable public network access." El private endpoint asigna a SalesDB una IP privada dentro de la VNet. Como sales-api usa VNet integration con vnet-prod/subnet-app, su tráfico saliente llega a SalesDB por esa IP privada a través de la red de Microsoft. Al desactivar el acceso de red público (Deny public network access) se garantiza que todo el tráfico de base de datos permanece privado y no puede usarse el endpoint público. Nota: crear el private endpoint no bloquea por sí solo el acceso público, por eso hay que desactivarlo explícitamente.

Incorrectas:
- "Allow Azure services and keep the public endpoint enabled." El tráfico sigue yendo al endpoint público, y esta opción permite conexiones desde cualquier recurso de Azure, incluso de otras suscripciones, lo que incumple el requisito.
- "Add IP firewall rules for the App Service outbound IP addresses." Las reglas de firewall IP siguen usando el endpoint público. Además las IP salientes de App Service son compartidas y pueden cambiar, lo que exige mantenimiento continuo.
- "Configure a database-level firewall rule for support IP addresses." Solo autoriza las IP de soporte, no resuelve la conectividad privada de sales-api y mantiene el endpoint público habilitado.

Reference:

https://learn.microsoft.com/en-us/azure/app-service/tutorial-connect-msi-azure-database
//learn.microsoft.com/en-us/azure/azure-sql/database/private-endpoint-overview
https://learn.microsoft.com/en-us/azure/azure-sql/database/connectivity-settings#deny-public-network-access

QUESTION 36

DRAG DROP -
You have a database named DB1. The schema is stored in a GitHub repository as an SDK-style SQL database project.

You use a feature branch workflow to deploy changes to DB1.
You need to update the local feature branch with the latest changes to main, and then create a pull request to merge the feature branch into main for review.

How should you complete the GitHub CLI script? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

DRAG DROP -

You have a database named DB1. The schema is stored in a GitHub repository as an SDK-style SQL database project.

You use a feature branch workflow to deploy changes to DB1.

You need to update the local feature branch with the latest changes to main, and then create a pull request to merge the feature branch into

How should you complete the GitHub CLI script? To answer, drag the appropriate values to the correct targets. Each value may be used once all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Values

- `gh pr create`
- `gh pr merge`
- `gh pr ready`
- `git checkout main`
- `git fetch origin`
- `git merge origin/main`
- `git pull origin main`

Answer Area

```
git checkout feature/db1-add-staticdata
[                                      ]
[                                      ]
[                                      ] \
--title "Feature Update: DB1" \
--body "Apply latest improvements and updates for review" \
--head feature/db1-add-staticdata \
--base main \
--repo <GitHubOwner>/DB1 \
--web
```

Answer:

DRAG DROP -

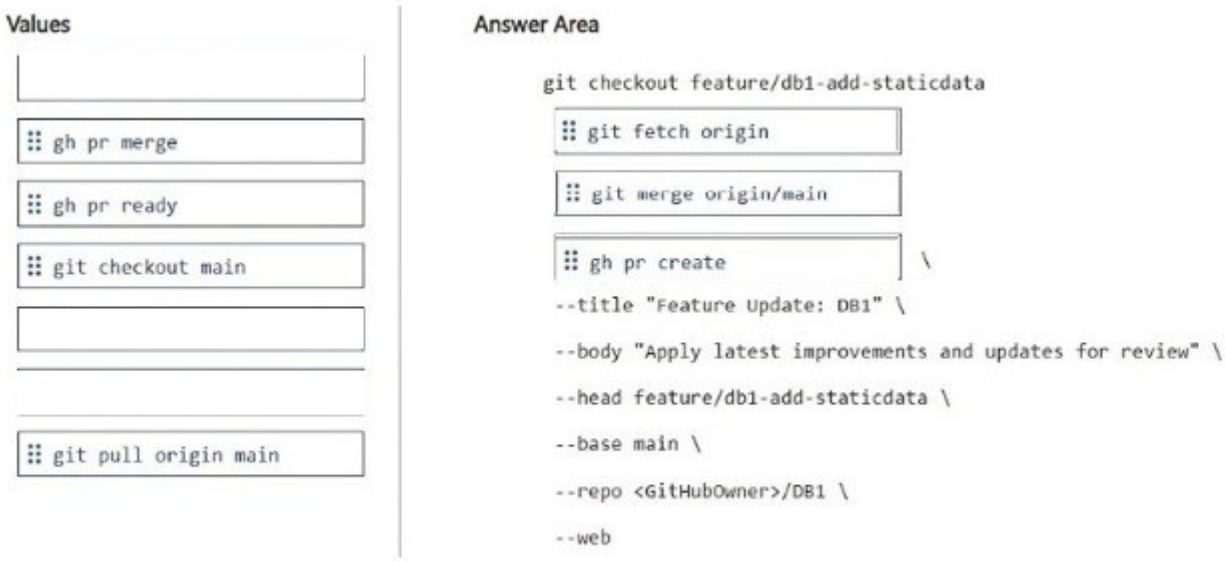
You have a database named DB1. The schema is stored in a GitHub repository as an SDK-style SQL database project.

You use a feature branch workflow to deploy changes to DB1.

You need to update the local feature branch with the latest changes to main, and then create a pull request to merge the feature branch into

How should you complete the GitHub CLI script? To answer, drag the appropriate values to the correct targets. Each value may be used once all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.



Section: (none)

Explanation/Reference:
Q36 T02

The correct answers are:

Blank 1: git fetch origin
Blank 2: git merge origin/main
Blank 3: git pr create

Solution

Given that git checkout feature/db1-add-staticdata is already provided in the script (switching to the feature branch), the three blanks need to accomplish:

Blank 1 - git fetch origin

Downloads the latest commits from the remote repository without modifying local branches
Updates the remote-tracking branch origin/main with the latest state

Blank 2 - git merge origin/main

Integrates the latest changes from the remote main branch into the local feature branch
Ensures the feature branch is up-to-date before creating the pull request

Blank 3 - git pr create

Creates the pull request using the GitHub CLI
Uses the parameters provided (--title, --body, --head, --base, --repo, --web) to configure the pull request

Why the other values are not used:

git pr merge: Used to merge an existing pull request, not create one.
git pr ready: Not a valid GitHub CLI command in this workflow context.
git checkout main: Would switch away from the feature branch, which is not needed here since you must stay on the feature branch to create the pull request from it.
git pull origin main: Combines fetch and merge into one command, but the script's design (with two separate blanks) indicates fetch and merge should be explicit separate steps.

References

Microsoft Learn - Adopt a Git branching strategy
URL: https://learn.microsoft.com/azure/devops/repos/git/git-branching-guidance?view=azure-devops
Topic: Feature branch workflow and best practices
Microsoft Learn - Update code with fetch, merge, and pull
URL: https://learn.microsoft.com/azure/devops/repos/git/pulling?view=azure-devops
Topic: Understanding fetch, merge, and pull operations
Microsoft Learn - Manage repository changes by using pull requests on GitHub
URL: https://learn.microsoft.com/training/modules/manage-changes-pull-requests-github/
Topic: Creating and managing pull requests
DP-800 Study Guide - Version Control with Git and GitHub
Topic: Understanding Git workflows and GitHub CLI for database schema management

QUESTION 37
You have an Azure SQL database that contains tables named dbo.ProductDocs and dbo.
ProductDocsEmbeddings. dbo.ProductDocs contains
product documentation and the following columns:
DocID (int)
Title (nvarchar(200))
Body (nvarchar(max))
LastModified (datetime2)
The documentation is edited throughout the day.
dbo.ProductDocsEmbeddings contains the following columns:
DocID (int)
ChunkOrder (int)
ChunkText (nvarchar(max))
Embedding (vector(1536))
The current embedding pipeline runs once per night.
You need to ensure that embeddings are updated every time the underlying documentation content changes. The solution must NOT require a
nightly batch process.
What should you include in the solution?

A. fixed-size chunking
B. a smaller embedding model
C. table triggers
D. change tracking on dbo.ProductDocs

Answer: D
Section: (none)

Explanation/Reference:
Q037 T02

Español

Respuesta

d. Change tracking en dbo.ProductDocs

Explicación

Por qué es la correcta: Change tracking es una característica del motor que registra qué filas de una tabla se han insertado, actualizado o eliminado desde una versión concreta. Con ella, un proceso de microlotes (un job programado, una Azure Function con SQL trigger binding o Logic Apps) consulta cada pocos segundos qué documentos han cambiado. Solo llama al modelo de embeddings para esos documentos, vuelve a dividir el texto en fragmentos y actualiza dbo.ProductDocsEmbeddings. Esto sustituye al lote nocturno: los embeddings se actualizan de forma continua, casi en tiempo real. Además tiene muy poca sobrecarga, y la llamada al modelo queda fuera de la transacción del usuario que edita el documento. Ese punto es importante aquí, porque la documentación se edita durante todo el día y cada documento genera varios fragmentos (ChunkOrder).

Por qué las demás no:

a. Fragmentación de tamaño fijo: define cómo se divide el texto en chunks, pero no detecta cambios ni dispara ninguna actualización. Sigue haciendo falta un mecanismo de detección.
b. Modelo de embeddings más pequeño: reduce dimensiones y coste, pero obligaría a regenerar todo, porque la columna es de 1536 dimensiones. No resuelve el problema de la frescura.
c. Triggers de tabla: técnicamente es posible, y Microsoft lo describe como la opción más simple. Pero el trigger forma parte de la transacción que modifica los datos, así que la llamada al modelo bloquea recursos y reduce la concurrencia. Microsoft lo recomienda sobre todo para pruebas de concepto o tablas con poco acceso concurrente. Con un Body grande, varios fragmentos por documento y ediciones constantes, no escala.

Referencias

Microsoft Learn:

Vector and embeddings frequently asked questions (FAQ) (apartado "How do I keep embeddings up to date?"): https://learn.microsoft.com/sql/sql-server/ai/vectors-frequently-asked-questions?view=sql-server-ver17#how-do-i-keep-embeddings-up-to-date
About Change Tracking (SQL Server): https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/about-change-tracking-sql-server?view=sql-server-ver16
Azure Functions SQL trigger binding: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-azure-sql-trigger

SQL Server:

Database and AI: solutions for keeping embeddings updated (blog oficial de Azure SQL, no es Learn): https://devblogs.microsoft.com/azure-sql/database-and-ai-solutions-for-keeping-embeddings-updated/

Clave DP-800

Para mantener embeddings al día sin lote nocturno y con mucha concurrencia, usa change tracking (detección incremental ligera) y no triggers que llamen al modelo dentro de la transacción.

English

Answer

d. Change tracking on dbo.ProductDocs

Explanation

Why it's correct: Change tracking records which rows were inserted, updated or deleted since a given version. A micro-batch process (a scheduled job, an Azure Function with the SQL trigger binding, or Logic Apps) polls for changed documents every few seconds. It calls the embedding model only for those

documents, re-chunks their text, and updates dbo.ProductDocsEmbeddings. This replaces the nightly batch, so embeddings are refreshed continuously, close to real time. It also has minimal overhead, and the model call stays outside the editing user's transaction. That matters here because documentation is edited all day and each document produces several chunks (ChunkOrder).

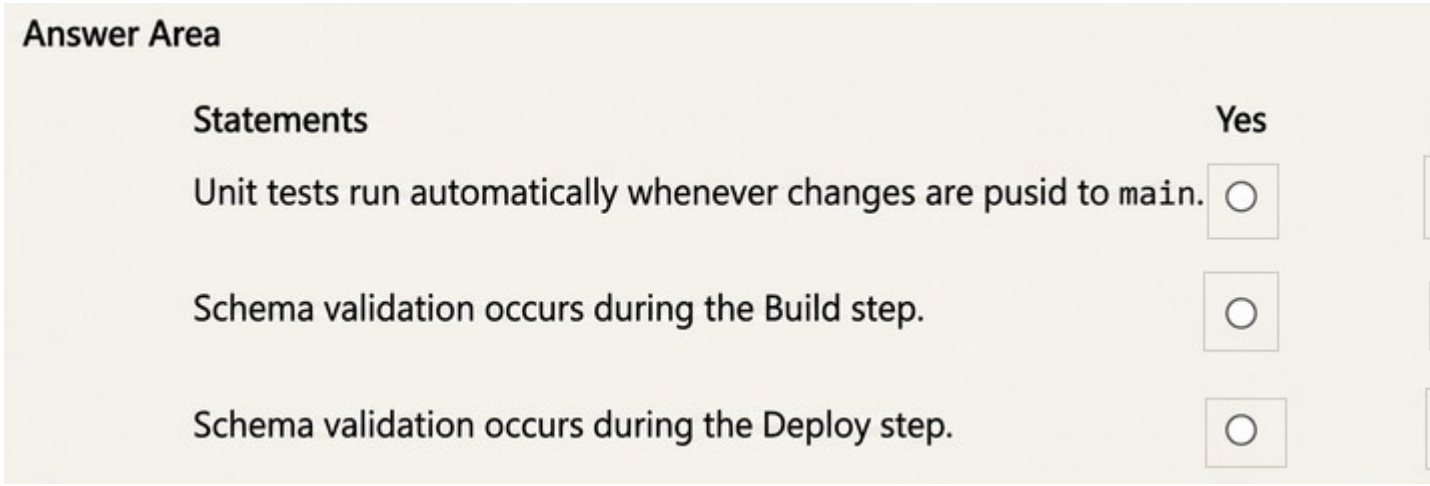
Why the others are wrong:

a. Fixed-size chunking: it defines how text is split, but it doesn't detect changes or trigger any refresh. You still need a change-detection mechanism.
b. Smaller embedding model: it lowers dimensions and cost, but the column is 1536-dimensional, so everything would have to be regenerated. It doesn't solve freshness.
c. Table triggers: technically possible, and Microsoft describes them as the simplest option. But the trigger runs inside the transaction that modifies the data, so the model call holds locks and reduces concurrency. Microsoft recommends this mainly for proofs of concept or tables with very limited concurrent access. With a large Body, multiple chunks per document and constant edits, it doesn't scale.

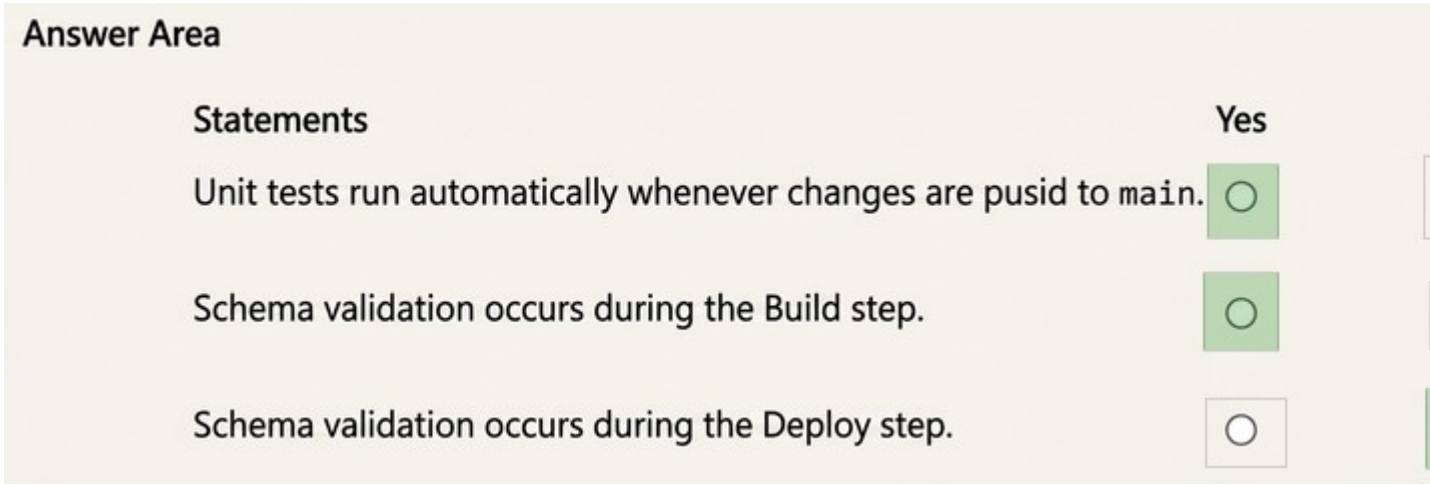
QUESTION 38
You have a database named db1. The schema is stored in a Git repository as an SDK-style SQL database project. The repository contains the following GitHub Action workflow.

```
name: Database CI/CD
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
 jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v3
    - name: Setup .NET
      uses: actions/setup-dotnet@v3
      with:
        dotnet-version: '8.x'
    - name: Build
      run: dotnet build db1.sqlproj --configuration Release
    - name: Deploy
      run: |
        SqlPackage /Action:Publish \
          /SourceFile:./bin/Release/db1.dacpac \
          /TargetConnectionString:"${{ secrets.Target_Connection_String }}"
  unit-tests:
    runs-on: ubuntu-latest
    needs: build-and-deploy
    if: github.ref == 'refs/heads/main'
    steps:
    - name: Checkout code
      uses: actions/checkout@v3
    - name: Setup .NET
      uses: actions/setup-dotnet@v3
      with:
        dotnet-version: '8.x'
    - name: Run unit tests
      run: dotnet test UnitTests.csproj --configuration Release
```

For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q38 T02

Correct Answer:
- Unit tests run automatically whenever changes are pushed to main. → Yes
- Schema validation occurs during the Build step. → Yes
- Schema validation occurs during the Deploy step. → No

Explanation:
El job unit-tests tiene la condición if: github.ref == 'refs/heads/main' y depende del job build-and-deploy. Cuando se hace push a la rama main, el workflow se dispara y los unit tests se ejecutan automáticamente. El proceso de Build (dotnet build) valida las relaciones entre objetos y la sintaxis del proyecto SQL frente a la plataforma de destino, generando el archivo .dacpac.
El paso Deploy (SqlPackage /Action:Publish) aplica el .dacpac a la base de datos de destino; no realiza la validación de esquema del proyecto (esa validación ya ocurrió en el Build).

English translation of the Explanation:
The unit-tests job has the condition if: github.ref == 'refs/heads/main' and depends on the build-and-deploy job. When a push is made to the main branch, the workflow is triggered and the unit tests run automatically. The Build process (dotnet build) validates the relationships between objects and the syntax of the SQL project against the target platform, generating the .dacpac file.
The Deploy step (SqlPackage /Action:Publish) applies the .dacpac to the target database; it does not perform the project schema validation (that validation already occurred in the Build).

Reference:

https://learn.microsoft.com/sql/tools/sql-database-projects/get-started?view=sql-server-ver17
https://learn.microsoft.com/sql/tools/sql-database-projects/tutorials/create-deploy-sql-project?view=sql-server-ver17

QUESTION 39
You have a Microsoft SQL Server 2025 instance that contains a database named SalesDB. SalesDB supports a Retrieval Augmented Generation (RAG) pattern for internal support tickets. The SQL Server instance runs without any outbound network connectivity.
You plan to generate embeddings inside the SQL Server instance and store them in a table for vector similarity queries.
You need to ensure that only a database user account named AIApplicationUser can run embedding generation by using the model.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Grant the CONTROL permission on SalesDB to AIApplicationUser.
B. Create a database audit specification on SalesDB owned by AIApplicationUser.
C. Grant the EXECUTE permission on the external model project to AIApplicationUser.
D. Create an external model project by using ONNX runtime and local paths.
E. Create an external model project that points to a Microsoft Foundry REST endpoint.

Answer: CD
Section: (none)

Explanation/Reference:
Q39 T02

Correct Answer:

C. Grant the EXECUTE permission on the external model project to AIApplicationUser.
D. Create an external model project by using ONNX runtime and local paths.

Explanation:

Para generar embeddings sin conectividad de red saliente, debe crearse un modelo externo que utilice ONNX Runtime y rutas locales. LOCATION indica la carpeta que contiene el modelo y LOCAL_RUNTIME_PATH la carpeta que contiene los archivos ejecutables de ONNX Runtime. De esta forma, SQL Server puede ejecutar el modelo localmente sin llamar a un endpoint REST externo.

Además, para que AIApplicationUser pueda utilizar el modelo externo mediante AI_GENERATE_EMBEDDINGS, debe concedérsele el permiso EXECUTE sobre ese modelo. El permiso CONTROL sobre toda la base de datos sería excesivo y no sigue el principio de privilegios mínimos.

Reference:

https://learn.microsoft.com/en-us/sql/t-sql/statements/create-external-model-transact-sql?view=sql-server-ver17#permissions

https://learn.microsoft.com/en-us/sql/t-sql/statements/create-external-model-transact-sql?view=sql-server-ver17#example-with-onnx-runtime-running-locally

QUESTION 40
You have an Azure SQL database that supports an OLTP application.

You need to write Transact-SQL code that returns blocking chain details. The output must return only sessions that are blocked or are blocking other sessions.

How should you complete the code? To answer, drag the appropriate values to the correct targets. Each

value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Values

- CROSS APPLY sys.dm_exec_sql_text (r.sql_handle)
- FROM sys.dm_exec_requests
- FROM sys.dm_tran_locks
- INNER JOIN sys.dm_tran_locks
- LEFT OUTER JOIN sys.dm_exec_requests
- OUTER APPLY sys.dm_exec_input_buffer(r.session_id, 0)
- OUTER APPLY sys.dm_exec_input_buffer(s.session_id, NULL)
- OUTER APPLY sys.dm_exec_sql_text (r.sql_handle)

Answer Area

```
WITH cteBL (session_id, blocking_these) AS
(
   SELECT
      s.session_id,
      blocking_these = x.blocking_these
   FROM sys.dm_exec_sessions AS s
   CROSS APPLY
   (
      SELECT
         ISNULL(CONVERT(varchar(6), er.session_id), '') + ', '
         [                                                    ]
      WHERE er.blocking_session_id = ISNULL(s.session_id, 0)
       AND er.blocking_session_id <> 0
      FOR XML PATH('')
   ) AS x(blocking_these)
)
SELECT
   s.session_id,
   blocked_by = r.blocking_session_id,
   bl.blocking_these,
   batch_text = t.text,
   input_buffer = ib.event_info
FROM sys.dm_exec_sessions AS s
   [                                                       ]
INNER JOIN cteBL AS bl ON s.session_id = bl.session_id
   [                                                       ]
   [                                                       ]
WHERE bl.blocking_these IS NOT NULL
   OR r.blocking_session_id > 0
ORDER BY LEN(bl.blocking_these) DESC, r.blocking_session_id DES
```

Answer:

Values

```
CROSS APPLY sys.dm_exec_sql_text (r.sql_handle)

FROM sys.dm_tran_locks

INNER JOIN sys.dm_tran_locks

OUTER APPLY sys.dm_exec_input_buffer(r.session_id, 0)
```

Answer Area

```
WITH cteBL (session_id, blocking_these) AS
(
   SELECT
      s.session_id,
      blocking_these = x.blocking_these
   FROM sys.dm_exec_sessions AS s
   CROSS APPLY
   (
      SELECT
         ISNULL(CONVERT(varchar(6), er.session_id), '') + ', '
         FROM sys.dm_exec_requests
      WHERE er.blocking_session_id = ISNULL(s.session_id, 0)
       AND er.blocking_session_id <> 0
      FOR XML PATH('')
   ) AS x(blocking_these)
)
SELECT
   s.session_id,
   blocked_by = r.blocking_session_id,
   bl.blocking_these,
   batch_text = t.text,
   input_buffer = ib.event_info
FROM sys.dm_exec_sessions AS s
   LEFT OUTER JOIN sys.dm_exec_requests
INNER JOIN cteBL AS bl ON s.session_id = bl.session_id
   OUTER APPLY sys.dm_exec_sql_text (r.sql_handle)
   OUTER APPLY sys.dm_exec_input_buffer(s.session_id, N
WHERE bl.blocking_these IS NOT NULL
   OR r.blocking_session_id > 0
ORDER BY LEN(bl.blocking_these) DESC, r.blocking_session_id DES
```

Section: (none)

Explanation/Reference:
Q40 T02

CORRECT ANSWER:

```
WITH cteBL (session_id, blocking_these) AS
(
   SELECT
      s.session_id,
      blocking_these = x.blocking_these
   FROM sys.dm_exec_sessions AS s
```

```
    CROSS APPLY
    (
        SELECT
            ISNULL(CONVERT(varchar(6), er.session_id), '') + ', '
        FROM sys.dm_exec_requests AS er
        WHERE er.blocking_session_id = ISNULL(s.session_id, 0)
          AND er.blocking_session_id <> 0
        FOR XML PATH('')
    ) AS x(blocking_these)
)
SELECT
    s.session_id,
    blocked_by = r.blocking_session_id,
    bl.blocking_these,
    batch_text = t.text,
    input_buffer = ib.event_info
FROM sys.dm_exec_sessions AS s
LEFT OUTER JOIN sys.dm_exec_requests AS r ON r.session_id = s.session_id
INNER JOIN cteBL AS bl ON s.session_id = bl.session_id
OUTER APPLY sys.dm_exec_sql_text (r.sql_handle) AS t
OUTER APPLY sys.dm_exec_input_buffer(s.session_id, NULL) AS ib
WHERE bl.blocking_these IS NOT NULL
   OR r.blocking_session_id > 0
ORDER BY LEN(bl.blocking_these) DESC, r.blocking_session_id DESC, r.session_id;
```

EXPLANATION:

This is Microsoft's blocking-chain script. Microsoft's Azure SQL Database article presents it as a query that identifies sessions blocking other sessions, including a list of the session_ids blocked per session_id. The exam version is nearly identical to the published one. The main difference is that the published version also returns extra columns with *. The WHERE and ORDER BY at the end are the same as in Microsoft's Azure Database Support blog, which filters on blocking_these is not null or r.blocking_session_id > 0 and orders by the length of the blocking list, then by blocker, then by session. That filter is what satisfies the requirement to return only sessions that are blocked or are blocking other sessions.

It works in two steps:

The CTE (cteBL) builds, for each session, a comma-separated list of the sessions it is blocking. It uses FOR XML PATH('') to concatenate the session_id values.
The main query joins that list to session and request information, and adds the SQL text (batch_text) and the last command sent (input_buffer).
Why each blank is correct

1. FROM sys.dm_exec_requests (inside the CROSS APPLY)
The subquery looks for every request whose blocking_session_id equals the current session's id, and that column belongs to sys.dm_exec_requests. It also uses the alias er, which the WHERE clause and the SELECT both reference (er.session_id, er.blocking_session_id).

2. LEFT OUTER JOIN sys.dm_exec_requests
sys.dm_exec_requests only contains sessions that currently have an active request. A session that blocks others is often idle, for example a session with an open transaction that is not running anything. With an INNER JOIN, those sessions would disappear, and they are usually the head of the blocking chain. The LEFT OUTER JOIN keeps every row from sys.dm_exec_sessions.

3. OUTER APPLY sys.dm_exec_sql_text (r.sql_handle)
When a session has no active request, r.sql_handle is NULL because of the left join above. OUTER APPLY keeps that row and returns NULL for batch_text. CROSS APPLY would remove it, which brings back the same problem as in blank 2.

4. OUTER APPLY sys.dm_exec_input_buffer(s.session_id, NULL)
According to the article, the event_info field of sys.dm_exec_input_buffer contains the last command string passed to the SQL engine. This is how you can see what an idle blocker last ran, even when it has no active

request. Two details matter here:

- It must use s.session_id, not r.session_id, because r.session_id is NULL when there is no request.
- The second parameter is the request id. Microsoft's documentation shows an example that cross-applies the function to sys.dm_exec_sessions and passes NULL as the request id. The other option, (r.session_id, 0), would fail for the reason above.

REFERENCES:
https://learn.microsoft.com/en-us/azure/azure-sql/database/understand-resolve-blocking?view=azuresql
https://learn.microsoft.com/en-us/troubleshoot/sql/database-engine/performance/understand-resolve-blocking
https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/sys-dm-exec-input-buffer-transact-sql?view=sql-server-ver16
https://techcommunity.microsoft.com/t5/azure-database-support-blog/can-certain-sql-user-roles-avoid-blocking/ba-p/3677550

QUESTION 41
You have an Azure SQL database that supports a customer-facing API. The API calls a stored procedure named dbo.GetCustomerOrders thousands of times per hour.

After a deployment that updated indexes and statistics, users report that the API endpoint backed by dbo. GetCustomerOrders is slower. In Query Store, the same query now has two persisted execution plans. During the last hour, the newer plan had a significantly higher average duration and CPU time than the older plan.

You need to restore the previous performance quickly, without changing the API code.

Which Transact-SQL command should you run?

A. EXEC sys.sp_query_store_set_hints
B. DBCC FREEPROCCACHE
C. EXEC sp_query_store_force_plan
D. ALTER DATABASE

Answer: C
Section: (none)

Explanation/Reference:
Q41 T02
1. ANÁLISIS

Concepto principal: regresión de plan de ejecución detectada en Query Store.

Servicio: Azure SQL Database + Query Store.

Escenario: después de cambios de índices/estadísticas aparecen dos planes; el nuevo es significativamente más lento. Se requiere forzar el plan anterior de forma inmediata y sin modificar el código de la API.
2. VERIFICACIÓN (MCP)

Documentación oficial de Microsoft Learn consultada sobre sp_query_store_force_plan y escenarios de regresión de planes en Query Store.
3. RESOLUCIÓN

C. EXEC sp_query_store_force_plan
4. JUSTIFICACIÓN

Correcta – C
Microsoft Learn documenta que sp_query_store_force_plan permite forzar un plan concreto ya capturado por Query Store para una consulta determinada. Al forzar el plan antiguo (el de menor duración y CPU), se

restaura el rendimiento anterior de forma inmediata y sin cambiar el código de la API ni el procedimiento almacenado.
Incorrectas
A. EXEC sys.sp_query_store_set_hints → se utiliza para aplicar hints (OPTION) a una consulta; no fuerza un plan de ejecución completo.
B. DBCC FREEPROCCACHE → solo limpia la caché de planes; no garantiza que se vuelva a elegir el plan antiguo y puede provocar recompilaciones adicionales.
D. ALTER DATABASE → se usa para cambiar opciones de base de datos (incluido el estado de Query Store), no para forzar un plan específico.

5. REFERENCIA

https://learn.microsoft.com/sql/relational-databases/system-stored-procedures/sp-query-store-force-plan-transact-sql

https://learn.microsoft.com/sql/relational-databases/performance/tune-performance-with-the-query-store

https://learn.microsoft.com/sql/relational-databases/performance/query-store-usage-scenarios

QUESTION 42
HOTSPOT -
You have an Azure SQL database that has Query Store enabled.
Query Performance Insight shows that one stored procedure has the longest runtime. The procedure runs the following parameterized query.

```
CREATE OR ALTER PROCEDURE dbo.GetRecentOrders
  @CustomerId int,
  @SinceDate  datetime2
AS
SELECT TOP (50)
    o.OrderId, o.OrderDate, o.Status, o.TotalAmount
FROM dbo.Orders AS o
WHERE o.CustomerId = @CustomerId
  AND o.OrderDate >= @SinceDate
ORDER BY o.OrderDate DESC;
```

The dbo.Orders table has approximately 120 million rows. CustomerId is highly selective, and OrderDate is used for range filtering and sorting.
You have the following indexes:
Clustered index: PK_Orders on (OrderId)
Nonclustered index: IX_Orders_OrderDate on (OrderDate) with no included columns
An actual execution plan captured from Query Store for slow runs shows the following:
An index seek on IX_Orders_OrderDate followed by a Key Lookup (Clustered) on PK_Orders for CustomerId, Status, and TotalAmount
A sort operator before Top (50), because the results are ordered by OrderDate DESC
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes
To avoid the explicit sort for the query, create a nonclustered index on (CustomerId, OrderDate DESC) that includes (Status, TotalAmount).	○
To eliminate the sort and make the query use an ordered seek, add CustomerId as an included column to IX_Orders_OrderDate.	○
The plan indicates a bottleneck from a suboptimal query plan, rather locking or blocking.	○

Answer:

Answer Area

Statements	Yes
To avoid the explicit sort for the query, create a nonclustered index on (CustomerId, OrderDate DESC) that includes (Status, TotalAmount).	○
To eliminate the sort and make the query use an ordered seek, add CustomerId as an included column to IX_Orders_OrderDate.	○
The plan indicates a bottleneck from a suboptimal query plan, rather locking or blocking.	○

Section: (none)

Explanation/Reference:
Q42 T02

Correct Anwser:
To avoid the explicit sort for the query, create a nonclustered index on (CustomerId, OrderDate DESC) that includes (Status, TotalAmount). Yes
To eliminate the sort and make the query use an ordered seek, add CustomerId as an included column to IX_Orders_OrderDate. No
The plan indicates a bottleneck from a suboptimal query plan, rather locking or blocking.Yes

Explanation:
Statement: To avoid the explicit sort for the query, create a nonclustered index on (CustomerId, OrderDate DESC) that includes (Status, TotalAmount). → Yes
Crear un índice no clusterizado compuesto:
SQLCREATE NONCLUSTERED INDEX IX_Orders_CustomerId_OrderDate
ON dbo.Orders (CustomerId, OrderDate DESC)
INCLUDE (Status, TotalAmount);
La clave (CustomerId, OrderDate DESC) permite un ordered index seek (igualdad en CustomerId + rango ordenado en OrderDate DESC).
Las columnas incluidas hacen que el índice sea de cobertura → se elimina el Key Lookup.
El orden físico del índice coincide con el ORDER BY OrderDate DESC, por lo que el optimizador puede eliminar el operador Sort.
Esto es exactamente lo que describe la guía oficial de diseño de índices de Microsoft.

Statement: To eliminate the sort and make the query use an ordered seek, add CustomerId as an included

column to IX_Orders_OrderDate. → No
Añadir solo CustomerId como columna incluida al índice existente IX_Orders_OrderDate (cuya clave es únicamente OrderDate) no cambia el orden de las claves.
El seek seguiría siendo por OrderDate.
El filtro de CustomerId se aplicaría como predicado residual o después del Key Lookup.
No se obtiene un ordered seek que satisfaga tanto el predicado de igualdad + rango como el ORDER BY.
El Sort sigue siendo necesario.

Statement: The plan indicates a bottleneck from a suboptimal query plan, rather locking or blocking → Yes
El plan capturado muestra claramente:
Index Seek → Key Lookup → Sort → Top.
Esto es un clásico síntoma de plan de ejecución subóptimo causado por la ausencia de un índice adecuado (falta de cobertura y orden incorrecto de las claves).
No hay ninguna mención en el enunciado de waits de bloqueo (LCK_*), deadlocks o contención de locks.
Query Store / Query Performance Insight están señalando un problema de diseño de índices, no de concurrency.

References: Fuentes oficiales exactas utilizadas (Microsoft Learn):

Index architecture and design guide
https://learn.microsoft.com/sql/relational-databases/sql-server-index-design-guide
(secciones “General index design guidelines” y “Use included columns in nonclustered indexes” – explica eliminación del Sort y covering indexes).
Create indexes with included columns
https://learn.microsoft.com/sql/relational-databases/indexes/create-indexes-with-included-columns
Documentación de operadores de plan de ejecución (Sort, Index Seek, Key Lookup) referenciada desde la misma guía de índices.

QUESTION 43
You have a database named DB1. The schema is stored in a Git repository as an SDK-style SQL database project.

You have a GitHub Actions workflow that already runs dotnet build and produces a database artifact.

You need to add a deployment step that publishes the .dacpac file to an Azure SQL database by using the secrets stored in GitHub repository secrets.

What should you include in the workflow?
```
env:
  SQL_CONNECTION_STRING: Server=tcp:myserver.database.windows.net;
...
steps:
```

A.
```
- name: Publish
  uses: azure/sql-action@v2
  with:
    action: publish
    path: bin/Debug/db1.dacpac
    connection-string: ${{ env.SQL_CONNECTION_STRING }}
```

B.
```
- name: Publish
  uses: azure/sql-action@v2
  with:
    action: extract
    path: bin/Debug/db1.dacpac
    connection-string: ${{ secrets.SQL_CONNECTION_STRING }}
```

C.
```
- name: Publish
  uses: azure/sql-action@v2
  with:
    action: publish
    path: bin/Debug/db1.dacpac
    connection-string: ${{ secrets.SQL_CONNECTION_STRING }}
```

D. - name: Publish
run: |
dotnet build db1.sqlproj \
/p:TargetConnectionString="${{ secrets.SQL_CONNECTION_STRING }}"

Answer: C
Section: (none)

Explanation/Reference:
Q017 #Topic 2

Correct Answer:
- name: Publish
uses: azure/sql-action@v2
with:
action: publish
path: bin/Debug/db1.dacpac
connection-string: ${{ secrets.SQL_CONNECTION_STRING }}
Technical Justification:

El workflow ya genera el artefacto .dacpac mediante dotnet build. Para publicarlo en Azure SQL Database usando un secreto del repositorio de GitHub se debe usar la acción oficial azure/sql-action@v2 con:

action: publish
path apuntando al .dacpac generado
connection-string: ${{ secrets.SQL_CONNECTION_STRING }}

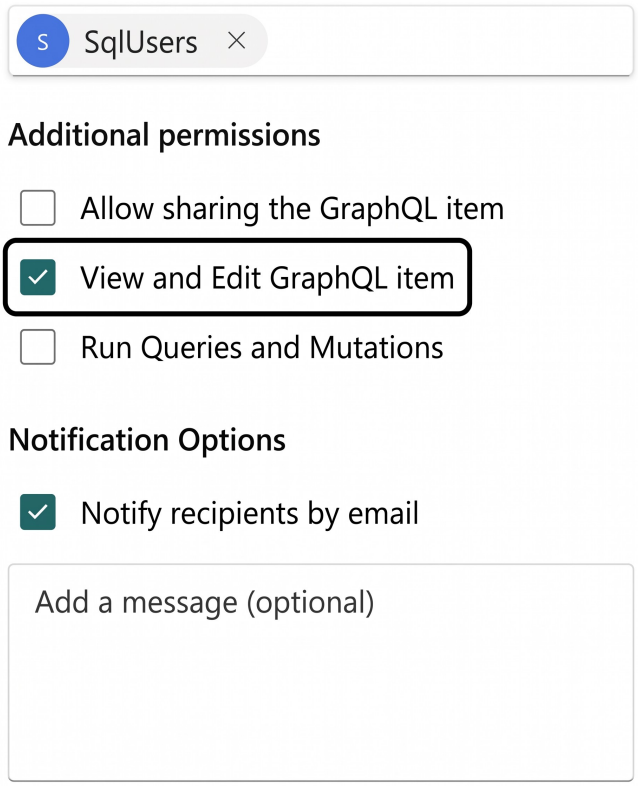
Esta es la sintaxis recomendada por Microsoft para desplegar un DACPAC desde GitHub Actions.
Why the other options are incorrect:

A: Usa una variable de entorno (env.SQL_CONNECTION_STRING) en lugar del secreto del repositorio (secrets).
B: Usa action: extract (incorrecto; debe ser publish).
D: Intenta usar dotnet build con /p:TargetConnectionString, que no es el método correcto ni soportado para publicar un .dacpac ya generado.

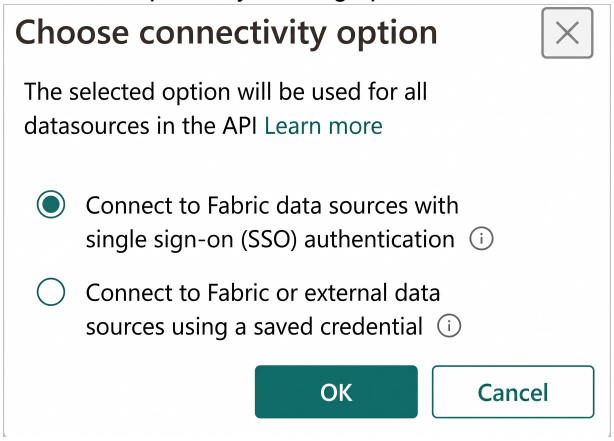
Official Microsoft references:

https://learn.microsoft.com/en-us/azure/azure-sql/database/connect-github-actions-sql-db
https://github.com/Azure/sql-action
https://learn.microsoft.com/en-us/sql/tools/sql-database-projects/sql-projects-automation

QUESTION 44
You have a Microsoft Fabric workspace named Workspace1 that contains a SQL database named SalesDB and an API for GraphQL item named SalesApi.
You have a Microsoft Entra group named SqlUsers.
From Workspace1, you assign permission to SalesApi as shown in the following exhibit:



From Workspace1, you assign permission to SalesApi as shown in the following exhibit:



SqlUsers has the Viewer role for Workspace1.
For each of the following statements, select Yes if the statement is true. Otherwise, select No.

NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
The members of SqlUsers can modify the data in SalesDB via SalesAPI.	○	○
The members of SqlUsers can view the data in SalesDB via SalesAPI.	○	○
The members of SqlUsers can change the field mappings of SalesAPI.	○	○

Answer:

Answer Area

Statements	Yes	No
The members of SqlUsers can modify the data in SalesDB via SalesAPI.	○	○
The members of SqlUsers can view the data in SalesDB via SalesAPI.	○	○
The members of SqlUsers can change the field mappings of SalesAPI.	○	○

Section: (none)

Explanation/Reference:
1. Respuesta Correcta:

The members of SqlUsers can modify the data in SalesDB via SalesAPI. → No
The members of SqlUsers can view the data in SalesDB via SalesAPI. → No
The members of SqlUsers can change the field mappings of SalesAPI. → Yes

2. Explicación Técnica Detallada:

Solo se otorgó el permiso View and Edit GraphQL item.
No se otorgó Run Queries and Mutations (permiso Execute).
La conectividad está configurada con SSO.
SqlUsers tiene solo el rol Viewer en el workspace (Read/ReadData en SalesDB, sin Write).

Por tanto:

No pueden ejecutar queries ni mutations a través de la API (falta Execute).
No pueden modificar datos (falta Execute + falta Write en SalesDB).
Sí pueden editar la definición del ítem GraphQL (incluyendo field mappings) gracias al permiso View and Edit GraphQL item.

3. Análisis de Opciones Incorrectas:
No aplica (pregunta de tipo Yes/No).

4. Referencias Oficiales de Microsoft & SQL Server:
Connect applications to Fabric API for GraphQL
https://learn.microsoft.com/en-us/fabric/data-engineering/connect-apps-api-graphql
Create and add data to an API for GraphQL
https://learn.microsoft.com/en-us/fabric/data-engineering/get-started-api-graphql
Microsoft Fabric API for GraphQL FAQ
https://learn.microsoft.com/en-us/fabric/data-engineering/graphql-faq
Permission model
https://learn.microsoft.com/en-us/fabric/security/permission-model

QUESTION 45
You have an Azure SQL database that supports the OLTP workload of an order-processing application. During a 10-minute incident window, you run a dynamic management view query and discover the following: Session 72 is sleeping with open_transaction_count = 1. Multiple other sessions show blocking_session_id = 72 in sys.dm_exec_requests. sys.dm_exec_input_buffer(72, NULL) returns only BEGIN TRANSACTION UPDATE Sales.Orders. Users report that updates to Sales.Orders intermittently time out during the incident window. The timeouts stop only after you manually terminate session 72. What is a possible cause of the blocking?

A. A long-running SELECT statement is blocking writers.
B. Session 72 caused a deadlock
C. An explicit transaction was started but not committed or rolled back.

D. A lock escalation occurred.

Answer: C
Section: (none)

Explanation/Reference:
#45 Q19-T02

Correct Answer:
An explicit transaction was started but not committed or rolled back.

Explanation:
La sesión 72 está en estado Sleeping pero con open_transaction_count = 1, y el input buffer solo muestra BEGIN TRANSACTION sin COMMIT/ROLLBACK. Esto indica una transacción explícita abierta y nunca cerrada, que retiene bloqueos sobre Sales.Orders hasta que se termina la sesión manualmente.

Why not the others:
A) una SELECT larga aparecería como Running, no Sleeping.
B) un deadlock lo resuelve el motor automáticamente.
D) una lock escalation no deja la sesión inactiva con una transacción sin cerrar.

Reference:
https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/sys-dm-exec-sessions-transact-sql

QUESTION 46
HOTSPOT - You have a SQL database in Microsoft Fabric that contains the following functions:

A multi-statement table-valued function (TVF) named Sales.mstvf_OrderStatus() that returns order status information

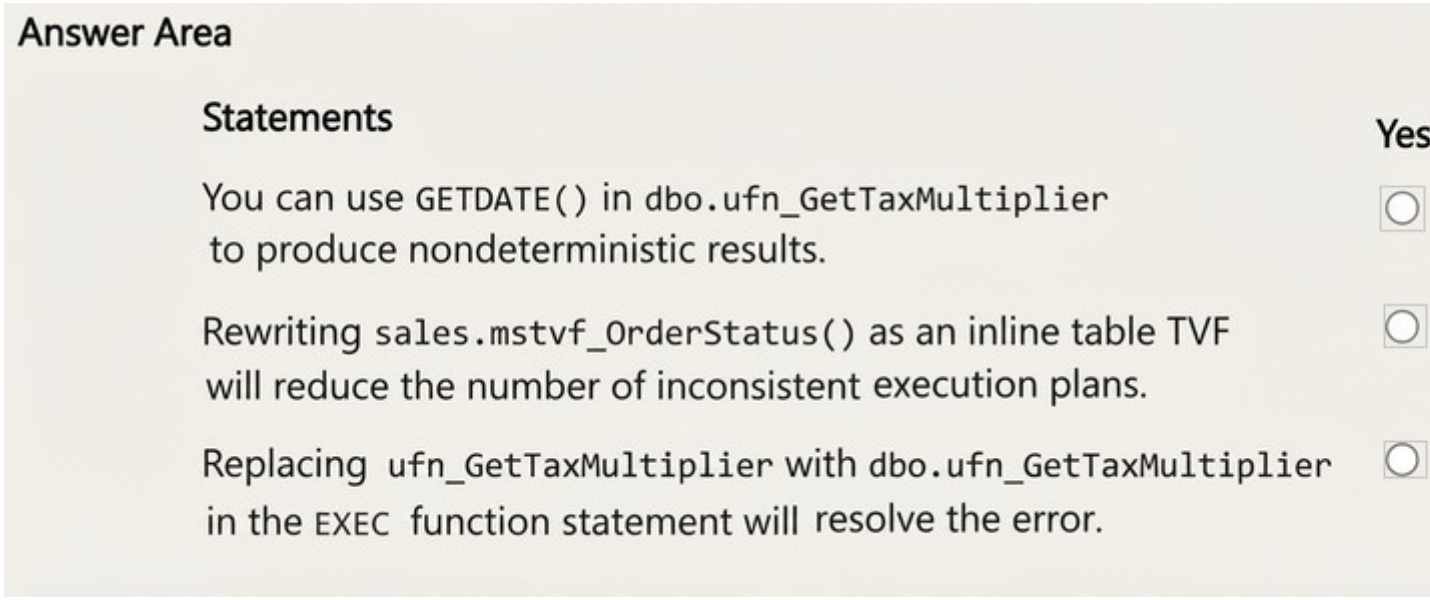
A scalar user-defined function (UDF) named dbo.ufn_GetTaxMultiplier (@TaxAmt money, @StateCode char(2)) that returns a numeric multiplier used in tax calculations

Reporting queries frequently join Sales.mstvf_OrderStatus() to a table named Sales.SalesOrderHeader and return large result sets. A performance review shows that the queries produce inconsistent execution plans.

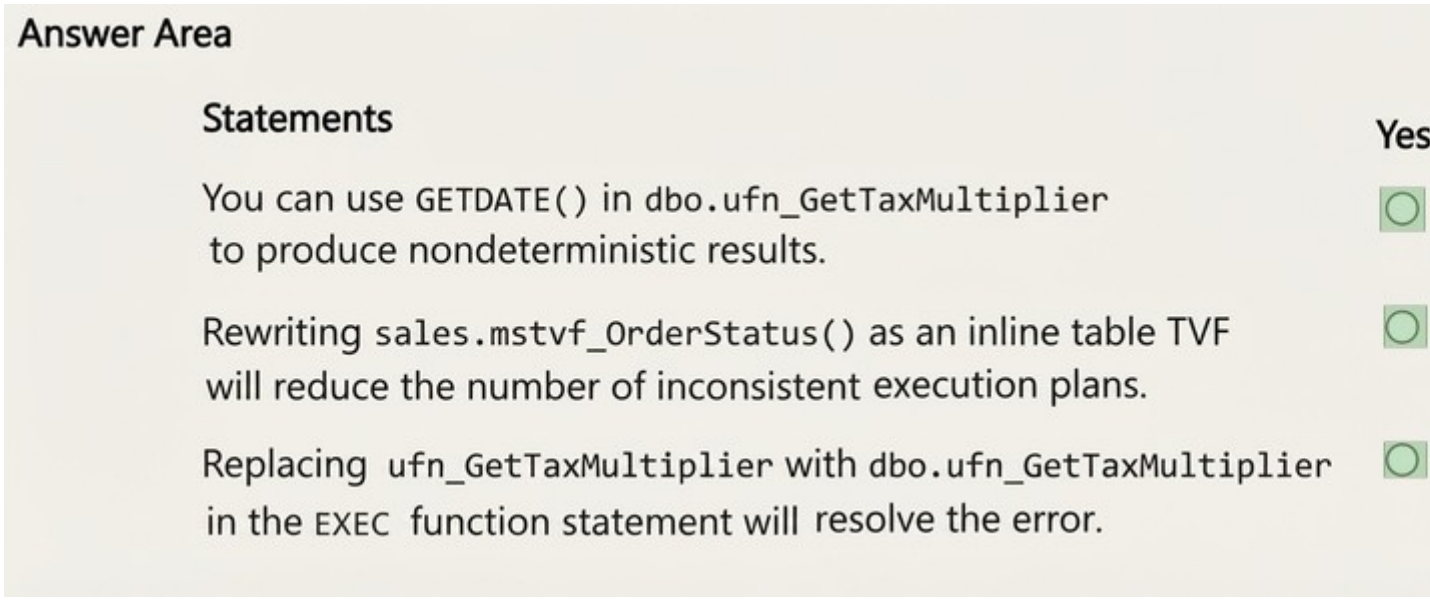
During a code review, a developer discovers that the following Transact-SQL statement produced an error.

EXEC @ret = ufn_GetTaxMultiplier @TaxAmt = 100.00, @StateCode = 'WA'; For each of the following statements, select Yes if the statement is true. Otherwise, select No.

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
VCE 46 / 020 Topic 02

Explanation

You can use GETDATE() in dbo.ufn_GetTaxMultiplier to produce nondeterministic results. → Yes

GETDATE() es una función del sistema no determinista y está explícitamente permitida dentro de funciones definidas por el usuario (UDFs) de Transact-SQL. Al usarla dentro de la scalar UDF, la función se vuelve no determinista (puede devolver resultados diferentes en cada ejecución aunque los parámetros de entrada y el estado de la base de datos sean los mismos).

Rewriting Sales.mstvf_OrderStatus() as an inline table TVF will reduce the number of inconsistent execution plans. → Yes

Las multi-statement table-valued functions (MSTVF) usan una estimación de cardinalidad fija (normalmente 100 filas a partir de SQL Server 2014). Esto provoca planes de ejecución subóptimos e inconsistentes

cuando se unen a tablas grandes. Al reescribirla como inline table-valued function, el optimizador de consultas puede expandirla como una vista, usar estadísticas reales de las tablas subyacentes y generar planes más consistentes y eficientes.

Replacing ufn_GetTaxMultiplier with dbo.ufn_GetTaxMultiplier in the EXEC function statement will resolve the error. → Yes

Las scalar user-defined functions deben invocarse con el nombre de dos partes (schema.function). El statement original EXEC @ret = ufn_GetTaxMultiplier ... falla porque falta el esquema. Cambiarlo a EXEC @ret = dbo.ufn_GetTaxMultiplier ... es la forma correcta (y documentada) de ejecutar una scalar UDF con la sintaxis EXEC.

Ninguna de las tres afirmaciones es falsa. Todas son verdaderas según la documentación oficial de SQL Server / Microsoft Fabric.

Reference

Requisito del nombre de dos partes y ejemplo de EXEC con scalar UDF.
- https://learn.microsoft.com/es-es/sql/relational-databases/user-defined-functions/execute-user-defined-functions?view=sql-server-ver17

GETDATE() es no determinista y está permitida en UDFs.
- https://learn.microsoft.com/es-es/sql/relational-databases/user-defined-functions/deterministic-and-nondeterministic-functions?view=sql-server-ver17

Lista de funciones no deterministas permitidas (incluye GETDATE).
- https://learn.microsoft.com/es-es/sql/relational-databases/user-defined-functions/user-defined-functions?view=sql-server-ver17

Diferencias de rendimiento y cardinalidad entre multi-statement TVF e inline TVF.
- https://learn.microsoft.com/es-es/sql/relational-databases/post-migration-validation-and-optimization-guide?view=sql-server-ver17

CREATE FUNCTION (Transact-SQL).
- https://learn.microsoft.com/es-es/sql/t-sql/statements/create-function-transact-sql?view=sql-server-ver17

QUESTION 47
You have an Azure SQL database named SalesDB on a logical server named sales-sql01.

You have an Azure App Service web app named OrderApi that connects to SalesDB by using SQL authentication.

You enable a user-assigned managed identity named OrderApi-Id for OrderApi.

You need to configure OrderApi to connect to SalesDB by using Microsoft Entra authentication. The managed identity must have read and write permissions to SalesDB.

Which Transact-SQL statements should you run in SalesDB?

A. CREATE LOGIN [OrderApi-Id] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id];
ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id];

B. CREATE USER [OrderApi-Id] WITH PASSWORD = 'P@ssw0rd!';
ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id];
ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id];

C. CREATE USER [OrderApi-Id] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id];
ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id];

D. CREATE LOGIN [OrderApi-Id] WITH PASSWORD = 'P@ssw0rd!';
ALTER SERVER ROLE sysadmin ADD MEMBER [OrderApi-Id];

Answer: C
Section: (none)

Explanation/Reference:
VCE 21 / 47 Topic 2

Correct Answer:
CREATE USER [OrderApi-Id] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id];
ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id];

Explanation:
Azure SQL Database (a single/pooled database) does not have a traditional master database that the app server can log into the way SQL Managed Instance or SQL Server does, so managed identities and other Microsoft Entra principals are added as contained database users directly inside the target database using CREATE USER ... FROM EXTERNAL PROVIDER. This creates a database principal tied to the Microsoft Entra identity (the managed identity's display name), without requiring a server-level login. Adding that user to the db_datareader and db_datawriter fixed database roles grants exactly the read and write permissions required.

CREATE LOGIN [OrderApi-Id] FROM EXTERNAL PROVIDER (with role assignments) is incorrect because CREATE LOGIN creates a server-level principal in the logical server's master database. While this is valid syntax for Managed Instance, in single Azure SQL Database the recommended and required approach for app/managed identity access is a contained database user created directly in the user database, not a server-level login.

CREATE USER ... WITH PASSWORD = '...' is incorrect because specifying a password creates a SQL authentication user, not a Microsoft Entra/managed identity-based user. Managed identities authenticate via Microsoft Entra tokens, not passwords.

CREATE LOGIN ... WITH PASSWORD plus ALTER SERVER ROLE sysadmin is incorrect on two counts: it uses SQL authentication (password) instead of Microsoft Entra authentication, and it grants full server administrative rights (sysadmin), which violates the principle of least privilege and is not even a valid server role in Azure SQL Database.

References:
https://learn.microsoft.com/azure/azure-sql/database/authentication-aad-configure?view=azuresql#create-microsoft-entra-principals-in-sql
https://learn.microsoft.com/sql/t-sql/statements/create-user-transact-sql?view=sql-server-ver17#remarks
https://learn.microsoft.com/azure/azure-sql/database/authentication-aad-overview?view=azuresql#microsoft-entra-principals

QUESTION 48
You have an Azure SQL database that contains a column named Notes.
A security review discovers that Notes contains sensitive data.
You need to ensure that the data is protected so that neither the stored values nor the query inputs reveal information about the actual data. The solution must prevent a user from inferring relationships or repetitions in the data based on the encrypted output.
Which should you use?

A. Always Encrypted with secure enclaves
B. Always Encrypted with randomized encryption
C. row-level security (RLS)
D. Always Encrypted with deterministic encryption

Answer: B
Section: (none)

Explanation/Reference:

#VCE 048 / 022 Topic 2
Correct answer: Always Encrypted with randomized encryption
Explanation:

Always Encrypted with randomized encryption is the right choice because it generates a different ciphertext every time the same plaintext value is encrypted. This fully hides any patterns, repetitions or relationships in the data, so an observer looking at the stored encrypted values (or at query parameters that also get encrypted client-side) cannot infer anything about the original content.
Always Encrypted with deterministic encryption always produces the same ciphertext for the same plaintext. That would allow an attacker to detect equal values and therefore infer relationships or repetitions, which the requirement explicitly forbids.
Always Encrypted with secure enclaves is an enhancement that lets the database engine perform richer operations (range comparisons, pattern matching, etc.) inside a trusted enclave on randomized columns; it is not required here because the question only demands protection against inference from the encrypted output, not advanced querying.
Row-level security (RLS) only filters which rows a user can see; it does not encrypt column values and therefore does not protect the sensitive data stored in Notes.
Reference:

https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-database-engine

https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-cryptography

https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-enclaves

QUESTION 49
You have an Azure SQL database that stores order data.

A reporting query aggregates monthly revenue per customer and runs frequently.

You need to reduce how long it takes to retrieve the calculated values. The solution must NOT alter any underlying table structure.

What should you do?

A. Create a view by using ORDER BY without TOP, and then create a unique clustered index on the view.
B. Create a view without using WITH SCHEMABINDING, and then create a nonclustered index on the view.
C. Create a view by using GROUP BY, and then create a unique clustered index on the view.
D. Create a view by using WITH SCHEMABINDING, include COUNT_BIG(*), and then create a unique clustered index on the view.

Answer: D
Section: (none)

Explanation/Reference:
#VCE 49 / 023 Topic 2

La respuesta correcta es la D: La solución adecuada es crear una vista indexada (indexed view) que materialice los resultados agregados de la consulta de reporting (ingresos mensuales por cliente). Así se evitan los cálculos pesados cada vez que se ejecuta la consulta, sin modificar la estructura de las tablas base.

Por qué D es correcta: Los requisitos oficiales de Microsoft para crear una vista indexada en Azure SQL Database / SQL Server son:

1. La vista debe crearse con WITH SCHEMABINDING.
2. Si la vista usa GROUP BY (necesario para agregaciones como SUM de ingresos mensuales), debe

incluir COUNT_BIG(*).
3. El primer índice que se crea sobre la vista debe ser un unique clustered index.

Esto materializa físicamente los resultados agregados, de forma que las consultas posteriores pueden leer directamente los datos precalculados (mucho más rápido).

Por qué las demás opciones son incorrectas:

A.“Create a view by using ORDER BY without TOP, and then create a unique clustered index on the view.”: Las vistas no pueden contener ORDER BY (salvo con TOP o OFFSET/FETCH en algunos casos muy limitados). Además, aunque se pudiera, faltan los requisitos obligatorios de SCHEMABINDING y COUNT_BIG(*).

B. “Create a view without using WITH SCHEMABINDING, and then create a nonclustered index on the view.”: Sin SCHEMABINDING no se puede crear ningún índice sobre una vista. El primer índice debe ser clustered (unique), no nonclustered.

C. “Create a view by using GROUP BY, and then create a unique clustered index on the view.”: Falta el WITH SCHEMABINDING (obligatorio). Falta el COUNT_BIG(*) (obligatorio cuando hay GROUP BY). Sin estos dos elementos, SQL Server rechaza la creación del índice.

Referencias
https://learn.microsoft.com/en-us/sql/relational-databases/views/create-indexed-views
https://learn.microsoft.com/en-us/sql/t-sql/statements/create-index-transact-sql (sección sobre vistas indexadas)
Documentación relacionada de COUNT_BIG: https://learn.microsoft.com/en-us/sql/t-sql/functions/count-big-transact-sql

QUESTION 50
You have a GitHub Actions workflow that builds and deploys an Azure SQL database. The schema is stored in a GitHub repository as an SDK-style SQL database project.
Following a code review, you discover that you need to generate a report that shows whether the production schema has diverged from the model in source control.
Which action should you add to the pipeline?

A. SqlPackage.exe /Action:DriftReport
B. SqlPackage.exe /Action:DeployReport
C. SqlPackage.exe /Action:Extract
D. SqlPackage.exe /Action:Script

Answer: A
Section: (none)

Explanation/Reference:
#VCE 50 / 024 Topic 2

Respuesta correcta:
SqlPackage.exe /Action:DriftReport

Explicación:
A: SqlPackage.exe /Action:DriftReport (correcta)

Por qué es la correcta:

DriftReport crea un informe XML con los cambios aplicados a una base de datos registrada desde la última vez que se registró. Eso es justo "producción ha divergido de lo que se desplegó desde el modelo".
microsoft
El enunciado pide un informe sobre divergencia (drift) del esquema de producción. Ninguna otra acción produce un informe de ese tipo.

Se puede ejecutar contra Azure SQL con /TargetConnectionString y /OutputPath para guardar el XML, y luego publicarlo como artefacto del workflow.
Requisito: la base de datos debe estar registrada como DAC (por ejemplo, desplegada con /p: RegisterDataTierApplication=True).

B: /Action:DeployReport (incorrecta)
DeployReport crea un informe XML de los cambios que realizaría una acción de publicación.
microsoft
Es una vista previa hacia adelante ("qué pasaría si despliego el .dacpac"), no una comprobación de si producción cambió por fuera del pipeline.
Es la trampa principal, porque también genera un informe XML. Pero compara un .dacpac con un destino para planificar un despliegue, no detecta cambios no controlados.

C: /Action:Extract (incorrecta)
Extract crea un archivo .dacpac con el esquema (o esquema y datos de usuario) de una base de datos SQL conectada.
microsoft
Produce un artefacto (snapshot), no un informe. Ese .dacpac serviría como material para comparar, pero por sí solo no dice si hay divergencia.

D: /Action:Script (incorrecta)
Script crea un script incremental de Transact-SQL que actualiza el esquema del destino para que coincida con el origen.
microsoft
Produce un script T-SQL de despliegue, no un informe. Sirve para revisar o aplicar cambios, no para reportar deriva.

Referencias:
Página específica de DeployReport y DriftReport (la más importante para esta pregunta: parámetros, propiedades y ejemplos):
https://learn.microsoft.com/en-us/sql/tools/sqlpackage/sqlpackage-deploy-drift-report
Visión general de SqlPackage (lista y descripción de todas las acciones: Extract, Publish, DeployReport, DriftReport, Script...):
https://learn.microsoft.com/en-us/sql/tools/sqlpackage/sqlpackage
Despliegue en Azure SQL Database con Azure Pipelines (muestra la sintaxis de Extract, DriftReport y Script con SqlPackage.exe /Action:..., que es el formato exacto de las opciones del examen):
https://learn.microsoft.com/en-us/azure/devops/pipelines/targets/azure-sqldb

QUESTION 51
Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.

After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.

You have a SQL database in Microsoft Fabric that contains a table named dbo.Orders. dbo.Orders has a clustered index, contains three years of data, and is partitioned by a column named OrderDate by month.

You need to remove all the rows for the oldest month. The solution must minimize the impact on other queries that access the data in dbo.Orders.

Solution: Identify the partition number for the oldest month, and then run the following Transact-SQL statement.

```
TRUNCATE TABLE dbo.Orders -
WITH (PARTITIONS (partition number));
```

Does this meet the goal?

A. Yes
B. No

Answer: A
Section: (none)

Explanation/Reference:
#VCE 51 / 025 Topic 2

Answer: A. Yes

Explanation:
TRUNCATE TABLE ... WITH (PARTITIONS (n)) elimina todas las filas de la partición indicada y nada más. Cumple el objetivo por estas razones:

Está soportado en esta plataforma. La sintaxis de SQL database in Microsoft Fabric (y de SQL Server 2016 + y Azure SQL Database) incluye WITH (PARTITIONS (...)). Solo el Warehouse de Fabric y Synapse tienen una sintaxis reducida sin esa cláusula. Esta pregunta usa una base de datos SQL en Fabric, así que la cláusula está disponible.
Elimina solo el mes más antiguo. dbo.Orders está particionada por mes sobre OrderDate, así que cada partición contiene exactamente un mes. Al truncar el número de partición del mes más antiguo, solo se borran esas filas y los demás meses no se tocan.
Tiene un impacto mínimo en otras consultas. TRUNCATE trabaja sobre metadatos: desasigna páginas sin registrar cada fila borrada en el log. Usa menos bloqueos, menos log de transacciones y menos recursos que un DELETE ... WHERE OrderDate < .... Ese DELETE fila a fila generaría mucho log, posible escalado de bloqueos, bloqueos entre consultas y fragmentación de índices.
Se cumple el requisito de alineación. Para truncar una tabla particionada, la tabla y sus índices deben estar alineados (particionados con la misma función de partición). El enunciado dice que la tabla tiene un índice clúster y está particionada por OrderDate, por lo que el índice clúster está alineado.
Es fácil obtener el número de partición. Puedes usar $PARTITION.<función_de_partición>(OrderDate) o consultar sys.partitions (columna partition_number).

References:
TRUNCATE TABLE (Transact-SQL), con la sintaxis para SQL Server, Azure SQL Database y Fabric SQL database, y el argumento WITH PARTITIONS: https://learn.microsoft.com/sql/t-sql/statements/truncate-table-transact-sql?view=sql-server-ver17
Ejemplo B, "Truncate table partitions": https://learn.microsoft.com/sql/t-sql/statements/truncate-table-transact-sql?view=sql-server-ver17#examples
$PARTITION (Transact-SQL), para obtener el número de partición de un valor: https://learn.microsoft.com/sql/t-sql/functions/partition-transact-sql?view=sql-server-ver17
sys.partitions (Transact-SQL), para la columna partition_number: https://learn.microsoft.com/sql/relational-databases/system-catalog-views/sys-partitions-transact-sql?view=sql-server-ver17
Manage retention of historical data (patrones de switch y merge de particiones): https://learn.microsoft.com/sql/relational-databases/tables/temporal/manage-retention?view=sql-server-ver17#use-table-partitioning

QUESTION 52
Question #26 Topic 2

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.

You have a SQL database in Microsoft Fabric that contains a table named dbo.Orders. dbo.Orders has a clustered index, contains three years of data, and is partitioned by a column named OrderDate by month.

You need to remove all the rows for the oldest month. The solution must minimize the impact on other queries that access the data in dbo.Orders.

Solution: Run the following Transact-SQL statement.

```
DELETE FROM dbo.Orders
WHERE OrderDate < DATEADD(month, -36, SYSUTCDATETIME());
```

Does this meet the goal?

A. Yes
B. No

Answer: B
Section: (none)

Explanation/Reference:
#VCE 52 / 026 Topic 2

1. Respuesta Oficial [DP-800]

La respuesta correcta es la B: **No**.

Un DELETE con filtro por fecha elimina las filas una a una y con logging completo, lo que genera bloqueos y crecimiento del log que afectan a las demás consultas sobre dbo.Orders. No aprovecha el particionamiento por mes para hacer una operación de metadatos.

2. Justificación Técnica y Arquitectura

DELETE FROM dbo.Orders
WHERE OrderDate < DATEADD(month, -36, SYSUTCDATETIME());

Operación fila por fila y fully logged. Cada fila borrada genera registros en el transaction log, y también se mantienen los índices (el clustered index y cualquier nonclustered). Con un mes de datos de una tabla de tres años, el volumen es considerable.
Locking y escalation. El DELETE toma locks de fila o página y puede escalar a un lock de tabla (lock escalation), bloqueando lecturas y escrituras concurrentes. Esto contradice directamente el requisito de "minimizar el impacto en otras consultas".
No usa el particionamiento como palanca. Aunque el predicado sobre OrderDate permite partition elimination para localizar las filas, el borrado sigue siendo por filas, no por partición.
Riesgo adicional en el predicado. OrderDate < DATEADD(month, -36, ...) no acota "el mes más antiguo": borra todo lo anterior al límite, y con SYSUTCDATETIME() el límite cambia a diario, por lo que el resultado no es determinista respecto a los límites de partición.

Las soluciones que sí cumplen el objetivo operan a nivel de partición (operaciones de metadatos, mínimo logging y locks):

```
-- Opción 1: TRUNCATE de una partición concreta
TRUNCATE TABLE dbo.Orders WITH (PARTITIONS (1));

-- Opción 2: SWITCH a una tabla de staging y luego eliminarla
ALTER TABLE dbo.Orders SWITCH PARTITION 1 TO dbo.Orders_Staging;
DROP TABLE dbo.Orders_Staging;
```

SWITCH requiere que la tabla de staging tenga la misma estructura, índices y filegroup, y un CHECK constraint compatible con el rango de la partición.

3. Desglose de Opciones
A. Yes (incorrecta): Funcionalmente eliminaría datos antiguos, pero no cumple la restricción clave de minimizar el impacto sobre otras consultas, por el logging masivo, los locks y el riesgo de lock escalation.
B. No (correcta): El DELETE masivo no es la técnica adecuada para retirar una partición completa. Lo apropiado es TRUNCATE TABLE ... WITH (PARTITIONS (n)) o ALTER TABLE ... SWITCH PARTITION.

4. Referencias Oficiales de Microsoft Learn
TRUNCATE TABLE (Transact-SQL), incluye la cláusula WITH (PARTITIONS ...): https://learn.microsoft.com/sql/t-sql/statements/truncate-table-transact-sql
ALTER TABLE (Transact-SQL), incluye SWITCH PARTITION: https://learn.microsoft.com/sql/t-sql/statements/alter-table-transact-sql
Partitioned tables and indexes: https://learn.microsoft.com/sql/relational-databases/partitions/partitioned-tables-and-indexes
Resolve blocking problems / lock escalation: https://learn.microsoft.com/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide

Ojo: en esta serie de preguntas suele aparecer la variante con TRUNCATE TABLE ... WITH (PARTITIONS ...) o con SWITCH PARTITION como solución correcta, así que si la ves con "Yes" ahí cuadra con lo anterior.

QUESTION 53
Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have a SQL database in Microsoft Fabric that contains a table named dbo.Orders. dbo.Orders has a clustered index, contains three years of data, and is partitioned by a column named OrderDate by month.
You need to remove all the rows for the oldest month. The solution must minimize the impact on other queries that access the data in dbo.Orders.
Solution: Identify the partition scheme for the oldest month, and then run the following Transact-SQL statement.

```
ALTER TABLE dbo.Orders -
DROP PARTITION SCHEME (partition_scheme_name);
```

Does this meet the goal?

A. Yes
B. No

Answer: B
Section: (none)

Explanation/Reference:
#VCE 53 / 027 Topic 2

Correct Answer:

B. No

Explanation:

ALTER TABLE ... DROP PARTITION SCHEME no es T-SQL válido. DROP PARTITION SCHEME es una sentencia independiente.
Un esquema de partición solo asigna particiones a filegroups y no contiene filas. Además, no se puede eliminar mientras la tabla lo use, así que daría error.
El esquema cubre toda la tabla, no solo el mes más antiguo, por lo que no cumple el objetivo ni minimiza el impacto.
Lo correcto sería TRUNCATE TABLE ... WITH (PARTITIONS (n)) o SWITCH PARTITION seguido de MERGE RANGE.

References:

Referencias (Microsoft Learn)

DROP PARTITION SCHEME (Transact-SQL): https://learn.microsoft.com/sql/t-sql/statements/drop-partition-scheme-transact-sql?view=sql-server-ver17

TRUNCATE TABLE (Transact-SQL), argumento WITH (PARTITIONS ...): https://learn.microsoft.com/sql/t-sql/statements/truncate-table-transact-sql?view=sql-server-ver17

Partitioned tables and indexes: https://learn.microsoft.com/sql/relational-databases/partitions/partitioned-tables-and-indexes?view=sql-server-ver17

Manage retention of historical data in system-versioned temporal tables (patrón de ventana deslizante con SWITCH PARTITION y MERGE RANGE): https://learn.microsoft.com/sql/relational-databases/tables/temporal/manage-retention?view=sql-server-ver17

QUESTION 54
Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.

After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have an SDK-style SQL database project stored in a Git repository. The project targets an Azure SQL database.
The CI build fails with unresolved reference errors when the project references system objects.
You need to update the SQL database project to ensure that dotnet build validates successfully by including the correct system objects in the database model for Azure SQL Database.
Solution: Add the Microsoft.SqlServer.Dacpacs.Azure.Master NuGet package to the project.

Does this meet the goal?

A. Yes
B. No

Answer: A
Section: (none)

Explanation/Reference:
#VCE 534 / 028 Topic 2

Correct answer:

a. Yes

Explanation:

Sí, la solución cumple el objetivo.
El punto fundamental del ejercicio es que los SQL Database Projects validan las referencias a objetos durante la compilación. De forma predeterminada, los objetos del sistema no están incluidos en el modelo del proyecto, por lo que una referencia a un objeto del sistema puede provocar errores de referencias no resueltas durante dotnet build. Microsoft indica que estos errores se solucionan incorporando una referencia al master.dacpac correspondiente a la plataforma de destino.

En este caso, el proyecto cumple dos condiciones importantes:
1. Es un proyecto SQL SDK-style.
Los proyectos basados en Microsoft.Build.Sql permiten utilizar paquetes NuGet como referencias de base de datos. Microsoft señala además que las referencias mediante paquetes son el método recomendado para nuevos desarrollos y que esta funcionalidad de NuGet está disponible para proyectos SQL SDK-style.
2. El destino es Azure SQL Database.
Por tanto, no debemos utilizar el master.dacpac genérico de SQL Server. Microsoft proporciona específicamente el paquete:
Microsoft.SqlServer.Dacpacs.Azure.Master

Este paquete contiene el modelo de objetos del sistema de master correspondiente a Azure SQL Database.
Microsoft proporciona incluso el siguiente comando como ejemplo para añadirlo:
dotnet add <path-to-sqlproj> package Microsoft.SqlServer.Dacpacs.Azure.Master

Esto genera una referencia equivalente a:

```
<ItemGroup>
  <PackageReference
    Include="Microsoft.SqlServer.Dacpacs.Azure.Master"
    Version="..." />
</ItemGroup>
```

De esta manera, durante dotnet build, el modelo del proyecto dispone de las definiciones de los objetos del sistema necesarias para validar las referencias.
Por qué "No" es incorrecto: rechazar la solución implicaría que Microsoft.SqlServer.Dacpacs.Azure.Master no es la referencia apropiada para incorporar los objetos del sistema de Azure SQL Database. Sin embargo, la documentación oficial de Microsoft identifica explícitamente ese paquete para este escenario.
Hay además una distinción importante para no confundir este ejercicio con Microsoft Fabric:
Azure SQL Database
↓
Microsoft.SqlServer.Dacpacs.Azure.Master

SQL Database in Microsoft Fabric
↓
Microsoft.SqlServer.Dacpacs.DbFabric

Microsoft documenta Microsoft.SqlServer.Dacpacs.DbFabric para los objetos del sistema de SQL Database in Fabric. Como el enunciado especifica Azure SQL Database, Microsoft.SqlServer.Dacpacs.Azure.Master es el paquete correspondiente.

References:

Microsoft Learn — SQL Projects System Objects
Es la referencia principal para este ejercicio. Explica exactamente el problema descrito: los objetos del sistema no se incluyen por defecto en el modelo, esto puede provocar errores de validación y, para Azure SQL Database, se puede añadir Microsoft.SqlServer.Dacpacs.Azure.Master.
https://learn.microsoft.com/en-us/sql/tools/sql-database-projects/concepts/system-objects?view=sql-server-ver17

Microsoft Learn — SQL Projects Package References
Confirma que los paquetes de bases de datos del sistema contienen sus esquemas y pueden utilizarse como referencias en proyectos SQL. También establece que las referencias NuGet están soportadas en proyectos SQL SDK-style.
https://learn.microsoft.com/en-us/sql/tools/sql-database-projects/concepts/package-references?view=sql-server-ver17

Microsoft Learn — What are SQL Database Projects?
Documenta que Microsoft.Build.Sql es el SDK utilizado por los proyectos SQL SDK-style y que el proceso de compilación valida las referencias entre objetos y genera un .dacpac.
https://learn.microsoft.com/en-us/sql/tools/sql-database-projects/sql-database-projects?view=sql-server-ver17

QUESTION 55
Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have an SDK-style SQL database project stored in a Git repository. The project targets an Azure SQL database.
The CI build fails with unresolved reference errors when the project references system objects.
You need to update the SQL database project to ensure that dotnet build validates successfully by including

the correct system objects in the database model for Azure SQL Database.
Solution: Add the Microsoft.SqlServer.Dacpacs.Master NuGet package to the project.
Does this meet the goal?

A. Yes
B. No

Answer: B
Section: (none)

Explanation/Reference:
#VCE 55 / 029 Topic 2

2. Correct Answer:
b.No

3. Explanation:
La respuesta correcta es No.
El proyecto SQL es de estilo SDK y tiene como plataforma de destino Azure SQL Database. Microsoft Learn indica que, durante la compilación, los proyectos SQL validan las referencias a objetos de base de datos. De forma predeterminada, los objetos del sistema no están incluidos en el modelo, lo que puede provocar errores de referencias no resueltas cuando el proyecto utiliza dichos objetos. Microsoft Learn
Para solucionar estos errores, se debe incluir una referencia al master.dacpac correspondiente específicamente a la plataforma de destino del proyecto. En el caso de Azure SQL Database, Microsoft indica que debe utilizarse el paquete NuGet Microsoft.SqlServer.Dacpacs.Azure.Master. Microsoft Learn
La solución propuesta utiliza Microsoft.SqlServer.Dacpacs.Master, pero Microsoft Learn identifica este paquete como el correspondiente a la base de datos master de SQL Server. Por ejemplo, la documentación lo muestra como el paquete master de SQL Server 2022. Microsoft Learn
Por tanto, Yes es incorrecta porque el paquete propuesto corresponde a SQL Server y no al master de Azure SQL Database. No es la respuesta correcta porque, para este escenario, debería agregarse Microsoft.SqlServer.Dacpacs.Azure.Master.
4. References:
Microsoft Learn — SQL Projects System Objects:
https://learn.microsoft.com/en-us/sql/tools/sql-database-projects/concepts/system-objects?view=sql-server-ver17
Microsoft Learn — SQL Projects Package References:
https://learn.microsoft.com/en-us/sql/tools/sql-database-projects/concepts/package-references?view=sql-server-ver17

QUESTION 56
You have an SDK-style SQL database project stored in a Git repository. The project targets an Azure SQL database.
The CI build fails with unresolved reference errors when the project references system objects.
You need to update the SQL database project to ensure that dotnet build validates successfully by including the correct system objects in the database model for Azure SQL Database.
Solution: Add an artifact reference to the Azure SQL Database master.dacpac file.
Does this meet the goal?

A. Yes
B. No

Answer: B
Section: (none)

Explanation/Reference:
#VCE 56 / 030 Topic 2
Respuesta Correcta:
No
Explicación:
La solución propuesta (añadir una referencia de artefacto al archivo master.dacpac de Azure SQL

Database) no cumple el objetivo de forma fiable en un proyecto SQL de estilo SDK.
En proyectos SQL de estilo SDK (Microsoft.Build.Sql), la forma recomendada y portable de incluir los objetos del sistema de Azure SQL Database es mediante una referencia de paquete NuGet:
dotnet add package Microsoft.SqlServer.Dacpacs.Azure.Master
Una ArtifactReference depende de una ruta local específica de Visual Studio (por ejemplo, $(DacPacRootPath)\Extensions\Microsoft\SQLDB\Extensions\SqlServer\AzureV12\SqlSchemas\master.dacpac). Esa ruta no está disponible de forma consistente en entornos de CI/CD (como GitHub Actions o Azure DevOps) donde solo se ejecuta dotnet build, por lo que el build seguiría fallando con errores de referencias no resueltas a objetos del sistema.
Por tanto, esta solución no garantiza que dotnet build valide correctamente el modelo de base de datos para Azure SQL Database.
Referencias:
https://learn.microsoft.com/en-us/sql/tools/sql-database-projects/concepts/system-objects

https://learn.microsoft.com/en-us/sql/tools/sql-database-projects/concepts/package-references

QUESTION 57
Case Study

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete al questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study

-

To display the first question in this case study, dick the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, dick the Question button to return to the question.

Existing Environment

-

Azure Environment

Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the flowing tables:
• Patients
• Employees
• Procedures
• Transactions
• UsefulPrompts
• ProcedureDocuments

You store a column master key as a secret in Azure Key Vault.

You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.

Problem Statements

Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.

Requirements

Planned Changes

Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.

Security Requirements

Fabrikam identifies the folowing security requirements:

• The TransactionProcessing application must use a passwordless connection to DB1. • The Employees table contains two columns named TaxID and Salary that must be encrypted at rest. • Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.

Database Performance Requirements Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs. AI Search, Embeddings, and Vector Indexing Fabrikam identifies the folowing AI-related requirements:

• Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
• Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
• The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.

Development Requirements

Fabrikam identifies the folowing development requirements:

• Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
• Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
o Read data from the procedures table without authentication.
o Read and insert data into the Transactions table once authenticated.
o Execute the sp_UpdateProcedurePatient stored procedure.
• Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
• Information for each medical procedure wil be stored in a table. The table will be used with a large language model (LLM) for user querying and wil have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities:

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
    }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You are evaluating the DAB configuration.
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
Applications can read data in the `Procedures` table without authentication.	○	○
Applications can read and update data in the `Transactions` table once authenticated.	○	○
Applications can execute the `sp_UpdateProcedurePatient` stored procedure.	○	○

Answer:

Answer Area

Statements	Yes	No
Applications can read data in the `Procedures` table without authentication.	○	○
Applications can read and update data in the `Transactions` table once authenticated.	○	○
Applications can execute the `sp_UpdateProcedurePatient` stored procedure.	○	○

Section: (none)

Explanation/Reference:
#VCE 057 / 031 Topic 02
Correct Answer:

Yes
No
Yes

Explanation:

1. Applications can read data in the Procedures table without authentication. → Yes (Verdadera)

En la configuración DAB, la entidad Procedures tiene el permiso:

"role": "anonymous", "actions": ["read"].

El rol anonymous permite acceso sin autenticación. Por eso las aplicaciones pueden leer la tabla Procedures sin credenciales.

2. Applications can read and update data in the Transactions table once authenticated. → No (Falsa)

La entidad Transactions tiene el permiso:

"role": "authenticated", "actions": ["read", "create"].

read permite lectura.
create permite solo inserción (INSERT).
No existe la acción update. Por lo tanto, las aplicaciones autenticadas no pueden actualizar datos; solo leer

e insertar. La afirmación es falsa porque dice “read and update”.

3. Applications can execute the sp_UpdateProcedurePatient stored procedure. → Yes (Verdadera)

La entidad UpdateProcedurePatient (que apunta a dbo.sp_UpdateProcedurePatient) tiene:

"role": "authenticated", "actions": ["execute"]

y está habilitada en REST con método POST.

Las aplicaciones autenticadas pueden ejecutar el procedimiento almacenado.

References:
https://learn.microsoft.com/azure/data-api-builder/concept/security/authorization-overview

https://learn.microsoft.com/azure/data-api-builder/configuration/entities#permissions-entity-name-entities

QUESTION 58

HOTSPOT

-

Case Study

-

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete ea
there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to co
included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Ca
contain exhibits and other resources that provide more information about the scenario that is described in the case study. Eac
independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes
the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study

-

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of
before you answer the questions. Clicking these buttons displays information such as business requirements, existing enviro
statements. If the case study has an All Information tab, note that the information displayed is identical to the information dis
subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment

-

Azure Environment

-

Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database nam
contains the following tables:

- Patients
- Employees
- Procedures
- Transactions
- UsefulPrompts
- ProcedureDocuments

You store a column master key as a secret in Azure Key Vault.

You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a conn
access DB1.

Problem Statements

-

Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the under

sometimes inconsistent.

Requirements

-

Planned Changes

-

Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request sul production will be validated before it can be merged. Deployments must use the Release configuration.

Security Requirements

-

Fabrikam identifies the following security requirements:

- The TransactionProcessing application must use a passwordless connection to DB1.
- The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
- Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.

Database Performance Requirements

Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored proce

AI Search, Embeddings, and Vector Indexing

Fabrikam identifies the following AI-related requirements:

- Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
- Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
- The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure O

Development Requirements

-

Fabrikam identifies the following development requirements:

• Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
• Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST A
o Read data from the procedures table without authentication.
o Read and insert data into the Transactions table once authenticated.
o Execute the sp_UpdateProcedurePatient stored procedure.
• Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
• Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for
will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

-

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You need to create a solution that meets the development requirements for retrieving the patient transaction data. The solution database developers use the resulting data to join to other tables.

How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
CREATE [ ▼ ]  dbo.CustomerTransactionsBetweenDates
(
  @CustomerId INT,
  @StartDate DATETIME2,
  @EndDate   DATETIME2
)
RETURNS TABLE
AS
RETURN
(
  SELECT
    t.TransactionId,
    t.CustomerId,
    t.TransactionDate,
    t.Amount,
    SUM(t.Amount) OVER (
        [ ▼ ]
        [ ▼ ]
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS RunningTotal
  FROM dbo.Transactions AS t
  WHERE t.CustomerId = @CustomerId
    AND t.TransactionDate >= @StartDate
    AND t.TransactionDate <=  @EndDate
);
```

- FUNCTION
- PROCEDURE
- VIEW

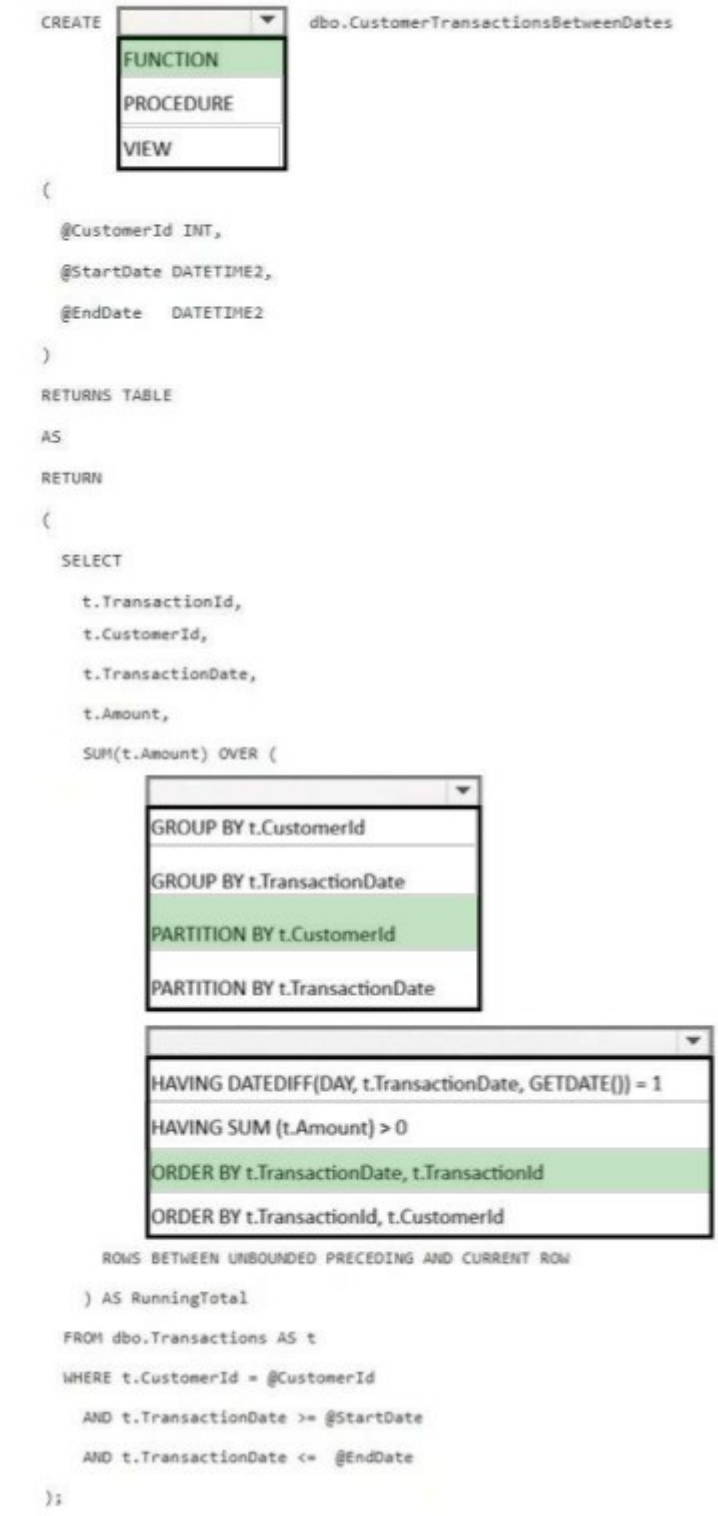
- GROUP BY t.CustomerId
- GROUP BY t.TransactionDate
- PARTITION BY t.CustomerId
- PARTITION BY t.TransactionDate

- HAVING DATEDIFF(DAY, t.TransactionDate, GETDATE()) = 1
- HAVING SUM (t.Amount) > 0
- ORDER BY t.TransactionDate, t.TransactionId
- ORDER BY t.TransactionId, t.CustomerId

Answer:

NOTE: Each correct selection is worth one point.

Answer Area



Section: (none)

Explanation/Reference:
VCE 58 / 032 Topic 2

Respuesta correcta:
CREATE FUNCTION dbo.CustomerTransactionsBetweenDates. Dentro de OVER, usar PARTITION BY t. CustomerId seguido de ORDER BY t.TransactionDate, t.TransactionId.

Explicación:
El código recibe parámetros (@CustomerId, @StartDate, @EndDate) y usa RETURNS TABLE AS RETURN (SELECT ...), que es la sintaxis de una función con valor de tabla en línea (inline table-valued function), por lo que el objeto debe crearse con CREATE FUNCTION. Después se consulta como una tabla: SELECT * FROM dbo.CustomerTransactionsBetweenDates(...).

Para el total acumulado, PARTITION BY t.CustomerId divide las filas en una ventana por cliente sin colapsarlas, de modo que cada transacción se mantiene en el resultado y RunningTotal se reinicia en cada cliente. El marco ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW requiere un ORDER BY dentro de OVER. Ordenar por t.TransactionDate hace que el total se acumule cronológicamente, y t.TransactionId actúa como desempate para que las transacciones de la misma fecha produzcan siempre un total acumulado determinista.

CREATE PROCEDURE es incorrecta porque un procedimiento almacenado no puede usarse en la cláusula FROM de una consulta y no utiliza la sintaxis RETURNS TABLE AS RETURN (...). CREATE VIEW es incorrecta porque una vista no acepta parámetros, y este objeto tiene tres (@CustomerId, @StartDate, @EndDate).

GROUP BY t.CustomerId y GROUP BY t.TransactionDate son incorrectas porque GROUP BY no es válido dentro de una cláusula OVER (error de sintaxis), y agrupar colapsaría las filas en lugar de conservar cada transacción. PARTITION BY t.TransactionDate es incorrecta porque el total acumulado se reiniciaría en cada fecha en lugar de acumularse por cliente.

HAVING DATEDIFF(DAY, t.TransactionDate, GETDATE()) = 1 y HAVING SUM(t.Amount) > 0 son incorrectas porque HAVING filtra grupos después de un GROUP BY y no es válido dentro de OVER; ninguna aporta el ORDER BY que exige el marco ROWS. ORDER BY t.TransactionId, t.CustomerId es incorrecta porque ordena primero por TransactionId, por lo que el total no se acumula en orden de fecha, y CustomerId sobra porque ya está en el PARTITION BY.

Referencias:
https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql
https://learn.microsoft.com/en-us/sql/t-sql/statements/create-function-transact-sql
https://learn.microsoft.com/en-us/sql/t-sql/statements/create-procedure-transact-sql
https://learn.microsoft.com/en-us/sql/t-sql/statements/create-view-transact-sql
https://learn.microsoft.com/en-us/sql/t-sql/queries/select-group-by-transact-sql
https://learn.microsoft.com/en-us/sql/t-sql/queries/select-having-transact-sql

QUESTION 59
What should you include in your recommendation?

Case Study -

This is a case study. Case studies are not timed separately. You can use as much exam time as you
there may be additional case studies and sections on this exam. You must manage your time to ens
included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is prov
contain exhibits and other resources that provide more information about the scenario that is descri
independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your ansv
the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study -

To display the first question in this case study, click the Next button. Use the buttons in the left pane
before you answer the questions. Clicking these buttons displays information such as business requ
statements. If the case study has an All Information tab, note that the information displayed is ident
subsequent tabs. When you are ready to answer a question, click the Question button to return to the

Existing Environment -

Azure Environment -

Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains a
contains the following tables:

- Patients
- Employees
- Procedures
- Transactions
- UsefulPrompts
- ProcedureDocuments

You store a column master key as a secret in Azure Key Vault.

You have an on-premises application named TransactionProcessing that uses a hard-coded usernam
access DB1.

Problem Statements -

Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPa
sometimes inconsistent.

Requirements -

Planned Changes -

Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in Git

production will be validated before it can be merged. Deployments must use the Release configuratio

Security Requirements -

Fabrikam identifies the following security requirements:

- The TransactionProcessing application must use a passwordless connection to DB1.
- The Employees table contains two columns named TaxID and Salary that must be encrypted at rest
- Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to

Database Performance Requirements

Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactio

AI Search, Embeddings, and Vector Indexing

Fabrikam identifies the following AI-related requirements:

- Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
- Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
- The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by

Development Requirements -

Fabrikam identifies the following development requirements:

• Provide the functionality to retrieve all the transactions of a given patient between two dates, showi
• Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following from the procedures table without authentication. o Read and insert data into the Transactions table sp_UpdateProcedurePatient stored procedure.
• Provide the functionality to retrieve a list of the names of patients who underwent medical procedu
• Information for each medical procedure will be stored in a table. The table will be used with a large will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB -

You create a DAB configuration file that meets the development requirements for DB1 and includes th

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
    }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You need to recommend a solution for the TransactionProcessing application that meets the securit

A. a service principal
B. a system-assigned managed identity
C. a shared access key
D. a user-assigned managed identity

Answer: A
Section: (none)

Explanation/Reference:
#VCE 59 / 033 Topic 02

Respuesta Correcta:

a service principal

Explicación:

a service principal es correcta porque el caso de estudio establece que Transaction Processing es una aplicación local (on-premises). Para cumplir con el requisito de seguridad de utilizar una conexión sin contraseña (passwordless) hacia la base de datos DB1 desde un entorno externo a Azure, la aplicación debe registrarse en Microsoft Entra ID, lo cual genera una entidad de servicio (Service Principal) que puede autenticarse de forma segura (por ejemplo, mediante certificados).

¿Por qué el resto son incorrectas?

a system-assigned managed identity: Las identidades administradas asignadas por el sistema están diseñadas exclusivamente para integrarse con recursos que se ejecutan de forma nativa dentro de Azure (como Máquinas Virtuales de Azure, App Services, etc.) o en servidores locales habilitados con Azure Arc. Puesto que la aplicación Transaction Processing es on-premises y no hay mención a Azure Arc en el escenario, el servidor no puede solicitar ni utilizar una identidad administrada.

a shared access key: Las claves de acceso compartido (Shared Access Keys) no son un método válido para la autenticación de usuarios o aplicaciones en Azure SQL Database (son propias de servicios como Azure Storage). Además, incrustar una clave compartida sería funcionalmente equivalente a usar una contraseña estática, lo que incumpliría el requerimiento explícito de usar una conexión "passwordless".

a user-assigned managed identity: Al igual que la opción a system-assigned managed identity, las identidades administradas asignadas por el usuario requieren que el recurso de cómputo subyacente esté en Azure o gestionado por Azure Arc para poder asignarles dicha identidad. No es un mecanismo que se pueda configurar para una aplicación on-premises tradicional.

Referencia:Microsoft Learn – Autenticación de Microsoft Entra con Azure SQL Database (Entidades de servicio): https://learn.microsoft.com/es-es/azure/azure-sql/database/authentication-aad-service-principal-tutorial

QUESTION 60
HOTSPOT
-

Case Study
-

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study
-

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment
-

Azure Environment
-

Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:

- Patients
- Employees
- Procedures
- Transactions
- UsefulPrompts
- ProcedureDocuments

You store a column master key as a secret in Azure Key Vault.

You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.

Problem Statements
-

Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.

Requirements
-

Planned Changes
-

Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.

Security Requirements
-

Fabrikam identifies the following security requirements:

- The TransactionProcessing application must use a passwordless connection to DB1.
- The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
- Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.

Database Performance Requirements

Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.

AI Search, Embeddings, and Vector Indexing
Fabrikam identifies the following AI-related requirements:

- Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
- Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
- The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.

Development Requirements
-

Fabrikam identifies the following development requirements:

- Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
- Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
  o Read data from the procedures table without authentication.
  o Read and insert data into the Transactions table once authenticated.
  o Execute the sp_UpdateProcedurePatient stored procedure.
- Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
- Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB
-

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

Question
You create an SDK-style SQL database project in Microsoft Visual Studio Code named Database.sqlproj and add the project to a GitHub repository.

You need to configure a GitHub Actions workflow to support the planned changes for DB1.

How should you complete the workflow? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
name: Validate SQL Project

on:
    [ ▼ ] :
        branches: [ "main" ]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
  - uses: actions/checkout@v4
      - name: Step 1
        run: dotnet [ ▼ ]
```

- merge
- pull_request
- push

- build Database.sqlproj -c Release
- pack Database.sqlproj
- publish Database.sqlproj -c Production
- publish Database.sqlproj -c Release

Answer:

Answer Area

```
name: Validate SQL Project

on:
    [ merge | pull_request | push ] :
      branches: [ "main" ]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
  - uses: actions/checkout@v4
      - name: Step 1
        run: dotnet [ build Database.sqlproj -c Release | pack Database.sqlproj | publish Database.sqlproj -c Production | publish Database.sqlproj -c Release ]
```

Section: (none)

Explanation/Reference:
Q60 T02

Correct Answer:

1. **push**

2. **build Database.sqlproj -c Release**

Explanation:

1. Correcta: "push". El workflow está filtrado con branches: ["main"], de modo que se ejecuta cada vez que se envían commits a la rama main, y valida el proyecto SQL en cada cambio. Es el mismo patrón que usa el workflow de ejemplo de Microsoft Learn para proyectos SQL (on: push con branches: ["main"]).

Incorrectas:

- "merge": no es un evento válido de GitHub Actions. No existe un trigger llamado merge; una fusión se refleja como un push a la rama destino.
- "pull_request": se dispara al abrir o actualizar una pull request, no al enviar commits a main. Es la opción que se usaría para validar antes de fusionar, pero la respuesta correcta de esta pregunta es push, igual que en el workflow de referencia de Microsoft Learn.

2. Correcta: "build Database.sqlproj -c Release". dotnet build compila el SDK-style SQL project (Microsoft. Build.Sql) y valida las referencias entre objetos y la sintaxis T-SQL contra la plataforma de destino, generando el .dacpac. El parámetro -c Release cumple el requisito del case study de que los despliegues deben usar la configuración Release.

Incorrectas:
- "pack Database.sqlproj": empaqueta el proyecto como paquete NuGet, no es el paso de validación de la compilación y no aplica el requisito de Release.
- "publish Database.sqlproj -c Production": Production no es una configuración de build estándar del proyecto (las estándar son Debug y Release), y publish no es el modo de validar un proyecto SQL. Los proyectos SQL se despliegan con SqlPackage o sql-action, no con dotnet publish.
- "publish Database.sqlproj -c Release": usa la configuración correcta, pero publish tampoco es el comando para validar un proyecto SQL, y el flujo de este workflow es solo validar, no desplegar a la base de datos.

References:
- https://learn.microsoft.com/en-us/training/modules/implement-cicd-sql-database-projects/02-create-build-validate-sql-database-projects
- https://learn.microsoft.com/en-us/sql/tools/sql-database-projects/tutorials/create-deploy-sql-project
- https://learn.microsoft.com/en-us/sql/tools/visual-studio-code-extensions/sql-database-projects/build-database-project-from-command-line

QUESTION 61
Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.

You have a SQL database in Microsoft Fabric that contains a table named dbo.Orders. dbo.Orders has a clustered index, contains three years of data, and is partitioned by a column named OrderDate by month.

You need to remove all the rows for the oldest month. The solution must minimize the impact on other queries that access the data in dbo.Orders.

Solution: Run the following Transact-SQL statement.

TRUNCATE TABLE dbo.Orders;

Does this meet the goal?

A. Yes
B. No

Answer: B
Section: (none)

Explanation/Reference:
Q61 T02

Respuesta Correcta - B (No)

Justificación: TRUNCATE TABLE dbo.Orders; sin la cláusula WITH (PARTITIONS (...)) elimina todas las filas de toda la tabla, es decir, los tres años de datos y no solo el mes más antiguo. Por eso no cumple el objetivo ni minimiza el impacto sobre otras consultas.

Solución

La instrucción propuesta no cumple el objetivo. Sin la cláusula WITH (PARTITIONS (...)), TRUNCATE TABLE elimina todas las filas de la tabla, así que se borrarían los tres años de datos en lugar de solo el mes más antiguo. Además, esto afectaría gravemente a todas las demás consultas que leen dbo.Orders.

La instrucción correcta apunta únicamente a la partición que contiene el mes más antiguo:

```
TRUNCATE TABLE dbo.Orders
WITH (PARTITIONS (1));
```

Por qué falla la solución propuesta

Requisito	¿TRUNCATE TABLE dbo.Orders; lo cumple?
Eliminar todas las filas del mes más antiguo	Sí, pero también elimina todos los demás meses
Mantener intactos el resto de los datos	No, se borran los tres años
Minimizar el impacto en otras consultas	No, se vacía la tabla completa

Cómo funciona el truncado a nivel de partición

Si no se incluye la cláusula WITH (PARTITIONS (...)), se trunca la tabla completa.
El truncado de particiones específicas está soportado en SQL Server 2016 y posteriores, Azure SQL Database y SQL database in Microsoft Fabric.
La tabla y sus índices deben estar alineados (particionados con la misma función de partición). Aquí se cumple, porque el índice clustered está particionado por OrderDate.
Solo se desasignan las páginas de la partición indicada, por lo que las demás particiones siguen disponibles.
TRUNCATE TABLE usa menos bloqueos y menos espacio del registro de transacciones que DELETE, porque solo registra las desasignaciones de páginas y no cada fila eliminada.

Otros enfoques válidos

ALTER TABLE ... SWITCH PARTITION mueve la partición más antigua a una tabla de archivo o de staging, que luego se puede eliminar o truncar.
DELETE ... WHERE OrderDate < ... funciona, pero es más lento, genera mucho más registro de transacciones y provoca más bloqueos, por lo que afecta más a las consultas concurrentes.

Referencias

TRUNCATE TABLE (Transact-SQL)
https://learn.microsoft.com/sql/t-sql/statements/truncate-table-transact-sql?view=sql-server-ver17

Sintaxis y comportamiento de WITH (PARTITIONS (...)), y truncado de toda la tabla cuando se omite la cláusula (aplica a SQL database in Microsoft Fabric).
TRUNCATE TABLE (Transact-SQL) - Ejemplos: Truncate table partitions
https://learn.microsoft.com/sql/t-sql/statements/truncate-table-transact-sql?view=sql-server-ver17#examples

Ejemplo de truncado de particiones específicas de una tabla particionada.
Partitioned tables and indexes
https://learn.microsoft.com/sql/relational-databases/partitions/partitioned-tables-and-indexes?view=sql-server-ver17

Beneficios del particionado, incluido el truncado de datos en una sola partición para retención de datos.
ALTER TABLE (Transact-SQL)

https://learn.microsoft.com/sql/t-sql/statements/alter-table-transact-sql?view=sql-server-ver17
Recomienda TRUNCATE TABLE ... WITH (PARTITIONS (...)) para eliminar todas las filas de particiones específicas sin eliminarlas ni hacer switch.

QUESTION 62
Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that
might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have an SDK-style SQL database project stored in a Git repository. The project targets an Azure SQL database.

The CI build fails with unresolved reference errors when the project references system objects.

You need to update the SQL database project to ensure that dotnet build validates successfully by including the correct system objects in the
database model for Azure SQL Database.

Solution: Build the project by running dotnet build -bl -flp:v=diag.

Does this meet the goal?

A. Yes
B. No

Answer: B
Section: (none)

Explanation/Reference:
Q62 T02

ES

Respuesta

B. No

Explicación

Por qué no cumple el objetivo: Por defecto, los proyectos SQL no incluyen los objetos del sistema en el modelo de base de datos. Por eso la validación falla al referenciar objetos de sistema. La solución es añadir una referencia de base de datos al `master.dacpac` de la plataforma de destino. En un proyecto SDK-style (Microsoft.Build.Sql) se hace con una referencia de paquete NuGet. Para Azure SQL Database el paquete es **Microsoft.SqlServer.Dacpacs.Azure.Master.

Ejecutar la compilación con registro binario y logging detallado solo genera diagnósticos. No modifica el proyecto ni el modelo, así que los objetos del sistema siguen sin resolverse y el error persiste. Sirve para investigar el fallo, no para corregirlo.

Conceptos a conocer:
- Referencia de paquete: es el método recomendado en desarrollo nuevo y solo funciona en proyectos SDK-style. Los proyectos originales (SSDT clásicos) no soportan paquetes NuGet y usan una referencia de artefacto al `master.dacpac`.
-Paquete según la plataforma: el de SQL Server (`Microsoft.SqlServer.Dacpacs.Master`) es distinto del de Azure SQL Database (`Microsoft.SqlServer.Dacpacs.Azure.Master`). Usar el equivocado da un modelo incorrecto para el destino.
- SQL database en Fabric: usa `Microsoft.SqlServer.Dacpacs.DbFabric` y requiere configurar además la propiedad `DatabaseVariableLiteralValue` con el valor `master`.

Referencias

- Microsoft Learn: [SQL projects system objects](https://learn.microsoft.com/sql/tools/sql-database-projects/concepts/system-objects?view=sql-server-ver17)
- Microsoft Learn: [SQL projects package references](https://learn.microsoft.com/sql/tools/sql-database-projects/concepts/package-references?view=sql-server-ver17)
- SQL Server: no aplica más allá de lo anterior.

Clave DP-800

Errores de referencias sin resolver a objetos del sistema se corrigen con la referencia de paquete al dacpac de `master` de la plataforma (Azure SQL → `Dacpacs.Azure.Master`), no con logs ni con otras opciones de compilación.

English version

Answer

B. No

Explanation

Why it doesn't meet the goal: By default, SQL projects don't include system objects in the database model, so validation fails when the project references them. The fix is a database reference to the target platform's `master.dacpac`. In SDK-style (Microsoft.Build.Sql) projects this is done with a NuGet package reference. For Azure SQL Database the package is Microsoft.SqlServer.Dacpacs.Azure.Master.

Running the build with a binary log and diagnostic verbosity only produces diagnostic output. It doesn't change the project or the model, so system objects remain unresolved and the error persists. It helps you investigate the failure, not fix it.

Concepts to know:
- Package reference: this is the recommended method for new development and only works in SDK-style projects. Original (classic SSDT) projects don't support NuGet packages and use an artifact reference to `master.dacpac` instead.
- Package per platform: the SQL Server package (`Microsoft.SqlServer.Dacpacs.Master`) differs from the Azure SQL Database package (`Microsoft.SqlServer.Dacpacs.Azure.Master`). Using the wrong one gives a model that doesn't match the target.
- SQL database in Fabric: it uses `Microsoft.SqlServer.Dacpacs.DbFabric` and also needs the `DatabaseVariableLiteralValue` property set to `master`.

References

- Microsoft Learn: [SQL projects system objects](https://learn.microsoft.com/sql/tools/sql-database-projects/concepts/system-objects?view=sql-server-ver17)
- Microsoft Learn: [SQL projects package references](https://learn.microsoft.com/sql/tools/sql-database-projects/concepts/package-references?view=sql-server-ver17)
- SQL Server: not applicable beyond the above.

DP-800 Key

Unresolved-reference errors on system objects are fixed by adding a package reference to the platform's `master` dacpac (Azure SQL → `Dacpacs.Azure.Master`), not by logging or other build options.

QUESTION 63
HOTSPOT

You have an SDK-style SQL database project named MyDatabaseProject.sqlproj stored in a private GitHub repository. The repository contains the following GitHub Actions workflow.

```yaml
name: Build and Deploy SQL Project
on:
  push:
    branches:
      - main
  workflow_dispatch:
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      - name: Build SQL project
        run: dotnet build MyDatabaseProject.sqlproj
      - name: Publish DACPAC
        uses: azure/sql-action@v2
        with:
          action: publish
          path: bin/Debug/MyDatabaseProject.dacpac
          connection-string: ${{ secrets.AZURE_SQL_CONNECTION_STRING }}
```

The repository contains the AZURE_SQL_CONNECTION_STRING secrets.

The target is an Azure SQL database that allows access to Azure services and is configured to support mixed authentication.

The workflow runs successfully.

For each of the following statements, select Yes if the statement is true. Otherwise, select No.

NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
The workflow relies on a Microsoft Entra workload identity to access the target Azure SQL database.	○	○
Changing the `Build SQL project` step to `run: dotnet build MyDatabaseProject.sqlproj -c Release` will result in a successful deployment.	○	○
The workflow can be triggered manually, without making changes to the repository.	○	○

Answer:

Answer Area

Statements	Yes	No
The workflow relies on a Microsoft Entra workload identity to access the target Azure SQL database.	○	●
Changing the `Build SQL project` step to `run: dotnet build MyDatabaseProject.sqlproj -c Release` will result in a successful deployment.	○	●
The workflow can be triggered manually, without making changes to the repository.	●	○

Section: (none)

Explanation/Reference:
Q63 T02

Correct Answer:
The workflow relies on a Microsoft Entra workload identity to access the target Azure SQL database. → No
Changing the Build SQL project step to run: dotnet build MyDatabaseProject.sqlproj -c Release will result in a successful deployment. → No
The workflow can be triggered manually, without making changes to the repository. → Yes

Explanation:
Aunque el workflow declara el permiso id-token: write, que haría falta para una federación OIDC con Microsoft Entra, no hay ningún paso azure/login que obtenga un token. El paso Publish DACPAC se autentica con la cadena de conexión guardada en el secreto AZURE_SQL_CONNECTION_STRING. Como

la base de datos admite autenticación mixta, esa cadena usa autenticación SQL (usuario y contraseña) y no una identidad de carga de trabajo de Microsoft Entra.
Con -c Release, dotnet build compila en la configuración Release y genera el .dacpac en bin/Release/MyDatabaseProject.dacpac. El paso Publish DACPAC sigue apuntando a bin/Debug/MyDatabaseProject.dacpac, así que no encuentra el archivo y el despliegue falla, salvo que también se cambie el valor de path.
El trigger workflow_dispatch permite lanzar el workflow a mano desde la pestaña Actions de GitHub, con la CLI o con la API REST, sin hacer ningún push ni cambio en el repositorio.

English translation of the Explanation:
Although the workflow declares the id-token: write permission, which would be required for OIDC federation with Microsoft Entra, there is no azure/login step that obtains a token. The Publish DACPAC step authenticates with the connection string stored in the AZURE_SQL_CONNECTION_STRING secret. Because the database supports mixed authentication, that connection string uses SQL authentication (username and password), not a Microsoft Entra workload identity.
With -c Release, dotnet build compiles in the Release configuration and outputs the .dacpac to bin/Release/MyDatabaseProject.dacpac. The Publish DACPAC step still points to bin/Debug/MyDatabaseProject.dacpac, so it cannot find the file and the deployment fails unless the path value is also changed.
The workflow_dispatch trigger lets the workflow be started manually from the GitHub Actions tab, the CLI or the REST API, without any push or change to the repository.

Reference:
https://github.com/Azure/sql-action
https://learn.microsoft.com/azure/developer/github/connect-from-azure-openid-connect
https://learn.microsoft.com/dotnet/core/tools/dotnet-build
https://docs.github.com/actions/using-workflows/events-that-trigger-workflows#workflow_dispatch
https://learn.microsoft.com/sql/tools/sql-database-projects/tutorials/create-deploy-sql-project?view=sql-server-ver17

QUESTION 64
You have an Azure SQL database that supports an AI-driven product search API.

You need to identify the top CPU-consuming queries from the last two hours by using Query Store data. The solution must aggregate CPU consumption across executions and return only the top 15 query hashes.

How should you complete the Transact-SQL code? Drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all.

NOTE: Each correct selection is worth one point.

Values

- DATEADD(DAY, -2, GETDATE())
- DATEADD(HOUR, -2, GETDATE())
- DATEADD(HOUR, -2, GETUTCDATE())
- rs.avg_cpu_time
- rs.last_cpu_time
- sys.dm_exec_query_stats
- sys.query_store_runtime_stats_interval

Answer Area

```
WITH AggregatedCPU AS
(
  SELECT
    q.query_hash,
    SUM(count_executions *
  FROM sys.query_store_query_text AS qt
  INNER JOIN sys.query_store_query AS q
    ON qt.query_text_id = q.query_text_id
  INNER JOIN sys.query_store_plan AS p
    ON q.query_id = p.query_id
  INNER JOIN sys.query_store_runtime_stats
    ON rs.plan_id = p.plan_id
  INNER JOIN
    ON rsi.runtime_stats_interval_id = rs.
  WHERE rs.execution_type_desc IN ('Regula
    AND rsi.start_time >=
  GROUP BY q.query_hash
),
OrderedCPU AS
(
  SELECT
    query_hash,
    total_cpu_ms,
    ROW_NUMBER() OVER (ORDER BY total_cpu_
  FROM AggregatedCPU
)
SELECT query_hash, total_cpu_ms
FROM OrderedCPU
WHERE rn <= 15
ORDER BY total_cpu_ms DESC;
```

Answer:

Values

- DATEADD(DAY, -2, GETDATE())
- DATEADD(HOUR, -2, GETDATE())
- rs.last_cpu_time
- sys.dm_exec_query_stats

Answer Area

```
WITH AggregatedCPU AS
(
  SELECT
    q.query_hash,
    SUM(count_executions * [ rs.avg_cpu_ti
  FROM sys.query_store_query_text AS qt
  INNER JOIN sys.query_store_query AS q
    ON qt.query_text_id = q.query_text_id
  INNER JOIN sys.query_store_plan AS p
    ON q.query_id = p.query_id
  INNER JOIN sys.query_store_runtime_stats
    ON rs.plan_id = p.plan_id
  INNER JOIN [ sys.query_store_runtime_stats_interval
    ON rsi.runtime_stats_interval_id = rs.
  WHERE rs.execution_type_desc IN ('Regula
    AND rsi.start_time >= [ DATEADD(HOUR,
  GROUP BY q.query_hash
),
OrderedCPU AS
(
  SELECT
    query_hash,
    total_cpu_ms,
    ROW_NUMBER() OVER (ORDER BY total_cpu_
  FROM AggregatedCPU
)
SELECT query_hash, total_cpu_ms
FROM OrderedCPU
WHERE rn <= 15
ORDER BY total_cpu_ms DESC;
```

Section: (none)

Explanation/Reference:

Q64 T02

Correct Answer:

1. rs.avg_cpu_time
2. sys.query_store_runtime_stats_interval
3. DATEADD(HOUR, -2, GETUTCDATE())

Explanation:

El primer hueco debe ser rs.avg_cpu_time. Query Store registra el tiempo medio de CPU por ejecución en microsegundos. Al multiplicarlo por count_executions y sumar los resultados, la consulta calcula el consumo acumulado de CPU; dividir entre 1000 lo expresa en milisegundos. rs.last_cpu_time solo representa la última ejecución.

El segundo hueco debe ser sys.query_store_runtime_stats_interval. Esta vista contiene los intervalos temporales asociados a las estadísticas de ejecución y permite consultar rsi.start_time. sys.dm_exec_query_stats no es la vista de intervalos de Query Store que necesita este JOIN.

El tercer hueco debe ser DATEADD(HOUR, -2, GETUTCDATE()). Resta dos horas a la hora UTC actual para establecer el límite temporal solicitado. DATEADD(DAY, -2, GETDATE()) abarcaría dos días, y GETDATE() no expresa explícitamente la hora UTC.

Por último, la consulta agrupa por query_hash, ordena los resultados por CPU total y conserva solo los primeros 15.

References:

Referencias (Microsoft Learn)

sys.query_store_runtime_stats (Transact-SQL): https://learn.microsoft.com/sql/relational-databases/system-catalog-views/sys-query-store-runtime-stats-transact-sql?view=sql-server-ver17

sys.query_store_runtime_stats_interval (Transact-SQL): https://learn.microsoft.com/sql/relational-databases/system-catalog-views/sys-query-store-runtime-stats-interval-transact-sql?view=sql-server-ver17

GETUTCDATE (Transact-SQL): https://learn.microsoft.com/sql/t-sql/functions/getutcdate-transact-sql?view=sql-server-ver17

DATEADD (Transact-SQL): https://learn.microsoft.com/sql/t-sql/functions/dateadd-transact-sql?view=sql-server-ver17

QUESTION 65
HOTSPOT -

You have an Azure SQL database that contains a table named Sales.Customer. Sales.Customer contains columns named CustomerId, FullName, Email, TaxID, and RegionId.

You have a database role named AppSupport that is used by a support application.
You need to implement a security solution for AppSupport that meets the following requirements:

- AppSupport must be prevented from viewing TaxID.
- AppSupport must be able to query Sales.Customer to troubleshoot issues.
- AppSupport must be able to run a stored procedure named Sales.usp_GetCustomerByCustomerId.

Which Transact-SQL statements should you include in the solution? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

To run Sales.usp_GetCustomerByCustomerId:

- DENY SELECT ON OBJECT::Sales.Customer TO AppSupport;
- DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;
- GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- REVOKE SELECT ON OBJECT::Sales.Customer(TaxID) FROM AppSupport;

To query Sales.Customer:

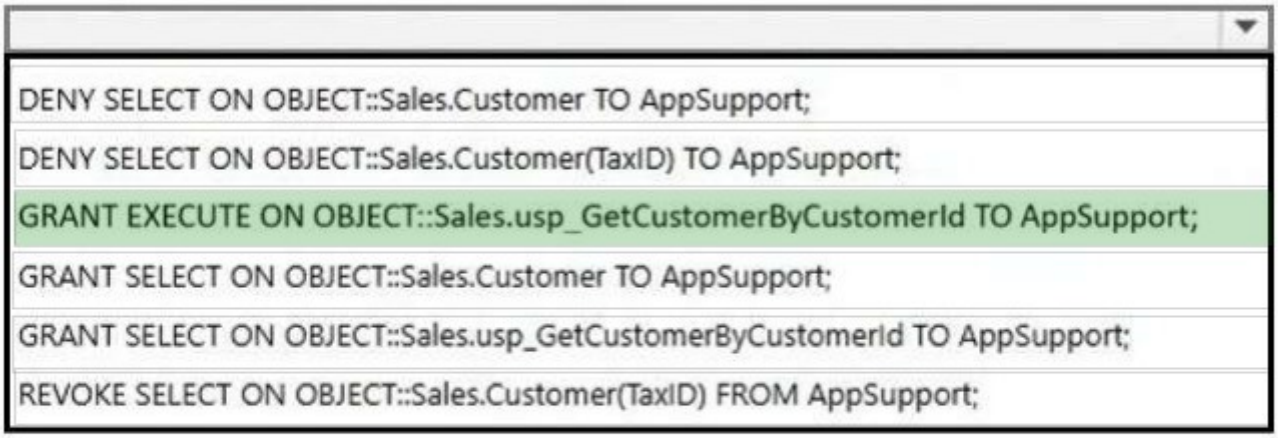
- DENY SELECT ON OBJECT::Sales.Customer TO AppSupport;
- DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;
- GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- REVOKE SELECT ON OBJECT::Sales.Customer(TaxID) FROM AppSupport;

To prevent viewing TaxID:

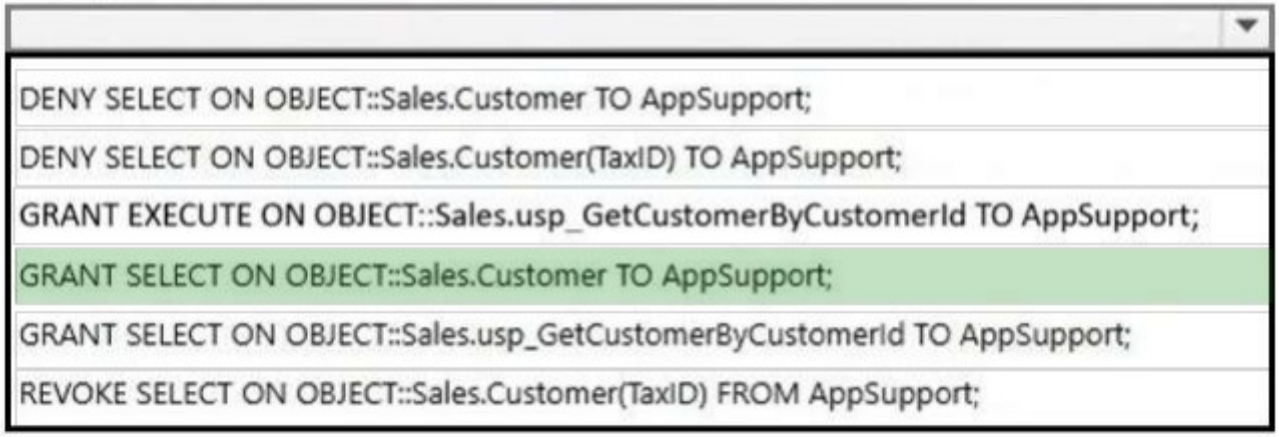
- DENY SELECT ON OBJECT::Sales.Customer TO AppSupport;
- DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;
- GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- REVOKE SELECT ON OBJECT::Sales.Customer(TaxID) FROM AppSupport;

Answer:

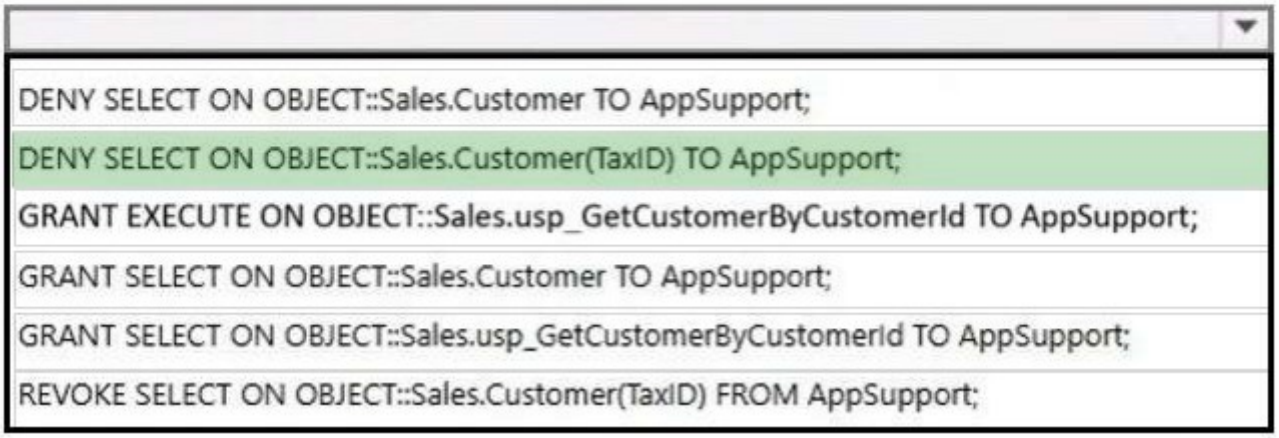
To run Sales.usp_GetCustomerByCustomerId:



To query Sales.Customer:



To prevent viewing TaxID:



Section: (none)

Explanation/Reference:
Q65 T02

CORRECT ANSWER:

- To run Sales.usp_GetCustomerByCustomerId: GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;

- To query Sales.Customer: GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;

- To prevent viewing TaxID: DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;

EXPLANATION:

- GRANT EXECUTE: Concede el permiso al rol AppSupport para poder ejecutar el procedimiento almacenado.

- GRANT SELECT: Otorga al rol AppSupport el derecho de consultar los datos de la tabla Sales.Customer.

- DENY SELECT: Bloquea explícitamente el acceso de lectura a la columna TaxID dentro de la tabla Sales. Customer para el rol AppSupport. En SQL Server y Azure SQL, un permiso DENY explícito a nivel de columna siempre anula un permiso GRANT concedido a nivel de tabla, permitiendo al usuario consultar el resto de la tabla sin exponer datos confidenciales.

REFERENCES:

GRANT - Permisos de objeto (Transact-SQL):
https://learn.microsoft.com/es-es/sql/t-sql/statements/grant-object-permissions-transact-sql

DENY - Permisos de objeto (Transact-SQL):
https://learn.microsoft.com/es-es/sql/t-sql/statements/deny-object-permissions-transact-sql

QUESTION 66
HOTSPOT

You have an Azure SQL database named **ProductsDB**.

You deploy Data API builder (DAB) to Azure Container Apps.

You discover that the container app cannot connect to ProductsDB.

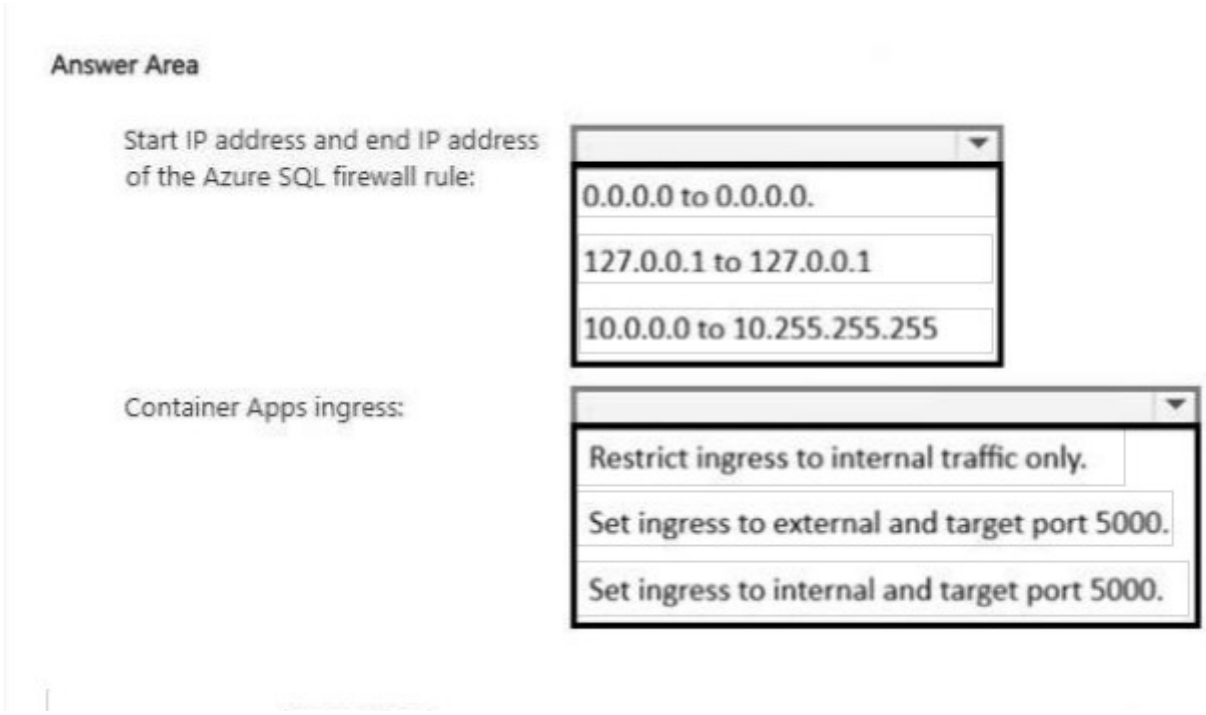
Your deployment team reports that the container app is unreachable from the internet for integration tests.

You need to update Azure SQL Database and Container Apps to meet the following requirements:

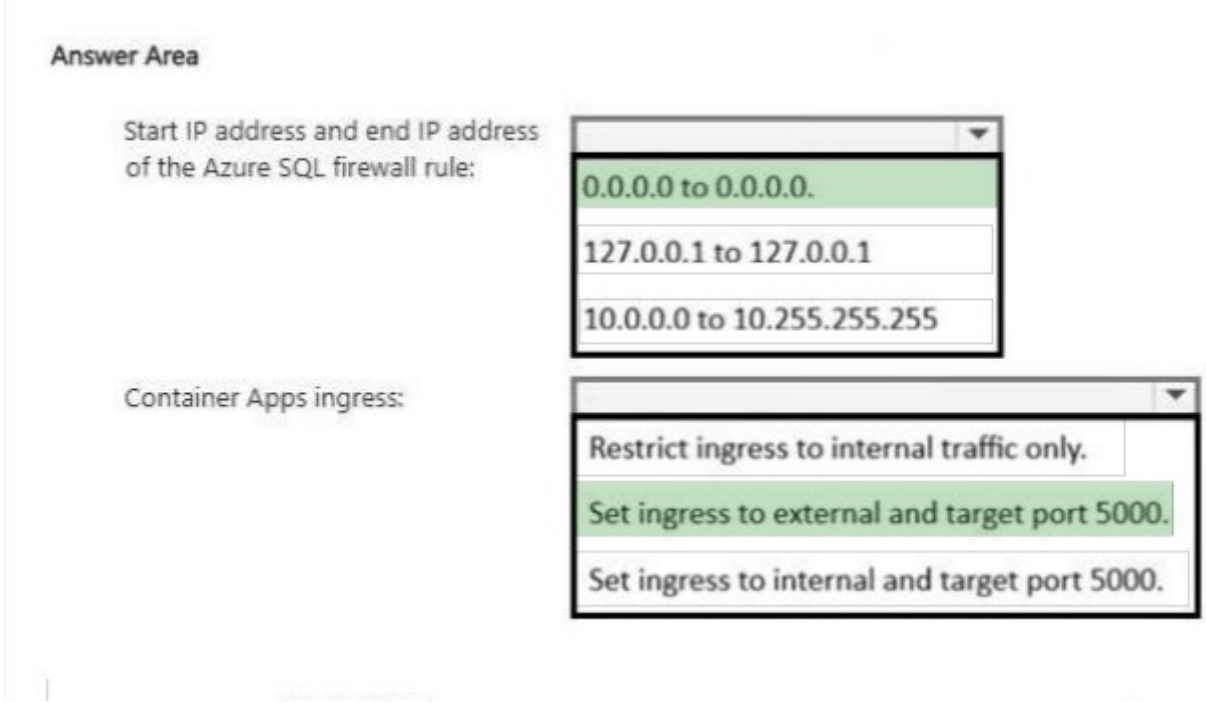
- Ensure that the Azure SQL logical server allows connections from Azure services.
- Ensure that the Container Apps environment accepts inbound requests from the public internet.

What should you configure? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q66 T02

1. ANÁLISIS

Concepto principal: configuración de conectividad de red.

Servicios: Azure SQL Database (firewall del logical server) + Azure Container Apps (ingress).

Escenario: DAB desplegado en Container Apps no puede conectar a ProductsDB y la app no es accesible desde Internet.

2. VERIFICACIÓN (MCP)

Documentación oficial de Microsoft Learn consultada sobre firewall de Azure SQL y ingress de Container Apps.
3. RESOLUCIÓN

Azure SQL logical server: Habilitar Allow Azure services and resources to access this server.
Container Apps environment / app: Configurar Ingress como External (Accepting traffic from anywhere / public internet).

4. JUSTIFICACIÓN

Azure SQL – Allow Azure services…
Microsoft Learn indica que esta opción crea la regla de firewall 0.0.0.0 – 0.0.0.0 (AllowAllWindowsAzureIps). Permite que cualquier recurso de Azure (incluido el Container App) se conecte al logical server sin necesidad de IPs específicas.
Container Apps – Ingress External
Según la documentación de Ingress, el tipo External publica la aplicación a través de la IP pública del entorno, permitiendo tráfico entrante desde Internet público. El tipo Internal solo permite acceso desde dentro del entorno.
Otras opciones (reglas IP específicas, Private Endpoint, Ingress Internal, etc.) no cumplen simultáneamente ambos requisitos de forma rápida y directa.

5. REFERENCIA

https://learn.microsoft.com/azure/azure-sql/database/firewall-configure

https://learn.microsoft.com/azure/azure-sql/database/network-access-controls-overview

https://learn.microsoft.com/azure/container-apps/ingress-overview

QUESTION 67
You have a database that contains production data. The schema is stored in a Git repository as an SDK-style SQL database project and contains the following reference data.

Name	Data type
RefId	int identity
Code	nvarchar(10)
CreateDate	datetime2
Description	nvarchar(255)

A deployment pipeline can be rerun automatically when a transient failure occurs.
You need to deploy the reference data as part of the same CI/CD process. Rerunning the pipeline must produce the same outcome and must NOT create duplicate rows.
What should you do?

A. Add a post-deployment script that inserts reference rows by using IF NOT EXISTS or MERGE logic.
B. Restore a backup after each deployment.
C. Store the reference values in GitHub repository secrets.

Answer: A
Section: (none)

Explanation/Reference:
Q67 T02

ANSWER: A

CORRECT ANSWER:
Add a post-deployment script that inserts reference rows by using IF NOT EXISTS or MERGE logic.

EXPLANATION:

respuesta correcta:

Los proyectos SQL de estilo SDK (.sqlproj con Microsoft.Build.Sql) permiten incluir un único script de post-deployment. Este script se ejecuta automáticamente después de aplicar el plan de despliegue del esquema (DACPAC).
La documentación oficial indica expresamente que los scripts de post-deployment deben ser idempotentes (seguros de ejecutar repetidamente) y que uno de los casos de uso principales es la gestión de datos de referencia / estáticos:

Usar IF NOT EXISTS o, preferiblemente, la sentencia MERGE para insertar/actualizar solo cuando sea necesario.
De esta forma, si el pipeline se reejecuta (por fallo transitorio o de forma intencionada), no se generan filas duplicadas y el resultado final es siempre el mismo.

Por qué las demás opciones son incorrectas:

B. Restore a backup after each deployment
Restaurar un backup sobrescribe todos los datos de producción (incluyendo datos transaccionales), no es parte del proceso de despliegue de esquema/datos de referencia y destruye cualquier cambio realizado después del backup. No cumple el requisito de idempotencia controlada ni forma parte del flujo CI/CD estándar de un SQL project.
C. Store the reference values in GitHub repository secrets
Los secrets de GitHub (o Azure DevOps) están diseñados para almacenar valores sensibles (cadenas de conexión, tokens, contraseñas). No son el mecanismo adecuado ni soportado para datos de referencia de negocio. Además, no se integran automáticamente en el despliegue del DACPAC.

REFERENCIA

Fuentes oficiales exactas de Microsoft Learn:

Pre-deployment and post-deployment scripts overview
https://learn.microsoft.com/sql/tools/sql-database-projects/concepts/pre-post-deployment-scripts
Manage static data with a post-deployment script (ejemplo explícito con MERGE)
https://learn.microsoft.com/fabric/database/sql/source-control#manage-static-data-with-a-post-deployment-script

QUESTION 68
You have an Azure SQL database named ProductsDB.

You deploy Data API builder (DAB) to Azure Container Apps by using the mcr.microsoft.com/azure-databases/data-api-builder:latest image.

The container app has the following configurations:

• Secrets: mssql-connection-string, dab-config-base64
• Environment variables:
• MSSQL_CONNECTION_STRING=secretref:mssql-connection-string
• DAB_CONFIG_BASE64=secretref:dab-config-base64
• Ingress: External on port 5000

Users report that the /health endpoint returns a healthy response, but all requests that query an entity named Products fail and generate a connection error.

You confirm that the SQL login in the connection string is correct and the database exists.

You need to ensure that the container app can establish connections to the Azure SQL logical server without changing the container app deployment settings or the DAB configuration file.

What should you do on the Azure SQL logical server?

A. Create an auto-failover group for ProductsDB.

B. Run DBCC CHECKDB on ProductsDB.
C. Create a firewall rule that allows a start and end IP address of 0.0.0.0.
D. Enable FORCE_LAST_GOOD_PLAN automatic tuning for ProductsDB.

Answer: C
Section: (none)

Explanation/Reference:
Q68 T02

2. Explicación Técnica Detallada:
El endpoint /health de Data API builder (DAB) responde "healthy" porque solo verifica que el contenedor esté en ejecución y el proceso escuche en el puerto 5000. Las peticiones a la entidad Products fallan con error de conexión porque DAB (ejecutado en Azure Container Apps) no puede alcanzar el Azure SQL logical server.
Por defecto, el firewall de Azure SQL Database bloquea todas las conexiones entrantes. Cuando un servicio de Azure (como Container Apps) intenta conectar, el firewall solo permite el tráfico si existe la regla especial de servidor con rango 0.0.0.0 – 0.0.0.0. Esta regla es la que se crea automáticamente al activar la opción "Allow Azure services and resources to access this server" (nombre interno AllowAllWindowsAzureIps).
Como el escenario prohíbe modificar la configuración del Container App ni el archivo de configuración de DAB, la única acción válida en el logical server es crear esa regla de firewall. Una vez creada, el tráfico saliente de Container Apps (que sale con IPs de Azure) es aceptado y las consultas a la entidad Products funcionan.

3. Análisis de Opciones Incorrectas:
A. Create an auto-failover group for ProductsDB.
Los Auto-Failover Groups proporcionan alta disponibilidad y recuperación ante desastres entre regiones. No tienen ninguna relación con el acceso de red ni resuelven errores de conexión por firewall.
B. Run DBCC CHECKDB on ProductsDB.
DBCC CHECKDB verifica la integridad lógica y física de la base de datos. El problema reportado es de conectividad de red, no de corrupción de datos. Además, el login y la existencia de la base de datos ya están confirmados.
D. Enable FORCE_LAST_GOOD_PLAN automatic tuning for ProductsDB.
Esta opción de Automatic Tuning fuerza el uso del último plan de ejecución "bueno" para mejorar el rendimiento de consultas. No afecta en absoluto a la conectividad ni a las reglas de firewall.

4. Referencias Oficiales de Microsoft & SQL Server:
IP Firewall Rules – Azure SQL Database
https://learn.microsoft.com/en-us/azure/azure-sql/database/firewall-configure
Create a server-level firewall rule
https://learn.microsoft.com/en-us/azure/azure-sql/database/firewall-create-server-level-portal-quickstart
sp_set_firewall_rule
https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-set-firewall-rule-azure-sql-database
SQL Server troubleshooting – Data API builder
https://learn.microsoft.com/en-us/azure/data-api-builder/troubleshooting/mssql
Tutorial: Deploy Data API builder to Azure Container Apps
https://learn.microsoft.com/en-us/azure/data-api-builder/tutorial-deploy-container-app-cli

QUESTION 69
You have an Azure SQL database named SalesDB and an Azure App Service app named sales-api.
SalesDB contains a table named dbo.Customers. dbo.Customers contains two columns named CreditCardNumber and TenantId.
Currently, sales-api connects to SalesDB by using SQL authentication with stored username and password.
You need to recommend a solution that meets the following requirements:

Provides a passwordless method for sales-api to access SalesDB.
Ensures that credit card numbers are NOT stored as plain text.

What should you include in the recommendation?

A. Azure Key Vault and Always Encrypted
B. Transparent Data Encryption (TDE) and row-level security (RLS)
C. managed identities and dynamic data masking
D. managed identities and Always Encrypted

Answer: D
Section: (none)

Explanation/Reference:
Q69 T02

1. Correct Answer:

D. managed identities and Always Encrypted
2. Technical Explanation:

La aplicación sales-api (Azure App Service) se conecta actualmente a SalesDB mediante autenticación SQL (usuario + contraseña).
Para cumplir el requisito de acceso passwordless:

Se debe usar una Managed Identity (system-assigned o user-assigned) en el App Service.
Se configura autenticación de Microsoft Entra ID en Azure SQL Database y se otorga al managed identity el rol adecuado (ej. db_datareader / db_datawriter o un rol personalizado).
La cadena de conexión deja de contener credenciales y usa Authentication=Active Directory Managed Identity (o Active Directory Default).

Para cumplir el requisito de que los números de tarjeta de crédito no se almacenen en texto plano:

Se implementa Always Encrypted en la columna CreditCardNumber.
Los datos se cifran en el cliente (driver .NET) antes de enviarse a SQL Server.
La clave de cifrado se almacena en Azure Key Vault (o en el certificado del cliente).
Ni siquiera los administradores de la base de datos pueden ver el valor en claro.

Esta combinación (Managed Identity + Always Encrypted) es la solución oficial recomendada por Microsoft para este escenario.
3. Analysis of Incorrect Options:

A. Azure Key Vault and Always Encrypted
Always Encrypted es correcto para proteger la columna, pero Azure Key Vault por sí solo no proporciona el método de autenticación passwordless. Key Vault se usa normalmente para almacenar la clave de Always Encrypted, pero no sustituye a Managed Identity para la conexión.
B. Transparent Data Encryption (TDE) and row-level security (RLS)
TDE cifra los datos en reposo (archivos, backups), pero cuando se consultan los datos se desencriptan y se devuelven en texto plano. No cumple el requisito de “NOT stored as plain text” desde la perspectiva de la aplicación.
RLS controla el acceso por filas (útil con TenantId), pero no cifra datos.

C. managed identities and dynamic data masking
Managed Identity es correcto para el acceso passwordless.
Dynamic Data Masking solo enmascara el valor en los resultados de consulta para usuarios no privilegiados. Los datos siguen almacenados en texto plano en la base de datos, por lo que no cumple el segundo requisito.

4. Official Microsoft References:

https://learn.microsoft.com/en-us/azure/azure-sql/database/authentication-azure-ad-user-assigned-managed-identity
https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-database-engine
https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-keystore-

providers
https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/transparent-data-encryption
https://learn.microsoft.com/en-us/sql/relational-databases/security/dynamic-data-masking

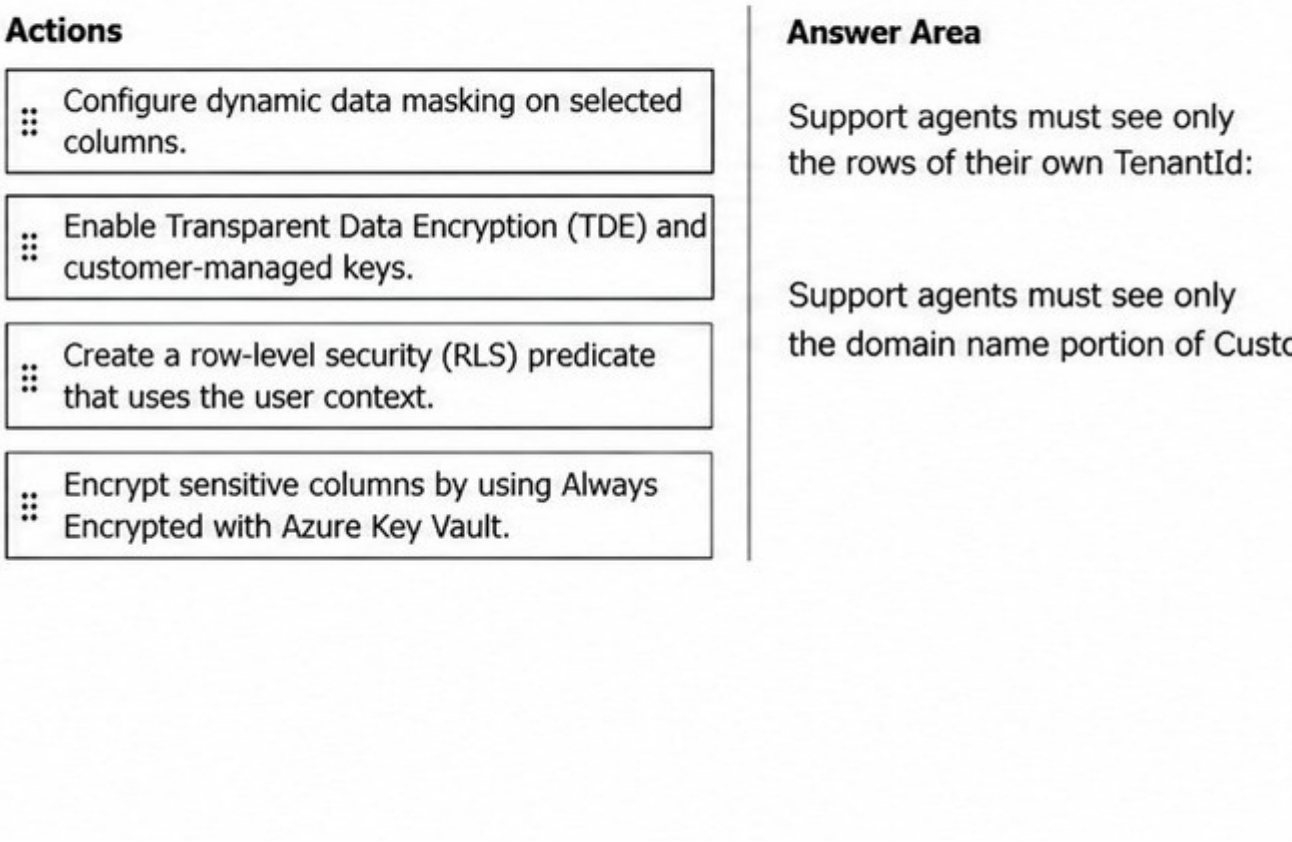
QUESTION 70
You have an Azure SQL database named sqldb-ai-prod that stores customer support tickets for a multitenant software as a service (SaaS) application. sqldb-ai-prod contains a table named Tickets. Tickets contains columns named TenantId, TicketId, CustomerEmail, CustomerPhone, and Notes.

You plan to harden data access, since a new support team will use ad hoc reporting tools that connect directly to sqldb-ai-prod.

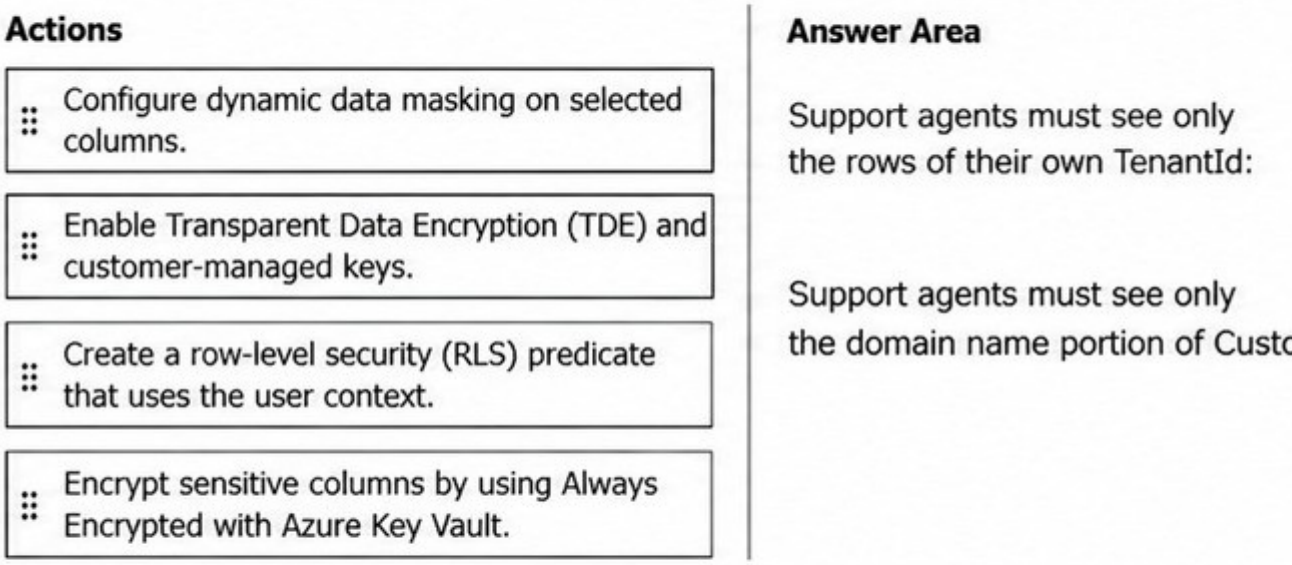
You need to configure security to meet the following requirements:

- Support agents must see only the rows of their own TenantId column.
- Support agents must see only the domain name portion of the CustomerEmail column.

What should you do for each requirement? To answer, drag the appropriate actions to the correct requirements. Each action may be used once, more than once, or not at all.



Answer:



Section: (none)

Explanation/Reference:
#Q70 T02

Correct Answer:

Support agents must see only the rows of their own TenantId: **Create a row-level security (RLS) predicate that uses the user context.**
Support agents must see only the domain name portion of CustomerEmail: **Configure dynamic data masking on selected columns.**

Explanation:

La clave está en que el primer requisito limita filas por tenant, y eso lo resuelve Row-Level Security (RLS) con un filter predicate basado en el contexto del usuario (por ejemplo SESSION_CONTEXT). El segundo requisito limita qué parte del valor de una columna se ve, y eso lo resuelve Dynamic Data Masking (DDM), que ofusca datos en el resultado de la consulta según permisos. Para mostrar solo el dominio se usa una regla de enmascarado sobre CustomerEmail, como el enmascarado basado en regex con REGEXP_REPLACE, que oculta el usuario y conserva el dominio.

Why not the others:
- TDE with customer-managed keys cifra los datos en reposo, pero no restringe lo que ve un usuario autorizado que consulta la base de datos.
- Always Encrypted cifra columnas de extremo a extremo y las herramientas de informes ad hoc no podrían mostrar el dominio. Además, no se puede definir una máscara DDM sobre columnas cifradas con Always Encrypted.

Reference:

Row-level security: https://learn.microsoft.com/sql/relational-databases/security/row-level-security?view=sql-server-ver17
Regex-based dynamic data masking in Azure SQL Database (preview): https://learn.microsoft.com/azure/azure-sql/database/dynamic-data-masking-regex?view=azuresql
Dynamic data masking (limitaciones, incluye Always Encrypted): https://learn.microsoft.com/sql/relational-databases/security/dynamic-data-masking?view=sql-server-ver17

QUESTION 71
You have a SQL database in Microsoft Fabric that uses the default settings for a newly created database and contains a table named Sales.Orders.

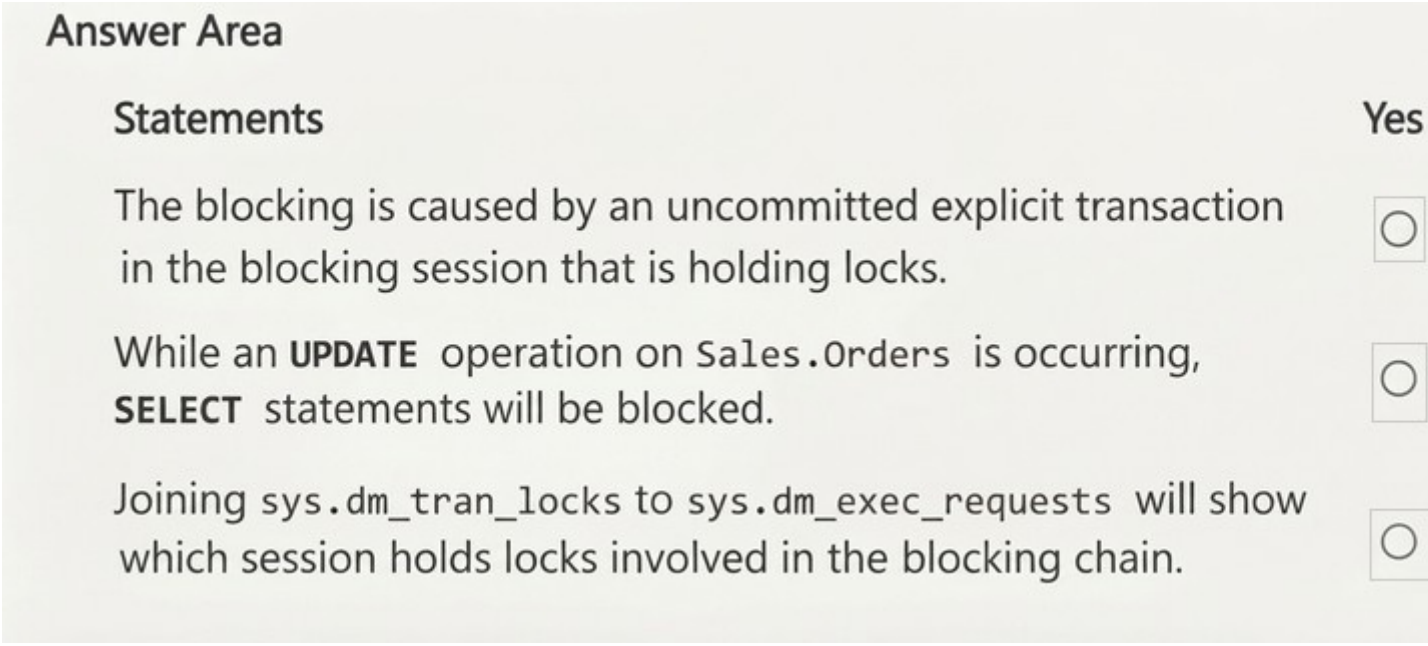
You have an application that uses two stored procedures to access Sales.Orders.

While monitoring database activity, you discover the following:

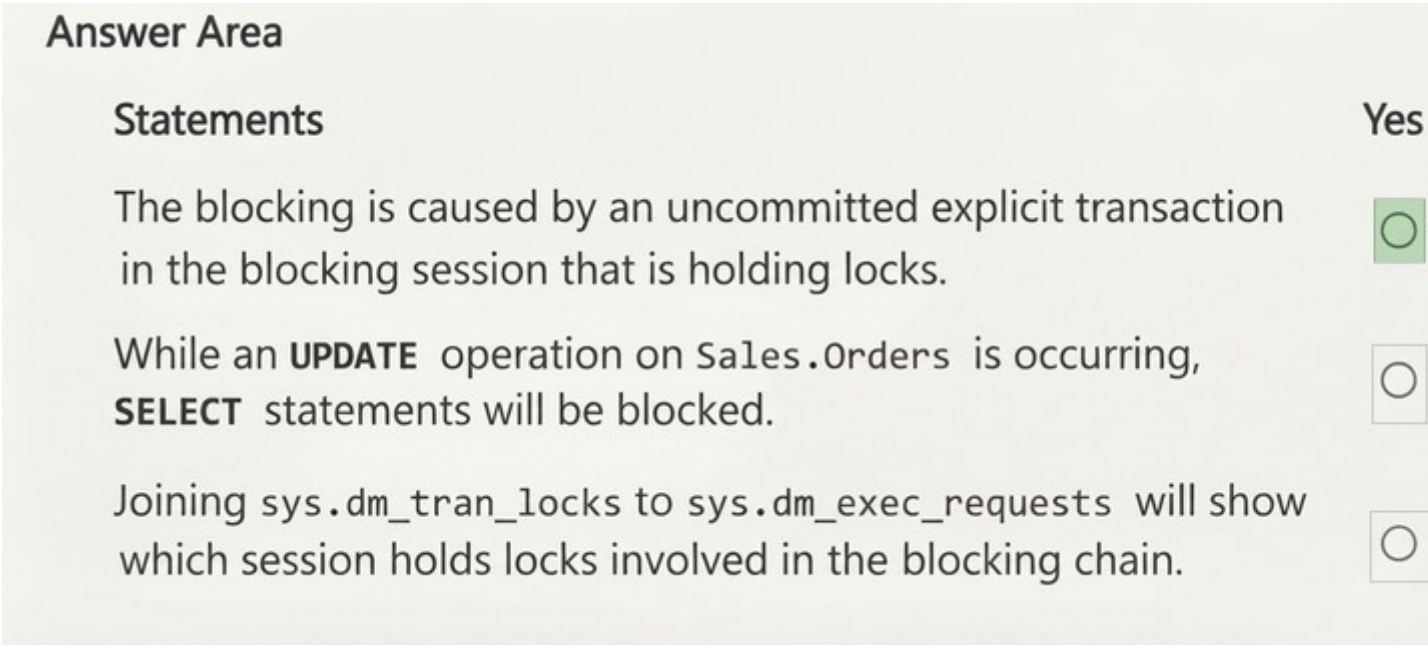
- sys.dm_exec_requests shows multiple sessions in a suspended state with wait_type = LCK_M_X. All the sessions show the same wait_resource, which maps to Sales.Orders, and the same nonzero blocking_session_id.
- sys.dm_exec_input_buffer(blocking_session_id, NULL) returns a last submitted command of BEGIN TRANSACTION UPDATE Sales.Orders.
- sys.dm_exec_sessions for the blocking session shows status = sleeping and open_transaction_count = 1

For each of the following statements, select Yes if the statement is true. Otherwise, select No

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q71 T02

Explanation

The blocking is caused by an uncommitted explicit transaction in the blocking session that is holding locks. → Yes
While an UPDATE operation on Sales.Orders is occurring, SELECT statements will be blocked. → No

Por qué es correcta la primera (Yes):

Los datos del escenario son claros:

sys.dm_exec_input_buffer muestra que la última instrucción enviada por la sesión bloqueante fue BEGIN TRANSACTION UPDATE Sales.Orders.
sys.dm_exec_sessions muestra status = sleeping y open_transaction_count = 1.

Esto es el patrón clásico de un sleeping head blocker: la sesión abrió una transacción explícita, ejecutó el UPDATE (adquiriendo locks exclusivos X), y luego se quedó en estado sleeping sin hacer COMMIT ni ROLLBACK. Mientras la transacción esté abierta, mantiene los locks, provocando que otras sesiones esperen con wait_type = LCK_M_X.

Por qué es falsa la segunda (No):

SQL database in Microsoft Fabric (basada en Azure SQL Database) tiene READ_COMMITTED_SNAPSHOT (RCSI) activado por defecto en las bases de datos nuevas.
Con RCSI:

Los lectores (SELECT) usan row versioning y no adquieren Shared (S) locks.
Por tanto, un UPDATE (que mantiene Exclusive locks) no bloquea a los SELECT.

Los sesiones suspendidas están esperando LCK_M_X (exclusive lock), lo que indica que son otras operaciones de escritura (UPDATE/INSERT/DELETE), no SELECT.

Justificación de por qué la 3 es falsa (No):
sys.dm_exec_requests solo muestra solicitudes activas (running, runnable, suspended, etc.).
En este escenario, la sesión bloqueante está en estado sleeping (status = sleeping) y tiene open_transaction_count = 1.
Las sesiones sleeping no aparecen en sys.dm_exec_requests porque no tienen ninguna request en

ejecución.
Por lo tanto, si haces un JOIN entre sys.dm_tran_locks y sys.dm_exec_requests, no vas a ver la sesión que está sosteniendo los locks (el head blocker), solo verás las sesiones que están esperando (las que están en estado suspended con LCK_M_X).
Para identificar correctamente quién sostiene los locks en este caso, hay que unir sys.dm_tran_locks con sys.dm_exec_sessions (o usar blocking_session_id desde sys.dm_exec_requests de las sesiones bloqueadas).

Reference

Troubleshoot Query Blocking in Fabric Data Warehouse (Microsoft Learn) : https://learn.microsoft.com/en-us/fabric/data-warehouse/troubleshoot-query-blocking

Understand and Resolve Blocking Problems - Azure SQL Database / SQL database in Microsoft Fabric: https://learn.microsoft.com/en-us/azure/azure-sql/database/understand-resolve-blocking?view=azuresql

Documentación de sys.dm_exec_sessions, sys.dm_exec_requests y sys.dm_exec_input_buffer (Microsoft Learn)

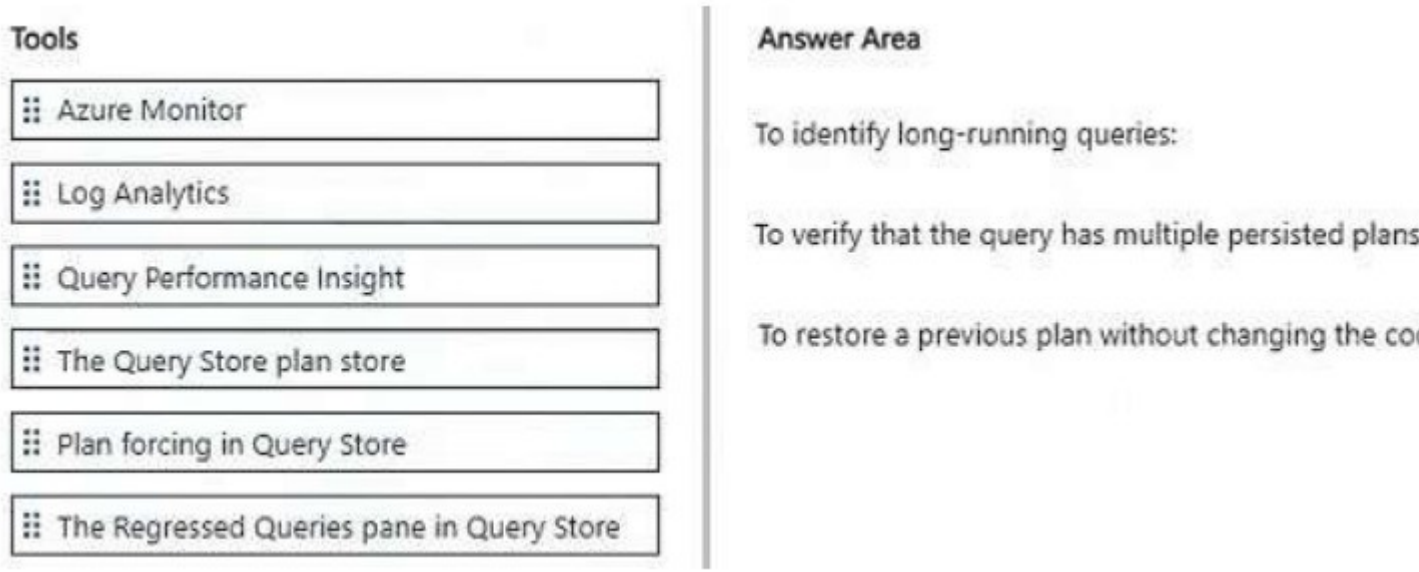
QUESTION 72
You have an Azure SQL database named SalesDB that has Query Store enabled. SalesDB supports an AI-driven product search API.

Users report that the latency of the API increased immediately after the API was deployed and remains high.

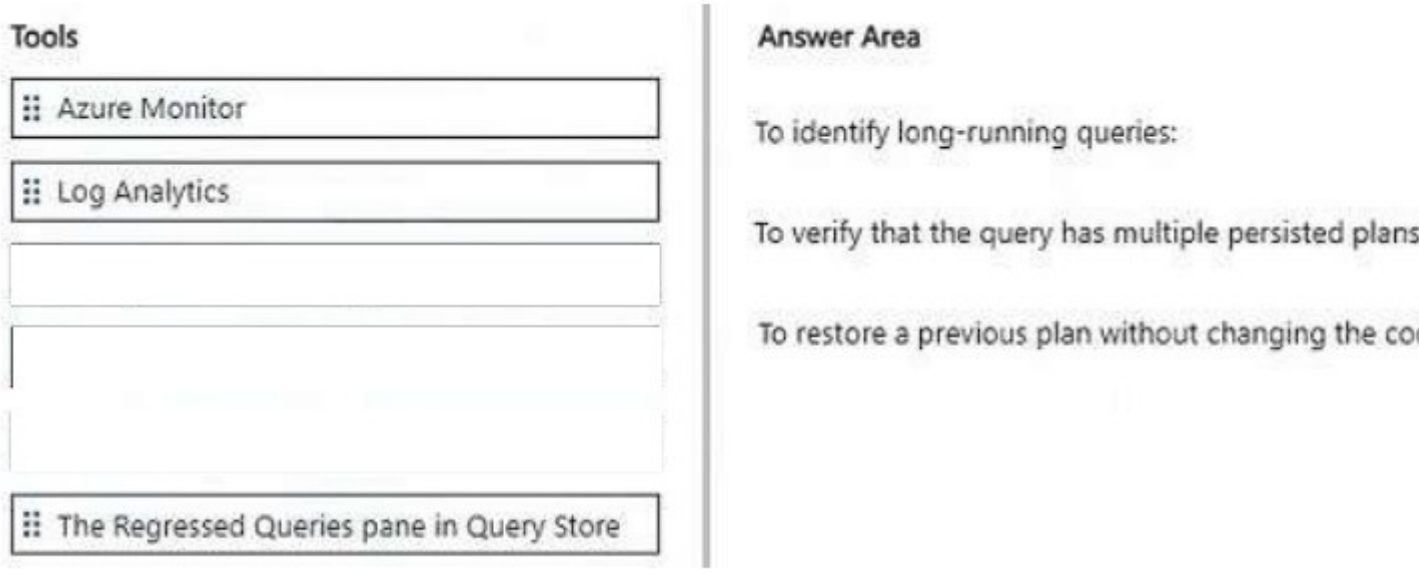
You need to identify whether the latency increase was caused by an execution plan regression and, if so, decide which corrective action will restore a previous plan. The solution must prevent any changes to the API code.

What should you use? To answer, drag the appropriate tools to the correct step. Each tool may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q72 T02

Correct Answer:
To identify long-running queries: Query Performance Insight
To verify that the query has multiple persisted plans over time: The Query Store plan store
To restore a previous plan without changing the code: Plan forcing in Query Store

Explanation:
Query Performance Insight is the right tool for finding long-running queries. In Azure SQL Database it shows the top resource-consuming and long-running queries, ranked by CPU, duration and execution count. It runs on top of Query Store, which is enabled by default.
Azure Monitor and Log Analytics are general-purpose telemetry and log tools. They can surface metrics and diagnostics, but they don't give a per-query view of durations and plans. They are also not the tool the documentation names for this step.
The Regressed Queries pane could also list slow queries, but it is meant for queries whose metrics have regressed. It is not the tool the scenario intends for this first step.
The Query Store plan store is where Query Store persists the execution plans for each query, so you can confirm that a query has more than one plan over time. Multiple plans for the same query are the sign of a plan-choice change, and the plan store's limit on plans per query is set by max_plans_per_query.
The Regressed Queries pane compares plans and shows how they performed over time, but the step asks where the multiple plans are persisted. That is the plan store.
The runtime stats store and wait stats store hold execution and wait metrics, not the plans themselves.
Azure Monitor and Log Analytics don't hold Query Store's persisted plan history.
Plan forcing in Query Store restores the earlier plan. The Query Store keeps several plans per query, so it can direct the optimizer to use a specific one. It works like a query hint but needs no change to the application, which meets the requirement to leave the API code untouched. You can only force plans that Query Store has already recorded for that query, which is why the previous plan can be restored. You do it with sp_query_store_force_plan or the Force Plan button in SSMS.
Azure Monitor, Log Analytics and Query Performance Insight are read-only monitoring tools, and none of them can change which plan the optimizer uses.
The Regressed Queries pane is where you would find the regression and click Force Plan. The action itself is plan forcing.

References:
https://learn.microsoft.com/azure/azure-sql/database/query-performance-insight-use?view=azuresql
https://learn.microsoft.com/sql/relational-databases/performance/monitoring-performance-by-using-the-query-store?view=sql-server-ver17#information-in-the-query-store
https://learn.microsoft.com/sql/relational-databases/performance/tune-performance-with-the-query-store?

view=sql-server-ver17#performance-tuning-sample-queries
https://learn.microsoft.com/sql/relational-databases/performance/query-store-usage-scenarios?view=sql-server-ver17#pinpoint-and-fix-queries-with-plan-choice-regressions

QUESTION 73
You have an Azure container app named app1-contoso-001 that hosts a Data API builder (DAB) container in front of an Azure SQL database.

You add an entity named Todo that uses a source named dbo.todos.

Which URL pattern should clients use to access the Todo REST endpoint?

A. https://app1-contoso-001.azurestaticapps.net/data-api/graphq1/Todo
B. https://app1-contoso-001.azurestaticapps.net/api/Todo
C. https://app1-contoso-001.azurestaticapps.net/data-api/Todo
D. https://app1-contoso-001.azurestaticapps.net/data-api/api/Todo

Answer: B
Section: (none)

Explanation/Reference:
Q73 T02

Correct answer: B) https://app1-contoso-001.azurestaticapps.net/api/Todo

Explanation:

When Data API builder (DAB) is hosted in a container (including Azure Container Apps), the default REST path configured in the runtime is /api. Combined with the entity name (Todo), clients reach the endpoint at /api/Todo.
The GraphQL-style path (/data-api/graphql/Todo) is incorrect because the question asks specifically for the REST endpoint, not GraphQL.

The path /data-api/Todo is incorrect because it omits the default REST root segment (/api) that DAB expects.

The path /data-api/api/Todo applies only to the Static Web Apps Database Connections feature (where a fixed /data-api prefix is added in front of the configured REST path). The scenario describes a Container App hosting a DAB container, so the Static Web Apps prefix is not used.
Reference:

https://learn.microsoft.com/azure/data-api-builder/concept/rest/overview
https://learn.microsoft.com/azure/data-api-builder/tutorial-deploy-container-app-cli
https://learn.microsoft.com/azure/data-api-builder/deployment/azure-container-apps

QUESTION 74
You have an Azure SQL database that contains a table named Customer. Customer contains the following columns:

NationalIDNumber is unique per customer.

InvestigationNotes contains free-form text.

You have a stored procedure that performs point lookups by NationalIDNumber and returns InvestigationNotes in the query results without filtering on InvestigationNotes.

You need to encrypt both columns by using Always Encrypted and the highest security possible.

Which type of Always Encrypted encryption should you use for each column?

A. deterministic for NationalIDNumber and randomized for InvestigationNotes
B. deterministic for NationalIDNumber and deterministic for InvestigationNotes
C. randomized for NationalIDNumber and randomized for InvestigationNotes
D. randomized for NationalIDNumber and deterministic for InvestigationNotes

Answer: A
Section: (none)

Explanation/Reference:
Q74 T02

Respuesta correcta: A. *deterministic for NationalIDNumber and randomized for InvestigationNotes*

Por qué es correcta la A:
En Always Encrypted (sin secure enclaves, que es el escenario estándar que asume la pregunta) hay dos tipos de cifrado:

Deterministic: genera siempre el mismo ciphertext para el mismo plaintext. Permite point lookups (búsquedas por igualdad, WHERE columna = @valor), equality joins, GROUP BY e indexación. Tiene una pequeña pérdida de información (se pueden detectar valores iguales por el patrón del ciphertext).
Randomized: genera un ciphertext diferente cada vez que se cifra el mismo valor. Ofrece mayor seguridad (no revela patrones de igualdad), pero no permite ninguna operación sobre la columna cifrada (ni equality searches, ni joins, ni indexes, ni filtros).

La pregunta dice explícitamente:
- Tienes un stored procedure que hace point lookups por NationalIDNumber.
- Devuelve InvestigationNotes sin filtrar sobre esa columna.
- Quieres la mayor seguridad posible.

Por tanto:

NationalIDNumber → deterministic (necesario para que el point lookup funcione).
InvestigationNotes → randomized (máxima seguridad, y no se necesita buscar/filtrar sobre ella).

Esto coincide exactamente con las recomendaciones oficiales de Microsoft.

Por qué las demás son incorrectas:

B: (deterministic + deterministic): InvestigationNotes no necesita deterministic. Usar deterministic en una columna de texto libre reduce la seguridad innecesariamente (permite detectar valores repetidos).
C (randomized + randomized): NationalIDNumber no podría usarse en point lookups. El stored procedure dejaría de funcionar.
D (randomized + deterministic): Al revés: NationalIDNumber no soportaría los point lookups, e InvestigationNotes tendría menos seguridad de la necesaria.

Referencias oficiales de Microsoft

- Always Encrypted overview (tipos de cifrado y operaciones soportadas):
https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-database-engine
- Recommend when to use Always Encrypted (ejemplo casi idéntico a la pregunta):
https://learn.microsoft.com/en-us/training/modules/security-azure-sql-database-azure-sql-managed-instance/9-always-encrypted
- Always Encrypted cryptography (detalle técnico de IV deterministic vs randomized):
https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-cryptography
- Tutorial oficial (ejemplo práctico con SSN = deterministic y Salary = randomized):
https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-tutorial-getting-started

QUESTION 75
Your development team uses Microsoft Visual Studio Code with the MSSQL extension and the GitHub Copilot Chat extension.

The team connects to an Azure SQL database by using individual database logins and uses the @mssql chat participant to generate and run Transact-SQL queries from prompts.

What is used to ensure that GitHub Copilot Chat-generated queries run in the context of the developer?

A. SQL permissions
B. Azure role-based access control (Azure RBAC) permissions
C. GitHub organization permissions
D. Shared MCP instruction files

Answer: A
Section: (none)

Explanation/Reference:
Q75 T02

Respuesta correcta:
SQL permissions.

Explicación:
El equipo se conecta a Azure SQL Database usando logins individuales de base de datos. Cuando un desarrollador usa el participante @mssql en GitHub Copilot Chat (dentro de la extensión MSSQL de VS Code), las consultas T-SQL que se generan y se ejecutan lo hacen bajo la conexión activa del desarrollador.
Eso implica que:

Copilot / @mssql no tiene una identidad propia ni permisos elevados.
Las consultas se ejecutan con el login/usuario de base de datos con el que el desarrollador se conectó.
Por tanto, lo que garantiza que las consultas se ejecuten “en el contexto del desarrollador” (y respeten exactamente lo que ese desarrollador puede o no puede hacer) son los SQL permissions de ese login/usuario.

Por qué no las otras opciones

Azure RBAC: controla el acceso a recursos de Azure (servidor, base de datos a nivel de Azure), no el contexto de ejecución de las consultas T-SQL una vez conectado con un login de base de datos.

GitHub organization permissions: controlan el acceso a Copilot/GitHub, no el contexto de ejecución en SQL.

shared MCP instruction files: se usan para dar contexto/instrucciones al modelo (AGENTS.md, CONSTITUTION.md, etc.), pero no definen el identity bajo el que se ejecutan las consultas.

Referencias

https://learn.microsoft.com/sql/tools/visual-studio-code-extensions/github-copilot/overview
https://learn.microsoft.com/sql/tools/visual-studio-code-extensions/github-copilot/limitations-and-known-issues
https://learn.microsoft.com/ssms/github-copilot/execution-context
https://learn.microsoft.com/ssms/github-copilot/chat

QUESTION 76

You have an Azure SQL database that supports an AI-enabled product catalog API. The database contains a table named Sales.Orders.
Sales.Orders contains 20 million rows and has a nonclustered index on a column named CreateDate.
The API passes a date parameter named @OrderDate.
You have a stored procedure named get_latest_day_orders that filters by running the following query:
WHERE CAST(CreateDate AS date) = @OrderDate.
A review of the query execution reveals that the query causes an index scan.
You need to modify the predicate to ensure that Microsoft SQL Server can use an index seek on CreateDate for a single day.
Which WHERE clause should you use?

A. WHERE CreateDate >= @OrderDate AND CreateDate < DATEADD(day, 1, @OrderDate)
B. WHERE CreateDate LIKE CONVERT(char(10), @OrderDate, 121) + '%'
C. WHERE LEFT(CONVERT(varchar(30), CreateDate, 121), 10) = CONVERT(char(10), @OrderDate, 121)
D. WHERE CONVERT(char(10), CreateDate, 121) = CONVERT(char(10), @OrderDate, 121)

Answer: A
Section: (none)

Explanation/Reference:
Q76 T02

Correct Answer: WHERE CreateDate >= @OrderDate AND CreateDate < DATEADD(day, 1, @OrderDate)

Explanation:
Para que SQL Server pueda hacer un index seek, el predicado debe ser SARGable (Search ARGument-able). Si se aplica una función o un cálculo sobre la columna indexada, el motor tiene que leer todos los valores, aplicar la función y solo después comparar. Eso produce un scan.

El predicado original, CAST(CreateDate AS date) = @OrderDate, envuelve la columna en una función. Por eso se escanean los 20 millones de filas del índice sobre CreateDate.

La opción A deja CreateDate sin tocar y mueve toda la lógica al lado del parámetro. Usa un rango semiabierto, [@OrderDate, @OrderDate + 1 día):

El optimizador puede ir directamente al primer valor >= @OrderDate en el índice no clustered y leer hacia adelante hasta < @OrderDate + 1 día.
Cubre el día completo, incluidas horas como 23:59:59.9999999. El patrón >= inicio AND < inicio + 1 día es el más seguro porque no depende de la precisión del tipo (datetime, datetime2, smalldatetime).

Por qué las otras son incorrectas:

(LIKE CONVERT(char(10), @OrderDate, 121) + '%'): LIKE es un operador de cadenas, así que CreateDate se convierte implícitamente a texto en cada fila. Eso lo hace no SARGable. El comodín al final solo ayuda cuando la columna ya es de tipo cadena.

(LEFT(CONVERT(varchar(30), CreateDate, 121), 10) = ...): aplica dos funciones (CONVERT y LEFT) sobre la columna, por lo que no es SARGable y produce un scan.

(CONVERT(char(10), CreateDate, 121) = ...): aplica CONVERT sobre la columna, así que sigue siendo no SARGable y produce un scan, con una comparación de cadenas aún más costosa que el CAST original.

Consejo de examen: cuando la pregunta pide un index seek, la respuesta es la que deja la columna "desnuda" a la izquierda de la comparación. Cualquier función o aritmética va en el lado del parámetro o la constante. Para filtrar por un día en una columna de fecha y hora, usa >= inicio AND < inicio + 1 día.

Como alternativa, si no se puede reescribir la consulta, se puede crear una columna calculada indexada con CAST(CreateDate AS date). No es una de las opciones de esta pregunta.

References:

SARGability: las funciones sobre columnas impiden el seek. Resolve blocking problems caused by lock escalation, "Prevent lock escalation": https://learn.microsoft.com/troubleshoot/sql/database-engine/performance/resolve-blocking-problems-caused-lock-escalation#prevent-lock-escalation
CONVERT() y CAST() en el WHERE causan scans; la columna calculada indexada como alternativa. Troubleshoot high-CPU-usage issues in SQL Server, Step 6: https://learn.microsoft.com/troubleshoot/sql/database-engine/performance/troubleshoot-high-cpu-usage-issues#step-6-investigate-and-resolve-sargability-issues
Ejemplos de predicados no SARGables (funciones, conversiones implícitas). Post-migration validation and optimization guide: https://learn.microsoft.com/sql/relational-databases/post-migration-validation-and-optimization-guide?view=sql-server-ver17#inability-to-use-predicates-to-filter-data
Diseño de índices y definición de columnas SARGables. SQL Server and Azure SQL index architecture and design guide: https://learn.microsoft.com/sql/relational-databases/sql-server-index-design-guide?view=sql-server-ver17#query-considerations

QUESTION 77
You need to generate embeddings to resolve the issues identified by the analysts.
Which column should you use?

Case Study Title (Case Study):
Case Study
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment
Azure Environment
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUtc (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN (nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(200)) • LastUpdatedUtc (datetime2) • EngineStatus (bit) • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:

AI workloads
Vector search
Modernized API access
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly:

SQL

SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealthFROM dbo.
VehicleHealthSummaryWHERE FleetId = @FleetIdORDER BY LastUpdatedUtc DESC;
You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.

Requirements
Planned Changes
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.

Security Requirements
Contoso identifies the following security requirements:

Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:

Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations.
latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:

Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:

Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity
Keyword filtering or boosting
Development Requirements
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.
Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:

Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.

Order the results by similarity score, with the highest score first.

A. VehicleLocation
B. IncidentDescription

C. IncidentType
D. SeverityScore

Answer: B
Section: (none)

Explanation/Reference:
Q77 - T03

RESPUESTA CORRECTA: ** IncidentDescription**

EXPLICACION:

IncidentDescription es el unico campo de **texto libre y de gran longitud (nvarchar(max))** en VehicleIncidentReports, por lo que es el campo de lenguaje natural que realmente contiene el detalle narrativo de clima, trafico y ubicacion que los analistas necesitan comparar semanticamente.

El requisito del caso de estudio es explicito en dos puntos:
- "VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details." -> el problema es de **busqueda semantica sobre texto descriptivo**, no de filtrado estructurado.
- "Embeddings must be generated from rich natural language fields." -> requisito explicito de la seccion AI Search, Embeddings, and Vector Indexing.

Revisando el esquema de VehicleIncidentReports:
- IncidentType: nvarchar(50) -> No es texto en lenguaje natural rico, es una categoria corta (taxonomia fija)
- VehicleLocation: nvarchar(200) -> No es texto en lenguaje natural rico, es un dato estructurado/geografico corto, no narrativo
- **IncidentDescription: nvarchar(max) -> SI es texto en lenguaje natural rico**, campo de texto libre, longitud ilimitada, redactado en prosa
- SeverityScore: int -> No es texto, es un valor numerico sin semantica textual

Solo IncidentDescription permite que el texto narrativo (que, segun el enunciado, suele mencionar clima, trafico y ubicacion) se convierta en un **vector semantico** con un modelo de embeddings (p. ej., text-embedding-3-large via **AI_GENERATE_EMBEDDINGS** o un endpoint de Azure OpenAI), para luego almacenarlo en una columna **VECTOR** (como ya existe el patron Embeddings vector(1536) en VehicleHealthSummary) y habilitar comparaciones por similitud coseno/producto punto entre incidentes.

POR QUE LAS OTRAS OPCIONES SON INCORRECTAS:

VehicleLocation: Es un campo corto (200 caracteres) con datos de ubicacion estructurados o semiestructurados (coordenadas, direccion), no una descripcion narrativa rica. Generar embeddings de este campo captura muy poca semantic contextual y no resuelve el problema de "encontrar incidentes similares por clima, trafico y ubicacion" descrito en prosa.

IncidentType: Es una categoria corta y acotada (nvarchar(50)), probablemente un valor de una lista fija (p. ej., "Collision", "Breakdown"). No aporta el detalle contextual necesario; ademas, para datos categoricos de baja cardinalidad, un **filtro exacto (WHERE IncidentType = ...)** es mas apropiado que una busqueda vectorial.

SeverityScore: Es un int, no texto. Los embeddings se generan a partir de lenguaje natural, no de valores numericos escalares; no existe una representacion semantica significativa para este campo mediante un modelo de embeddings de texto.

REFERENCIAS:
- Generate embeddings in SQL Server/Azure SQL Database: https://learn.microsoft.com/sql/relational-databases/vectors/vectors-sql-server
- AI_GENERATE_EMBEDDINGS (Transact-SQL): https://learn.microsoft.com/sql/t-sql/functions/ai-generate-embeddings-transact-sql
- Vector similarity search in SQL Server: https://learn.microsoft.com/sql/relational-databases/vectors/vector-search
- Exact nearest neighbor and vector indexes overview: https://learn.microsoft.com/sql/relational-databases/

vectors/vector-indexes

QUESTION 78
You need to enable similarity search to provide the analysts with the ability to retrieve the most relevant health summary reports. The solution must minimize latency.
What should you include in the solution?

Case Study Title (Case Study):
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment
Azure Environment
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUtc (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN (nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(200)) • LastUpdatedUtc (datetime2) • EngineStatus (bit) • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.

Problem Statements
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:

AI workloads

Vector search

Modernized API access

Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.

Requirements
Planned Changes
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.

Security Requirements
Contoso identifies the following security requirements:

Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.

Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.

Database Performance and Requirements
Contoso identifies the following telemetry requirements:

Telemetry data must be stored in a partitioned table.

Telemetry data must provide predictable performance for ingestion and retention operations.

latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.

Contoso identifies the following maintenance data requirements:

Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.

Avoid recursive updates.

AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:

Embeddings must be stored in dedicated Azure SQL Database tables.

Embeddings must be generated from rich natural language fields.

Chunking must preserve semantic coherence.

Hybrid search must combine the following:

Vector similarity

Keyword filtering or boosting

Development Requirements
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.
Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:

Extract the customer feedback text from the JSON document.

Filter rows where the JSON text contains a keyword.

Calculate a fuzzy similarity score between the feedback text and a known issue description.

Order the results by similarity score, with the highest score first.

A. a computed column that manually compares vector values
B. a standard nonclustered index on the Embeddings (vector (1536)) column
C. a full-text index on the Embeddings (vector (1536)) column
D. a vector index on the Embeddings (vector (1536)) column

Answer: D
Section: (none)

Explanation/Reference:
Q78 T03

Correct Answer:
a vector index on the Embeddings (vector (1536)) column

Explanation:
El requisito es buscar por similitud minimizando la latencia. CREATE VECTOR INDEX crea un índice aproximado (DiskANN) que permite búsqueda ANN con SELECT TOP (N) WITH APPROXIMATE ... FROM VECTOR_SEARCH(...). Sin él, la búsqueda es un kNN que recorre toda la tabla. Disponible de forma oficial en Azure SQL Database y en SQL database en Fabric.

a computed column that manually compares vector values. Incorrecta. Columna calculada que compara vectores: Los vectores no admiten operadores de comparación ni aritméticos, y una columna calculada no es una estructura de búsqueda. Sería una comparación fila a fila (kNN exacto), lo contrario de baja latencia.
a standard nonclustered index on the Embeddings (vector (1536)) column. Incorrecta. Índice no clustered estándar: No se pueden usar índices B-tree ni columnstore sobre columnas vector. Solo pueden ir como columna INCLUDE, lo que no ayuda a buscar por similitud, porque un vector no tiene orden ni igualdad.
a full-text index on the Embeddings (vector (1536)) column. Incorrecta. Índice full-text: Indexa texto para búsquedas por palabra clave. Embeddings es un vector(1536), no texto. Serviría para la parte de keyword de una búsqueda híbrida, no para la similitud vectorial.

References:

CREATE VECTOR INDEX: https://learn.microsoft.com/sql/t-sql/statements/create-vector-index-transact-sql?view=sql-server-ver17
VECTOR_SEARCH: https://learn.microsoft.com/sql/t-sql/functions/vector-search-transact-sql?view=sql-server-ver17
Vector search y vector indexes: https://learn.microsoft.com/sql/sql-server/ai/vectors?view=sql-server-ver17
Vector data type, sección Limitations > Indexes: https://learn.microsoft.com/sql/t-sql/data-types/vector-data-type?view=sql-server-ver17#limitations

QUESTION 79
You need to recommend a solution for the development team to retrieve the live metadata.
The solution must meet the development requirements.

What should you include in the recommendation?

Case Study Title (Case Study):
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment

Azure Environment

Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUtc (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN (nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(200)) • LastUpdatedUtc (datetime2) • EngineStatus (bit) • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
   "text": "The battery drains too fast when driving uphill.",
   "category": "Battery",
   "metadata": {
      "appVersion": "5.2.1",
      "device": "Android",
      "language": "en-GB"
   }
}
```

The support staff at Contoso never has the UNMASK permission.

Problem Statements

Contoso is deploying a new Azure SQL database that will become the authoritative data store for the

following:

AI workloads
Vector search
Modernized API access
Retrieval Augmented Generation (RAG) pipelines

Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.

The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.

VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.

Requirements

Planned Changes

Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.

Security Requirements

Contoso identifies the following security requirements:

Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.

Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.

Database Performance and Requirements

Contoso identifies the following telemetry requirements:

Telemetry data must be stored in a partitioned table.

Telemetry data must provide predictable performance for ingestion and retention operations.
latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.

Contoso identifies the following maintenance data requirements:

Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity
Keyword filtering or boosting
Development Requirements
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.

A. Export the database schema as a .dacpac file and load the schema into a GitHub Copilot context window.
B. Add the schema to a GitHub Copilot instruction file.
C. Use an MCP server.
D. Include the database project in the code repository.

Answer: C
Section: (none)

Explanation/Reference:
Q79 T03

Correct Answer

Use an MCP server.

Explanation

La respuesta correcta es Use an MCP server porque el requisito clave del escenario es que el equipo utilizará Visual Studio Code y GitHub Copilot y necesita recuperar metadatos en vivo de las bases de datos.

MCP (Model Context Protocol) proporciona una arquitectura cliente-servidor mediante la cual aplicaciones de IA, como GitHub Copilot en Agent Mode, pueden conectarse a servicios externos y utilizar herramientas proporcionadas por un servidor MCP. El servidor puede consultar directamente el sistema de origen y devolver información actualizada en el momento de la petición. Microsoft documenta específicamente la integración de servidores MCP con GitHub Copilot en Visual Studio Code. Microsoft Learn

Para este escenario resulta especialmente relevante la implementación de SQL MCP Server. Microsoft muestra que un servidor MCP conectado a una base de datos SQL puede exponerse a GitHub Copilot en Visual Studio Code y permite realizar peticiones como consultar qué tablas están disponibles. Es decir, Copilot obtiene información directamente de la base de datos en lugar de depender de una copia estática del esquema. Microsoft Learn
Además, la integración actual de GitHub Copilot con la extensión MSSQL confirma el mismo principio: cuando existe conexión con la base de datos, las herramientas de Agent Mode pueden trabajar con el esquema real y devolver información procedente de la base de datos conectada. Microsoft Learn
Respecto a las demás respuestas:

- Export the database schema as a .dacpac file and load the schema into a GitHub Copilot context window: un archivo DACPAC representa una instantánea del esquema en el momento en que se genera. Si posteriormente cambia la base de datos, el DACPAC no se actualiza automáticamente. Por tanto, no proporciona live metadata.

- Add the schema to a GitHub Copilot instruction file: los archivos de instrucciones sirven para proporcionar reglas, convenciones y contexto a Copilot. Incluir manualmente el esquema proporcionaría información estática, pero no crea una conexión con la base de datos ni recupera los cambios del esquema en tiempo real.

- Include the database project in the code repository: un database project permite mantener el modelo de la base de datos como código y controlar sus cambios mediante Git, pero representa el estado definido en el repositorio. No garantiza que ese esquema sea exactamente el que está desplegado actualmente en la base de datos y, por tanto, tampoco satisface por sí solo el requisito de recuperar metadatos en vivo.

- Use an MCP server: permite que GitHub Copilot invoque herramientas que se comunican con la fuente de datos real. Por ello es la única opción de las propuestas específicamente orientada a proporcionar acceso

dinámico a sistemas externos y recuperar información actualizada.

References

Microsoft Learn — What is the Azure MCP Server?
Explica la arquitectura MCP y cómo Visual Studio Code actúa como host, GitHub Copilot Agent Mode como cliente MCP y los servidores MCP proporcionan herramientas y acceso a recursos externos.
https://learn.microsoft.com/en-us/azure/developer/azure-mcp-server/overview

Microsoft Learn — Get started with the Azure MCP Server using Visual Studio Code
Documenta la conexión entre GitHub Copilot Agent Mode, Visual Studio Code y Azure MCP Server, permitiendo que Copilot interactúe con recursos externos mediante herramientas MCP.
https://learn.microsoft.com/en-us/azure/developer/azure-mcp-server/get-started/tools/visual-studio-code

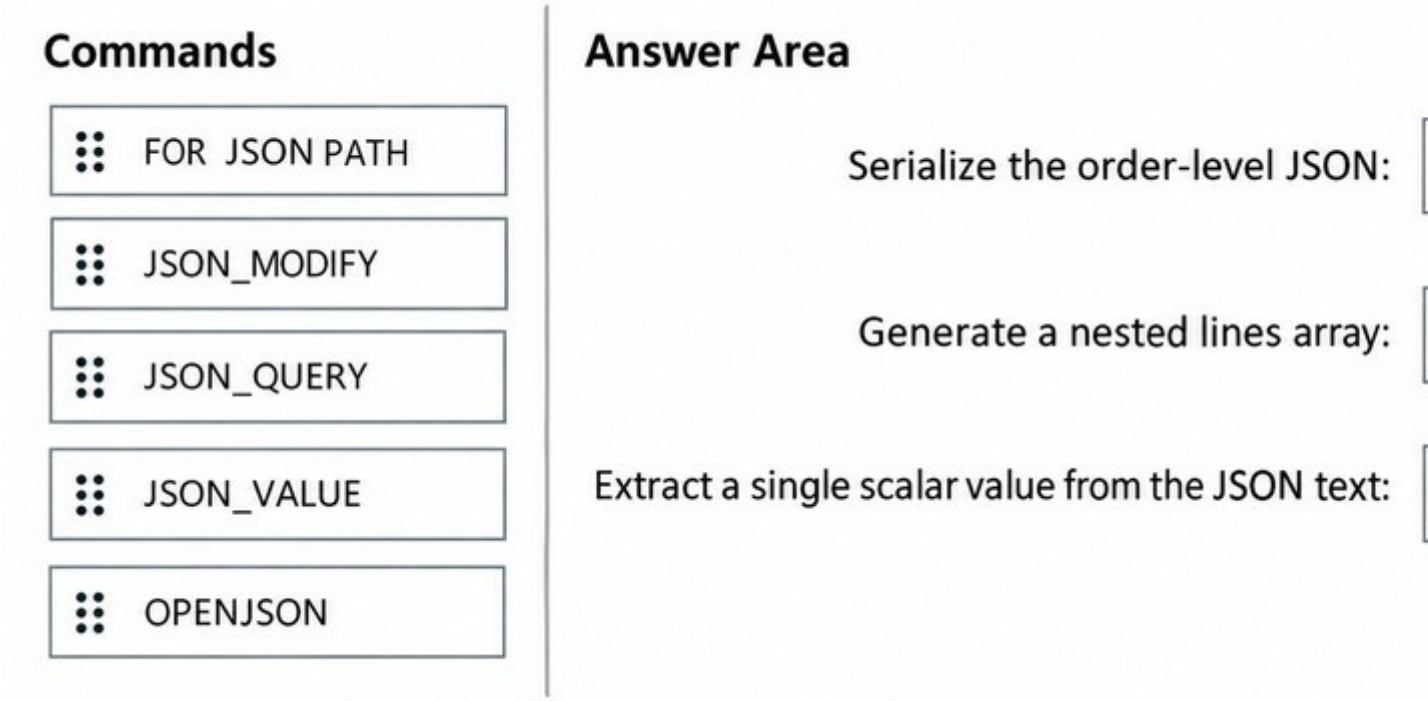
Microsoft Learn — Quickstart: Create a Hosted MCP Server in Connector Namespace
Es la referencia más directa para esta pregunta: muestra un SQL MCP Server conectado a GitHub Copilot en Visual Studio Code y utiliza como ejemplo la consulta “What tables are available?”, demostrando la recuperación de información directamente desde SQL.
https://learn.microsoft.com/en-us/azure/connector-namespace/hosted-mcp-quickstart

Microsoft Learn — How GitHub Copilot works with the MSSQL Extension
Explica que las herramientas de Agent Mode tienen conocimiento del esquema mediante llamadas a herramientas y que los resultados proceden de la base de datos conectada, a diferencia del contexto puramente estático.
https://learn.microsoft.com/en-us/sql/tools/visual-studio-code-extensions/github-copilot/how-it-works?view=sql-server-ver17

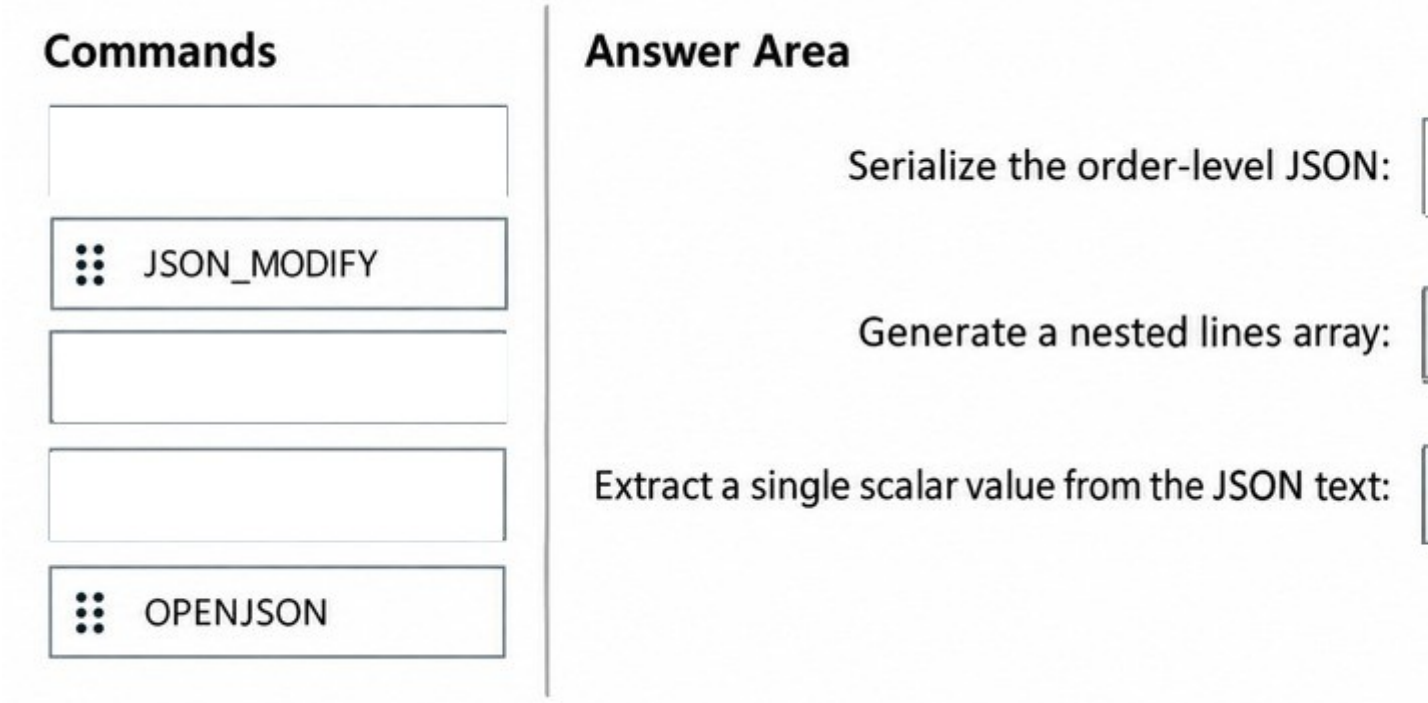
QUESTION 80
You have an Azure SQL database named SalesDB that contains tables named Sales.Order and Sales.OrderLines. Both tables contain sales data.
You have a Retrieval Augmented Generation (RAG) service that queries SalesDB to retrieve order details and passes the results to a large language model (LLM) as JSON text. The following is a simple of the JSON.


```
[
  {
    "orderHeaderId": 102348,
    "orderNumber": "SO-2026-000912",
    "orderDateUtc": "2026-01-28T14:22:09Z",
    "customerId": 77821,
    "currencyCode": "EUR",
    "orderTotal": 149.97,
    "lines": [
      {
        "lineNumber": 1,
        "productId": 5012,
        "sku": "CBL-USB-C-1M",
        "quantity": 2,
        "unitPrice": 19.99,
        "lineTotal": 39.98
      },
      {
        "lineNumber": 2,
        "productId": 8841,
        "sku": "HUB-USB-C-7P",
        "quantity": 1,
        "unitPrice": 109.99,
        "lineTotal": 109.99
      }
    ]
  }
]
```

You need to return one JSON document per order that includes the order header fields and an array of related order lines. The LLM must receive a single JSON array of orders, where each order contains a lines property that is a JSON array of line items.
Which Transact-SQL commands should you use to produce the required JSON shape from the relational tables? To answer, drag the appropriate commands to the correct operations. Each command may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q80 T03
2**. Correct Answer:**
Serialize the order-level JSON: JSON_QUERY
Generate a nested lines array: FOR JSON PATH
Extract a single scalar value from the JSON text: JSON_VALUE

3. Explanation:
Para Serialize the order-level JSON, la respuesta es JSON_QUERY. JSON_QUERY trabaja con fragmentos JSON que representan objetos o matrices. Además, cuando se combina con FOR JSON, permite que un fragmento que ya contiene JSON válido se incluya como JSON y no como una cadena de texto escapada. Esto es importante para conservar correctamente la estructura JSON que recibirá el LLM. Microsoft Learn
Para Generate a nested lines array, la respuesta es FOR JSON PATH. Esta cláusula convierte los resultados de una consulta SELECT en JSON. Por tanto, una subconsulta sobre Sales.OrderLines con FOR JSON PATH permite transformar las filas relacionadas de cada pedido en una matriz JSON de líneas, que puede incorporarse como la propiedad lines del pedido. Microsoft Learn
Para Extract a single scalar value from the JSON text, la respuesta es JSON_VALUE. Esta función está diseñada específicamente para extraer un valor escalar de un documento JSON, como un identificador, una fecha, un código o un importe. Microsoft distingue expresamente JSON_VALUE, que devuelve escalares, de JSON_QUERY, que devuelve objetos o matrices. Microsoft Learn
Las demás opciones no corresponden a estas operaciones. JSON_MODIFY se utiliza para modificar valores o propiedades de un documento JSON, no para extraer un escalar ni para generar una matriz a partir de filas relacionales. OPENJSON realiza fundamentalmente la operación inversa: analiza JSON y lo transforma en filas y columnas que pueden procesarse mediante Transact-SQL. Microsoft Learn

4. **References**:
Microsoft Learn — JSON_QUERY (Transact-SQL):
https://learn.microsoft.com/en-us/sql/t-sql/functions/json-query-transact-sql?view=sql-server-ver17
Microsoft Learn — Work with JSON data in SQL Server:
https://learn.microsoft.com/en-us/sql/relational-databases/json/json-data-sql-server?view=sql-server-ver17
Microsoft Learn — Validate, query, and change JSON data with built-in functions:
https://learn.microsoft.com/en-us/sql/relational-databases/json/validate-query-and-change-json-data-with-built-in-functions-sql-server?view=sql-server-ver17
Microsoft Learn — JSON functions (Transact-SQL):
https://learn.microsoft.com/en-us/sql/t-sql/functions/json-functions-transact-sql?view=sql-server-ver17

QUESTION 81

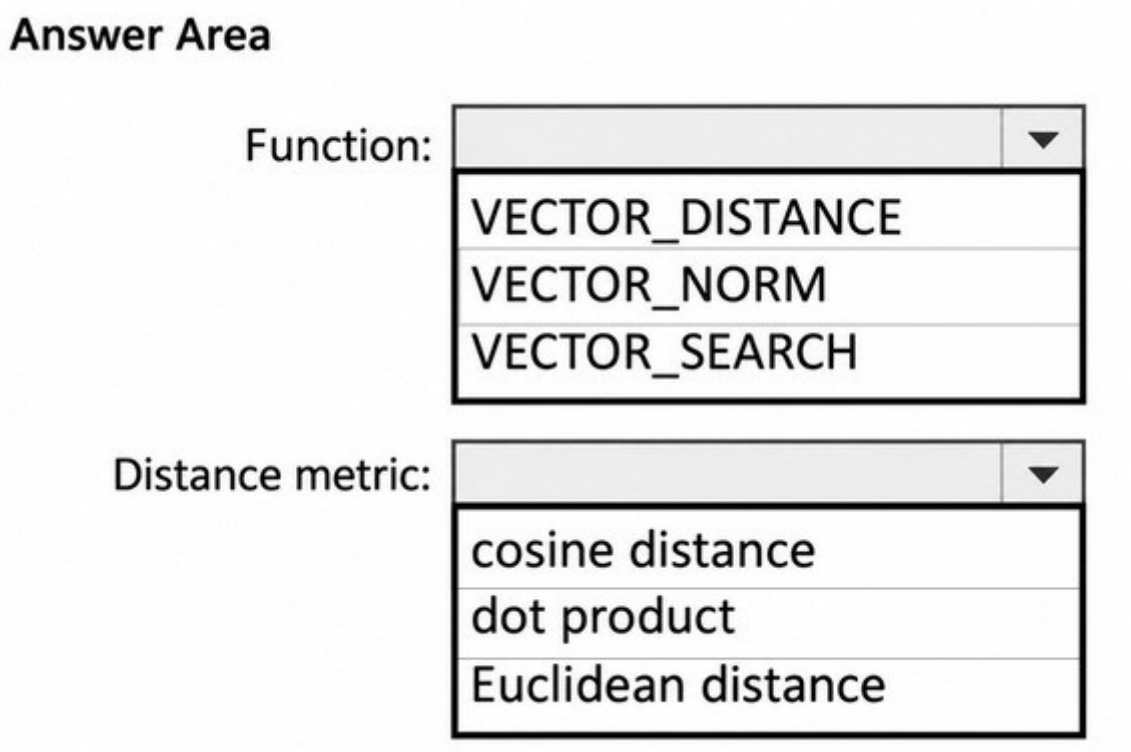
You have an Azure SQL database that contains the following tables and columns.

Tables	Columns
Articles	• ArticleId (int) • Title (nvarchar(200)) • Notes (nvarchar(max)) • Description (nvarchar(max))
NotesEmbeddings	• ArticleId (int) • ChunkIndex (int) • Embedding (vector(1536))
DescriptionEmbeddings	• ArticleId (int) • ChunkIndex (int) • Embedding (vector(1536))

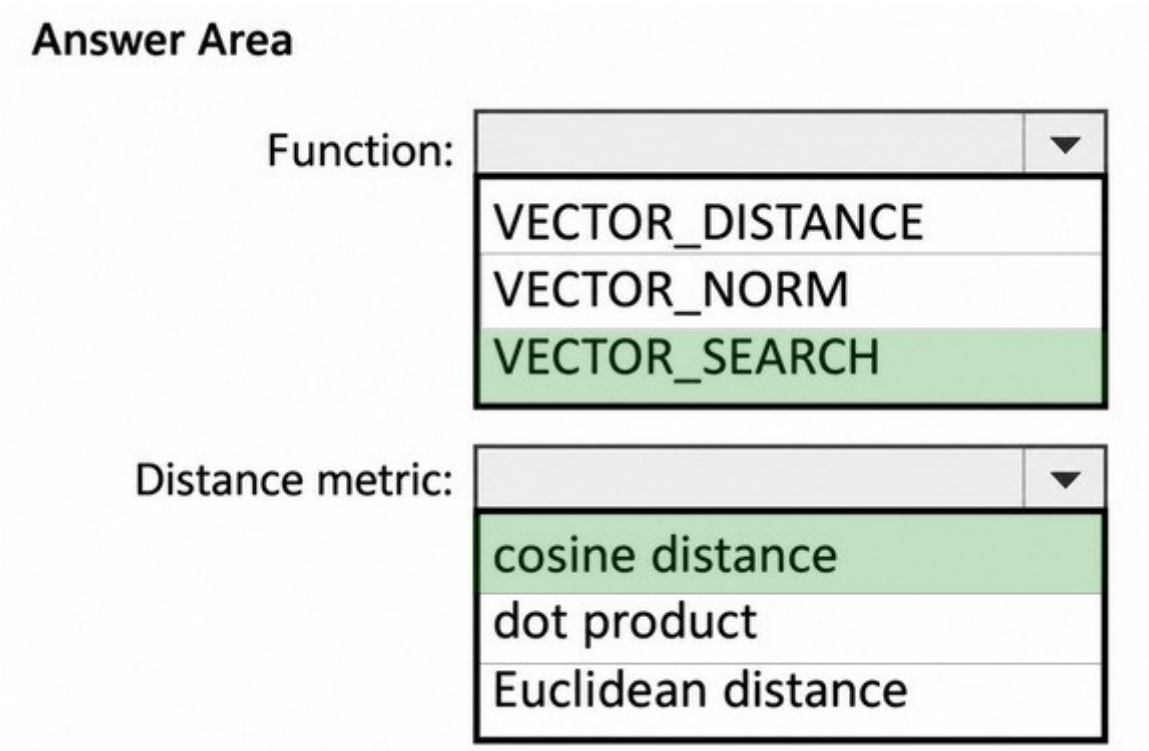
Embeddings in the NotesEmbeddings and DescriptionEmbeddings tables have been generated from values in the Description and Notes columns of the Articles table by using different chunk sizes.

You need to perform approximate nearest neighbor (ANN) queries across both embedding tables. The solution must minimize the impact of using different chunk sizes.

What should you use? To answer, select the appropriate options in the answer area.



Answer:



Section: (none)

Explanation/Reference:
Q81 T03

Respuesta Correcta:
Function: VECTOR_SEARCH

Distance metric: cosine distance

Explicación:
Para realizar consultas de Approximate Nearest Neighbor (ANN) en Azure SQL Database sobre tablas de embeddings, se debe usar la función VECTOR_SEARCH. Esta función aprovecha los índices vectoriales para devolver resultados aproximados de forma eficiente, a diferencia de VECTOR_DISTANCE (que realiza un cálculo exacto de distancia y no usa índices) o VECTOR_NORM (que solo normaliza un vector). Cuando los embeddings se generan a partir de campos de texto con diferentes tamaños de chunk (como Notes y Description), la métrica cosine distance es la más adecuada. Cosine mide la similitud angular entre vectores e ignora diferencias de magnitud que pueden introducirse por el chunking, minimizando así el impacto de usar tamaños de fragmento distintos. Dot product y Euclidean distance son más sensibles a la magnitud y no minimizan ese impacto de la misma manera.

Referencias:
https://learn.microsoft.com/en-us/sql/t-sql/functions/vector-search-transact-sql

https://learn.microsoft.com/en-us/sql/sql-server/ai/vectors

https://learn.microsoft.com/en-us/sql/t-sql/functions/vector-distance-transact-sql

QUESTION 82
HOTSPOT
You have a SQL database in Microsoft Fabric named SalesD8 that contains a table named dbo.Products.
You need to modify SalesDB to meet the following requirements:

Create a vector index on the appropriate column.

Use a supplied natural language query vector. How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

```
CREATE VECTOR INDEX idx_products_embedding ON dbo.Products (
WITH (METRIC = 'cosine', TYPE = 'DISKANN');
DECLARE @query_vector VECTOR(1536) = @SuppliedVector
SELECT
    t.product_id,
    t.product_name,
    s.distance
FROM
```

[▼] (

- VECTOR_DISTANCE
- VECTOR_NORMALIZE
- VECTOR_SEARCH

```
    TABLE = dbo.Products AS t,
    COLUMN = embedding,
    SIMILAR_TO = @query_vector,
    METRIC = 'cosine',
    TOP_N = 10
) AS s
```

[▼]

- GROUP BY s.distance
- ORDER BY s.distance
- PARTITION BY s.distance

[▼]

- distance
- embedding
- product_name

Answer:

Answer Area

```
CREATE VECTOR INDEX idx_products_embedding ON dbo.Products (
WITH (METRIC = 'cosine', TYPE = 'DISKANN');
DECLARE @query_vector VECTOR(1536) = @SuppliedVector
SELECT
    t.product_id,
    t.product_name,
    s.distance
FROM
```

[▼] (

- VECTOR_DISTANCE
- VECTOR_NORMALIZE
- VECTOR_SEARCH

```
    TABLE = dbo.Products AS t,
    COLUMN = embedding,
    SIMILAR_TO = @query_vector,
    METRIC = 'cosine',
    TOP_N = 10
) AS s
```

[▼]

- GROUP BY s.distance
- ORDER BY s.distance
- PARTITION BY s.distance

[▼]

- distance
- embedding
- product_name

Section: (none)

Explanation/Reference:
Q82 T02

Correct Answer:
embedding

VECTOR_SEARCH

ORDER BY s.distance

Explanation:
Para crear un índice vectorial sobre la columna adecuada que almacena los embeddings (columna de tipo VECTOR), se debe especificar la columna embedding en la instrucción CREATE VECTOR INDEX.
Esto coincide con la sintaxis documentada CREATE VECTOR INDEX index_name ON object (vector_column) y permite realizar búsquedas aproximadas de vecinos más cercanos utilizando el índice DiskANN con la métrica cosine.
La consulta debe utilizar la función con valores de tabla VECTOR_SEARCH para ejecutar la búsqueda aproximada de similitud vectorial sobre la columna indexada.
La función recibe la tabla, la columna vectorial, el vector de consulta suministrado, la métrica y (en la sintaxis utilizada en esta pregunta) el parámetro TOP_N, y devuelve las filas coincidentes junto con una columna calculada distance.
Por último, los resultados deben ordenarse por la distancia devuelta por VECTOR_SEARCH. La documentación indica que la columna distance es la única clave de ordenación válida para los resultados de búsqueda vectorial aproximada y que debe ordenarse en orden ascendente.
distance es incorrecta para la creación del índice porque es una columna de resultado calculada generada por VECTOR_SEARCH, no una columna que exista en la tabla.

product_name es incorrecta para la creación del índice porque es una columna de tipo cadena normal y no se puede utilizar para crear un índice vectorial.
VECTOR_DISTANCE es incorrecta porque es una función escalar que calcula la distancia entre dos vectores individuales; no realiza una búsqueda aproximada de vecinos más cercanos sobre una tabla indexada.

VECTOR_NORMALIZE es incorrecta porque es una función escalar que normaliza un único vector; no interviene en la búsqueda sobre un índice.
GROUP BY s.distance es incorrecta porque agrupar por la columna de distancia no es necesario y colapsaría las filas distintas de vecinos más cercanos.

PARTITION BY s.distance es incorrecta porque es una cláusula de función de ventana y no ordena el conjunto de resultados final.
References:
https://learn.microsoft.com/sql/t-sql/statements/create-vector-index-transact-sql

https://learn.microsoft.com/sql/t-sql/functions/vector-search-transact-sql

QUESTION 83
You have an Azure SQL database that contains a table named dbo.Products. dbo.Products contains three columns named Embedding, Category, and Price. The Embedding column is defined as VECTOR(1536).
You use AI_GENERATE_EMBEDDINGS and VECTOR_SEARCH to support semantic search and apply additional filters on two columns named Category and Price.
You plan to change the embedding model from text-embedding-ada-002 to text-embedding-3-small.
Existing rows already contain embeddings in the Embedding column.
You need to implement the model change. Applications must be able to use VECTOR_SEARCH without runtime errors.
What should you do first?

A. Regenerate embeddings for the existing rows.
B. Normalize the vector lengths before storing new embeddings.
C. Convert the Embedding column to nvarchar(max).
D. Create a vector index on dbo.Products.Embedding.

Answer: A
Section: (none)

Explanation/Reference:
Q83 T03

Respuesta correcta:
Regenerar los embeddings de las filas existentes.

Explicación:
La columna Embedding es VECTOR(1536), y tanto text-embedding-ada-002 como text-embedding-3-small devuelven vectores de 1.536 dimensiones. Por eso la columna no necesita cambios de tipo ni de tamaño. El problema es otro: los vectores que ya están almacenados los generó ada-002, y los vectores de consulta que se generarán con AI_GENERATE_EMBEDDINGS usarán text-embedding-3-small. Cada modelo produce su propio espacio vectorial, así que comparar vectores de modelos distintos no da distancias con sentido. Microsoft indica expresamente que no se puede "actualizar" entre modelos de embeddings y que, para cambiar de ada-002 a otro modelo, hay que generar embeddings nuevos. Por tanto, lo primero es apuntar el modelo externo al nuevo modelo y volver a calcular la columna Embedding de todas las filas con AI_GENERATE_EMBEDDINGS (por ejemplo, con un UPDATE sobre dbo.Products). Después, VECTOR_SEARCH compara vectores de la misma procedencia y los filtros por Category y Price se aplican sobre resultados coherentes.

Normalize the vector lengths before storing new embeddings. es incorrecta porque no hay ninguna diferencia de longitud entre los vectores que corregir: ambos modelos generan 1.536 dimensiones. Además, normalizar la longitud de un vector no lo traslada al espacio del otro modelo, así que los embeddings antiguos seguirían siendo incompatibles con los nuevos.

Convert the Embedding column to nvarchar(max). es incorrecta porque VECTOR_SEARCH exige que la columna de búsqueda sea del tipo de datos vector, y que SIMILAR_TO sea una variable o columna de tipo vector. Convertir Embedding a nvarchar(max) haría que VECTOR_SEARCH dejara de funcionar sobre esa columna y perdería las ventajas del tipo vector.

Create a vector index on dbo.Products.Embedding. es incorrecta porque un índice vectorial no es un requisito previo: VECTOR_SEARCH también funciona sin índice, haciendo una búsqueda exacta (kNN) sobre todas las filas. El índice solo acelera la búsqueda aproximada sobre los vectores almacenados. Crearlo ahora indexaría los embeddings generados con el modelo anterior y no resolvería la incompatibilidad con el nuevo modelo, así que, en todo caso, sería un paso posterior a regenerar los datos.

Referencias:
https://learn.microsoft.com/en-us/sql/t-sql/functions/ai-generate-embeddings-transact-sql
https://learn.microsoft.com/en-us/sql/t-sql/functions/vector-search-transact-sql
https://learn.microsoft.com/en-us/sql/t-sql/data-types/vector-data-type
https://learn.microsoft.com/en-us/sql/t-sql/statements/create-vector-index-transact-sql
https://learn.microsoft.com/en-us/sql/t-sql/statements/alter-external-model-transact-sql
https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/models-sold-directly-by-azure

QUESTION 84
DRAG DROP

You have an Azure SQL database named DB1 that contains two tables named knowledge_base and query_cache. knowledge_base contains support articles and embeddings. query_cache contains chat questions, responses, and embeddings. DB1 supports an AI-enabled chat agent.

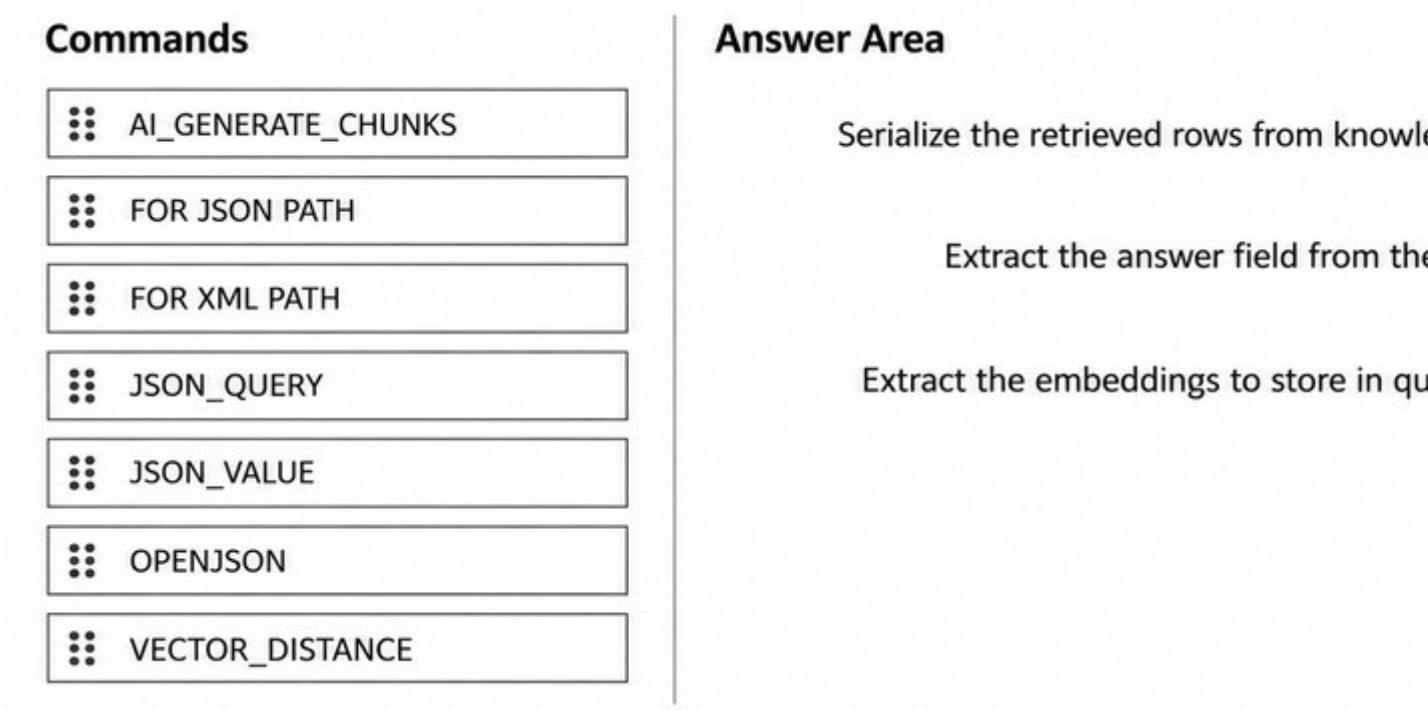
You need to design a solution that meets the following requirements:

- Serializes the retrieved rows from knowledge_base
- Extracts the answer field from the response
- Extracts the embeddings to store in query_cache
- You will call the external large language model (LLM) by using the sp_invoke_external_rest_endpoint stored procedure.

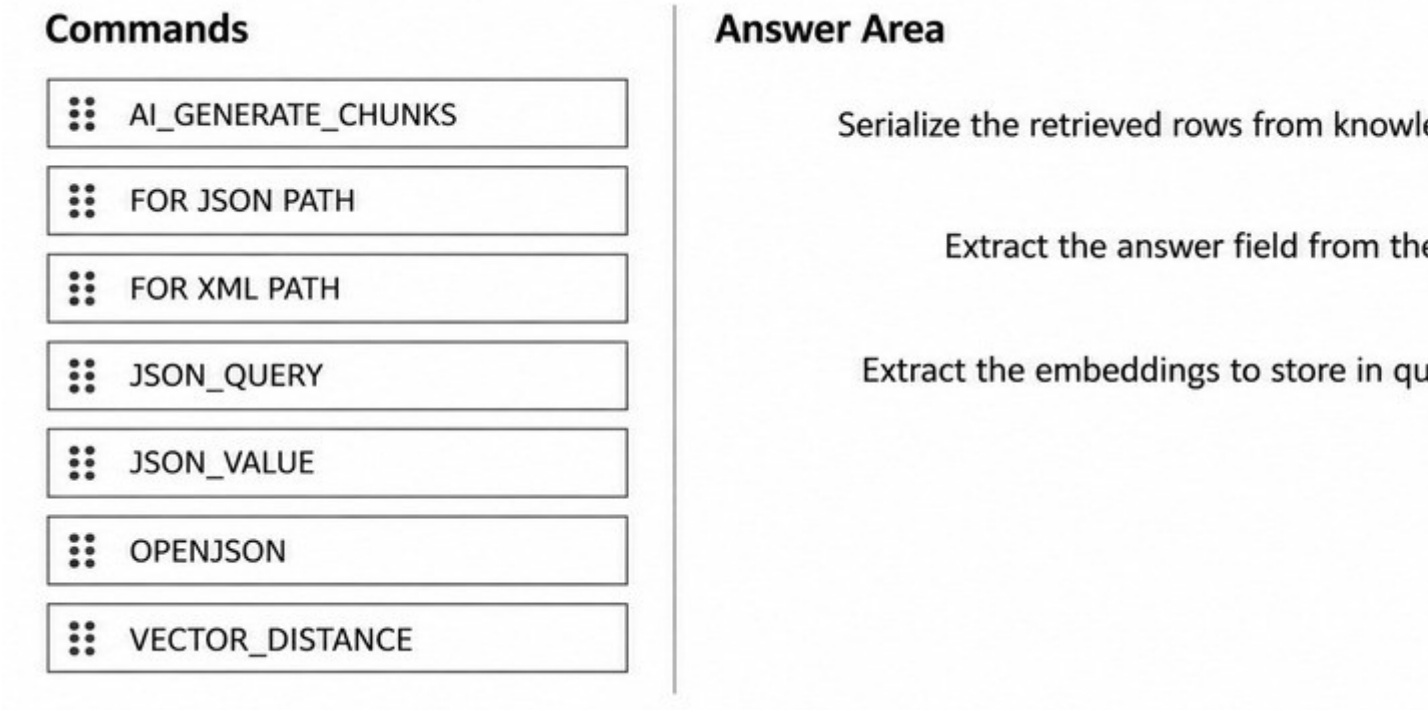
Which Transact-SQL commands should you use for each requirement? To answer, drag the appropriate commands to the correct requirements.
Each command may be used once, more than once, or not at all. You may need to drag the split bar

between panes or scroll to view content.

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
VCE 84 / Topic 03

Respuestas Correcta:

- Serialize the retrieved rows from knowledge_base: FOR JSON PATH

- Extract the answer field from the response: JSON_VALUE
- Extract the embeddings to store in query_cache: JSON_QUERY

Explanation:

Para construir el payload que se enviará al LLM mediante sp_invoke_external_rest_endpoint, hay que convertir el conjunto de filas recuperado de knowledge_base en un único documento JSON. FOR JSON PATH es la cláusula Transact-SQL diseñada precisamente para serializar los resultados de una consulta (varias filas) como un documento JSON, lo que además evita la sobrecarga de hacer una llamada HTTPS por cada fila.

Una vez que sp_invoke_external_rest_endpoint devuelve la respuesta del modelo en el parámetro @response, hay que extraer el campo de la respuesta (answer/content), que es un valor escalar (una cadena de texto). Para extraer valores escalares de un documento JSON se usa JSON_VALUE.

Por último, para extraer los embeddings que se van a guardar en query_cache, hay que tener en cuenta que un embedding no es un valor escalar, sino un array/objeto JSON (un vector de números). Cuando se necesita extraer un objeto o una matriz JSON en lugar de un valor único, la función adecuada es JSON_QUERY, mientras que JSON_VALUE fallaría o devolvería NULL al intentar extraer una estructura compleja.

Las demás opciones de la lista no encajan en ninguno de los tres requisitos:

- AI_GENERATE_CHUNKS: se usa para dividir texto en fragmentos (chunking) antes de generar embeddings, no para serializar filas ni para parsear respuestas JSON.
- FOR XML PATH: serializa resultados como XML, no como JSON, por lo que no es compatible con el formato que espera el endpoint REST.
- OPENJSON: convierte un documento JSON en un conjunto de filas (filas/columnas), es decir, hace el proceso inverso al requerido para serializar knowledge_base.
- VECTOR_DISTANCE: calcula la distancia/similitud entre dos vectores, útil para la búsqueda vectorial, pero no sirve para serializar ni para extraer campos de un JSON.

Referencias:

Microsoft Learn – sys.sp_invoke_external_rest_endpoint (Transact-SQL), apartado "Best practices – Use a batching technique": "it's recommended to batch the rows into a single JSON document... This can be done using the FOR JSON statement"; ejemplo con JSON_QUERY(@response, '$.result.data[0].embedding') para extraer un embedding de la respuesta.
https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-invoke-external-rest-endpoint-transact-sql?view=sql-server-ver17
Microsoft Learn – Training: Generar y procesar respuestas RAG (módulo "Design and implement RAG with Azure SQL"): "Para llegar allí, use JSON_VALUE, que extrae valores escalares de JSON... Si alguna vez necesita extraer un objeto JSON o una matriz en lugar de un solo valor, use JSON_QUERY en su lugar."
https://learn.microsoft.com/es-es/training/modules/design-implement-rag-with-sql/5-generate-process-rag-responses

QUESTION 85
HOTSPOT

You have an Azure AI Search service and an index named hotels that includes a vector field named DescriptionVector.
You query hotels by using the Search Documents REST API.

You add semantic ranking to the hybrid search query and discover that some queries return fewer results than expected, and captions and answers are missing.

You need to complete the hybrid search request to meet the following requirements:
Include more documents when ranking.
Always include captions and answers.

How should you complete the REST request body? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
{
    "search":  "ocean view",
    "vectorQueries": [
        {
            "vector": [/* embedding array */],
            "fields": "DescriptionVector",
            "k": [ ▼ ]
                -50,
                10,
                50,
            "kind": "vector"
        }
    ],
    "queryType": [ ▼ ]
                "full",
                "semantic",
                "simple",
    "semanticConfiguration": "hotels",
    "captions": [ ▼ ]
                "extractive",
                "generative",
                "none",
    "answers": [ ▼ ]
                "extractive",
                "generative",
                "none",
    "top": 10,
    "hybridSearch": { "maxTextRecallSize": 50 }
}
```

Answer:

Answer Area

```
{
      "search":  "ocean view",
      "vectorQueries": [
          {
                "vector": [/* embedding array */],
                "fields": "DescriptionVector",
                "k": [ -50, | 10, | 50, ]
                "kind": "vector"
          }
      ],
      "queryType": [ "full", | "semantic", | "simple", ]
      "semanticConfiguration": "hotels",
      "captions": [ "extractive", | "generative", | "none", ]
      "answers": [ "extractive", | "generative", | "none", ]
      "top": 10,
      "hybridSearch": { "maxTextRecallSize": 50 }
}
```

Section: (none)

Explanation/Reference:
Q85 T03

Answers:

1. **50**
2. **semantic**
3. **extractive**
4. **extractive**

Explanation:

1.
- "k": **50** es correcto. El ranker semántico solo reordena los 50 mejores resultados de la consulta inicial (la fusión RRF de la búsqueda de texto y la vectorial). Si k es menor, el ranker recibe menos documentos candidatos y las consultas devuelven menos resultados de los esperados. Con k = 50 se incluyen más documentos en el ranking, y es coherente con "maxTextRecallSize": 50.

- Por eso 10 no es correcto: limita los candidatos.

- -50 tampoco lo es: k debe ser un entero positivo y un valor negativo no es válido.

2.
- "queryType": "**semantic**" es correcto. Para que se aplique el reranking semántico (y se generen captions y answers) la solicitud debe usar queryType "semantic" junto con una semanticConfiguration, que aquí es "hotels".

- Con "simple" o "full" solo se usa el analizador de consultas léxico (sintaxis simple o Lucene completa) y no se invoca el ranker semántico, por lo que seguirían sin aparecer captions ni answers.

3.
- "captions": "**extractive**" es correcto. Los captions semánticos se obtienen extrayendo pasajes relevantes del contenido, y para que se devuelvan siempre hay que establecer el parámetro en "extractive".

- "none" desactiva los captions, que es justo el problema descrito.

- "generative" no es un valor admitido por este parámetro en la API de búsqueda de documentos: los valores son "extractive" y "none".

4.
- "answers": "**extractive**" es correcto. Las respuestas semánticas se extraen de los documentos mejor clasificados y solo se devuelven si answers se establece en "extractive".

- "none" no devuelve respuestas, y "generative" no es un valor admitido.

Referencias:

Microsoft Learn - Configure semantic ranker and return captions in search results (semantic ranking, queryType, captions y answers):
https://learn.microsoft.com/en-us/azure/search/semantic-how-to-query-request

Microsoft Learn - Create a hybrid query in Azure AI Search (k, maxTextRecallSize y RRF):
https://learn.microsoft.com/en-us/azure/search/hybrid-search-how-to-query

Microsoft Learn - Documents - Search Post (REST API):
https://learn.microsoft.com/en-us/rest/api/searchservice/documents/search-post

QUESTION 86
You have an Azure SQL database that contains a table named dbo.ManualChunks. dbo.ManualChunks contains product manuals.
A retrieval query already returns the top five matching chunks as nvarchar(max) text.

You need to call an Azure OpenAI REST endpoint for chat completions. The request body must include both the user question and the retrieved chunks.
You write the following Transact-SQL code.

```
01  CREATE DATABASE SCOPED CREDENTIAL AzureOpenAIHeaders
02  WITH IDENTITY = 'HTTPEndpointHeaders',
03    SECRET = N'{"api-key":"<YOUR_AZURE_OPENAI_API_KEY>"}';
04  GO
05  CREATE OR ALTER PROCEDURE dbo.AskManuals
06    . . .
07    SELECT @chunks =
08    (
09      SELECT TOP (5)
10        mc.ChunkText AS [text]
11      FROM dbo.ManualChunks AS mc
12      ORDER BY mc.Score DESC
13      FOR JSON PATH
14    );
15    SET @payload =
16    (
17      SELECT
18        'system' AS [messages[0].role],
19        'Use only the provided manual chunks.' AS [messages[0].content],
20        'user' AS [messages[1].role],
21        CONCAT(@question, CHAR(10), JSON_QUERY(@chunks)) AS [messages[1].content]
22
23    );
24
25    EXEC @retval =
26    . . .
27  END;
28  GO
```

What should you insert at line 22?

A. FOR XML AUTO, TYPE, XMLSCHEMA
B. FOR JSON AUTO, INCLUDE_NULL_VALUES
C. FOR XML PATH, INCLUDE_NULL_VALUES
D. FOR JSON PATH, WITHOUT_ARRAY_WRAPPER

Answer: D
Section: (none)

Explanation/Reference:
Q86 T03

Respuesta

La respuesta correcta es:

FOR JSON PATH, WITHOUT_ARRAY_WRAPPER

Solución

En la línea 17-23 se está construyendo el @payload que debe enviarse a la API REST de Azure OpenAI para chat completions. El cuerpo (body) de una solicitud a esa API tiene este formato:

```
{
  "messages": [
    { "role": "system", "content": "..." },
    { "role": "user", "content": "..." }
```

```
  ]
}
```

Es decir, debe ser un único objeto JSON, no un array de objetos. Al usar la sintaxis de alias con notación de punto ([messages[0].role], [messages[1].content], etc.) junto con FOR JSON PATH, SQL Server construye automáticamente la estructura anidada messages: [...] a partir de los nombres de columna.

El problema es que, por defecto, FOR JSON PATH envuelve siempre el resultado entre corchetes [ ] (como si fuera un array), incluso cuando solo hay una fila. Como aquí SELECT devuelve una sola fila (es un SELECT sin agrupamiento, construyendo directamente el payload), hace falta eliminar ese wrapper de array externo para obtener un objeto JSON válido tal como lo espera la API de Azure OpenAI.

Por eso se necesita la opción WITHOUT_ARRAY_WRAPPER, que elimina los corchetes que rodean la salida de FOR JSON por defecto, generando un único objeto JSON como salida en lugar de un array.

Explicación
Por qué es correcta
FOR JSON PATH permite construir JSON anidado usando notación de puntos en los alias de columna (p. ej. [messages[0].role]), lo cual es justo lo que se necesita para construir la estructura messages: [{role, content}, {role, content}].
WITHOUT_ARRAY_WRAPPER quita los corchetes [ ] que FOR JSON añade por defecto, produciendo un objeto JSON único en vez de un array. Esto es necesario porque el body de la API de Azure OpenAI espera un objeto JSON ({...}), no un array ([{...}]).
Ejemplo de la documentación oficial: una consulta con FOR JSON PATH, WITHOUT_ARRAY_WRAPPER sobre una fila produce {"year":2015,"month":12,"day":15} en lugar de [{"year":2015,"month":12,"day":15}].

Por qué las otras opciones son incorrectas

FOR XML AUTO, TYPE, XMLSCHEMA: Genera XML, no JSON. La API de Azure OpenAI espera un cuerpo en formato JSON, así que esta opción no serviría en absoluto.

FOR JSON AUTO, INCLUDE_NULL_VALUES: FOR JSON AUTO determina la estructura del JSON automáticamente a partir de la jerarquía de tablas/columnas del SELECT, en lugar de dejar control total como PATH. Aquí se necesita PATH para poder construir manualmente el array anidado messages[0], messages[1] mediante los alias. Además, INCLUDE_NULL_VALUES solo sirve para incluir propiedades con valor NULL, no resuelve el problema del wrapper de array.

FOR XML PATH, INCLUDE_NULL_VALUES: De nuevo genera XML en lugar de JSON, por lo que no es compatible con el formato que espera la API REST.
Resultado esperado

Con FOR JSON PATH, WITHOUT_ARRAY_WRAPPER, el @payload resultante sería un objeto JSON único como:

```
{
  "messages": [
    { "role": "system", "content": "Use only the provided manual chunks." },
    { "role": "user", "content": "<pregunta del usuario>\n<chunks recuperados>" }
  ]
}
```

Esto coincide exactamente con el formato que exige el endpoint de chat completions de Azure OpenAI.

Referencias
SELECT - FOR clause (Transact-SQL)
https://learn.microsoft.com/sql/t-sql/queries/select-for-clause-transact-sql?view=sql-server-ver17#for-json

Describe las opciones AUTO, PATH, INCLUDE_NULL_VALUES, ROOT y WITHOUT_ARRAY_WRAPPER de la cláusula FOR JSON.
Remove Square Brackets from JSON - WITHOUT_ARRAY_WRAPPER Option

https://learn.microsoft.com/sql/relational-databases/json/remove-square-brackets-from-json-without-array-wrapper-option?view=sql-server-ver17

Ejemplos de uso de WITHOUT_ARRAY_WRAPPER para producir un único objeto JSON en lugar de un array.
Format query results as JSON with FOR JSON
https://learn.microsoft.com/sql/relational-databases/json/format-query-results-as-json-with-for-json-sql-server?view=sql-server-ver17

Explica las opciones adicionales de FOR JSON, incluida WITHOUT_ARRAY_WRAPPER, usada "para generar un único objeto JSON como salida de un resultado de una sola fila".
Get started with JSON features (Azure SQL)
https://learn.microsoft.com/azure/azure-sql/database/json-features?view=azuresql#format-relational-data-in-json-format

Muestra ejemplos prácticos de FOR JSON PATH, WITHOUT_ARRAY_WRAPPER para devolver un único objeto JSON.

QUESTION 87
HOTSPOT
You have an Azure SQL database that contains a table named knowledge_base. knowledge_base stores human resources (HR) policy documents
and contains columns named title, content, category, and embedding.

You have an application named App1. App1 queries two relational tables named employee_profiles and benefits_enrollment that contain HR data.
App1 hosts a chatbot that calls a large language model (LLM) directly.

Users report that the chatbot answers general HR questions correctly but provides outdated or incorrect answers when policies change. The
chatbot also fails to answer questions that reference internal policy documents by title or category.
You need to recommend a Retrieval Augmented Generation (RAG) solution to resolve the chatbot issues.

What should you recommend? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

Retrieve grounding data from:
employee_profiles and benefits_enrollment
knowledge_base
PDF exports of the policies
The LLM training data

Inference step to perform the retrieval:
Perform keyword searches.
Call the LLM first, and then store the response.
Fine-tune the LLM by using the data in knowledge_base
Generate query embeddings, and then run a vector sim

Answer:

Answer Area

Retrieve grounding data from:
employee_profiles and benefits_enrollment
knowledge_base
PDF exports of the policies
The LLM training data

Inference step to perform the retrieval:
Perform keyword searches.
Call the LLM first, and then store the response.
Fine-tune the LLM by using the data in knowledge_base
Generate query embeddings, and then run a vector sim

Section: (none)

Explanation/Reference:
#Q87-T03
Respuesta
Retrieve grounding data from: knowledge_base
Inference step to perform the retrieval: Generate query embeddings, and then run a vector similarity search.

Explicación

Por qué es correcta. El chatbot falla porque llama al LLM directamente, sin contexto. Solo conoce lo que vio en su entrenamiento, así que se equivoca cuando cambian las políticas y no puede responder sobre documentos internos. RAG resuelve esto recuperando datos de tu base antes de generar la respuesta, de modo que esta se apoya en datos actuales y propios (según Microsoft Learn).

Fuente: knowledge_base ya guarda las políticas de RRHH (title, content, category) y tiene una columna embedding. Es la fuente de conocimiento ya preparada para búsqueda semántica. Si se actualiza una política, se actualiza la fila y el chatbot responde con lo nuevo, sin reentrenar nada.
Recuperación: el flujo RAG típico es fragmentar, generar embeddings, almacenarlos y, en la consulta, vectorizar la pregunta con el mismo modelo y buscar los fragmentos más similares. Después se añade ese contexto al prompt y se genera la respuesta. Azure SQL admite el tipo vector y la función de distancia vectorial para esto. Que la pregunta use palabras distintas a las del documento no importa, porque se compara el significado.

Por qué las demás no son correctas.

employee_profiles y benefits_enrollment: son datos transaccionales de empleados, no documentos de políticas. Sirven para enriquecer el contexto con un join, pero no para responder preguntas sobre políticas ni sobre documentos por título o categoría.
PDF exportados de las políticas: duplican el contenido que ya está en knowledge_base. Habría que extraer, fragmentar y vectorizar de nuevo, y se desincronizarían de la base.
Datos de entrenamiento del LLM: son justo la causa del problema, porque están desactualizados y no contienen documentos internos.
Búsqueda por palabras clave: es una recuperación válida, pero no aprovecha la columna embedding y falla con sinónimos o preguntas formuladas distinto. Los embeddings son lo que RAG usa aquí.
Llamar primero al LLM y guardar la respuesta: invierte el patrón. La recuperación debe ir antes de la generación, no después.
Ajustar (fine-tune) el LLM con knowledge_base: es costoso y deja el conocimiento fijado en el modelo, así que se queda obsoleto en cuanto cambia una política. Además no es una etapa de recuperación.
Referencias

Microsoft Learn:

Intelligent applications and AI (Azure SQL Database): https://learn.microsoft.com/azure/azure-sql/database/ai-artificial-intelligence-intelligent-applications?view=azuresql
Use SQL database in AI applications (patrón RAG, Fabric): https://learn.microsoft.com/fabric/database/sql/use-case-ai-application
Retrieval-augmented generation (RAG) in Azure AI Search: https://learn.microsoft.com/azure/search/retrieval-augmented-generation-overview

SQL Server:

Vector search and vector indexes in the SQL Database Engine: https://learn.microsoft.com/sql/sql-server/ai/vectors?view=sql-server-ver17

GB English version
Answer
Retrieve grounding data from: knowledge_base
Inference step to perform the retrieval: Generate query embeddings, and then run a vector similarity search.
Explanation

Why it's correct. The chatbot fails because it calls the LLM directly with no context. It only knows what it saw

during training, so it gives wrong answers when policies change and can't answer questions about internal documents. RAG fixes this by retrieving data from your own database before generating the answer, so the response rests on current, domain-specific facts (per Microsoft Learn).

Source: knowledge_base already stores the HR policies (title, content, category) and has an embedding column. It is the knowledge source already prepared for semantic search. When a policy changes, you update the row and the chatbot answers with the new content, with no retraining.
Retrieval: the typical RAG flow is chunk, embed, store, then at query time embed the question with the same model and find the most similar chunks. That context is added to the prompt and the answer is generated. Azure SQL supports the vector data type and a vector distance function for this. It doesn't matter if the question uses different words than the document, because meaning is compared.

Why the others are wrong.

employee_profiles and benefits_enrollment: these are transactional employee data, not policy documents. They can enrich the context through a join, but they can't answer questions about policies or documents by title or category.
PDF exports of the policies: they duplicate what's already in knowledge_base. You would have to extract, chunk and vectorize again, and they would drift out of sync with the database.
The LLM training data: this is the cause of the problem, since it's outdated and contains no internal documents.
Keyword searches: a valid retrieval method, but it ignores the embedding column and misses synonyms and differently phrased questions. Embeddings are what RAG uses here.
Call the LLM first, then store the response: this reverses the pattern. Retrieval must come before generation, not after.
Fine-tune the LLM with knowledge_base: it's costly and bakes knowledge into the model, so it goes stale as soon as a policy changes. It is also not a retrieval step.
References

Microsoft Learn:

Intelligent applications and AI (Azure SQL Database): https://learn.microsoft.com/azure/azure-sql/database/ai-artificial-intelligence-intelligent-applications?view=azuresql
Use SQL database in AI applications (RAG pattern, Fabric): https://learn.microsoft.com/fabric/database/sql/use-case-ai-application
Retrieval-augmented generation (RAG) in Azure AI Search: https://learn.microsoft.com/azure/search/retrieval-augmented-generation-overview

SQL Server:

Vector search and vector indexes in the SQL Database Engine: https://learn.microsoft.com/sql/sql-server/ai/vectors?view=sql-server-ver17

QUESTION 88
HOTSPOT

Your company has an ecommerce catalog in a Microsoft SQL Server 2025 database named SalesDB. SalesDB contains a table named products.

products contains the following columns:
product_id (int)
product_name (nvarchar(200))
description (nvarchar(max))
category (nvarchar(50))
brand (nvarchar(50))
price (decimal)
sku (nvarchar(40))

The description fields are updated daily by a content pipeline, and price can change multiple times per day.

You want customers to be able to submit natural language queries and apply structured filters for brand and price.

You plan to store embeddings in a new VECTOR(1536) column and use VECTOR_SEARCH(... METRIC='cosine' ...).

For each of the following statements, select Yes if the statement is true. Otherwise, select No.

NOTE: Each correct selection is worth one point.

Answer Area

Statements	
Generating an embedding by concatenating product_name, category, ar will support the customer requirements.	
Including price in the text used to generate embeddings is required.	
The underlying base type of the embeddings will be float(32).	

Answer:

Answer Area

Statements	
Generating an embedding by concatenating product_name, category, ar will support the customer requirements.	
Including price in the text used to generate embeddings is required.	
The underlying base type of the embeddings will be float(32).	

Section: (none)

Explanation/Reference:
Q88 T03

Correct Answer
Generar el embedding concatenando product_name, category y description cubre los requisitos --> Yes
Incluir price en el texto usado para generar los embeddings es obligatorio --> No
El tipo base de los embeddings será float(32) --> Yes

Explanation
Statement 1: Yes. La búsqueda en lenguaje natural necesita que el vector capture el significado del producto, y ese significado está en las columnas de texto descriptivo. La documentación de vectores de SQL contempla dos opciones cuando hay varias columnas: un embedding por columna, o concatenarlas en una sola cadena y generar un único embedding. brand no hace falta en el texto porque el requisito lo pide como filtro estructurado, no como criterio semántico.
Statement 2: No. El precio se resuelve con un predicado relacional en el WHERE, que es exacto y puede

usar un índice B-tree. Meterlo en el embedding es contraproducente por dos motivos:

- Un embedding no representa bien comparaciones numéricas ni rangos como "menos de 50".
- El precio cambia varias veces al día, así que habría que regenerar el vector en cada cambio, con el coste de llamadas al modelo que eso supone.

La documentación describe justo este patrón: embeddings y datos relacionales viven en la misma base de datos y se combinan similitud vectorial y filtros WHERE en la misma consulta.
Statement 3: Yes. VECTOR(1536) no especifica tipo base, y la documentación indica que por defecto es float32 (precisión simple, 4 bytes por elemento). float16 solo se usa si se declara explícitamente como VECTOR(1536, float16), y en SQL Server 2025 requiere además PREVIEW_FEATURES. Las 1.536 dimensiones caben en el máximo de 1.998 de float32.

```
-- Columna de embeddings: base type float32 por defecto
ALTER TABLE dbo.products ADD embedding VECTOR(1536) NULL;
-- Embedding solo con el texto semántico (sin price ni brand)
UPDATE dbo.products
SET embedding = AI_GENERATE_EMBEDDINGS(
   CONCAT_WS(N' | ', product_name, category, description)
   USE MODEL MyEmbeddingModel);
-- Consulta: similitud semántica + filtros estructurados
DECLARE @qv VECTOR(1536) =
   AI_GENERATE_EMBEDDINGS(N'zapatillas ligeras para correr por montaña' USE MODEL
MyEmbeddingModel);
SELECT TOP (10) p.product_id, p.product_name, p.brand, p.price, r.distance
FROM VECTOR_SEARCH(
      TABLE = dbo.products AS p,
      COLUMN = embedding,
      SIMILAR_TO = @qv,
      METRIC = 'cosine',
      TOP_N = 100          -- sobremuestreo: el filtro se aplica después
   ) AS r
WHERE p.brand = N'Contoso' AND p.price <= 120
ORDER BY r.distance;
```

Notas críticas

- **Redacción de float(32)**. La pregunta se refiere al tipo base float32 del vector. Tomado literalmente como tipo escalar de T-SQL, float(32) sería un valor de 8 bytes (doble precisión, porque n está entre 25 y 53). Es un defecto de redacción; la respuesta defendible sigue siendo Yes.
- **Post-filtering en SQL Server 2025**. Según la documentación actual, la versión más reciente del índice vectorial (filtrado iterativo, DML completo, sintaxis TOP (N) WITH APPROXIMATE) está en Azure SQL Database, SQL database in Fabric y Managed Instance con Always-up-to-date. En SQL Server 2025 el índice y VECTOR_SEARCH siguen en preview, y con la versión anterior el WHERE se aplica después de recuperar los TOP_N vecinos. Por eso el ejemplo sobremuestrea.
- **Tabla de solo lectura**. Con esa versión anterior, una tabla con índice vectorial queda en solo lectura salvo que se active ALLOW_STALE_VECTOR_INDEX. Como price cambia varias veces al día, en producción conviene separar los embeddings en una tabla propia (product_embeddings) y unirla por product_id.
- **Regeneración diaria**. Cuando el pipeline actualiza description, hay que regenerar el embedding de esas filas; un vector desactualizado devuelve resultados semánticos incorrectos sin dar error.

References
https://learn.microsoft.com/es-es/sql/t-sql/data-types/vector-data-type?view=sql-server-ver17&tabs=csharp
https://learn.microsoft.com/es-es/sql/t-sql/data-types/vector-data-type-half-precision-float?view=sql-server-ver17
https://learn.microsoft.com/es-es/sql/t-sql/functions/vector-search-transact-sql?view=sql-server-ver17
https://learn.microsoft.com/es-es/sql/t-sql/statements/create-vector-index-transact-sql?view=sql-server-ver17
https://learn.microsoft.com/es-es/sql/sql-server/ai/vectors-frequently-asked-questions?view=sql-server-ver17

QUESTION 89
You need to design a generative AI solution that uses a Microsoft SQL Server 2025 database named DB1 as a data source. The solution must generate responses that meet the following requirements:

Are grounded in the latest transactional and reference data stored in DB1
Do NOT require retraining or fine-tuning the language model when the data changes
Can include citations or references to the source data used in the response
Which scenario is the best use case for implementing a Retrieval Augmented Generation (RAG) pattern?
More than one answer choice may achieve the goal. Select the BEST answer.

A. summarizing free-form user input text
B. training a custom language model on historical database data
C. answering user questions based on company-specific knowledge
D. generating marketing slogans based on user sentiment analysis

Answer: C
Section: (none)

Explanation/Reference:
Q89 / T03

CORRECT ANSWER:
Answering user questions based on company-specific knowledge

EXPLANATION:
• answering user questions based on company-specific knowledge es correcta: RAG recupera información relevante de DB1 y la proporciona al
modelo como contexto para responder preguntas sobre datos específicos de la
empresa. Permite utilizar datos actualizados sin reentrenar ni ajustar el modelo y
puede incluir referencias a las fuentes utilizadas.

• summarizing free-form user input text es incorrecta: Resumir texto proporcionado por el usuario no requiere recuperar
conocimiento empresarial desde DB1.
• training a custom language model on historical database data es incorrecta: Entrenar un modelo con datos históricos no cumple el requisito de
utilizar información actualizada sin reentrenamiento.
• generating marketing slogans based on user sentiment analysis es incorrecta: Generar eslóganes a partir del sentimiento del usuario no es el
caso que mejor encaja con los requisitos de conocimiento empresarial actualizado y
referencias.

REFERENCES:
https://learn.microsoft.com/es-es/dotnet/ai/conceptual/rag
https://learn.microsoft.com/es-es/azure/databricks/agents/retrieval-augmented-generation

QUESTION 90
HOTSPOT

You have an Azure SQL database that contains a table named stores. stores contains a column named description and a vector column named embedding.

You need to implement a hybrid search query that meets the following requirements:
- Uses full-text search on description for the keyword portion
- Returns the top 20 results based on a combined score that uses a weighted formula of 60% vector distance and 40% full-text rank

How should you configure the query components?

To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Semantic query operator/function:

- VECTOR_DISTANCE an
- VECTOR_SEARCH with
- VECTORPROPERTY to

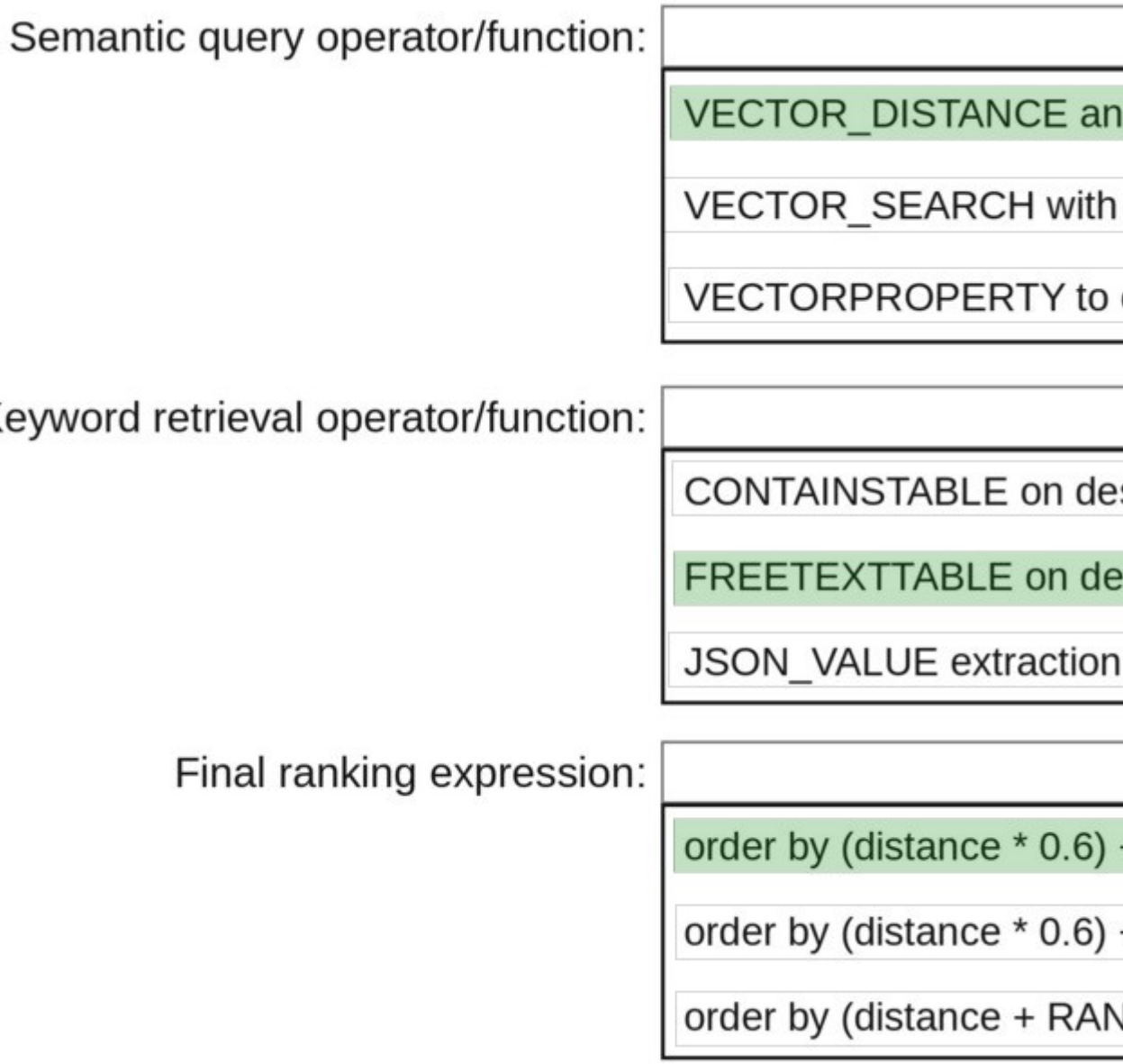
Keyword retrieval operator/function:

- CONTAINSTABLE on de
- FREETEXTTABLE on de
- JSON_VALUE extraction

Final ranking expression:

- order by (distance * 0.6)
- order by (distance * 0.6)
- order by (distance + RAN

Answer:



Section: (none)

Explanation/Reference:
Q90 / T03

Correct Answer:

Semantic query operator/function:
VECTOR_DISTANCE and order by distance ascending

Keyword retrieval operator/function:
FREETEXTTABLE on description for keyword scoring

Final ranking expression:
order by (distance * 0.6) + ((1.0 - RANK/1000.0) * 0.4)

****Explanation:****

Semantic query operator/function: En Azure SQL Database, la función nativa `VECTOR_DISTANCE` se utiliza para calcular la distancia geométrica entre dos vectores. Debido a que una distancia menor indica

una mayor similitud, la consulta debe ordenar los resultados de forma ascendente para priorizar las mejores coincidencias.

Keyword retrieval operator/function: `FREETEXTTABLE` se utiliza para búsquedas de texto completo (Full-Text Search). A diferencia de otras funciones que buscan coincidencias exactas, `FREETEXTTABLE` busca palabras con el mismo significado y devuelve una columna `RANK` que puntúa la relevancia de cada coincidencia, lo cual es esencial para calcular el peso de la palabra clave en una búsqueda híbrida.

Final ranking expression: Para combinar correctamente ambas métricas en una búsqueda híbrida, deben compartir la misma escala direccional. Mientras que en `VECTOR_DISTANCE` un valor cercano a `0` es mejor, el `RANK` de texto completo funciona a la inversa, otorgando valores más altos (hasta 1000) a las mejores coincidencias. Para unificarlos, la puntuación de texto completo se normaliza y se invierte mediante `(1.0 - RANK/1000.0)`. Después, se aplican los porcentajes requeridos multiplicando la distancia vectorial por `0.6` (60%) y el rango de texto normalizado por `0.4` (40%).

****References:****

Información general sobre vectores: https://learn.microsoft.com/es-es/sql/sql-server/ai/vectors
Documentación técnica de VECTOR_DISTANCE: https://learn.microsoft.com/es-es/sql/t-sql/functions/vector-distance-transact-sql

Documentación técnica de FREETEXTTABLE: https://learn.microsoft.com/es-es/sql/relational-databases/system-functions/freetexttable-transact-sql

QUESTION 91
You need to use the UsefulPrompts table as defined in the AI requirements.

Which stored procedure should you use?

Case Study Title (Case Study):
Case Study

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment

Azure Environment

Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
Patients
Employees
Procedures
Transactions
UsefulPrompts
ProcedureDocuments

You store a column master key as a secret in Azure Key Vault.

You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.

Problem Statements

Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.

Requirements

Planned Changes

Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.

Security Requirements

Fabrikam identifies the following security requirements:

The TransactionProcessing application must use a passwordless connection to DB1.

The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.

Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.

Database Performance Requirements

Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.

AI Search, Embeddings, and Vector Indexing

Fabrikam identifies the following AI-related requirements:

Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).

Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.

The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.

Development Requirements

Fabrikam identifies the following development requirements:

Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.

Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:

o Read data from the procedures table without authentication.

o Read and insert data into the Transactions table once authenticated.

o Execute the sp_UpdateProcedurePatient stored procedure.

Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.

Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentID INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(20) NOT NULL,
    Content NVARCHAR(MAX) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  )
```

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
{
  "entities": {
    "Procedures": {
      "source": "dbo.Procedures",
      "rest": true,
      "graphql": true,
      "permissions": [
        {
          "role": "anonymous",
          "actions": [ "read" ]
        }
      ]
    },
    {,
    "Transactions": {
      "source": "dbo.Transactions",
      "rest": true,
      "graphql": true,
      "permissions": [
        {
          "role": "authenticated",
          "actions": [ "read", "create" ]
        }
      ]
    },
    {
    "UpdateProcedurePatient": {
      "source": "dbo.sp_UpdateProcedurePatient",
      "rest": {
        "enabled": true,
        "method": "post",
        "path": "/procedurepatient"
      },
      "graphql": false,
      "permissions": [
        {
          "role": "authenticated",
          "actions": [ "execute" ]
        }
      ]
    }
  }
```

A. sp_addendpoint
B. xp_cmdshell
C. sp_OACreate
D. sp_invoke_external_rest_endpoint

Answer: D

Section: (none)

Explanation/Reference:
Q91 T03

CORRECTA
sp_invoke_external_rest_endpoint

ANÁLISIS

Concepto principal: invocación de un endpoint REST externo (Azure OpenAI) desde Azure SQL Database para cumplir el requisito de AI relacionado con la tabla UsefulPrompts.

Servicio: Azure SQL Database + Azure OpenAI.

Escenario: la tabla UsefulPrompts debe almacenar prompts que se conectan a un endpoint de Azure OpenAI para ayudar en el diagnóstico.

VERIFICACIÓN (MCP)

Documentación oficial de Microsoft Learn consultada sobre sp_invoke_external_rest_endpoint y su uso con Azure OpenAI.

JUSTIFICACIÓN

Microsoft Learn documenta que sp_invoke_external_rest_endpoint permite invocar endpoints HTTPS REST directamente desde T-SQL en Azure SQL Database. Azure OpenAI está explícitamente en la lista de endpoints permitidos (*.openai.azure.com). Este procedimiento es el mecanismo oficial para conectar la lógica de UsefulPrompts con el endpoint de Azure OpenAI.

Incorrectas
sp_addendpoint → Se usa para crear endpoints de Service Broker, no para llamadas REST externas.
xp_cmdshell → Ejecuta comandos del sistema operativo; no está diseñado ni permitido para invocar servicios REST de forma segura.
sp_OACreate → Forma parte de OLE Automation (obsoleto y restringido); no es el método soportado para integraciones modernas con Azure OpenAI.

REFERENCIA

https://learn.microsoft.com/sql/relational-databases/system-stored-procedures/sp-invoke-external-rest-endpoint-transact-sql

https://learn.microsoft.com/azure/azure-sql/database/ai-artificial-intelligence-intelligent-applications

QUESTION 92
You implement ProcedureDocuments to support the planned changes.

When users consume data through the Retrieval Augmented Generation (RAG) pattern, they experience data retrieval delays.

You need to improve the data retrieval performance and reduce the number of tokens per retrieval.

What should you implement?

Case Study Title (Case Study):
Case Study

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You

must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.

To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.

At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment

Azure Environment

Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:

- Patients

- Employees

- Procedures

- Transactions

- UsefulPrompts

- ProcedureDocuments

You store a column master key as a secret in Azure Key Vault.

You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.

Problem Statements

Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.

Requirements

Planned Changes

Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.

Security Requirements

Fabrikam identifies the following security requirements:

- The TransactionProcessing application must use a passwordless connection to DB1.

- The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.

- Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.

Database Performance Requirements

Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.

AI Search, Embeddings, and Vector Indexing

Fabrikam identifies the following AI-related requirements:

- Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).

- Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.

- The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.

Development Requirements

- Fabrikam identifies the following development requirements:

- Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.

- Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:

-- Read data from the procedures table without authentication.

-- Read and insert data into the Transactions table once authenticated.

- Execute the sp_UpdateProcedurePatient stored procedure.

- Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.

- Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
      }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

A. chunking
B. embeddings
C. a small language model (SLM)
D. JSON content

Answer: A
Section: (none)

Explanation/Reference:
Q92 T03

CORRECT ANSWER:
chunking

EXPLANATION:

La documentación oficial de Microsoft establece claramente que el chunking (particionar documentos grandes en trozos más pequeños y semánticamente coherentes) es la técnica fundamental para:

Mantener el texto por debajo de los límites de tokens de los modelos de embedding y de chat completion (ej. text-embedding-3-small tiene un límite de 8.191 tokens).
Reducir el número de tokens enviados al LLM en cada recuperación RAG.
Mejorar la precisión y el rendimiento de la búsqueda vectorial, porque se recuperan fragmentos más relevantes y pequeños en lugar de documentos completos o demasiado grandes.
Evitar truncamiento y pérdida de información.

En el caso de estudio, la tabla ProcedureDocuments ya tiene la columna Embedding VECTOR(1536). Por tanto, los embeddings ya están implementados. El problema descrito (retrasos + alto consumo de tokens) se resuelve aplicando chunking al contenido (Content) antes de generar/almacenar los embeddings.

Por qué las demás opciones son incorrectas:

embeddings
Ya están presentes en el esquema (Embedding VECTOR(1536)). Añadir o mejorar embeddings no resuelve por sí solo el problema de tokens excesivos ni de retrasos cuando se recuperan documentos grandes.

a small language model (SLM)
Un modelo de lenguaje más pequeño puede reducir coste/latencia de generación, pero no aborda el problema raíz de recuperación (cantidad de tokens enviados al modelo ni la granularidad de los resultados del vector search).

JSON content
No es una técnica de optimización de RAG ni de reducción de tokens. No tiene relación con el problema descrito.

REFERENCIA
Fuentes oficiales exactas de Microsoft Learn:

Chunk large documents for RAG and vector search in Azure AI Search
https://learn.microsoft.com/azure/search/vector-search-how-to-chunk-documents
(Explica explícitamente que el chunking reduce tokens, evita límites de modelos y mejora el rendimiento de RAG).
Retrieval-augmented generation (RAG) in Azure AI Search – Content preparation for RAG
https://learn.microsoft.com/azure/search/retrieval-augmented-generation-overview
Guía de arquitectura RAG (fase de chunking)
https://learn.microsoft.com/azure/architecture/ai-ml/guide/rag/rag-chunking-phase

QUESTION 93
Your team is developing an Azure SQL database solution from a locally cloned GitHub repository by using Microsoft Visual Studio Code and GitHub Copilot Chat.

You need to ensure that GitHub Copilot Chat uses the team's coding standards when generating Transact-SQL code in Visual Studio Code.

A. .github/copilot-instructions.md
B. .vscode/settings.json
C. %APPDATA%\Code\User\settings.json
D. %APPDATA%\Code\User\copilot-instructions.md

Answer: A
Section: (none)

Explanation/Reference:
Q93 T03

1. Respuesta Correcta:
github/copilot-instructions.md

2. Explicación Técnica Detallada:

El archivo .github/copilot-instructions.md (ubicado en la raíz del repositorio, dentro de la carpeta .github) es el mecanismo oficial de custom instructions a nivel de repositorio para GitHub Copilot Chat.
Cuando el equipo trabaja en un repositorio clonado localmente con Visual Studio Code + GitHub Copilot Chat, este archivo se incluye automáticamente en el contexto de todas las solicitudes de chat (incluyendo generación de código Transact-SQL).
Esto permite definir de forma centralizada y versionada (en Git) los estándares de codificación del equipo: convenciones de nomenclatura T-SQL, uso de tipos de datos, patrones de procedimientos almacenados, formato de scripts, comentarios obligatorios, compatibilidad Azure SQL / SQL Server, etc.
Como el archivo forma parte del repositorio, todos los miembros del equipo reciben las mismas instrucciones de forma automática, sin necesidad de configurar nada a nivel de usuario o máquina individual. Es la solución diseñada precisamente para escenarios de desarrollo colaborativo de soluciones Azure SQL Database.

3. Análisis de Opciones Incorrectas:
.vscode/settings.json
Archivo de configuración de workspace de VS Code. Se puede usar para apuntar a archivos de instrucciones adicionales mediante la setting github.copilot.chat.codeGeneration.instructions, pero no es el mecanismo principal ni el recomendado para definir los estándares del equipo de forma automática y compartida. No sustituye a .github/copilot-instructions.md.

%APPDATA%\Code\User\settings.json
Settings de usuario de VS Code (ámbito máquina/usuario). Cualquier instrucción definida aquí es personal y no se comparte con el equipo a través del repositorio. No cumple el requisito de aplicar los estándares del equipo de forma consistente.

%APPDATA%\Code\User\copilot-instructions.md
Ruta de usuario (no existe como convención oficial de repositorio). Las instrucciones personales se gestionan de otra forma y no se propagan al resto del equipo ni se versionan en el repositorio GitHub clonado.

4. Referencias Oficiales de Microsoft & SQL Server:

Customize GitHub Copilot Responses using Instruction Files (Microsoft Learn – módulo oficial):
https://learn.microsoft.com/en-us/training/modules/configure-customize-github-copilot-visual-studio-code/3-customize-copilot-responses-instruction-files
Customize GitHub Copilot Chat Responses (Microsoft Learn):
https://learn.microsoft.com/en-us/training/modules/introduction-vibe-coding/6-customize-github-copilot-responses
Quickstart: Custom Instructions for GitHub Copilot and T-SQL (MSSQL Extension for Visual Studio Code):
https://learn.microsoft.com/en-us/sql/tools/visual-studio-code-extensions/github-copilot/custom-instructions?view=sql-server-ver17
Adding repository custom instructions for GitHub Copilot (GitHub Docs – documentación oficial referenciada por Microsoft):
https://docs.github.com/en/copilot/customizing-copilot/adding-custom-instructions-for-github-copilot
Use custom instructions in VS Code (documentación VS Code):
https://code.visualstudio.com/docs/copilot/copilot-customization

QUESTION 94
You have a Microsoft SQL Server 2025 database that contains a table named products. products contains two columns named description and embedding. embedding is generated from description.
You have an application named App1. App1 has a feature that uses the following query.
VECTOR_DISTANCE('cosine', @query_vector, embedding)
Users report that the feature is slow during peak usage times.
You discover that during peak usage times, the semantic search latency increases and CPU utilization spikes.
You need to reduce latency and CPU utilization related to the App1 feature.
What should you do?

A. Create a vector index on products.embedding and change the query to use VECTOR_NORMALIZE and NORM_TYPE = 'norm1'.
B. Use CONTAINSTABLE against description only and remove the vector search.
C. Create a vector index on products.embedding and change the query to use VECTOR_SEARCH and METRIC = 'cosine'.
D. Add a nonclustered index on products.embedding.
E. Change embedding to VARBINARY(8000).

Answer: C
Section: (none)

Explanation/Reference:
Q94 T03

1. Correct Answer:
Create a vector index on products.embedding and change the query to use VECTOR_SEARCH and METRIC = 'cosine'.

2. Technical Explanation:

La consulta actual usa VECTOR_DISTANCE('cosine', @query_vector, embedding), que realiza una búsqueda exacta de vecinos más cercanos (exact kNN). Esto implica escanear todas las filas de la tabla, calcular la distancia coseno para cada vector y ordenar los resultados. En tablas grandes y durante picos de uso, genera alta latencia y picos de CPU.
En SQL Server 2025 la solución recomendada es:

Crear un vector index (algoritmo DiskANN) sobre la columna embedding especificando la misma métrica (METRIC = 'cosine').
Cambiar la consulta para usar la función VECTOR_SEARCH con METRIC = 'cosine'.

VECTOR_SEARCH realiza una búsqueda aproximada de vecinos más cercanos (ANN). El índice vectorial permite encontrar los vectores más similares de forma muy eficiente, reduciendo drásticamente el uso de CPU y la latencia, manteniendo una alta precisión (recall normalmente > 0.95).

3. Analysis of Incorrect Options:

Create a vector index on products.embedding and change the query to use VECTOR_NORMALIZE and NORM_TYPE = 'norm1'.
VECTOR_NORMALIZE normaliza vectores, pero no es la función de búsqueda. Además, no se combina de esa forma con el índice vectorial. La función correcta para aprovechar el índice es VECTOR_SEARCH.

Use CONTAINSTABLE against description only and remove the vector search.

CONTAINSTABLE es full-text search tradicional (basado en texto). Elimina la capacidad de búsqueda semántica por embeddings, que es el requisito de la aplicación.
Add a nonclustered index on products.embedding.
No se pueden crear índices B-tree (nonclustered tradicionales) sobre columnas de tipo vector. Solo se admiten vector indexes específicos (CREATE VECTOR INDEX).

Change embedding to VARBINARY(8000).
Cambiar el tipo de dato a VARBINARY elimina el soporte nativo de vectores y las funciones VECTOR_DISTANCE / VECTOR_SEARCH. No soluciona el problema de rendimiento.

4. Official Microsoft References:

https://learn.microsoft.com/en-us/sql/t-sql/functions/vector-search-transact-sql
https://learn.microsoft.com/en-us/sql/t-sql/statements/create-vector-index-transact-sql
https://learn.microsoft.com/en-us/sql/sql-server/ai/vectors
https://learn.microsoft.com/en-us/training/modules/build-ai-solutions-sql-server/3-implement-vector

QUESTION 95
You have a SQL database in Microsoft Fabric that contains a table named dbo.Products. dbo.Products contains product catalog data.

You need to create a stored procedure that performs hybrid search. The solution must meet the following requirements:

Use approximate nearest neighbor (ANN) to retrieve the top 20 candidate products.
Re-rank only the candidates that also match a full-text query.
Generate the query embedding.

How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

```
CREATE OR ALTER PROCEDURE dbo.SearchProducts
    @query_text  NVARCHAR(4000),
    @keywords    NVARCHAR(4000)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @qv VECTOR(1536) =
        [                                ▼] (@query_text USE MOD
        AI_GENERATE_EMBEDDINGS
        SEMANTICKEYPHRASETABLE
        VECTOR_NORMALIZE
        VECTOR_SEARCH

    ;WITH ann AS
    (
        SELECT t.product_id, t.product_name, s.distance
        FROM   [                                ▼] (
               CONTAINSTABLE
               SEMANTICSIMILARITYTABLE
               VECTOR_DISTANCE
               VECTOR_SEARCH
            TABLE  =  dbo.Products AS t,
            COLUMN  =  embedding,
            SIMILAR_TO = @qv,
            TOP_N = 20,
            METRIC = 'cosine'
        ) AS s
    )
```

Answer:

```
CREATE OR ALTER PROCEDURE dbo.SearchProducts
    @query_text  NVARCHAR(4000),
    @keywords    NVARCHAR(4000)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @qv VECTOR(1536) =
        [▼]                                   (@query_text USE MOD
        AI_GENERATE_EMBEDDINGS
        SEMANTICKEYPHRASETABLE
        VECTOR_NORMALIZE
        VECTOR_SEARCH

    ;WITH ann AS
    (
        SELECT t.product_id, t.product_name, s.distance
        FROM   [▼]                                   (
               CONTAINSTABLE
               SEMANTICSIMILARITYTABLE
               VECTOR_DISTANCE
               VECTOR_SEARCH
            TABLE  =  dbo.Products AS t,
            COLUMN  =  embedding,
            SIMILAR_TO = @qv,
            TOP_N = 20,
            METRIC = 'cosine'
        ) AS s
    )
```

Section: (none)

Explanation/Reference:
Q95 T03

Correct Answer:

AI_GENERATE_EMBEDDINGS
FROM VECTOR_SEARCH (
JOIN CONTAINSTABLE (dbo.Products, description, @keywords) AS fts

Explanation:

AI_GENERATE_EMBEDDINGS: la clave es "Generate the query embedding" y la sintaxis (@query_text USE MODEL Ada2Embeddings). Esa es la forma de la función, y devuelve un VECTOR. Microsoft Learn usa este mismo patrón: DECLARE @qv VECTOR(1536) = AI_GENERATE_EMBEDDINGS(... USE MODEL Ada2Embeddings).

VECTOR_SEARCH: la clave es "approximate nearest neighbor (ANN)" y los parámetros TABLE, COLUMN, SIMILAR_TO, METRIC y TOP_N. VECTOR_SEARCH es la función de ANN, y TOP_N = 20 da los 20 candidatos.

CONTAINSTABLE: la clave es la tabla full-text con RANK y [KEY] para hacer el JOIN con los candidatos de ANN, y el parámetro se llama @keywords. CONTAINSTABLE hace coincidencia de palabras y frases (keyword matching) y devuelve KEY y RANK. Con el JOIN (inner), solo sobreviven los candidatos que también cumplen el full-text.

Why not the others:

AI_GENERATE_EMBEDDINGS (1er desplegable):
SEMANTICKEYPHRASETABLE extrae frases clave de documentos.
VECTOR_NORMALIZE normaliza un vector existente, no genera embeddings. VECTOR_SEARCH busca, no genera.

VECTOR_SEARCH (2º desplegable):
CONTAINSTABLE es full-text.
SEMANTICSIMILARITYTABLE es semantic search sobre documentos.
VECTOR_DISTANCE calcula la distancia entre dos vectores, no es una función de tabla con TABLE/COLUMN/SIMILAR_TO.

CONTAINSTABLE (3er desplegable):
SEMANTICKEYPHRASETABLE y SEMANTICSIMILARITYDETAILSTABLE son de Semantic Search (otra característica), no de búsqueda por palabras clave.
FREETEXTTABLE es el único distractor sintácticamente válido: devuelve también KEY y RANK, pero hace coincidencia por significado (stemming, sinónimos), no por palabras exactas. La documentación no distingue más allá de eso, así que la elección de CONTAINSTABLE se apoya en el nombre @keywords y en que la parte semántica ya la cubre el vector search. Es también la respuesta que suele aparecer en los bancos de práctica.

Nota Fabric/versiones: TOP_N en VECTOR_SEARCH es la sintaxis de índices vectoriales de versión anterior, ahora deprecated. La versión más reciente usa SELECT TOP (N) WITH APPROXIMATE sin TOP_N. La pregunta usa TOP_N, así que responde según el código mostrado.

Reference:

VECTOR_SEARCH (Transact-SQL): https://learn.microsoft.com/sql/t-sql/functions/vector-search-transact-sql?view=sql-server-ver17
Vector search and vector indexes in the SQL Database Engine: https://learn.microsoft.com/sql/sql-server/ai/vectors?view=sql-server-ver17
Query with Full-Text Search (CONTAINSTABLE vs FREETEXTTABLE): https://learn.microsoft.com/sql/relational-databases/search/query-with-full-text-search?view=sql-server-ver17

QUESTION 96
You have a GitHub Enterprise subscription.
Your team is developing an Azure SQL dataset solution from a locally cloned GitHub repository by using Microsoft Visual Studio Code and GitHub Copilot Chat.
A mix of GitHub Copilot instructions is configured at different levels, including organization-wide, repository-wide, agent-specific, and personal.
Which instructions will take precedence over the others?

A. repository-wide
B. personal
C. organization-wide
D. agent-specific

Answer: B
Section: (none)

Explanation/Reference:
Q96 / T03

La respuesta correcta es "personal".

Según la documentación oficial de GitHub, el orden de precedencia de las instrucciones personalizadas de GitHub Copilot (de mayor a menor prioridad) es el siguiente:

Personal instructions (las de mayor prioridad)
Repository custom instructions:
Path-specific (archivos .github/instructions/**/*.instructions.md)
Repository-wide (archivo .github/copilot-instructions.md)
Agent instructions (por ejemplo, AGENTS.md)

Organization custom instructions (las de menor prioridad)

Por tanto, las instrucciones personales tienen precedencia sobre todas las demás (repository-wide, organization-wide y agent-specific). Todas las instrucciones relevantes se envían a Copilot, pero en caso de conflicto, las personales prevalecen.

Por qué las otras opciones son incorrectas:

repository-wide: Tienen prioridad media. Están por debajo de las personales y de las path-specific, pero por encima de las agent-specific y de las organization-wide.
organization-wide: Son las de menor prioridad. Se aplican a todos los miembros de la organización, pero se sobrescriben fácilmente por las de repositorio o personales.
agent-specific: Forman parte del grupo de repository custom instructions y están en el nivel más bajo de ese grupo (después de path-specific y repository-wide). No tienen la máxima prioridad.

REFERENCIAS:

- sys.dm_exec_requests: https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-objects/sys-dm-exec-requests-transact-sql?view=sql-server-ver17
- sys.dm_exec_sessions: https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-objects/sys-dm-exec-sessions-transact-sql?view=sql-server-ver17
- Troubleshoot Query Blocking in Fabric Data Warehouse: https://learn.microsoft.com/en-us/fabric/data-warehouse/troubleshoot-query-blocking

QUESTION 97

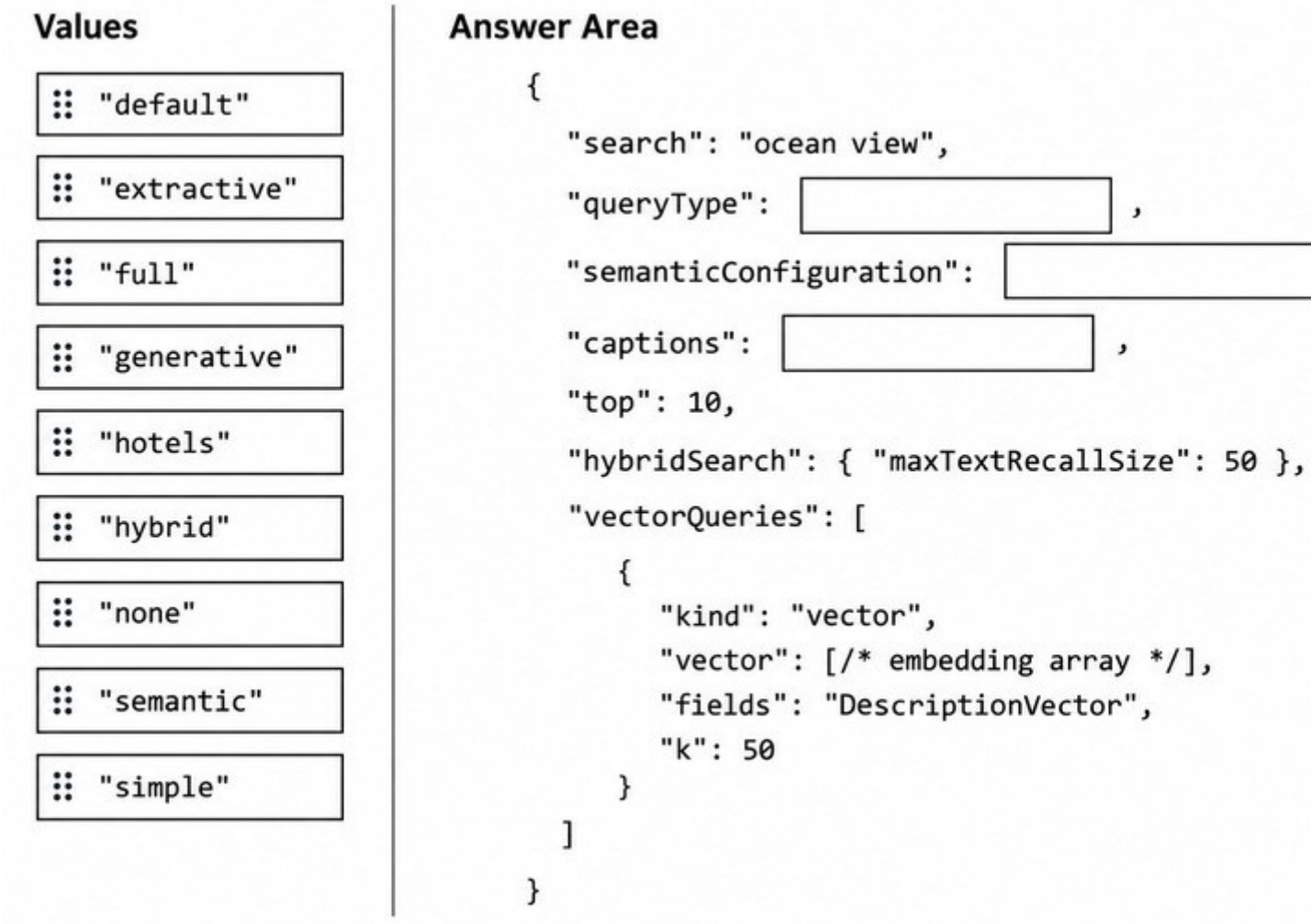
You have an Azure AI Search service and an index named hotels that includes a vector field named DescriptionVector.

You query hotels by using the Search Documents REST API.

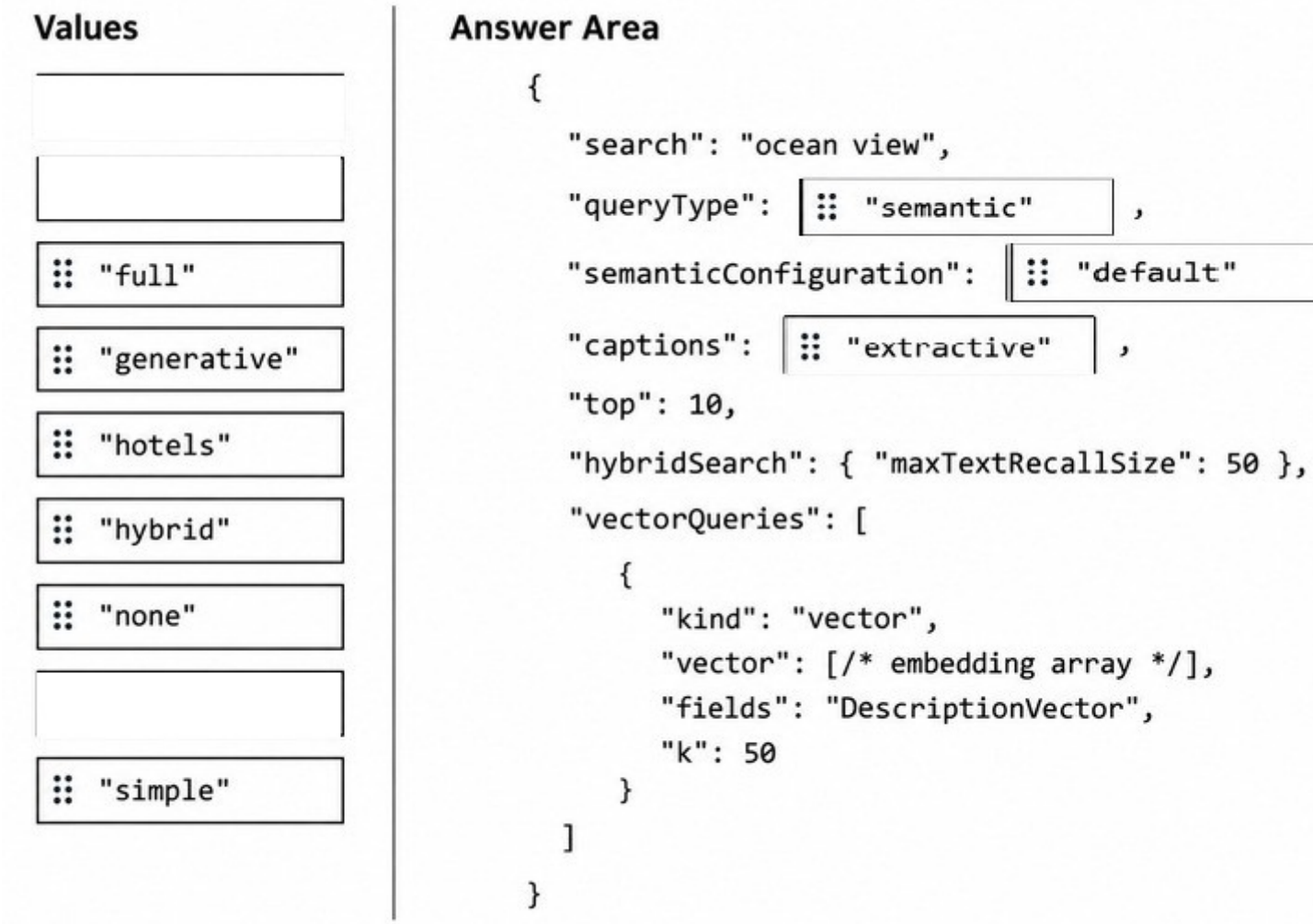
You need to implement a hybrid search query that uses DescriptionVector and includes captions.

How should you complete the REST request body? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.



Answer:



Section: (none)

Explanation/Reference:
Q97 T03

Correct Answers:
"queryType": "semantic"
"semanticConfiguration": "default"
"captions": "extractive"

Explanation:
"queryType": "semantic"
The request has both search ("ocean view") and vectorQueries, so it is already a hybrid query. The text and vector results are merged with RRF.
Captions are a semantic ranker feature, so the semantic ranker has to run over the merged results. queryType must be semantic for that.
"simple" is the default BM25 text-query mode and cannot return captions. "full" is Lucene syntax, which semantic ranking does not support through queryType.
"hybrid", "extractive", "generative", "none" and "default" are not valid queryType values. A query is hybrid because it combines search with vectorQueries, not because of this property.
"semanticConfiguration": "default"

When queryType is semantic, semanticConfiguration is required. It must name a semantic configuration defined in the index.
"default" is the only value in the list that can be a configuration name.
"hotels" is the name of the index, not of a semantic configuration. It is a distractor.
The other values are keywords for other parameters, not configuration names.
"captions": "extractive"

The valid values for captions are extractive and none. The default is none, so you must set extractive to get captions.
The semantic ranker extracts verbatim passages from the top-ranked documents, and they appear in @search.captions.
"none" disables captions. "generative" does not exist in this API, because captions and answers are extracted and never composed by an LLM.
"semantic", "hybrid", "simple", "full" and "default" are not valid caption values.
The rest of the body already follows best practice. k: 50 gives the semantic ranker its maximum of 50 inputs, and maxTextRecallSize controls how many text results go into the merge.

Resources:
https://learn.microsoft.com/azure/search/semantic-how-to-query-request#set-up-the-query
https://learn.microsoft.com/azure/search/hybrid-search-how-to-query#examples-of-hybrid-queries
https://learn.microsoft.com/azure/search/search-get-started-semantic#understand-the-code
https://learn.microsoft.com/rest/api/searchservice/documents/search-post

QUESTION 98
You have an Azure SQL database that contains tables named dbo.Tickets and dbo.TicketNotes. dbo. Tickets cotains support tickets and dbo.TicketNotes contains ticket notes.
A retrieval query returns the top five relevant ticket notes for a user question.
You plan to implement a Retrieval Augmented Generation (RAG) pattern that meets the following requirements:
- Formats the retrieved relational data for large language model (LLM) processing
- Sends the user question and retrieved context to an Azure OpenAI REST endpoint for chat completions
- Extracts the response text from the LLM response

Which Transact-SQL function should you use to extract the response text?

A. JSON_MODIFY(@response, '$.result')
B. OPENJSON(@response)
C. JSON_QUERY(@response, '$.choices')
D. JSON_VALUE(@response, '$.choices[0].message.content')

Answer: D
Section: (none)

Explanation/Reference:
Q98 T03
Correct answer: JSON_VALUE(@response, '$.choices[0].message.content')
Explanation:

La función correcta para extraer el texto de la respuesta del LLM es JSON_VALUE(@response, '$.choices [0].message.content').
Cuando se llama al endpoint de Azure OpenAI (chat completions) mediante sp_invoke_external_rest_endpoint, la respuesta llega en formato JSON. El texto generado por el modelo (el contenido del mensaje del assistant) se encuentra en la ruta choices[0].message.content. JSON_VALUE está diseñada precisamente para extraer un valor escalar (string) de un documento JSON a partir de una expresión de ruta, que es exactamente lo que se necesita para obtener el texto de la respuesta del LLM.
Las demás opciones no sirven para este propósito:

JSON_MODIFY(@response, '$.result') no extrae nada: esta función se usa para modificar (actualizar, insertar o eliminar) valores dentro de un JSON y devolver el JSON modificado, no para leer un valor.
OPENJSON(@response) convierte un documento JSON en un conjunto de filas (tabla relacional). Es útil para descomponer arrays u objetos, pero no extrae directamente el texto escalar de la respuesta del modelo.
JSON_QUERY(@response, '$.choices') extrae un objeto o un array completo (en este caso devolvería el array choices). No devuelve el valor escalar del contenido del mensaje. Además, la ruta es incompleta: no llega hasta .message.content.

Por tanto, solo JSON_VALUE con la ruta correcta al contenido del mensaje cumple el requisito de “extraer el texto de la respuesta del LLM”.

Reference:

https://learn.microsoft.com/en-us/training/modules/design-implement-rag-with-sql/5-generate-process-rag-responses
https://learn.microsoft.com/en-us/sql/t-sql/functions/json-value-transact-sql
https://learn.microsoft.com/en-us/sql/t-sql/functions/json-query-transact-sql

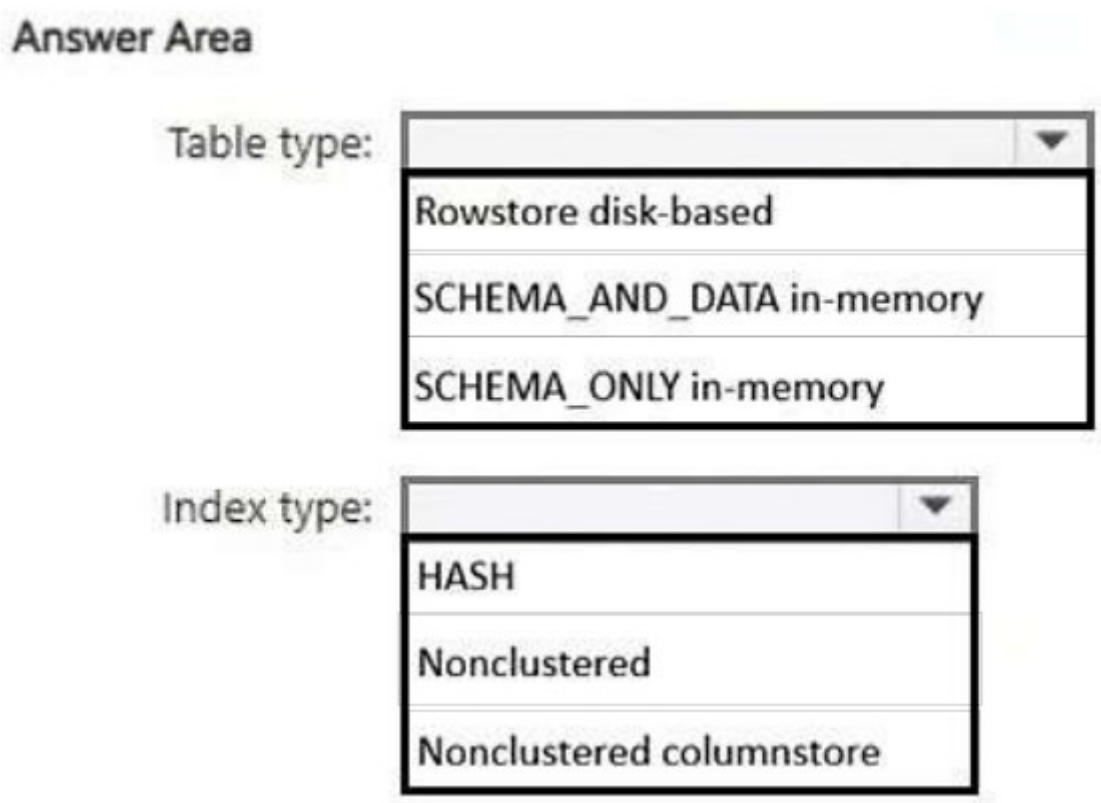
QUESTION 99
You have an Azure SQL managed instance that supports a gaming leaderboard API and contains a table named dbo.Leaderboard.

You plan to reduce write latency during peak events of dbo.Leaderboard.

You need to ensure that dbo.Leaderboard supports point lookups. The leaderboard information does NOT need to persist after a restart.

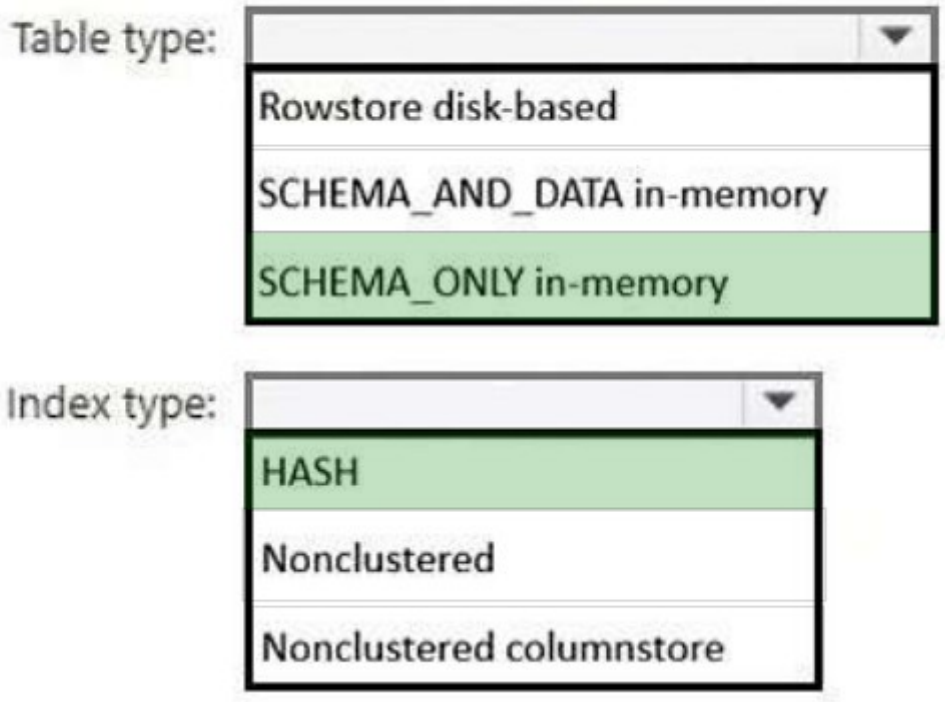
Which type of table and index should you configure? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.



Answer:

Answer Area



Section: (none)

Explanation/Reference:
Q99 / T03

1. Respuesta correcta.

Table type: SCHEMA_ONLY in-memory
Index type: HASH

2. Explicación de por qué esa es la respuesta correcta y por qué las otras son incorrectas.

El escenario requiere:

Reducir la latencia de escritura durante picos de eventos (alto throughput de inserts/updates en un leaderboard de gaming).
Soporte de point lookups (búsquedas por igualdad, típicas en un ranking: “dame la puntuación del jugador X”).
Que la información no necesite persistir después de un reinicio.

Las tablas memory-optimized (In-Memory OLTP) de Azure SQL Managed Instance están diseñadas exactamente para este tipo de cargas: eliminan latches y bloqueos tradicionales, mantienen los datos en memoria y ofrecen latencias de escritura mucho más bajas.
Dentro de las tablas memory-optimized existen dos opciones de durabilidad:

SCHEMA_AND_DATA: persiste tanto el esquema como los datos (sobreviven a reinicios).
SCHEMA_ONLY: solo persiste el esquema; los datos se pierden en un reinicio o failover.

Como el enunciado indica explícitamente que “the leaderboard information does NOT need to persist after a restart”, la opción correcta es SCHEMA_ONLY in-memory.

Para el índice:

Los HASH indexes en tablas memory-optimized están optimizados para point lookups (predicado de igualdad en todas las columnas de la clave). Proporcionan un acceso de tiempo constante y son la elección recomendada cuando solo se necesitan búsquedas por igualdad.
Un índice Nonclustered (B-tree) es mejor para rangos y ordenaciones, no el caso principal aquí.
Un Nonclustered columnstore está pensado para analítica, no para OLTP de alta concurrencia con point lookups.

Por tanto, la combinación correcta es SCHEMA_ONLY in-memory + HASH.

Por qué las demás opciones son incorrectas:

Rowstore disk-based: tablas tradicionales en disco. No aprovechan In-Memory OLTP, por lo que no reducen la latencia de escritura de forma significativa en picos.
SCHEMA_AND_DATA in-memory: persistiría los datos, lo cual viola el requisito de no necesitar persistencia tras reinicio.
Nonclustered: útil para rangos, no óptimo para point lookups puros.
Nonclustered columnstore: orientado a cargas analíticas, no a un leaderboard OLTP de baja latencia.

3. Referencias:

https://learn.microsoft.com/en-us/sql/relational-databases/in-memory-oltp/defining-durability-for-memory-optimized-objects
https://learn.microsoft.com/en-us/sql/relational-databases/in-memory-oltp/indexes-for-memory-optimized-tables
https://learn.microsoft.com/en-us/azure/azure-sql/managed-instance/in-memory-oltp-overview

QUESTION 100
You have an Azure SQL database named SalesDB. SalesDB contains a table named dbo.Articles. dbo. Articles contains the following columns:

• ArticleId

• Title

• Body

• LastModifiedUtc

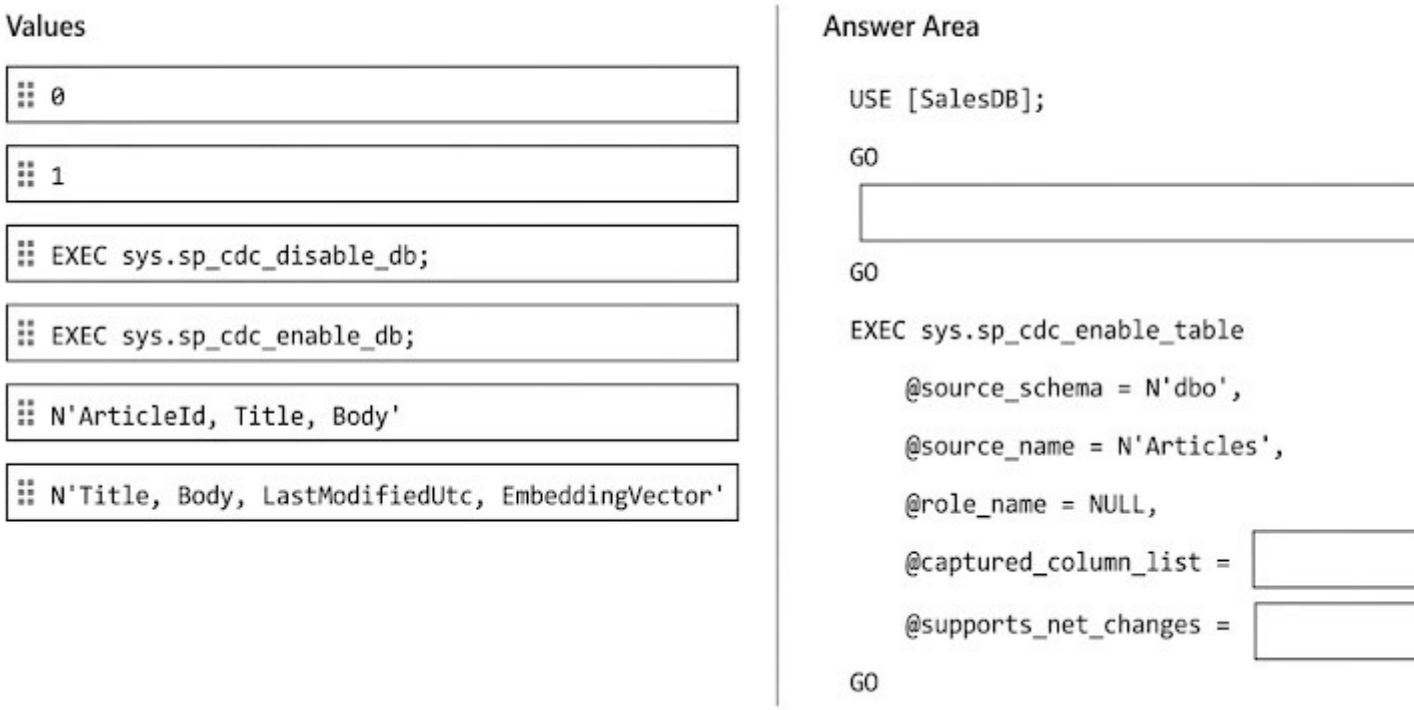
• EmbeddingVector

You have an application that generates embeddings from the concatenation of Title and Body and stores the results in EmbeddingVector.

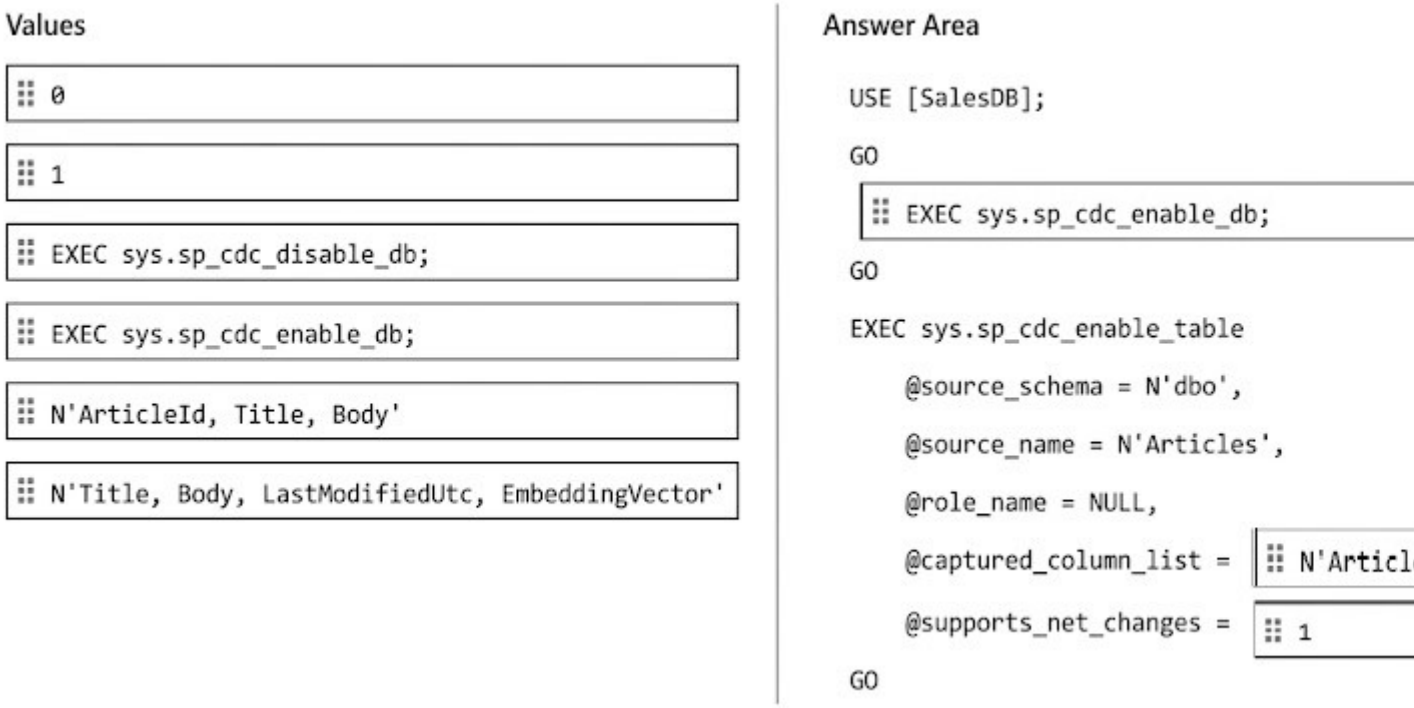
You plan to implement an incremental embedding maintenance method that will use change data capture (CDC) to update embeddings only for rows that change, without scanning the entire table.

You need to ensure that only the columns required to generate the embeddings are captured. The solution must support querying net changes.

How should you complete the Transact-SQL script? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.



Answer:



Section: (none)

Explanation/Reference:
Q100 T03

<u>Los tres huecos del drag and drop se rellenan así:</u>
Primer hueco (después del primer GO):
EXEC sys.sp_cdc_enable_db;

Segundo hueco (@captured_column_list):
N'ArticleId, Title, Body'

Tercer hueco (@supports_net_changes):
1

Por qué estos tres valores son los correctos:
EXEC sys.sp_cdc_enable_db es el único valor que habilita CDC a nivel de base de datos, paso previo obligatorio antes de poder habilitarlo en una tabla.
N'ArticleId, Title, Body' captura únicamente las columnas necesarias para generar el embedding (Title y Body) e incluye ArticleId porque es la clave primaria, requisito indispensable cuando se activa el soporte de net changes.
El valor 1 activa el soporte de consultas de net changes, tal como exige el enunciado.

Por qué las demás opciones no se usan:
0 desactiva el soporte de net changes, lo que va en contra del requisito.
EXEC sys.sp_cdc_disable_db deshabilita CDC en toda la base de datos, que es lo contrario de lo que se necesita.
N'Title, Body, LastModifiedUtc, EmbeddingVector' incluye columnas que no intervienen en la generación del embedding y, además, omite ArticleId, que es obligatorio para poder usar net changes.

Referencias oficiales de Microsoft Learn:
Documentación de sys.sp_cdc_enable_table:

https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sys-sp-cdc-enable-table-transact-sql
Cómo habilitar y deshabilitar Change Data Capture:

https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/enable-and-disable-change-data-capture-sql-server
Change Data Capture en Azure SQL Database:

https://learn.microsoft.com/en-us/azure/azure-sql/database/change-data-capture-overview

QUESTION 101
You have an Azure SQL database that contains a table named Tickets_Embeddings. Tickets_Embeddings contains a VECTOR(1536) column.

Embeddings are generated by using text-embedding-ada-002 and the /openai/v1/embeddings endpoint.

After a review of the current environment, the following changes are requested:

- The AI architect has recommended switching to text-embedding-3-small.
- The security team requires authentication by using Microsoft Entra managed identities only. Storing API keys in an application is prohibited.

Which two actions should you include in the solution? Each correct answer presents a complete solution.

NOTE: Each correct selection is worth one point.

A. Alter the database tables.
B. Regenerate all the embeddings.
C. Change the endpoint.
D. Change the primary keys for the tickets.
E. Add a firewall rule.

Answer: BC
Section: (none)

Explanation/Reference:
Q101 T03

Respuesta Correcta:

Regenerate all the embeddings.

Change the endpoint.

Explanation:

Regenerate all the embeddings. Los vectores generados por modelos de embeddings distintos pertenecen a espacios vectoriales diferentes y no son comparables entre sí. Si se cambia de text-embedding-ada-002 a text-embedding-3-small, hay que regenerar todos los embeddings ya almacenados. De lo contrario, la búsqueda de similitud (por ejemplo con VECTOR_DISTANCE) mezclaría vectores de dos modelos distintos y devolvería resultados incoherentes.

Change the endpoint. Al cambiar de modelo hay que apuntar a la nueva implementación (deployment) de text-embedding-3-small en Azure OpenAI, y por tanto cambiar el endpoint de embeddings. Además, el requisito de seguridad obliga a autenticarse solo con una identidad administrada de Microsoft Entra (sin claves de API), por lo que el modelo externo se define con la URL del nuevo endpoint y una credencial de ámbito de base de datos de tipo Managed Identity.

Incorrectas
Alter the database tables. text-embedding-3-small devuelve por defecto vectores de 1536 dimensiones, las mismas que text-embedding-ada-002, por lo que la columna VECTOR(1536) sigue siendo válida y no hace falta modificar las tablas.

Change the primary keys for the tickets. Las claves primarias de los tickets no tienen relación con el modelo de embeddings ni con el método de autenticación, así que cambiarlas no aporta nada a la solución.

Add a firewall rule. Una regla de firewall controla el acceso de red al servidor, pero no cambia el modelo de embeddings ni el método de autenticación. No cumple ninguno de los dos requisitos.

Referencias:
Microsoft Learn - Azure OpenAI embeddings (modelos text-embedding-3-small y ada-002, dimensiones):
https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/understand-embeddings

Microsoft Learn - CREATE EXTERNAL MODEL (Transact-SQL), con endpoint y credencial Managed Identity:
https://learn.microsoft.com/en-us/sql/t-sql/statements/create-external-model-transact-sql

Microsoft Learn - Vector data type (Transact-SQL):
https://learn.microsoft.com/en-us/sql/t-sql/data-types/vector-data-type